LÖVE WIKI · PART 4 OF 9

Types (2/2)

639 pages

wiki pages 651–1289

Contents of this part

Mercurial
Mesh
Mesh:attachAttribute
Mesh:detachAttribute
Mesh:flush
Mesh:getDraw Mode
Mesh:getDraw Range
Mesh:getImage
Mesh:getTexture
Mesh:getVertex
Mesh:getVertexAttribute
Mesh:getVertexCount
Mesh:getVertexFormat
Mesh:getVertexMap
Mesh:getVertices
Mesh:hasVertexColors
Mesh:isAttributeEnabled
Mesh:setAttributeEnabled
Mesh:setDraw Mode
Mesh:setDraw Range
Mesh:setImage
Mesh:setTexture
Mesh:setVertex
Mesh:setVertexAttribute
Mesh:setVertexColors
Mesh:setVertexMap
Mesh:setVertices
MeshDraw Mode
MessageBoxType
MGL
MiddleClass
MiniFS
MipmapMode
Missilium
ModifierKey
Monocle
Moses
MotorJoint
MotorJoint:getAngularOffset
MotorJoint:getLinearOffset
MotorJoint:setAngularOffset
MotorJoint:setLinearOffset
MountPermissions
MouseConstant
MouseJoint
MouseJoint:getDampingRatio
MouseJoint:getFrequency
MouseJoint:getMaxForce
MouseJoint:getTarget
MouseJoint:setDampingRatio
MouseJoint:setFrequency
MouseJoint:setMaxForce
MouseJoint:setTarget
Mr. Rescue
MyPaint
N.T.U.I.
Netw orking
New Pages
NoHighDPI
NoiseWrapper
Noobhub
Object
Object:release
Object:type
Object:typeOf
Objects
ONET
OpenCanvas
Optimising
os.time
ParticleInsertMode
ParticleSystem
ParticleSystem:clone
ParticleSystem:count
ParticleSystem:emit
ParticleSystem:getAreaSpread
ParticleSystem:getBufferSize
ParticleSystem:getColors
ParticleSystem:getCount
ParticleSystem:getDirection
ParticleSystem:getEmissionArea
ParticleSystem:getEmissionRate
ParticleSystem:getEmitterLifetime
ParticleSystem:getImage
ParticleSystem:getInsertMode
ParticleSystem:getLinearAcceleration
ParticleSystem:getLinearDamping
ParticleSystem:getOffset
ParticleSystem:getOffsetX
ParticleSystem:getOffsetY
ParticleSystem:getParticleLifetime
ParticleSystem:getPosition
ParticleSystem:getQuads
ParticleSystem:getRadialAcceleration
ParticleSystem:getRotation
ParticleSystem:getSizes
ParticleSystem:getSizeVariation
ParticleSystem:getSpeed
ParticleSystem:getSpin
ParticleSystem:getSpinVariation
ParticleSystem:getSpread
ParticleSystem:getTangentialAcceleration
ParticleSystem:getTexture
ParticleSystem:getX
ParticleSystem:getY
ParticleSystem:hasRelativeRotation
ParticleSystem:isActive
ParticleSystem:isEmpty
ParticleSystem:isFull
ParticleSystem:isPaused
ParticleSystem:isStopped
ParticleSystem:moveTo
ParticleSystem:pause
ParticleSystem:reset
ParticleSystem:setAreaSpread
ParticleSystem:setBufferSize
ParticleSystem:setColor
ParticleSystem:setColors
ParticleSystem:setDirection
ParticleSystem:setEmissionArea
ParticleSystem:setEmissionRate
ParticleSystem:setEmitterLifetime
ParticleSystem:setGravity
ParticleSystem:setImage
ParticleSystem:setInsertMode
ParticleSystem:setLifetime
ParticleSystem:setLinearAcceleration
ParticleSystem:setLinearDamping
ParticleSystem:setOffset
ParticleSystem:setParticleLife
ParticleSystem:setParticleLifetime
ParticleSystem:setPosition
ParticleSystem:setQuads
ParticleSystem:setRadialAcceleration
ParticleSystem:setRelativeRotation
ParticleSystem:setRotation
ParticleSystem:setSize
ParticleSystem:setSizes
ParticleSystem:setSizeVariation
ParticleSystem:setSpeed
ParticleSystem:setSpin
ParticleSystem:setSpinVariation
ParticleSystem:setSpread
ParticleSystem:setSprite
ParticleSystem:setTangentialAcceleration
ParticleSystem:setTexture
ParticleSystem:start
ParticleSystem:stop
ParticleSystem:update
PATH
Physics
PixelEffect
PixelEffect:getWarnings
PixelEffect:send
PixelFormat
Pnorar
PointInConcavePolygon
PointInConvexPolygon
PointStyle
PointWithinShape
Polybius
PolygonShape
PolygonShape:getPoints
PolygonShape:validate
Popo
Pow erState
PrismaticJoint
PrismaticJoint:areLimitsEnabled
PrismaticJoint:enableLimit
PrismaticJoint:enableMotor
PrismaticJoint:getAxis
PrismaticJoint:getJointSpeed
PrismaticJoint:getJointTranslation
PrismaticJoint:getLimits
PrismaticJoint:getLow erLimit
PrismaticJoint:getMaxMotorForce
PrismaticJoint:getMotorForce
PrismaticJoint:getMotorSpeed
PrismaticJoint:getReferenceAngle
PrismaticJoint:getUpperLimit
PrismaticJoint:hasLimitsEnabled
PrismaticJoint:isLimitEnabled
PrismaticJoint:isLimitsEnabled
PrismaticJoint:isMotorEnabled
PrismaticJoint:setLimits
PrismaticJoint:setLimitsEnabled
PrismaticJoint:setLow erLimit
PrismaticJoint:setMaxMotorForce
PrismaticJoint:setMotorEnabled
PrismaticJoint:setMotorSpeed
PrismaticJoint:setUpperLimit
Pseudo3D
PulleyJoint
PulleyJoint:getConstant
PulleyJoint:getGroundAnchors
PulleyJoint:getLength1
PulleyJoint:getLength2
PulleyJoint:getLengthA
PulleyJoint:getLengthB
PulleyJoint:getMaxLengths
PulleyJoint:getRatio
PulleyJoint:setConstant
PulleyJoint:setMaxLengths
PulleyJoint:setRatio
PyCharm
Quad
Quad:flip
Quad:getLayer
Quad:getTextureDimensions
Quad:getView port
Quad:setLayer
Quad:setView port
Quasimofo
Quickie
RandomGenerator
RandomGenerator:getSeed
RandomGenerator:getState
RandomGenerator:random
RandomGenerator:randomNormal
RandomGenerator:setSeed
RandomGenerator:setState
RandomLua
Rasterizer
Rasterizer:getAdvance
Rasterizer:getAscent
Rasterizer:getDescent
Rasterizer:getGlyphCount
Rasterizer:getGlyphData
Rasterizer:getHeight
Rasterizer:getLineHeight
Rasterizer:hasGlyphs

Rebound
RecordingDevice
RecordingDevice:getBitDepth
RecordingDevice:getChannelCount
RecordingDevice:getData
RecordingDevice:getName
RecordingDevice:getSampleCount
RecordingDevice:getSampleRate
RecordingDevice:isRecording
RecordingDevice:start
RecordingDevice:stop
RevoluteJoint
RevoluteJoint:areLimitsEnabled
RevoluteJoint:enableLimit
RevoluteJoint:enableMotor
RevoluteJoint:getJointAngle
RevoluteJoint:getJointSpeed
RevoluteJoint:getLimits
RevoluteJoint:getLow erLimit
RevoluteJoint:getMaxMotorTorque
RevoluteJoint:getMotorSpeed
RevoluteJoint:getMotorTorque
RevoluteJoint:getReferenceAngle
RevoluteJoint:getUpperLimit
RevoluteJoint:hasLimitsEnabled
RevoluteJoint:isLimitsEnabled
RevoluteJoint:isMotorEnabled
RevoluteJoint:setLimits
RevoluteJoint:setLimitsEnabled
RevoluteJoint:setLow erLimit
RevoluteJoint:setMaxMotorTorque
RevoluteJoint:setMotorEnabled
RevoluteJoint:setMotorSpeed
RevoluteJoint:setUpperLimit
RichText
RopeJoint
RopeJoint:getMaxLength
RopeJoint:setMaxLength
RotLove
Ryomaechizen
Scancode
SciTE
SensorType
Ser
Serial
Serialization
Serializing
ServerObject
ServerObject.tcp:acceptAll
ServerObject:checkPing
ServerObject:Init
ServerObject:receive
ServerObject:respondToServBrow ser
ServerObject:send
ServerObject:setCallback
ServerObject:setHandshake
ServerObject:setPing
ServerObject:startserver
ServerObject:update
sfxr.lua
Shader
Shader:getDebugName
Shader:getExternVariable
Shader:getWarnings
Shader:hasStage
Shader:hasUniform
Shader:send
Shader:sendColor
ShaderStage
Shape
Shape:computeAABB
Shape:computeMass
Shape:destroy
Shape:getBody
Shape:getBoundingBox
Shape:getCategory
Shape:getCategoryBits
Shape:getChildCount
Shape:getData
Shape:getDensity
Shape:getFilterData
Shape:getFriction
Shape:getMask
Shape:getMaskBits
Shape:getRadius
Shape:getRestitution
Shape:getType
Shape:isSensor
Shape:rayCast
Shape:setCategory
Shape:setData
Shape:setDensity
Shape:setFilterData
Shape:setFriction
Shape:setMask
Shape:setRestitution
Shape:setSensor
Shape:testPoint
Shape:testSegment
ShapeType
Shear
SICK
SLAM
SoundData
SoundData:copyFrom
SoundData:getBitDepth
SoundData:getBits
SoundData:getChannelCount
SoundData:getChannels
SoundData:getDuration
SoundData:getSample
SoundData:getSampleCount
SoundData:getSampleRate
SoundData:setSample
SoundData:slice
SoundManager
Source
Source:clone
Source:getActiveEffects
Source:getAirAbsorption
Source:getAttenuationDistances
Source:getChannelCount
Source:getChannels
Source:getCone
Source:getDirection
Source:getDistance
Source:getDuration
Source:getEffect
Source:getFilter
Source:getFreeBufferCount
Source:getPitch
Source:getPosition
Source:getRolloff
Source:getType
Source:getVelocity
Source:getVolume
Source:getVolumeLimits
Source:isLooping
Source:isPaused
Source:isPlaying
Source:isRelative
Source:isStatic
Source:isStopped
Source:pause
Source:play
Source:queue
Source:resume
Source:rew ind
Source:seek
Source:setAirAbsorption
Source:setAttenuationDistances
Source:setCone
Source:setDirection
Source:setDistance
Source:setEffect
Source:setFilter
Source:setLooping
Source:setPitch
Source:setPosition
Source:setRelative
Source:setRolloff
Source:setVelocity
Source:setVolume
Source:setVolumeLimits
Source:stop
Source:tell
SourceTree
SourceType
Space
Spookyard
SpriteBatch
SpriteBatch:add
SpriteBatch:addLayer
SpriteBatch:addq
SpriteBatch:attachAttribute
SpriteBatch:bind
SpriteBatch:clear
SpriteBatch:flush
SpriteBatch:getBufferSize
SpriteBatch:getColor
SpriteBatch:getCount
SpriteBatch:getImage
SpriteBatch:getTexture
SpriteBatch:lock
SpriteBatch:set
SpriteBatch:setBufferSize
SpriteBatch:setColor
SpriteBatch:setDraw Range
SpriteBatch:setImage
SpriteBatch:setLayer
SpriteBatch:setq
SpriteBatch:setTexture
SpriteBatch:unbind
SpriteBatch:unlock
SQLite3
SquareWorld
sstrict.lua
StackingSceneMgr
StackType
stateful.lua
Stateswitcher
StencilAction
StreamType
SUIT
Sukippu
SURVIVE
Swingers
SystemTheme
Tactile
Talkies.lua
Terus
TEsound
Text
text.lua
Text:add
Text:addf
Text:clear
Text:getDimensions
Text:getFont
Text:getHeight
Text:getWidth
Text:set
Text:setf
Text:setFont
TextBatch
TextBatch:add
TextBatch:addf
TextBatch:clear
TextBatch:getDimensions
TextBatch:getFont
TextBatch:getHeight
TextBatch:getWidth
TextBatch:set
TextBatch:setf
TextBatch:setFont
TextInputField
Texture
Texture:generateMipmaps
Texture:getDebugName
Texture:getDepth

Texture:getDepthSampleMode
Texture:getDimensions
Texture:getDPIScale
Texture:getFilter
Texture:getFormat
Texture:getHeight
Texture:getLayerCount
Texture:getMipmapCount
Texture:getMipmapFilter
Texture:getMSAA
Texture:getPixelDimensions
Texture:getPixelHeight
Texture:getPixelWidth
Texture:getTextureType
Texture:getWidth
Texture:getWrap
Texture:isCanvas
Texture:isComputeWritable
Texture:isFormatLinear
Texture:isReadable
Texture:renderTo
Texture:replacePixels
Texture:setDepthSampleMode
Texture:setFilter
Texture:setMipmapFilter
Texture:setWrap
TexturedPolygon
TextureFormat
TextureType
Thranduil
Thread
Thread:demand
Thread:get
Thread:getError
Thread:getKeys
Thread:getName
Thread:isRunning
Thread:kill
Thread:peek
Thread:receive
Thread:send
Thread:set
Thread:start
Thread:w ait
Tiled
TiledMapLoader
TileMerging
TimeUnit
TLbind
TLfres
TLpath
TLTools
TouchDeviceType
Transform
Transform:apply
Transform:clone
Transform:getMatrix
Transform:inverse
Transform:inverseTransformPoint
Transform:isAffine2DTransform
Transform:reset
Transform:rotate
Transform:scale
Transform:setMatrix
Transform:setTransformation
Transform:shear
Transform:transformPoint
Transform:translate
TransitionMgr
Tserial
TutorialTestPage
tw een.lua
Types
UaLove
UbuntuSecurity
Underlife
Unrequited
Variant
Vectors
VertexAttributeStep
VertexWinding
Video
Video:getDimensions
Video:getFilter
Video:getHeight
Video:getSource
Video:getStream
Video:getWidth
Video:isPlaying
Video:pause
Video:play
Video:rew ind
Video:seek
Video:setFilter
Video:setSource
Video:tell
VideoStream
VideoStream:getFilename
VideoStream:isPlaying
VideoStream:pause
VideoStream:play
VideoStream:rew ind
VideoStream:seek
VideoStream:setSync
VideoStream:tell
Wasx
WeldJoint
WeldJoint:getDampingRatio
WeldJoint:getFrequency
WeldJoint:getReferenceAngle
WeldJoint:setDampingRatio
WeldJoint:setFrequency
WheelJoint
WheelJoint:enableMotor
WheelJoint:getAxis
WheelJoint:getJointSpeed
WheelJoint:getJointTranslation
WheelJoint:getMaxMotorTorque
WheelJoint:getMotorSpeed
WheelJoint:getMotorTorque
WheelJoint:getSpringDampingRatio
WheelJoint:getSpringFrequency
WheelJoint:isMotorEnabled
WheelJoint:setMaxMotorTorque
WheelJoint:setMotorEnabled
WheelJoint:setMotorSpeed
WheelJoint:setSpringDampingRatio
WheelJoint:setSpringFrequency
World
World:destroy
World:getAllow Sleeping
World:getBodies
World:getBodyCount
World:getBodyList
World:getCallbacks
World:getContactCount
World:getContactFilter
World:getContactList
World:getContacts
World:getGravity
World:getJointCount
World:getJointList
World:getJoints
World:getMeter
World:getShapesInArea
World:isAllow Sleep
World:isDestroyed
World:isLocked
World:isSleepingAllow ed
World:queryBoundingBox
World:queryShapesInArea
World:rayCast
World:rayCastAny
World:rayCastClosest
World:setAllow Sleep
World:setAllow Sleeping
World:setCallbacks
World:setContactFilter
World:setGravity
World:setMeter
World:setSleepingAllow ed
World:translateOrigin
World:update
WrapMode
Yonder
Zoetrope
Zuli
ZURUECKgefilmt

Mercurial

Mercurial · revision 11676 · last edited 2021-03-14

Mercurial is a free, distributed source control management tool. It offers you the power to efficiently handle projects of any size while using an intuitive interface. It is easy to use and hard to break, making it ideal for anyone working with versioned files.

Mesh

Mesh · revision 27747 · last edited 2026-04-11

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This type is not supported in earlier versions.

A polygon mesh (http://en.wikipedia.org/wiki/Polygon_mesh) used for drawing arbitrary textured shapes.

Constructors

love.graphics.newMesh	Creates a new Mesh.	Added since 0.9.0

Functions

Mesh:attachAttribute	Attaches a vertex attribute from a different Mesh onto this Mesh, for use when drawing. Optionally allows per-instance attributes.	Added since 0.10.0	
Mesh:detachAttribute	Removes a previously attached vertex attribute from this Mesh.	Added since 11.0	
Mesh:flush	Immediately sends all modified vertex data in the Mesh to the graphics card.	Added since 0.10.0	
Mesh:getDrawMode	Gets the mode used when drawing the Mesh.	Added since 0.9.0	
Mesh:getDrawRange	Gets the range of vertices used when drawing the Mesh.	Added since 0.9.1	
Mesh:getImage	Gets the Image used when drawing the Mesh.	Added since 0.9.0	Removed in 0.10.0
Mesh:getTexture	Gets the texture (Image or Canvas) used when drawing the Mesh.	Added since 0.9.1	
Mesh:getVertex	Gets the properties of a vertex in the Mesh.	Added since 0.9.0	
Mesh:getVertexAttribute	Gets the properties of a specific attribute within a vertex in the Mesh.	Added since 0.10.0	
Mesh:getVertexCount	Gets the total number of vertices in the Mesh.	Added since 0.9.0	
Mesh:getVertexFormat	Gets the vertex format that the Mesh was created with.	Added since 0.10.0	
Mesh:getVertexMap	Gets the vertex map for the Mesh.	Added since 0.9.0	
Mesh:getVertices	Gets all the vertices in the Mesh.	Added since 0.9.0	Removed in 0.10.0
Mesh:hasVertexColors	Gets whether per-vertex colors are used when drawing the Mesh.	Added since 0.9.0	Removed in 0.10.0
Mesh:isAttributeEnabled	Gets whether a specific vertex attribute in the Mesh is enabled.	Added since 0.10.0	
Mesh:setAttributeEnabled	Enables or disables a specific vertex attribute in the Mesh.	Added since 0.10.0	
Mesh:setDrawMode	Sets the mode used when drawing the Mesh.	Added since 0.9.0	
Mesh:setDrawRange	Restricts the drawn vertices of the Mesh to a subset of the total.	Added since 0.9.1	
Mesh:setImage	Sets the Image used when drawing the Mesh.	Added since 0.9.0	Removed in 0.10.0
Mesh:setTexture	Sets the texture (Image or Canvas) used when drawing the Mesh.	Added since 0.9.1	
Mesh:setVertex	Sets the properties of a vertex in the Mesh.	Added since 0.9.0	

Mesh:setVertexAttribute	Sets the properties of a specific attribute within a vertex in the Mesh.	Added since 0.10.0	
Mesh:setVertexColors	Sets whether per-vertex colors are used instead of the constant color when drawing the Mesh.	Added since 0.9.0	Removed in 0.10.0
Mesh:setVertexMap	Sets the vertex map for the Mesh.	Added since 0.9.0	
Mesh:setVertices	Replaces a range of vertices in the Mesh with new ones.	Added since 0.9.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		

Enums

AttributeDataType	Data types used in a Mesh's vertex format.	Added since 0.9.0
BufferDataUsage	Usage hints for SpriteBatches, Meshes, and GraphicsBuffers to optimize data storage and access.	Added since 0.8.0
IndexDataType	Vertex map datatype.	Added since 11.0
MeshDrawMode	How a Mesh's vertices are used when drawing.	Added since 0.9.0
VertexAttributeStep	The frequency at which a vertex shader fetches the vertex attribute's data from the Mesh when it's drawn.	Added since 11.0

Supertypes

- Drawable
- Object

See Also

- love.graphics
- love.graphics.draw
- SpriteBatch:attachAttribute

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:attachAttribute

Mesh:attachAttribute · revision 27032 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Attaches a vertex attribute from a different Mesh onto this Mesh, for use when drawing. This can be used to share vertex attribute data between several different Meshes.

Function

Synopsis

```
Mesh:attachAttribute( name, mesh )
```

Arguments

`string name`
: The name of the vertex attribute to attach.

`Mesh mesh`
: The Mesh to get the vertex attribute from.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
Mesh:attachAttribute( name, mesh, step, attachname )
```

Arguments

`string name`
: The name of the vertex attribute to attach.

`Mesh mesh`
: The Mesh to get the vertex attribute from.

`VertexAttributeStep step ("pervertex")`
: Whether the attribute will be per-vertex or per-instance when the mesh is drawn.

`string attachname (name)`

The name of the attribute to use in shader code. Defaults to the name of the attribute in the given mesh. Can be used to use a different name for this attribute when rendering.

Returns

Nothing.

Notes

If a Mesh wasn't created with a custom vertex format, it will have 3 vertex attributes named `VertexPosition`, `VertexTexCoord`, and `VertexColor`.

Custom named attributes can be accessed in a vertex shader by declaring them as `attribute vec4 MyCustomAttributeName;` at the top-level of the vertex shader code. The name must match what was specified in the Mesh's vertex format and in the `name` argument of Mesh:attachAttribute.

See Also

- Mesh
- Mesh:getVertexFormat
- Mesh:detachAttribute
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:detachAttribute

Mesh:detachAttribute · revision 22630 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Removes a previously attached vertex attribute from this Mesh.

Function

Synopsis

```
success = Mesh:detachAttribute( name )
```

Arguments

`string name`
: The name of the attached vertex attribute to detach.

Returns

`boolean success`
: Whether the attribute was successfully detached.

See Also

- Mesh
- Mesh:attachAttribute

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:flush

Mesh:flush · revision 15116 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Immediately sends all modified vertex data in the Mesh to the graphics card.

Normally it isn't necessary to call this method as love.graphics.draw(mesh, ...) will do it automatically if needed, but explicitly using Mesh:flush gives more control over when the work happens.

If this method is used, it generally shouldn't be called more than once (at most) between love.graphics.draw(mesh, ...) calls.

Function

Synopsis

```
Mesh:flush( )
```

Arguments

None.

Returns

Nothing.

See Also

- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getDrawMode

Mesh:getDrawMode · revision 11783 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the mode used when drawing the Mesh.

Function

Synopsis

```
mode = Mesh:getDrawMode( )
```

Arguments

None.

Returns

`MeshDrawMode mode`

The mode used when drawing the Mesh.

See Also

- Mesh
- love.graphics.newMesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getDrawRange

Mesh:getDrawRange · revision 12978 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets the range of vertices used when drawing the Mesh.

Function

Synopsis

```
min, max = Mesh:getDrawRange( )
```

Arguments

None.

Returns

`number min (nil)`

The index of the first vertex used when drawing, or the index of the first value in the vertex map used if one is set for this Mesh.

`number max (nil)`

The index of the last vertex used when drawing, or the index of the last value in the vertex map used if one is set for this Mesh.

Notes

If the Mesh's draw range has not been set previously with Mesh:setDrawRange, this function will return nil.

See Also

- Mesh
- Mesh:setDrawRange

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getImage

Mesh:getImage · revision 15737 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Removed in LÖVE 0.10.0

Use Mesh:getTexture instead.

Gets the Image used when drawing the Mesh.

Function

Synopsis

```
image = Mesh:getImage( )
```

Arguments

None.

Returns

`Image image (nil)`

The Image used to texture the Mesh when drawing. May be nil if no Image is being used with the Mesh.

See Also

- Mesh
- Mesh:setImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getTexture

Mesh:getTexture · revision 17613 · last edited 2026-06-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets the texture (Image or Canvas) used when drawing the Mesh.

Function

Synopsis

```
texture = Mesh:getTexture( )
```

Arguments

None.

Returns

`Texture texture (nil)`

The Image or Canvas to texture the Mesh with when drawing, or nil if none is set.

See Also

- Mesh
- Mesh:setTexture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getVertex

Mesh:getVertex · revision 21269 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the properties of a vertex in the Mesh.

In versions prior to 11.0, color and byte component values were within the range of 0 to 255 instead of 0 to 1.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
attributecomponent, ... = Mesh:getVertex( index )
```

Arguments

number index
: The one-based index of the vertex you want to retrieve the information for.

Returns

number attributecomponent
: The first component of the first vertex attribute in the specified vertex.

number ...
: Additional components of all vertex attributes in the specified vertex.

Notes

The values are returned in the same order as the vertex attributes in the Mesh's vertex format. A standard Mesh that wasn't created with a custom vertex format will return two position numbers, two texture coordinate numbers, and four color components: x, y, u, v, r, g, b, a.

Function

Gets the vertex components of a Mesh that wasn't created with a custom vertex format.

Synopsis

```
x, y, u, v, r, g, b, a = Mesh:getVertex( index )
```

Arguments

number index
: The index of the vertex you want to retrieve the information for.

Returns

number x
: The position of the vertex on the x-axis.

number y

The position of the vertex on the y-axis.

`number u`

The horizontal component of the texture coordinate.

`number v`

The vertical component of the texture coordinate.

`number r`

The red component of the vertex's color.

`number g`

The green component of the vertex's color.

`number b`

The blue component of the vertex's color.

`number a`

The alpha component of the vertex's color.

See Also

- Mesh
- Mesh:setVertex
- Mesh:getVertexCount
- Mesh:getVertexAttribute
- Mesh:getVertexFormat
- love.graphics.newMesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getVertexAttribute

Mesh:getVertexAttribute · revision 21457 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the properties of a specific attribute within a vertex in the Mesh.

Meshes without a custom vertex format specified in love.graphics.newMesh have position as their first attribute, texture coordinates as their second attribute, and color as their third attribute.

Function

Synopsis

```
value1, value2, ... = Mesh:getVertexAttribute( vertexindex, attributeindex )
```

Arguments

number vertexindex
: The index of the the vertex you want to retrieve the attribute for (one-based).

number attributeindex
: The index of the attribute within the vertex to be retrieved (one-based).

Returns

number value1
: The value of the first component of the attribute.

number value2
: The value of the second component of the attribute.

number ...
: Any additional vertex attribute components.

See Also

- Mesh
- Mesh:setVertexAttribute
- Mesh:getVertexCount
- Mesh:getVertexFormat
- Mesh:getVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getVertexCount

Mesh:getVertexCount · revision 11777 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the total number of vertices in the Mesh.

Function

Synopsis

```
count = Mesh:getVertexCount( )
```

Arguments

None.

Returns

`number count`

The total number of vertices in the mesh.

See Also

- Mesh
- Mesh:getVertex
- Mesh:setVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getVertexFormat

Mesh:getVertexFormat · revision 15791 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the vertex format that the Mesh was created with.

Function

Synopsis

```
format = Mesh:getVertexFormat( )
```

Arguments

None.

Returns

table format

The vertex format of the Mesh, which is a table containing tables for each vertex attribute the Mesh was created with, in the form of `{attribute, ...}`.

table attribute

A table containing the attribute's name, it's data type, and the number of components in the attribute, in the form of `{name, datatype, components}`.

table ...

Additional vertex attributes in the Mesh.

Notes

If a Mesh wasn't created with a custom vertex format, it will have the following vertex format:

```
defaultformat = {
    {"VertexPosition", "float", 2}, -- The x,y position of each vertex.
    {"VertexTexCoord", "float", 2}, -- The u,v texture coordinates of each vertex.
    {"VertexColor", "byte", 4} -- The r,g,b,a color of each vertex.
}
```

See Also

- Mesh
- love.graphics.newMesh
- AttributeDataType
- Mesh:setVertex
- Mesh:setVertexAttribute
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Mesh:getVertexMap

Mesh:getVertexMap · revision 17753 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the vertex map for the Mesh. The vertex map describes the order in which the vertices are used when the Mesh is drawn. The vertices, vertex map, and mesh draw mode work together to determine what exactly is displayed on the screen.

If no vertex map has been set previously via Mesh:setVertexMap, then this function will return nil in LÖVE 0.10.0+, or an empty table in 0.9.2 and older.

Function

Synopsis

```
map = Mesh:getVertexMap( )
```

Arguments

None.

Returns

`table map`

A table containing the list of vertex indices used when drawing.

See Also

- Mesh
- Mesh:setVertexMap

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:getVertices

Mesh:getVertices · revision 15921 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

Use Mesh:getVertex in a loop instead.

Gets all the vertices in the Mesh.



This method can be slow if the Mesh has a large number of vertices. Keep the original table used to create the Mesh around and update it when necessary instead of using this method frequently, if possible.

Function

Synopsis

```
vertices = Mesh:getVertices( )
```

Arguments

None.

Returns

`table vertices`

The table filled with vertex information tables for each vertex as follows:

`number [1]`

The position of the vertex on the x-axis.

`number [2]`

The position of the vertex on the y-axis.

`number [3]`

The horizontal component of the texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

`number [4]`

The vertical component of the texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

`number [5] (255)`

The red color component.

`number [6] (255)`

The green color component.

`number [7] (255)`

The blue color component.

`number [8] (255)`

The alpha color component.

See Also

- Mesh
- Mesh:setVertices
- Mesh:getVertex

- Mesh:getVertexCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:hasVertexColors

Mesh:hasVertexColors · revision 15920 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

It has been replaced by Mesh:isAttributeEnabled("VertexColor").

Gets whether per-vertex colors are used instead of the constant color when drawing the Mesh (constant color being love.graphics.setColor.)

Per-vertex colors are enabled by default for a Mesh if at least one vertex color was not the default (255, 255, 255, 255) when the Mesh was created.

Function

Synopsis

```
vertexcolors = Mesh:hasVertexColors( )
```

Arguments

None.

Returns

`boolean vertexcolors`

True if per-vertex coloring is used, otherwise love.graphics.setColor is used when drawing the Mesh.

See Also

- Mesh
- Mesh:setVertexColors
- love.graphics.newMesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:isAttributeEnabled

Mesh:isAttributeEnabled · revision 21458 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whether a specific vertex attribute in the Mesh is enabled. Vertex data from disabled attributes is not used when drawing the Mesh.

Function

Synopsis

```
enabled = Mesh:isAttributeEnabled( name )
```

Arguments

`string name`
: The name of the vertex attribute to be checked.

Returns

`boolean enabled`
: Whether the vertex attribute is used when drawing this Mesh.

Notes

If a Mesh wasn't created with a custom vertex format, it will have 3 vertex attributes named `VertexPosition`, `VertexTexCoord`, and `VertexColor`. Otherwise the attribute name must either match one of the vertex attributes specified in the vertex format when creating the Mesh, or must match a vertex attribute from another Mesh attached to this Mesh via Mesh:attachAttribute.

See Also

- Mesh
- Mesh:setAttributeEnabled
- Mesh:getVertexFormat
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setAttributeEnabled

Mesh:setAttributeEnabled · revision 15643 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Enables or disables a specific vertex attribute in the Mesh. Vertex data from disabled attributes is not used when drawing the Mesh.

Function

Synopsis

```
Mesh:setAttributeEnabled( name, enable )
```

Arguments

`string name`
: The name of the vertex attribute to enable or disable.

`boolean enable`
: Whether the vertex attribute is used when drawing this Mesh.

Returns

Nothing.

Notes

If a Mesh wasn't created with a custom vertex format, it will have 3 vertex attributes named `VertexPosition`, `VertexTexCoord`, and `VertexColor`. Otherwise the attribute name must either match one of the vertex attributes specified in the vertex format when creating the Mesh, or must match a vertex attribute from another Mesh attached to this Mesh via Mesh:attachAttribute.

See Also

- Mesh
- Mesh:isAttributeEnabled
- Mesh:getVertexFormat
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setDrawMode

Mesh:setDrawMode · revision 11791 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the mode used when drawing the Mesh.

Function

Synopsis

```
Mesh:setDrawMode( mode )
```

Arguments

`MeshDrawMode mode`

The mode to use when drawing the Mesh.

Returns

Nothing.

See Also

- Mesh
- Mesh:getDrawMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setDrawRange

Mesh:setDrawRange · revision 29388 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Restricts the drawn vertices of the Mesh to a subset of the total.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
Mesh:setDrawRange( start, count )
```

Arguments

`number start`
: The index of the first vertex to use when drawing, or the index of the first value in the vertex map to use if one is set for this Mesh.

`number count`
: The number of vertices to use when drawing, or number of values in the vertex map to use if one is set for this Mesh.

Returns

Nothing.

Function

Allows all vertices in the Mesh to be drawn.

Synopsis

```
Mesh:setDrawRange( )
```

Arguments

None.

Returns

Nothing.

Function

Removed in LÖVE 11.0

This variant is not supported in that and later versions.

Synopsis

```
Mesh:setDrawRange( min, max )
```

Arguments

number min
: The index of the first vertex to use when drawing, or the index of the first value in the vertex map to use if one is set for this Mesh.

number max
: The index of the last vertex to use when drawing, or the index of the last value in the vertex map to use if one is set for this Mesh.

Returns

Nothing.

Notes

If a vertex map is used with the Mesh, this method will set a subset of the values in the vertex map array to use, instead of a subset of the total vertices in the Mesh.

For example, if `Mesh:setVertexMap(1, 2, 3, 1, 3, 4)` and `Mesh:setDrawRange(4, 3)` are called, vertices 1, 3, and 4 will be drawn.

See Also

- Mesh
- Mesh:getDrawRange
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setImage

Mesh:setImage · revision 15736 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Removed in LÖVE 0.10.0

Use Mesh:setTexture instead.



Sets the Image used when drawing the Mesh.

Function

Synopsis

```
Mesh:setImage( image )
```

Arguments

`Image image`
: The Image to texture the Mesh with when drawing.

Returns

Nothing.

Function

Synopsis

```
Mesh:setImage( )
```

Arguments

None.

Returns

Nothing.

Notes

Disables any Image from being used when drawing the Mesh. Untextured meshes have a white color by default.

See Also

- Mesh

- Mesh:getImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setTexture

Mesh:setTexture · revision 26217 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Sets the texture (Image or Canvas) used when drawing the Mesh.

Function

Synopsis

```
Mesh:setTexture( texture )
```

Arguments

`Texture texture`
: The Image or Canvas to texture the Mesh with when drawing.

Returns

Nothing.

Function

Synopsis

```
Mesh:setTexture( )
```

Arguments

None.

Returns

Nothing.

Notes

Disables any texture from being used when drawing the Mesh. Untextured meshes have a white color by default.

Notes

In version 0.9.1 Textures are not released from memory automatically when nil-ing meshes. Call Mesh:setTexture() to manually clear. This is fixed in later versions.

See Also

- Mesh

- Mesh:getTexture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setVertex

Mesh:setVertex · revision 21268 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the properties of a vertex in the Mesh.

In versions prior to 11.0, color and byte component values were within the range of 0 to 255 instead of 0 to 1.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
Mesh:setVertex( index, attributecomponent, ... )
```

Arguments

`number index`
: The index of the the vertex you want to modify (one-based).

`number attributecomponent`
: The first component of the first vertex attribute in the specified vertex.

`number ...`
: Additional components of all vertex attributes in the specified vertex.

Returns

Nothing.

Notes

The arguments are in the same order as the vertex attributes in the Mesh's vertex format. A standard Mesh that wasn't created with a custom vertex format will use two position numbers, two texture coordinate numbers, and four color components per vertex: x, y, u, v, r, g, b, a.

If no value is supplied for a specific vertex attribute component, it will be set to a default value of 0 if its data type is "float", or 1 if its data type is "byte".

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
Mesh:setVertex( index, vertex )
```

Arguments

number index
The index of the the vertex you want to modify (one-based).

table vertex
A table with vertex information, in the form of `{attributecomponent, ...}`.

number attributecomponent
The first component of the first vertex attribute in the specified vertex.

number ...
Additional components of all vertex attributes in the specified vertex.

Returns

Nothing.

Notes

The table indices are in the same order as the vertex attributes in the Mesh's vertex format. A standard Mesh that wasn't created with a custom vertex format will use two position numbers, two texture coordinate numbers, and four color components per vertex: x, y, u, v, r, g, b, a.

If no value is supplied for a specific vertex attribute component, it will be set to a default value of 0 if its data type is "float", or 1 if its data type is "byte".

Function

Sets the vertex components of a Mesh that wasn't created with a custom vertex format.

Synopsis

```
Mesh:setVertex( index, x, y, u, v, r, g, b, a )
```

Arguments

number index
The index of the the vertex you want to modify (one-based).

number x
The position of the vertex on the x-axis.

number y
The position of the vertex on the y-axis.

number u
The horizontal component of the texture coordinate.

number v
The vertical component of the texture coordinate.

number r (1)
The red component of the vertex's color.

number g (1)
The green component of the vertex's color.

number b (1)
The blue component of the vertex's color.

number a (1)

The alpha component of the vertex's color.

Returns

Nothing.

Function

Sets the vertex components of a Mesh that wasn't created with a custom vertex format.

Synopsis

```
Mesh:setVertex( index, vertex )
```

Arguments

number index
The index of the the vertex you want to modify (one-based).

table vertex
A table with vertex information.

number [1]
The position of the vertex on the x-axis.

number [2]
The position of the vertex on the y-axis.

number [3]
The u texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

number [4]
The v texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

number [5] (1)
The red color component.

number [6] (1)
The green color component.

number [7] (1)
The blue color component.

number [8] (1)
The alpha color component.

Returns

Nothing.

See Also

- Mesh
- Mesh:getVertex
- Mesh:getVertexCount
- Mesh:getVertexFormat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setVertexAttribute

Mesh:setVertexAttribute · revision 25340 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets the properties of a specific attribute within a vertex in the Mesh.

Meshes without a custom vertex format specified in love.graphics.newMesh have position as their first attribute, texture coordinates as their second attribute, and color as their third attribute.

Function

Synopsis

```
Mesh:setVertexAttribute( vertexindex, attributeindex, value1, value2, ... )
```

Arguments

number vertexindex
: The index of the the vertex to be modified (one-based).

number attributeindex
: The index of the attribute within the vertex to be modified (one-based).

number value1
: The new value for the first component of the attribute.

number value2
: The new value for the second component of the attribute.

number ...
: Any additional vertex attribute components.

Returns

Nothing.

Notes

Attribute components which exist within the attribute but are not specified as arguments default to 0 for attributes with the `float` data type, and 1 for the `byte` data type.

Examples

Modify the colors of a standard mesh after it's created.

```
-- Standard mesh with position, texture coordinates, and color attributes.
mesh = love.graphics.newMesh {
    {0,   0,    0, 0,  1, 1, 1, 1}, -- first vertex positioned at (0, 0)
    {400, 0,    0, 0,  1, 1, 1, 1}, -- second vertex positioned at (400, 0)
    {200, 400,  0, 0,  1, 1, 1, 1}, -- third vertex positioned at (200, 400)
}

local time = 0

function love.update(dt)
    time = time + dt

    for i = 1, mesh:getVertexCount() do
        -- The 3rd vertex attribute for a standard mesh is its color.
        mesh:setVertexAttribute(i, 3, (time * 10) % 1, 1, 1)
```

```
    end
end

function love.draw()
    love.graphics.draw(mesh)
end
```

See Also

- Mesh
- Mesh:getVertexAttribute
- Mesh:getVertexCount
- Mesh:getVertexFormat
- Mesh:setVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setVertexColors

Mesh:setVertexColors · revision 15919 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

It has been replaced by Mesh:setAttributeEnabled("VertexColor", enable).

Sets whether per-vertex colors are used instead of the constant color when drawing the Mesh (constant color being love.graphics.setColor.)

Per-vertex colors are enabled by default for a Mesh if at least one vertex color was not the default (255, 255, 255, 255) when the Mesh was created.

Function

Synopsis

```
Mesh:setVertexColors( on )
```

Arguments

`boolean on`

True to use per-vertex coloring, otherwise love.graphics.setColor is used when drawing.

Returns

Nothing.

See Also

- Mesh
- Mesh:hasVertexColors
- love.graphics.newMesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setVertexMap

Mesh:setVertexMap · revision 27359 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the vertex map for the Mesh. The vertex map describes the order in which the vertices are used when the Mesh is drawn. The vertices, vertex map, and mesh draw mode work together to determine what exactly is displayed on the screen.

The vertex map allows you to re-order or reuse vertices when drawing without changing the actual vertex parameters or duplicating vertices. It is especially useful when combined with different Mesh Draw Modes.

Function

Synopsis

```
Mesh:setVertexMap( map )
```

Arguments

`table map`
: A table containing a list of vertex indices to use when drawing. Values must be in the range of [1, Mesh:getVertexCount()].

Returns

Nothing.

Function

Synopsis

```
Mesh:setVertexMap( vi1, vi2, vi3, ... )
```

Arguments

`number vi1`
: The index of the first vertex to use when drawing. Must be in the range of [1, Mesh:getVertexCount()].

`number vi2`
: The index of the second vertex to use when drawing.

`number vi3`
: The index of the third vertex to use when drawing.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
Mesh:setVertexMap( data, datatype )
```

Arguments

`Data data`
Array of vertex indices to use when drawing. Values must be in the range of [0, Mesh:getVertexCount()-1]

`IndexDataType datatype`
Datatype of the vertex indices array above.

Returns

Nothing.

Function

Disables the vertex map for this mesh.

Synopsis

```
Mesh:setVertexMap( )
```

Arguments

None.

Returns

Nothing.

Examples

Use a vertex map to fix a visual glitch without copy/pasting vertices.

```
function love.load()
    image = love.graphics.newImage("pig.png")
    local w,h = image:getDimensions()

    -- We want to make a Mesh with 1 vertex in the middle of the image, and 4 at the corners.
    local vertices = {
        {w/2, h/2,   0.5, 0.5,   1, 0, 0}, -- Center vertex, with a red tint.
        {0,   0,     0,   0,     1, 1, 1}, -- Top left.
        {w,   0,     1,   0,     1, 1, 1}, -- Top right.
        {w,   h,     1,   1,     1, 1, 1}, -- Bottom right.
        {0,   h,     0,   1,     1, 1, 1}, -- Bottom left.
    }

    -- But there's a problem! The drawn mesh will have a big triangle missing on its left side.
    -- This is because, in the default mesh draw mode ("fan"), the vertices don't "loop": the top left vertex (#2) is unconnected to the
bottom left one (#5).
    mesh = love.graphics.newMesh(vertices)
    mesh:setTexture(image)

    -- We could copy/paste the second vertex onto the end of the table of vertices.
    -- But instead we can just change the vertex map!
    mesh:setVertexMap(1, 2, 3, 4, 5, 2)
end

function love.draw()
```

```
    love.graphics.draw(mesh, 0, 0)
end
```

See Also

- Mesh
- Mesh:getVertexMap
- MeshDrawMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mesh:setVertices

Mesh:setVertices · revision 25444 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Replaces a range of vertices in the Mesh with new ones. The total number of vertices in a Mesh cannot be changed after it has been created. This is often more efficient than calling Mesh:setVertex in a loop.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
Mesh:setVertices( vertices, startvertex, count )
```

Arguments

table vertices

The table filled with vertex information tables for each vertex, in the form of `{vertex, ...}` where each vertex is a table in the form of `{attributecomponent, ...}`.

number attributecomponent

The first component of the first vertex attribute in the vertex.

number ...

Additional components of all vertex attributes in the vertex.

number startvertex (1)

The index of the first vertex to replace.

Available since LÖVE 11.3

number count (all)

Amount of vertices to replace.

Returns

Nothing.

Notes

The values in each vertex table are in the same order as the vertex attributes in the Mesh's vertex format. A standard Mesh that wasn't created with a custom vertex format will use two position numbers, two texture coordinate numbers, and four color components per vertex: x, y, u, v, r, g, b, a.

If no value is supplied for a specific vertex attribute component, it will be set to a default value of 0 if its data type is "float", or 255 if its data type is "byte".

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Sets the vertex components of the Mesh by copying directly from the memory of a Data object.

Synopsis

```
Mesh:setVertices( data, startvertex, count )
```

Arguments

Data data
A Data object to copy from. The contents of the Data must match the layout of this Mesh's vertex format.

number startvertex (1)
The index of the first vertex to replace.

Available since LÖVE 11.3

number count (all)
Amount of vertices to replace.

Returns

Nothing.

Notes

If LuaJIT's FFI (http://luajit.org/ext_ffi_api.html) is used to populate the Data object via Data:getPointer and ffi.cast (http://luajit.org/ext_ffi_api.html#ffi_cast), this variant can be drastically more efficient than other methods of setting Mesh vertex data.

Function

Sets the vertex components of a Mesh that wasn't created with a custom vertex format.

Synopsis

```
Mesh:setVertices( vertices )
```

Arguments

table vertices
The table filled with vertex information tables for each vertex as follows:

number [1]
The position of the vertex on the x-axis.

number [2]
The position of the vertex on the y-axis.

number [3]
The horizontal component of the texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode).

number [4]
The vertical component of the texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode).

number [5] (1)
The red color component.

number [6] (1)
The green color component.

number [7] (1)
The blue color component.

number [8] (1)

The alpha color component.

Returns

Nothing.

See Also

- Mesh
- Mesh:getVertex
- Mesh:getVertexCount
- Mesh:getVertexFormat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MeshDrawMode

MeshDrawMode · revision 14875 · last edited 2026-04-10

Available since LÖVE 0.9.0
This enum is not supported in earlier versions.

How a Mesh's vertices are used when drawing.

Constants

fan
: The vertices create a "fan" shape with the first vertex acting as the hub point. Can be easily used to draw simple convex polygons.

strip
: The vertices create a series of connected triangles using vertices 1, 2, 3, then 3, 2, 4 (note the order), then 3, 4, 5, and so on.

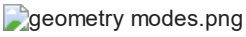
triangles
: The vertices create unconnected triangles.

points
: The vertices are drawn as unconnected points (see love.graphics.setPointSize.)

Notes

If the Mesh has a custom vertex map, then that will determine the order in which the vertices are interpreted by the draw mode.

The vertex map can also be used to make the draw mode reuse vertices, for example if the Mesh has 4 vertices and is using the "triangles" draw mode, the vertex map could be set to {1, 2, 3, 1, 3, 4} to draw 2 triangles using only 4 vertices, effectively drawing a quad using triangles.



See Also

- love.graphics
- love.graphics.newMesh
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MessageBoxType

MessageBoxType · revision 13771 · last edited 2023-07-05

Available since LÖVE 0.9.2

This enum is not supported in earlier versions.

Types of message box dialogs. Different types may have slightly different looks.

Constants

info

Informational dialog.

warning

Warning dialog.

error

Error dialog.

See Also

- love.window
- love.window.showMessageBox

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MGL

MGL · revision 25395 · last edited 2023-07-05

Mathematics for Graphics in pure Lua.

Library for graphics computation (vector, matrix, 2D, 3D, etc.).

- Project and documentation: https://github.com/ImagicTheCat/MGL ↗
- Forum topic (https://love2d.org/forums/viewtopic.php?f=5&p=234291)

MiddleClass

MiddleClass · revision 14685 · last edited 2023-07-05

https://github.com/kikito/middleclass ↗

A clean object-orientation framework for Lua, well-documented and tested.

Features

- Metamethod support(except __index and __newindex)
- Straightforward and fast
- Class-Commons support (https://github.com/kikito/middleclass-commons)

Docs

The documentation can be found on the Github wiki (https://github.com/kikito/middleclass/wiki)

See Also

- Libraries

MiniFS

MiniFS · revision 16432 · last edited 2023-07-05

MiniFS is a small but powerful module for easy filesystem access. It uses LuaFileSystem, the Lua standard I/O module and the standard OS module to form a convenient API.

MiniFS is written in MoonScript, a language that compiles to Lua. There is a pre-compiled version in the repository if you don't want to embed Moonscript in your Lua programs.

You can grab it from GitLab (http://gitlab.com/zatherz/minifs). Documentation of the module can be seen here (https://zatherz.gitlab.io/minifs). You can also install it through LuaRocks (https://luarocks.org/modules/zatherz/minifs).

Examples

Moonscript

```
fs = require("minifs")
print(fs.system(true)) -- print the operating system
fs.mkdir("test")
fs.write("test/test.txt", "Hello") -- write to a file in one line without file handles
fs.mkdir("test/test2")
fs.move("test/test.txt", "test/test2/test.txt")
fs.usetmp((path) ->
    -- path is a temporary file that is removed when the function ends
    print(path)
)
fs.rmdir("test", true) -- recursive remove
```

Lua

```
fs = require("minifs")
print(fs.system(true)) -- print the operating system
fs.mkdir("test")
fs.write("test/test.txt", "Hello") -- write to a file in one line without file handles
fs.mkdir("test/test2")
fs.move("test/test.txt", "test/test2/test.txt")
fs.usetmp(function(path)
    -- path is a temporary file that is removed when the function ends
    print(path)
end)
fs.rmdir("test", true) -- recursive remove
```

MipmapMode

MipmapMode · revision 22745 · last edited 2026-04-09

Available since LÖVE 11.0
This enum is not supported in earlier versions.

Controls whether a Canvas has mipmaps, and its behaviour when it does.

Constants

none
: The Canvas has no mipmaps.

manual
: The Canvas has mipmaps. love.graphics.setCanvas can be used to render to a specific mipmap level, or Canvas:generateMipmaps can (re-)compute all mipmap levels based on the base level.

auto
: The Canvas has mipmaps, and all mipmap levels will automatically be recomputed when switching away from the Canvas with love.graphics.setCanvas.

See Also

- love.graphics
- love.graphics.newCanvas
- Canvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

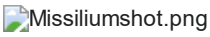
Missilium

Missilium · revision 12308 · last edited 2021-03-14

Project page and demo download: http://www.bitoutsidethebox.com (http://w w w.bitoutsidethebox.com)

You are the commander of the new "Missilium" project, a missile launcher and interceptor that promises to bring great advantage in future wars.

But the future is closer than everyone thought, a surprise attack of the enemy requires you use the newly created Missilium to defend and win this unexpected war.



- Battle over 15 levels in campaign mode, with two last very special and different levels.
- Fight along 4 different scenarios, in day and night modes.
- Different win and loose conditions in all battles.
- Unlimited hours of game play with Free Play mode.
- Use your game score to acquire extras and powerups to aid you in your mission.

ModifierKey

ModifierKey · revision 28067 · last edited 2023-07-05

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Modifier keys are keys that have "active" state in addition of pressed/released state, thus modifier keys are small subset of KeyConstant.

Constants

capslock
: Caps-lock key.

numlock
: Num-lock key.

scrolllock
: Scroll-lock key.

mode
: Mode key.

See Also

- love.keyboard
- love.keyboard.isModifierActive

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Monocle

Monocle · revision 13579 · last edited 2023-07-05

Monocleis a simple, but very helpful debugging library for Love2D.
Monocle can watch variables and expressions and keep them pinned to the top left of the screen.
If a problem occurs, the game will issue an error, but the watched expressions stay on the screen.
Best of all, the game will watch files and automatically reload the game when those files have been changed.

Basic implementation:

```
require 'monocle/monocle'
Monocle.new({})

-- The most basic way to watch any expression or variable:
Monocle.watch("FPS", function() return math.floor(1/love.timer.getDelta()) end)

function love.update(dt)
    Monocle.update()
end

function love.draw()
    Monocle.draw()
end

function love.textinput(t)
    Monocle.textinput(t)
end

function love.keypressed(text)
    Monocle.keypressed(text)
end
```

Monocle can also be started with options in mind that are tailored to your needs!

```
Monocle.new({       -- ALL of these parameters are optional!
   isActive=true,          -- Whether the debugger is initially active
   customPrinter=false,    -- Whether Monocle prints status messages to the output
   printColor = {51,51,51},-- Color to print with
   debugToggle='`',        -- The keyboard button for toggling Monocle
   filesToWatch=           -- Files that, when edited, cause the game to reload automatically
      {
         'main.lua'
      }
})
```

You can find it at [1] (https://github.com/kurtjarvi/monocle).

Moses

Moses · revision 9688 · last edited 2023-07-05

Moses is a utility library which provides a set of functions acting as shortcuts for common programming tasks,
and support for functional programming (http://en.w ikipedia.org/w iki/Functional_programming). It aims to extend the built-in Lua table library, facilitate operations on
arrays, lists, collections, objects, through **85** weird, strange, bizarre, odd functions.

Moses was heavily inspired by Underscore.js (http://documentcloud.github.com/underscore/).

Sample

```lua
local _ = require 'moses'
-- let's have fun!

print('Creating an array of integers')
local array = _.range(0,12,3)
_.each(array,print)

print('Shuffles the array with a random seed')
array = _.shuffle(array, os.time())
_.each(array, print)

print('Slicing values between 2 and 5 keys')
array = _.slice(array,2,5)
_.each(array,print)

print('Count values by parity')
local parityCount = _.countBy(array,function(i,v)
  return v%2==0 and 'even' or 'odd'
end)
_.each(parityCount,print)
```

A possible output would be:

```
-- Creating an array of integers
-- 1    0
-- 2    3
-- 3    6
-- 4    9
-- 5    12

-- Shuffles the array with a random seed
-- 1    3
-- 2    0
-- 3    6
-- 4    12
-- 5    9

-- Slicing values between 2 and 5 keys
-- 1    0
-- 2    6
-- 3    12
-- 4    9

-- Count values by parity
-- odd  1
-- even 3
```

Grab it on Github (https://github.com/Yonaba/Moses).

- Source (https://github.com/Yonaba/Moses/blob/master/moses.lua)

- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=10170)
- HTML Documentation (https://github.com/Yonaba/Moses/dow nloads)
- Complete set of samples (https://github.com/Yonaba/Moses/blob/master/docs/moses.md)
- Specification tests (https://github.com/Yonaba/Moses/tree/master/specs)

MotorJoint

MotorJoint · revision 16466 · last edited 2023-07-05

Available since LÖVE 0.9.0
This type is not supported in earlier versions.

Controls the relative motion between two Bodies. Position and rotation offsets can be specified, as well as the maximum motor force and torque that will be applied to reach the target offsets.

Constructors

love.physics.newMotorJoint	Creates a joint between two bodies which controls the relative motion between them.	Added since 0.9.0

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
MotorJoint:getAngularOffset	Gets the target angular offset between the two Bodies the Joint is attached to.	Added since 0.9.0	
MotorJoint:getLinearOffset	Gets the target linear offset between the two Bodies the Joint is attached to.	Added since 0.9.0	
MotorJoint:setAngularOffset	Sets the target angular offset between the two Bodies the Joint is attached to.	Added since 0.9.0	
MotorJoint:setLinearOffset	Sets the target linear offset between the two Bodies the Joint is attached to.	Added since 0.9.0	

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MotorJoint:getAngularOffset

MotorJoint:getAngularOffset · revision 18727 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This method is not supported in earlier versions.

Gets the target angular offset between the two Bodies the Joint is attached to.

Function

Synopsis

```
angleoffset = MotorJoint:getAngularOffset( )
```

Arguments

None.

Returns

`number angleoffset`

The target angular offset in radians: the second body's angle minus the first body's angle.

See Also

- MotorJoint
- MotorJoint:setAngularOffset

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MotorJoint:getLinearOffset

MotorJoint:getLinearOffset · revision 16172 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This method is not supported in earlier versions.

Gets the target linear offset between the two Bodies the Joint is attached to.

Function

Synopsis

```
x, y = MotorJoint:getLinearOffset( )
```

Arguments

None.

Returns

number x
: The x component of the target linear offset, relative to the first Body.

number y
: The y component of the target linear offset, relative to the first Body.

See Also

- MotorJoint
- MotorJoint:setLinearOffset

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MotorJoint:setAngularOffset

MotorJoint:setAngularOffset · revision 16177 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This method is not supported in earlier versions.

Sets the target angluar offset between the two Bodies the Joint is attached to.

Function

Synopsis

```
MotorJoint:setAngularOffset( angleoffset )
```

Arguments

`number angleoffset`
: The target angular offset in radians: the second body's angle minus the first body's angle.

Returns

Nothing.

See Also

- MotorJoint
- MotorJoint:getAngularOffset

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MotorJoint:setLinearOffset

MotorJoint:setLinearOffset · revision 16171 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This method is not supported in earlier versions.

Sets the target linear offset between the two Bodies the Joint is attached to.

Function

Synopsis

```
MotorJoint:setLinearOffset( x, y )
```

Arguments

`number x`
: The x component of the target linear offset, relative to the first Body.

`number y`
: The y component of the target linear offset, relative to the first Body.

Returns

Nothing.

See Also

- MotorJoint
- MotorJoint:getLinearOffset

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MountPermissions

MountPermissions · revision 30596 · last edited 2026-02-04

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Enumeration representing the desired permissions when creating a mount.

Constants

read
: Set mount to only allow read operations.

readwrite
: Set mount to allow both read and write operations.

See Also

- love.filesystem
- love.filesystem.mountCommonPath
- love.filesystem.unmountCommonPath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseConstant

MouseConstant · revision 16017 · last edited 2025-03-25

Removed in LÖVE 0.10.0
Mouse button constants have been replaced with button index numbers.

Mouse buttons.

Constants

l
: Left Mouse Button.

m
: Middle Mouse Button.

r
: Right Mouse Button.

wd
: Mouse Wheel Down.

wu
: Mouse Wheel Up.

x1
: Mouse X1 (also known as button 4).

x2
: Mouse X2 (also known as button 5).

See Also

- love.mouse
- love.mouse.isDown
- love.mousepressed
- love.mousereleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint

MouseJoint · revision 10238 · last edited 2023-07-05

For controlling objects with the mouse.

Constructors

love.physics.newMouseJoint	Create a joint between a body and the mouse.

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
MouseJoint:getDampingRatio	Returns the damping ratio.	Added since 0.8.0	
MouseJoint:getFrequency	Returns the frequency.	Added since 0.8.0	
MouseJoint:getMaxForce	Gets the highest allowed force.		
MouseJoint:getTarget	Gets the target point.		
MouseJoint:setDampingRatio	Sets a new damping ratio.	Added since 0.8.0	
MouseJoint:setFrequency	Sets a new frequency.	Added since 0.8.0	
MouseJoint:setMaxForce	Sets the highest allowed force.		
MouseJoint:setTarget	Sets the target point.		

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:getDampingRatio

MouseJoint:getDampingRatio · revision 8543 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the damping ratio.

Function

Synopsis

```
ratio = MouseJoint:getDampingRatio( )
```

Arguments

None.

Returns

`number ratio`

The new damping ratio.

See Also

- MouseJoint
- MouseJoint:setDampingRatio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:getFrequency

MouseJoint:getFrequency · revision 8541 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the frequency.

Function

Synopsis

```
freq = MouseJoint:getFrequency( )
```

Arguments

None.

Returns

`number freq`
: The frequency in hertz.

See Also

- MouseJoint
- MouseJoint:setFrequency

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:getMaxForce

MouseJoint:getMaxForce · revision 6080 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the highest allowed force.

Function

Synopsis

```
f = MouseJoint:getMaxForce( )
```

Arguments

None.

Returns

`number f`
The max allowed force.

See Also

- MouseJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:getTarget

MouseJoint:getTarget · revision 6081 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the target point.

Function

Synopsis

```
x, y = MouseJoint:getTarget( )
```

Arguments

None.

Returns

number x
: The x-component of the target.

number y
: The x-component of the target.

See Also

- MouseJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:setDampingRatio

MouseJoint:setDampingRatio · revision 8542 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new damping ratio.

Function

Synopsis

```
MouseJoint:setDampingRatio( ratio )
```

Arguments

`number ratio`
: The new damping ratio.

Returns

Nothing.

See Also

- MouseJoint
- MouseJoint:getDampingRatio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:setFrequency

MouseJoint:setFrequency · revision 8540 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new frequency.

Function

Synopsis

```
MouseJoint:setFrequency( freq )
```

Arguments

`number freq`
: The new frequency in hertz.

Returns

Nothing.

See Also

- MouseJoint
- MouseJoint:getFrequency

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:setMaxForce

MouseJoint:setMaxForce · revision 6082 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the highest allowed force.

Function

Synopsis

```
MouseJoint:setMaxForce( f )
```

Arguments

number f
: The max allowed force.

Returns

Nothing.

See Also

- MouseJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

MouseJoint:setTarget

MouseJoint:setTarget · revision 8325 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the target point.

Function

Synopsis

```
MouseJoint:setTarget( x, y )
```

Arguments

number x
: The x-component of the target.

number y
: The y-component of the target.

Returns

Nothing.

See Also

- MouseJoint

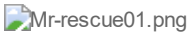
Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Mr. Rescue

Mr. Rescue · revision 10845 · last edited 2021-03-14

Mr. Rescue is an arcade styled 2d action game centered around evacuating civilians from burning buildings.



The game features fast paced fire extinguishing action, intense boss battles, a catchy soundtrack and lots of throwing people around in pseudo-randomly generated buildings.

Download latest version (http://tangramgames.dk/games/mrrescue/).

MyPaint

MyPaint · revision 7087 · last edited 2021-03-14

MyPaint is a fast and easy open-source graphics application for digital painters. It lets you focus on the art instead of the program. You work on your canvas with minimum distractions, bringing up the interface only when you need it.

[1] (http://mypaint.intilinux.com/)

N.T.U.I.

N.T.U.I. · revision 10634 · last edited 2021-03-14

On this page

- Officiel website
- Using...
- Follow us! (in French only...)
- The project
- The story
- Screenshots

Officiel website

http://40.innovgame.efrei.fr/en/ ↗



The first level

Using...

Zoetrope 1.3.1

TEsound

Follow us! (in French only...)

Google+: https://plus.google.com/103461791473292628133?prsrc=2 ↗

Facebook: https://www.facebook.com/pages/Sea-Sharp/334047836708045 ↗

The project

NTUI is a project developed for INNOV'Game 2013, a competition organised by a french engineer school, the EFREI, and **won the 2nd price**! You can freely download and distribute it here: http://40.innovgame.efrei.fr/en/?Download ↗

The story

N.T.U.I. is the acronym of Never Trust User's Input, a computer science term which defines a conductive path in development.

A team of 2 developers and an artist known under the name of Sea Sharp decided to make this idea a game by immersing you in the heart of a video game! In fact, you will embody a hero, revolted by having been put in a totally bugged video game ! Then, the hero decides to fight against his creators by using the source code to meet them and wreak his revenge! However, he has to take into consideration the twists and turns that the addicted players of this studio's games will mak him go through. Fortunately, or not, our hero will be able to exploit the numerous game bugs to get the advantage... but if he abused of them... it could be tragic... If you want to read more about the game's concept (and speak a little French!), we encourage you to read the Innov'Game project presentation file of N.T.U.I.!

Screenshots

You can find some screenshots and artowrks here : http://40.innovgame.efrei.fr/en/?Medias ↗

Networking

Networking · revision 25314 · last edited 2023-07-05

Networking Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
Grease	The one and (no longer) only networking library		Any	
Noobhub	Multiplayer and network messaging		Any	
lua-https	A simple Lua HTTPS module using native platform backends where applicable.	Yes (12.0)/No (C module)	12.0	Any
lua-enet	Multiplayer networking module for games.		0.9.0	
lua-enet (简体中文)	用于多人游戏的网络模块。		0.9.0	
socket	Module for HTTP, TCP, and UDP networking.		0.5.0	
socket (简体中文)	HTTP、TCP和UDP网络模块。		0.5.0	
LoverNet	Module designed to make networking easier		0.10.x	0.10.x
ONET	A Object Oriented Networking Library		0.10.2	

NewPages

NewPages · revision 27861 · last edited 2024-06-11

This is a list of new pages that aren't in the docs themselves yet (because they belong to an unreleased version). This page is here to help people who want to document for the next version.

Note: if function from unreleased version appear in "Functions" list (same for enums, ...), add `[[Concept:Current]]` to `{{#ask: ... }}` in listing (as done here (https://love2d.org/w/index.php?title=love.system&type=revision&diff=27816&oldid=22358))

Another note: if some function became deprecated (same for enums, ...) and list ("Functions", ...) does not show that it is deprecated, add `| ?PrettyDeprecated` to end of `{{#ask: ... }}` in listing

Body:getShape	Gets the first Shape attached to the Body.
Body:getShapes	Returns a table with all Shapes attached to the Body.
Body:hasCustomMassData	Gets whether the Body's mass data has been set to custom values.
BufferDataFormat	Data formats of individual array elements GraphicsBuffers and Meshes.
ByteData:setString	Replaces all or part of the ByteData's memory with the contents of a string.
CommonPath	Commonly used operating system directories.
Contact:getShapes	Gets the two Shapess that are in contact.
Data:getString	Gets the full Data as a string.
Data:getString (Deutsch)	Ruft die vollständigen Daten als Zeichenkette ab.
DeviceType	Types of hardware cursors.
DeviceType (Deutsch)	Arten von Hardwarezeigern.
FileDialogType	Possible types of file dialogs.
GamepadType	Types of gamepad controllers.
GamepadType (日本語)	ゲームパッドコントローラーの機種。
GraphicsBuffer	Low-level data stored in graphics memory, including arrays of vertices, vertex indices, and custom collections of data accessible in Shaders.
GraphicsBuffer (Français)	Données bas-niveau stockées en mémoire graphique, incluant les tables de vecteurs, les indices de vecteurs, et les collections personnalisées de données accessibles au sein des Shaders (ombrages).
GraphicsBuffer:clear	Clears the entire GraphicsBuffer or a specified byte range within it to zero.
GraphicsBuffer:getElementCount	Gets the total number of array elements in this GraphicsBuffer.
GraphicsBuffer:getElementStride	Gets the size in bytes used by one array element of this GraphicsBuffer.
GraphicsBuffer:getSize	Gets the total size in bytes of the GraphicsBuffer's contents.
Joystick:getGamepadType	Gets the GamepadType of the Joystick, if it's recognized as a gamepad.
Joystick:getGamepadType (日本語)	ゲームパッドとして認識されている場合に、ジョイスティックの GamepadType を取得します。
Joystick:getJoystickType	Gets the JoystickType of the Joystick.
Joystick:getJoystickType (日本語)	ジョイスティックの JoystickType を取得します。
Joystick:getPlayerIndex	Gets the player index of this Joystick. This corresponds to an indicator light on many common gamepads.
Joystick:getPlayerIndex (日本語)	この Joystick のプレイヤーインデックスを取得します。これは一般的なゲームパッドに搭載されているインジケータライトと対応関係にあります。
Joystick:getSensorData	Gets the latest data for the given sensor type for this Joystick
Joystick:hasSensor	Gets whether the specified sensor exists in the Joystick.
Joystick:isSensorEnabled	Gets whether the specified sensor is currently enabled on this Joystick.
Joystick:setPlayerIndex	Sets the player index of this Joystick. This corresponds to an indicator light on many common gamepads.
Joystick:setPlayerIndex (日本語)	この Joystick のプレイヤーインデックスを設定します。これは一般的なゲームパッドに搭載されているインジケータライトと対応関係にあります。
Joystick:setSensorEnabled	Enables or disables the specified sensor on this Joystick.
JoystickType	Types of Joysticks.
JoystickType (日本語)	ジョイスティックの種類。
LoadMode	Possible load modes for love.filesystem.load.
ModifierKey	Modifier keys.

MountPermissions	Permission settings for use when mounting.
SensorType	Kinds of sensors.
Shader:getDebugName	Gets the debug name that was set in love.graphics.newShader.
Shader:hasStage	Gets whether the Shader was created with the given shader stage.
Shader:hasStage (Français)	Retourne si le Shader a été créé avec l'étape de shader donné.
ShaderStage	Spots within the graphics pipeline where a Shader runs its code.
SoundData:copyFrom	Copies the specified section of the given SoundData into this one.
SoundData:slice	Creates a new copy of a section of this SoundData.
StreamType	Types of audio sources.
SystemTheme	System themes.
TextBatch	One or more precalculated strings of text stored in a Drawable object.
TextBatch:add	Adds additional colored text to the TextBatch object at the specified position.
TextBatch:addf	Adds additional formatted / colored text to the TextBatch object at the specified position.
TextBatch:clear	Clears the contents of the TextBatch object.
... further results	

NoHighDPI

NoHighDPI · revision 25349 · last edited 2023-07-05

This library allows you to disable High DPI scaling on Android devices.

How to use

You just need to do this:

```
nohdpi = require "nohighdpi"

function love.load()
    --Pass "graphics", "mouse" and/or "touch"
    --as arguments to replace the given modules.
    --If you don't give any arguments, then
    --all modules will be replaced.
    nohighdpi:replace()
end
```

And voilà, you can use LÖVE normally without worrying about DPI scaling!

- Forum Thread (https://love2d.org/forums/viewtopic.php?f=5&t=87083)
- Github (https://github.com/FloatingBanana/NoHighDPI)

This library is useless since LÖVE 11.3 allows to disable automomatic DPI scaling by setting t.window.usedpiscale = false in conf.lua

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

NoiseWrapper

NoiseWrapper · revision 19068 · last edited 2021-03-14

Generates either a normal noise (normalized from -1 to 1) or a fractal noise depending on how many arguments you give it.

```
local function clamp(v, M, m)
    return (v > M and M or v) < m and m or v
end

local function fractalNoise2(x, y, n, a, f)
    local ca = 1
    local cf = 1
    local val = 0
    for _=1, n do
        val = val + (love.math.noise(x*cf, y*cf)*2-1)*ca
        ca = ca * a
        cf = cf * f
    end
    return clamp(val, 1, -1)
end

local function fractalNoise3(x, y, z, n, a, f)
    local ca = 1
    local cf = 1
    local val = 0
    for _=1, n do
        val = val + (love.math.noise(x*cf, y*cf, z*cf)*2-1)*ca
        ca = ca * a
        cf = cf * f
    end
    return clamp(val, 1, -1)
end

local function fractalNoise4(x, y, z, w, n, a, f)
    local ca = 1
    local cf = 1
    local val = 0
    for _=1, n do
        val = val + (love.math.noise(x*cf, y*cf, z*cf, w*cf)*2-1)*ca
        ca = ca * a
        cf = cf * f
    end
    return clamp(val, 1, -1)
end

-- Giving less than or equal to 2 arguments will generate normal simplex noise
-- normalized from -1 to 1. For 2 or 3 parameters, it will generate Perlin
-- noise. Giving more than 4 arguments will generate fractal noise.
-- When generating fractal noise, the last 3 arguments will be the
-- amount of octaves, the amplitude scale factor, and the frequency scale
-- factor, respectively.
--
-- noiseWrapper(x, [y, [z, [w, [n, a, f]]]])
function noiseWrapper(...)
    local l = #{...}
    if l <= 4 then
        return love.math.noise(...)*2-1
    elseif l == 5 then
        return fractalNoise2(...)
    elseif l == 6 then
        return fractalNoise3(...)
    elseif l == 7 then
        return fractalNoise4(...)
    else return nil end
end
```

Noobhub

Noobhub · revision 16311 · last edited 2023-07-05

Noobhub is a node.js-based multiplayer and network messaging for CoronaSDK, Moai, Gideros & LÖVE

Features

- Connections are routed through socket server with minimum latency, ideal for action games.
- Simple interface. Publish/subscribe paradigm in action.
- Server written on blazing fast node.js.
- Socket connections, works great through any NAT (local area network), messages delivery is reliable and fast.

Repo includes server code (so you can use your own server) and CoronaSDK/Moai/Gideros client. More clients to come. You can test on my server, credentials are hardcoded in demo project!

Lua code may serve as an example of how LuaSocket library works.

Examples

client.lua

```
-- connect to the server
hub = noobhub.new({ server = "127.0.0.1"; port = 1337; });

-- subscribe to a channel and receive callbacks when new json messages arrive
hub:subscribe({
  channel = "hello-world";
    callback = function(message)

        if(message.action == "ping")   then
            print("Pong!")
        end;

    end;
});

-- say something to everybody on the channel
hub:publish({
    message = {
        action  =  "ping",
        timestamp = love.timer.getTime()
    }
});
```

Links

- Forum thread (https://love2d.org/forums/viewtopic.php?f=3&t=76953)
- Source code (https://github.com/Overtorment/NoobHub)

Object

Object · revision 22351 · last edited 2023-07-05

The superclass of all LÖVE types.

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Subtypes

BezierCurve	A Bézier curve object that can evaluate and render Bézier curves of arbitrary degree.	Added since 0.9.0	
Body	Bodies are objects with velocity and position.		
ByteData	Data object containing arbitrary bytes in an contiguous memory.	Added since 11.0	
Canvas	Off-screen render target.	Added since 0.8.0	
ChainShape	A ChainShape consists of multiple line segments.		
Channel	An object which can be used to send and receive data between different threads.	Added since 0.9.0	
CircleShape	Circle extends Shape and adds a radius and a local position.		
CompressedData	Byte data compressed using a specific algorithm.	Added since 0.10.0	
CompressedImageData	Compressed image data designed to stay compressed in RAM and on the GPU.	Added since 0.9.0	
Contact	Contacts are objects created to manage collisions in worlds.		
Cursor	Represents a hardware cursor.	Added since 0.9.0	
Data	The superclass of all data.		
Decoder	An object which can gradually decode a sound file.		
DistanceJoint	Keeps two bodies at the same distance.		
Drawable	Superclass for all things that can be drawn on screen.		
DroppedFile	Represents a file dropped from the window.	Added since 0.10.0	
EdgeShape	EdgeShape is a line segment.		
File	Represents a file on the filesystem.		
FileData	Data representing the contents of a file.		
Fixture	Fixtures attach shapes to bodies.	Added since 0.8.0	
Font	Defines the shape of characters than can be drawn onto the screen.		
FontData	A FontData represents a font.	Added since 0.7.0	Removed in 0.8.0
Framebuffer	Off-screen render target.	Added since 0.7.0	Removed in 0.8.0
FrictionJoint	A FrictionJoint applies friction to a body.	Added since 0.8.0	
GearJoint	Keeps bodies together in such a way that they act like gears.		
GlyphData	A GlyphData represents a drawable symbol of a font.	Added since 0.7.0	
Image	Drawable image type.		
ImageData	Raw (decoded) image data.		
Joint	Attach multiple bodies together to interact in unique ways.		
Joystick	Represents a physical joystick.	Added since 0.9.0	
Mesh	A 2D polygon mesh used for drawing arbitrary textured shapes.	Added since 0.9.0	
MotorJoint	Controls the relative motion between two Bodies	Added since 0.9.0	
MouseJoint	For controlling objects with the mouse.		
ParticleSystem	Used to create cool effects, like fire.		
PixelEffect	Pixel shader effect.	Added since 0.8.0	Removed in 0.9.0
PolygonShape	Polygon is a convex polygon with up to 8 sides.		
PrismaticJoint	Restricts relative motion between Bodies to one shared axis.		
PulleyJoint	Allows you to simulate bodies connected through pulleys.		
Quad	A quadrilateral with texture coordinate information.		
RandomGenerator	A random number generation object which has its own random state.	Added since 0.9.0	
Rasterizer	A Rasterizer represents font data and glyphs.	Added since 0.7.0	
RecordingDevice	Represents an audio input device capable of recording sounds.	Added since 11.0	
RevoluteJoint	Allow two Bodies to revolve around a shared point.		
RopeJoint	Enforces a maximum distance between two points on two bodies.	Added since 0.8.0	
Shader	Shader effect.	Added since 0.9.0	
Shape	Shapes are objects used to control mass and collisions.		

SoundData	Contains raw audio samples.	
Source	A Source represents audio you can play back.	
SpriteBatch	Store image positions in a buffer, and draw it in one call.	
Text	Drawable text.	Added since 0.10.0
Texture	Superclass for drawable objects which represent a texture.	Added since 0.9.1
Thread	A Thread represents a thread.	Added since 0.7.0
Transform	Object containing a coordinate system transformation.	Added since 11.0
Video	A drawable video.	Added since 0.10.0
VideoStream	An object which decodes, streams, and controls Videos.	Added since 0.10.0
WeldJoint	A WeldJoint essentially glues two bodies together.	Added since 0.8.0
WheelJoint	Restricts a point on the second body to a line on the first body.	
World	A world is an object that contains all bodies and joints.	

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Object:release

Object:release · revision 21304 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Destroys the object's Lua reference. The object will be completely deleted if it's not referenced by any other LÖVE object or thread.

This method can be used to immediately clean up resources without waiting for Lua's garbage collector.



After this method has been called, attempting to call any other method on the object or using the object as an argument in another LÖVE API will cause an error.

Function

Synopsis

```
success = Object:release( )
```

Arguments

None.

Returns

`boolean success`

True if the object was released by this call, false if it had been previously released.

See Also

- Object

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Object:type

Object:type · revision 28133 · last edited 2025-10-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Printing the type of an object
- See Also
- Other Languages

Gets the type of the object as a string.

Function

Synopsis

```
type = Object:type()
```

Arguments

None.

Returns

`string type`
: The type as a string.

Examples

Printing the type of an object

```
image = love.graphics.newImage("test.png")
print(image:type()) -- outputs: Texture
source = love.audio.newSource("test.ogg")
print(source:type()) -- outputs: Source
```

See Also

- Object

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Object:typeOf

Object:typeOf · revision 12109 · last edited 2025-10-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Checking the type of an object
- See Also
- Other Languages

Checks whether an object is of a certain type. If the object has the type with the specified name in its hierarchy, this function will return true.

Function

Synopsis

```
b = Object:typeOf( name )
```

Arguments

`string name`
: The name of the type to check for.

Returns

`boolean b`
: True if the object is of the specified type, false otherwise.

Examples

Checking the type of an object

```
image = love.graphics.newImage("test.png")
print(image:typeOf("Object")) -- outputs: true
print(image:typeOf("Drawable")) -- outputs: true
print(image:typeOf("Image")) -- outputs: true
print(image:typeOf("MouseJoint")) -- outputs: false
```

See Also

- Object

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Objects

Objects · revision 28217 · last edited 2023-02-20

You can create own classes and make objects like:

```lua
 -- objects.lua
local Objects = {}

function Objects:new(obj)
  obj = obj or {}
  setmetatable(obj, self)
  self.__index = self
  return obj
end

function Objects:update(dt)
  -- update object properties
end

function Objects:draw()
  -- draw object on screen
end

return Objects
```

ONET

ONET · revision 24597 · last edited 2023-07-05

ONET is a object oriented networking library that can be used to synchronize objects over network, it works over enet.

- You can find more information about this library on the following link.
  - https://github.com/matiasah/onet/wiki ↗
- The code repository link.
  - https://github.com/matiasah/onet ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

OpenCanvas

OpenCanvas · revision 2470 · last edited 2021-03-14

OpenCanvas is a popular computer graphics software program created by Portalgraphics. It was specifically designed for use with pressure-sensitive drawing tablets; it is a sketch program rather than a general-purpose graphic design program.

Its popular for being fast and lightwieght, while still being powerful enough for many works. the paintchat feature can be used for graphical Collaboration, as well as simple rapid Ideation

It was originally released as freeware, but the authors moved to a demo/retail model starting with version 2. openCancas 1.1 b72, the last free version, is available to download.

- 1.1b72 (freeware) Version Archive Page (http://w istinga.online.fr/opencanvas/)
- Official OpenCanvas Website (http://w w w.portalgraphics.net/en/oc/)

Optimising

Optimising · revision 29430 · last edited 2025-01-10

On this page

- Root of all evil
- Be a sniper, not a carpet bomber
- Keep loops short and sw eet
- Prefer numerical fors, and ipairs
- Use Locals
- Cull the herd
- Avoid Overdraw
- Watch the hidden loops

Optimising in Lua is often counter-intuitive. Never assume a change has improved performance, even if you "know", logically, that it would have. Always check. Always.

General Optimising Tips

Root of all evil

Donald Knuth said,

> *" premature optimization is the root of all evil"*

In other words, don't do heavy optimisation until you see things slowing down.

That is all.

Be a sniper, not a carpet bomber

Hit the target, not the square mile its in.

Much like the CPU cycles you are trying to save, your time is valuable. Don't waste it optimising code that isn't slowing you down. Profile your code, figure out where your bottlenecks are, and optimise *those*.

General Lua Optimisation

Keep loops short and sweet

While loops are useful and important construct in any program, they are also *repetitious*, and like to magnify the effects of any statements within them. Every performance tip in here applies double to loops.

When designing loops, consider if each individual statement *really* needs to be in the body of the loop, everything that can be legitimately moved outside the body of the loop will reduce the time it takes for the loop to complete.

Prefer numerical *for*s, and *ipairs*

Lua's *table*s are powerful useful and, to Lua's credit, fairly fast. The *pairs* iterator allows you to easily loop through all of the elements in a *table*, rather than *ipairs* merely looping through the elements with integral numeric keys in 1 to n. It is, however, not the fastest way to loop through a table, *ipairs* is faster, but the fastest is the humble numerical for; "for i=1,n do ... end". Yes, its not as elegant, but its still the fastest.

Use *Local*s

In Lua, Variables are generally either *global*s (the default), or *local*s. Lua keeps a short list of *local* variables that it checks before checking the global environment. Generally this means that *local* look-ups are "just plain faster" than *global*s. Normally this performance gain is splitting hairs, but there's a common situation where it can be noticeable.

Consider the following:

```
--WRONG WAY
  for i=1,1000000 do
    localx = math.sin(i)
  end

--RIGHT WAY
  local sin = math.sin
  for i=1, 1000000 do
```

```
    localx = sin(i)
end
```

- In the **WRONG WAY** example, each use of *math.sin()* would require two look-ups, once in *_G*, looking for *math*, and again in *math* looking for *sin*; and this would happen every iteration of the loop (so about 2,000,000 look-ups).
- In the **RIGHT WAY** example, we begin by creating a local containing the *math.sin()* function. its this value which is then used in the loop. Immediately its obvious that we've basically halved the number of look-ups required by this section code, and that's without considering that local look-ups are cheaper than global look ups.

Love-Specific Tips

Cull the herd

It can be tempting to design drawing functions that make little or no attempt to consider the visibility of game elements, relying instead on the graphics hardware's basic clipping (https://en.w ikipedia.org/w iki/Clipping_(computer_graphics)) capabilities. if nothing else, this can often give simpler and easier to understand functions.

But even if these elements aren't visible, they still need to be transformed and processed, and their data sent to the card's hardware. Depending on your game's design, you might do additional calculations for that element, such as calculating look-ats, or blending colors. All of which are unnecessary, give that the element isn't actually visible, and on non-trivial projects, this can be a performance cost you can ill-afford.

Refactor (https://en.w ikipedia.org/w iki/Code_refactoring) your code to avoid calculations that only need to be done visible elements, and avoid sending any data to the hardware about objects you can determine won't be visible.

Avoid Overdraw

Overdraw is when you draw over part of the screen that you had already drawn to in the current frame. much like in the *Cull the Herd* tip above, this is wasted effort. Why spend time drawing something that won't be seen?

The old axiom from the early days of graphical gaming was "touch every pixel only once", (I.E. zero-overdraw). fortunately hardware has come a long way since then, and some slovenliness is perfectly reasonable if it offers some other advantage (such as simpler code), but should avoid engine designs that are likely to create large amounts of overdraw.

Watch the hidden loops

Love calls the functions *love.update(dt)* and *love.draw()* frequently during the lifetime of a Love game. in fact, they are the body of loops hidden in *love.run()*. All the performance considerations that apply to loops, apply to these two callbacks as well.

Further Tips

- LuaPerformance (http://trac.caspring.org/w iki/LuaPerformance) at trac

os.time

os.time · revision 6851 · last edited 2021-03-14

Part of the Os Library built into Lua, returns the time in system seconds. `os.time()` is very useful in conjunction with `math.randomseed()`, since time will always be different each time it is used (essentially guaranteeing a unique seed value).

Examples

Assign a random number based on the time to var and print it

```
math.randomseed(os.time())
var = math.random()
function love.draw()
   love.graphics.print(var,5,5)
end
```

See Also

- love.timer

ParticleInsertMode

ParticleInsertMode · revision 11057 · last edited 2023-07-05

Available since LÖVE 0.9.0
This enum is not supported in earlier versions.

How newly created particles are added to the ParticleSystem.

Constants

top
: Particles are inserted at the top of the ParticleSystem's list of particles.

bottom
: Particles are inserted at the bottom of the ParticleSystem's list of particles.

random
: Particles are inserted at random positions in the ParticleSystem's list of particles.

See Also

- ParticleSystem
- ParticleSystem:setInsertMode
- ParticleSystem:getInsertMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem

ParticleSystem · revision 28596 · last edited 2024-10-15

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

A ParticleSystem can be used to create particle effects like fire or smoke.

The particle system has to be created using love.graphics.newParticleSystem. Just like any other Drawable it can be drawn to the screen using love.graphics.draw. You also have to update it in the update callback to see any changes in the particles emitted.

The particle system won't create any particles unless you call setParticleLifetime and setEmissionRate.

Constructors

love.graphics.newParticleSystem	Creates a new ParticleSystem.

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
ParticleSystem:clone	Creates an identical copy of the ParticleSystem in the stopped state.	Added since 0.9.1	
ParticleSystem:count	Gets the amount of particles that are currently in the system.		Removed in 0.9.0
ParticleSystem:emit	Emits a burst of particles from the particle emitter.	Added since 0.9.0	
ParticleSystem:getAreaSpread	Gets the area-based spawn parameters for the particles.	Added since 0.9.0	Deprecated in 11.0
ParticleSystem:getBufferSize	Gets the maximum number of particles the ParticleSystem can have at once.	Added since 0.9.0	
ParticleSystem:getColors	Gets the colors to apply to the particle sprite.	Added since 0.9.0	
ParticleSystem:getCount	Gets the number of particles that are currently in the system.	Added since 0.9.0	
ParticleSystem:getDirection	Gets the direction of the particle emitter (in radians).		
ParticleSystem:getEmissionArea	Gets the area-based spawn parameters for the particles.	Added since 11.0	
ParticleSystem:getEmissionRate	Gets the amount of particles emitted per second.	Added since 0.9.0	
ParticleSystem:getEmitterLifetime	Gets how long the particle system will emit particles	Added since 0.9.0	
ParticleSystem:getImage	Gets the image used for the particles.	Added since 0.9.0	Removed in 0.10.0
ParticleSystem:getInsertMode	Gets the mode used when the ParticleSystem adds new particles.	Added since 0.9.0	
ParticleSystem:getLinearAcceleration	Gets the linear acceleration (acceleration along the x and y axes) for particles.	Added since 0.9.0	
ParticleSystem:getLinearDamping	Gets the amount of linear damping (constant deceleration) for particles.	Added since 0.9.2	
ParticleSystem:getOffset	Gets the particle image's draw offset.	Added since 0.9.0	
ParticleSystem:getOffsetX	Get the x coordinate of the particle rotation offset.		Removed in 0.9.0
ParticleSystem:getOffsetY	Get the y coordinate of the particle rotation offset.		Removed in 0.9.0
ParticleSystem:getParticleLifetime	Gets the lifetime of the particles.	Added since 0.9.0	
ParticleSystem:getPosition	Gets the position of the emitter.		

ParticleSystem:getQuads	Gets a series of Quads used for the particle sprites.	Added since 0.9.2	
ParticleSystem:getRadialAcceleration	Gets the radial acceleration (away from the emitter).	Added since 0.9.0	
ParticleSystem:getRotation	Gets the rotation of the image upon particle creation (in radians).	Added since 0.9.0	
ParticleSystem:getSizeVariation	Gets the amount of size variation.	Added since 0.9.0	
ParticleSystem:getSizes	Gets the sizes of the particle over its lifetime.	Added since 0.9.0	
ParticleSystem:getSpeed	Gets the speed of the particles.	Added since 0.9.0	
ParticleSystem:getSpin	Gets the spin of the sprite.	Added since 0.9.0	
ParticleSystem:getSpinVariation	Gets the amount of spin variation.	Added since 0.9.0	
ParticleSystem:getSpread	Gets the amount of directional spread of the particle emitter (in radians).		
ParticleSystem:getTangentialAcceleration	Gets the tangential acceleration (acceleration perpendicular to the particle's direction).	Added since 0.9.0	
ParticleSystem:getTexture	Gets the texture (Image or Canvas) used for the particles.	Added since 0.9.1	
ParticleSystem:getX	Gets the x-coordinate of the particle emitter's position.		Removed in 0.9.0
ParticleSystem:getY	Gets the y-coordinate of the particle emitter's position.		Removed in 0.9.0
ParticleSystem:hasRelativeRotation	Gets whether particle angles and rotations are relative to their velocities.	Added since 0.9.1	
ParticleSystem:isActive	Checks whether the particle system is actively emitting particles.		
ParticleSystem:isEmpty	Checks whether the particle system is empty of particles.		Removed in 0.9.0
ParticleSystem:isFull	Checks whether the particle system is full of particles.		Removed in 0.9.0
ParticleSystem:isPaused	Checks whether the particle system is paused.	Added since 0.9.0	
ParticleSystem:isStopped	Checks whether the particle system is stopped.	Added since 0.9.0	
ParticleSystem:moveTo	Moves the position of the emitter.	Added since 0.9.1	
ParticleSystem:pause	Pauses the particle emitter.		
ParticleSystem:reset	Resets the particle emitter, removing existing particles and resetting the lifetime counter.		
ParticleSystem:setAreaSpread	Sets area-based spawn parameters for the particles.	Added since 0.9.0	Deprecated in 11.0
ParticleSystem:setBufferSize	Sets the size of the buffer (the max allowed amount of particles in the system).		
ParticleSystem:setColor	Sets the color of the image.		Removed in 0.8.0
ParticleSystem:setColors	Sets the colors to apply to the particle sprite.	Added since 0.8.0	
ParticleSystem:setDirection	Sets the direction the particles will be emitted in.		
ParticleSystem:setEmissionArea	Sets area-based spawn parameters for the particles.	Added since 11.0	
ParticleSystem:setEmissionRate	Sets the amount of particles emitted per second.		
ParticleSystem:setEmitterLifetime	Sets how long the particle system should emit particles	Added since 0.9.0	
ParticleSystem:setGravity	Sets the gravity affecting the particles (acceleration along the y-axis).		Removed in 0.9.0
ParticleSystem:setImage	Sets the image to be used for the particles.	Added since 0.9.0	Removed in 0.10.0
ParticleSystem:setInsertMode	Sets the mode to use when the ParticleSystem adds new particles.	Added since 0.9.0	
ParticleSystem:setLifetime	Sets how long the particle system should emit particles (if -1 then it emits particles forever).		Removed in 0.9.0

ParticleSystem:setLinearAcceleration	Sets the linear acceleration (acceleration along the x and y axes) for particles.	Added since 0.9.0	
ParticleSystem:setLinearDamping	Sets the amount of linear damping (constant deceleration) for particles.	Added since 0.9.2	
ParticleSystem:setOffset	Set the offset position which the particle sprite is rotated around.		
ParticleSystem:setParticleLife	Sets the life of the particles.		Removed in 0.9.0
ParticleSystem:setParticleLifetime	Sets the lifetime of the particles.	Added since 0.9.0	
ParticleSystem:setPosition	Sets the position of the emitter.		
ParticleSystem:setQuads	Sets a series of Quads to use for the particle sprites.	Added since 0.9.2	
ParticleSystem:setRadialAcceleration	Set the radial acceleration (away from the emitter).		
ParticleSystem:setRelativeRotation	Sets whether particle angles and rotations are relative to their velocities.	Added since 0.9.1	
ParticleSystem:setRotation	Sets the rotation of the image upon particle creation (in radians).		
ParticleSystem:setSize	Sets the size of the particle (1.0 being normal size).		Removed in 0.8.0
ParticleSystem:setSizeVariation	Sets the amount of size variation.		
ParticleSystem:setSizes	Sets the sizes of the particle over its lifetime.	Added since 0.8.0	
ParticleSystem:setSpeed	Sets the speed of the particles.		
ParticleSystem:setSpin	Sets the spin of the sprite.		
ParticleSystem:setSpinVariation	Sets the amount of spin variation.		
ParticleSystem:setSpread	Sets the amount of spread for the system.		
ParticleSystem:setSprite	Sets the image which is to be emitted.		Removed in 0.9.0
ParticleSystem:setTangentialAcceleration	Sets the tangential acceleration (acceleration perpendicular to the particle's direction).		
ParticleSystem:setTexture	Sets the texture (Image or Canvas) to be used for the particles.	Added since 0.9.1	
ParticleSystem:start	Starts the particle emitter.		
ParticleSystem:stop	Stops the particle emitter, resetting the lifetime counter.		
ParticleSystem:update	Updates the particle system; moving, creating and killing particles.		

Enums

AreaSpreadDistribution	Types of particle area spread distribution.	Added since 0.9.0
ParticleInsertMode	How newly created particles are added to the ParticleSystem.	Added since 0.9.0

Supertypes

- Drawable
- Object

See Also

- love.graphics

Particle editors:

- https://love2d.org/forums/viewtopic.php?f=4&t=2110
- https://love2d.org/forums/viewtopic.php?f=5&t=32954
- https://love2d.org/forums/viewtopic.php?f=5&t=76986

- https://love2d.org/forums/viewtopic.php?f=5&t=88860

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:clone

ParticleSystem:clone · revision 12785 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Creates an identical copy of the ParticleSystem in the stopped state.

Function

Synopsis

```
particlesystem = ParticleSystem:clone( )
```

Arguments

None.

Returns

`ParticleSystem particlesystem`

The new identical copy of this ParticleSystem.

Notes

Cloned ParticleSystem inherit all the set-able state of the original ParticleSystem, but they are initialized stopped.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:count

ParticleSystem:count · revision 11158 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to ParticleSystem:getCount.

Gets the amount of particles that are currently in the system.

Function

Synopsis

```
count = ParticleSystem:count( )
```

Arguments

None.

Returns

`number count`

The current number of live particles.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:emit

ParticleSystem:emit · revision 17791 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Spawn a bunch of particles
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Emits a burst of particles from the particle emitter.

Function

Synopsis

```
ParticleSystem:emit( numparticles )
```

Arguments

number numparticles

The amount of particles to emit. The number of emitted particles will be truncated if the particle system's max buffer size is reached.

Returns

Nothing.

Examples

Spawn a bunch of particles

This example will create a burst of particles when the spacebar is pressed. You can use the löve logo as an example image.

```
function love.load()
    local img = love.graphics.newImage('logo.png')

    psystem = love.graphics.newParticleSystem(img, 32)
    psystem:setParticleLifetime(2, 5) -- Particles live at least 2s and at most 5s.
    psystem:setLinearAcceleration(-5, -5, 50, 100) -- Randomized movement towards the bottom of the screen.
    psystem:setColors(255, 255, 255, 255, 255, 255, 255, 0) -- Fade to black.
end

function love.draw()
    -- Draw the particle system at the center of the game window.
    love.graphics.draw(psystem, love.graphics.getWidth() * 0.5, love.graphics.getHeight() * 0.5)
end

function love.update(dt)
    psystem:update(dt)
end

function love.keypressed(key)
    if key == 'space' then
        psystem:emit(32)
    end
end
```

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getAreaSpread

ParticleSystem:getAreaSpread · revision 22738 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function has been renamed to ParticleSystem:getEmissionArea.

Gets the area-based spawn parameters for the particles.

Function

Synopsis

```
distribution, dx, dy = ParticleSystem:getAreaSpread( )
```

Arguments

None.

Returns

`AreaSpreadDistribution distribution`
: The type of distribution for new particles.

`number dx`
: The maximum spawn distance from the emitter along the x-axis for uniform distribution, or the standard deviation along the x-axis for normal distribution.

`number dy`
: The maximum spawn distance from the emitter along the y-axis for uniform distribution, or the standard deviation along the y-axis for normal distribution.

See Also

- ParticleSystem
- ParticleSystem:setAreaSpread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getBufferSize

ParticleSystem:getBufferSize · revision 11773 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the maximum number of particles the ParticleSystem can have at once.

Function

Synopsis

```
size = ParticleSystem:getBufferSize( )
```

Arguments

None.

Returns

`number size`

The maximum number of particles.

See Also

- ParticleSystem
- ParticleSystem:setBufferSize
- ParticleSystem:getCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getColors

ParticleSystem:getColors · revision 25353 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the series of colors applied to the particle sprite.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
rgba1, rgba2, ..., rgba8 = ParticleSystem:getColors( )
```

Arguments

Nothing.

Returns

`table rgba1`
: First color, a numerical indexed table with the red, green, blue and alpha values as numbers (0-1).

`table rgba2`
: Second color, a numerical indexed table with the red, green, blue and alpha values as numbers (0-1).

`table rgba8`
: Eighth color, a numerical indexed table with the red, green, blue and alpha values as numbers (0-1).

See Also

- ParticleSystem
- ParticleSystem:setColors

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getCount

ParticleSystem:getCount · revision 11774 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from ParticleSystem:count.

Gets the number of particles that are currently in the system.

Function

Synopsis

```
count = ParticleSystem:getCount( )
```

Arguments

None.

Returns

`number count`

The current number of live particles.

See Also

- ParticleSystem
- ParticleSystem:getBufferSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getDirection

ParticleSystem:getDirection · revision 5820 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the direction of the particle emitter (in radians).

Function

Synopsis

```
direction = ParticleSystem:getDirection( )
```

Arguments

None.

Returns

`number direction`
: The direction of the emitter (radians).

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getEmissionArea

ParticleSystem:getEmissionArea · revision 22658 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from ParticleSystem:getAreaSpread.

Gets the area-based spawn parameters for the particles.

Function

Synopsis

```
distribution, dx, dy, angle, directionRelativeToCenter = ParticleSystem:getEmissionArea( )
```

Arguments

None.

Returns

`AreaSpreadDistribution distribution`
: The type of distribution for new particles.

`number dx`
: The maximum spawn distance from the emitter along the x-axis for uniform distribution, or the standard deviation along the x-axis for normal distribution.

`number dy`
: The maximum spawn distance from the emitter along the y-axis for uniform distribution, or the standard deviation along the y-axis for normal distribution.

`number angle`
: The angle in radians of the emission area.

`boolean directionRelativeToCenter`
: True if newly spawned particles will be oriented relative to the center of the emission area, false otherwise.

See Also

- ParticleSystem
- ParticleSystem:setEmissionArea

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getEmissionRate

ParticleSystem:getEmissionRate · revision 13791 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the amount of particles emitted per second.

Function

Synopsis

```
rate = ParticleSystem:getEmissionRate( )
```

Arguments

Nothing.

Returns

`number rate`
: The amount of particles per second.

See Also

- ParticleSystem
- ParticleSystem:setEmissionRate

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getEmitterLifetime

ParticleSystem:getEmitterLifetime · revision 13792 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets how long the particle system will emit particles (if -1 then it emits particles forever).

Function

Synopsis

```
life = ParticleSystem:getEmitterLifetime( )
```

Arguments

Nothing.

Returns

`number life`

The lifetime of the emitter (in seconds).

See Also

- ParticleSystem
- ParticleSystem:setEmitterLifetime

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getImage

ParticleSystem:getImage · revision 15741 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Removed in LÖVE 0.10.0

Use ParticleSystem:getTexture instead.

Gets the image used for the particles.

Function

Synopsis

```
image = ParticleSystem:getImage( )
```

Arguments

Nothing.

Returns

`Image image`

An Image to use for the particles.

See Also

- ParticleSystem
- ParticleSystem:setImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getInsertMode

ParticleSystem:getInsertMode · revision 11061 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the mode used when the ParticleSystem adds new particles.

Function

Synopsis

```
mode = ParticleSystem:getInsertMode( )
```

Arguments

None.

Returns

`ParticleInsertMode mode`

The mode used when the ParticleSystem adds new particles.

See Also

- ParticleSystem
- ParticleSystem:setInsertMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getLinearAcceleration

ParticleSystem:getLinearAcceleration · revision 12387 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the linear acceleration (acceleration along the x and y axes) for particles.

Every particle created will accelerate along the x and y axes between xmin,ymin and xmax,ymax.

Function

Synopsis

```
xmin, ymin, xmax, ymax = ParticleSystem:getLinearAcceleration( )
```

Arguments

Nothing.

Returns

number xmin
: The minimum acceleration along the x axis.

number ymin
: The minimum acceleration along the y axis.

number xmax (xmin)
: The maximum acceleration along the x axis.

number ymax (ymin)
: The maximum acceleration along the y axis.

See Also

- ParticleSystem
- ParticleSystem:setLinearAcceleration

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getLinearDamping

ParticleSystem:getLinearDamping · revision 14280 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the amount of linear damping (constant deceleration) for particles.

Function

Synopsis

```
min, max = ParticleSystem:getLinearDamping( )
```

Arguments

None.

Returns

number min
: The minimum amount of linear damping applied to particles.

number max
: The maximum amount of linear damping applied to particles.

See Also

- ParticleSystem
- ParticleSystem:setLinearDamping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getOffset

ParticleSystem:getOffset · revision 12376 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

It has been moved from ParticleSystem:getOffsetX and ParticleSystem:getOffsetY.

Gets the particle image's draw offset.

Function

Synopsis

```
ox, oy = ParticleSystem:getOffset( )
```

Arguments

None.

Returns

number ox
: The x coordinate of the particle image's draw offset.

number oy
: The y coordinate of the particle image's draw offset.

See Also

- ParticleSystem
- ParticleSystem:setOffset

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getOffsetX

ParticleSystem:getOffsetX · revision 12374 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been moved to ParticleSystem:getOffset.

Get the x coordinate of the particle rotation offset.

Function

Synopsis

```
xOffset = ParticleSystem:getOffsetX( )
```

Arguments

None.

Returns

`number xOffset`

The x coordinate of the rotation offset.

See Also

- ParticleSystem
- ParticleSystem:getOffsetY
- ParticleSystem:setOffset

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getOffsetY

ParticleSystem:getOffsetY · revision 12375 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been moved to ParticleSystem:getOffset.

Get the y coordinate of the particle rotation offset.

Function

Synopsis

```
yOffset = ParticleSystem:getOffsetY( )
```

Arguments

None.

Returns

`number yOffset`

The y coordinate of the rotation offset.

See Also

- ParticleSystem
- ParticleSystem:getOffsetX
- ParticleSystem:setOffset

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

ParticleSystem:getParticleLifetime

ParticleSystem:getParticleLifetime · revision 12388 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the lifetime of the particles.

Function

Synopsis

```
min, max = ParticleSystem:getParticleLifetime( )
```

Arguments

Nothing.

Returns

`number min`
: The minimum life of the particles (in seconds).

`number max`
: The maximum life of the particles (in seconds).

See Also

- ParticleSystem
- ParticleSystem:setParticleLifetime

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getPosition

ParticleSystem:getPosition · revision 9650 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the position of the emitter.

Function

Synopsis

```
x, y = ParticleSystem:getPosition( )
```

Arguments

None.

Returns

number x
: Position along x-axis.

number y
: Position along y-axis.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getQuads

ParticleSystem:getQuads · revision 20214 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the series of Quads used for the particle sprites.

Function

Synopsis

```
quads = ParticleSystem:getQuads( )
```

Arguments

Nothing.

Returns

`table quads`

A table containing the Quads used.

See Also

- ParticleSystem
- ParticleSystem:setQuads

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getRadialAcceleration

ParticleSystem:getRadialAcceleration · revision 17792 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the radial acceleration (away from the emitter).

Function

Synopsis

```
min, max = ParticleSystem:getRadialAcceleration( )
```

Arguments

Nothing.

Returns

number min
: The minimum acceleration.

number max
: The maximum acceleration.

See Also

- ParticleSystem
- ParticleSystem:setRadialAcceleration

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getRotation

ParticleSystem:getRotation · revision 12390 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the rotation of the image upon particle creation (in radians).

Function

Synopsis

```
min, max = ParticleSystem:getRotation( )
```

Arguments

Nothing.

Returns

`number min`
: The minimum initial angle (radians).

`number max`
: The maximum initial angle (radians).

See Also

- ParticleSystem
- ParticleSystem:setRotation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getSizes

ParticleSystem:getSizes · revision 13793 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the series of sizes by which the sprite is scaled. 1.0 is normal size. The particle system will interpolate between each size evenly over the particle's lifetime.

Function

Synopsis

```
size1, size1, ..., size8 = ParticleSystem:getSizes( )
```

Arguments

Nothing.

Returns

`number size1`
: The first size.

`number size2`
: The second size.

`number size8`
: The eighth size.

See Also

- ParticleSystem
- ParticleSystem:setSizes

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getSizeVariation

ParticleSystem:getSizeVariation · revision 12391 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the amount of size variation (0 meaning no variation and 1 meaning full variation between start and end).

Function

Synopsis

```
variation = ParticleSystem:getSizeVariation( )
```

Arguments

Nothing.

Returns

`number variation`

The amount of variation (0 meaning no variation and 1 meaning full variation between start and end).

See Also

- ParticleSystem
- ParticleSystem:setSizeVariation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getSpeed

ParticleSystem:getSpeed · revision 12393 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the speed of the particles.

Function

Synopsis

```
min, max = ParticleSystem:getSpeed( )
```

Arguments

Nothing.

Returns

`number min`
: The minimum linear speed of the particles.

`number max`
: The maximum linear speed of the particles.

See Also

- ParticleSystem
- ParticleSystem:setSpeed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getSpin

ParticleSystem:getSpin · revision 12394 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the spin of the sprite.

Function

Synopsis

```
min, max, variation = ParticleSystem:getSpin( )
```

Arguments

Nothing.

Returns

`number min`
: The minimum spin (radians per second).

`number max (min)`
: The maximum spin (radians per second).

`number variation (0)`
: The degree of variation (0 meaning no variation and 1 meaning full variation between start and end).

See Also

- ParticleSystem
- ParticleSystem:setSpin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getSpinVariation

ParticleSystem:getSpinVariation · revision 12395 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the amount of spin variation (0 meaning no variation and 1 meaning full variation between start and end).

Function

Synopsis

```
variation = ParticleSystem:getSpinVariation( )
```

Arguments

Nothing.

Returns

`number variation`

The amount of variation (0 meaning no variation and 1 meaning full variation between start and end).

See Also

- ParticleSystem
- ParticleSystem:setSpinVariation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getSpread

ParticleSystem:getSpread · revision 5823 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the amount of directional spread of the particle emitter (in radians).

Function

Synopsis

```
spread = ParticleSystem:getSpread( )
```

Arguments

None.

Returns

`number spread`
: The spread of the emitter (radians).

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getTangentialAcceleration

ParticleSystem:getTangentialAcceleration · revision 12385 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the tangential acceleration (acceleration perpendicular to the particle's direction).

Function

Synopsis

```
min, max = ParticleSystem:getTangentialAcceleration( )
```

Arguments

Nothing.

Returns

number min
: The minimum acceleration.

number max
: The maximum acceleration.

See Also

- ParticleSystem
- ParticleSystem:setTangentialAcceleration

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getTexture

ParticleSystem:getTexture · revision 13691 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

It has been renamed from ParticleSystem:getImage.

Gets the texture (Image or Canvas) used for the particles.

Function

Synopsis

```
texture = ParticleSystem:getTexture( )
```

Arguments

None.

Returns

`Texture texture`

The Image or Canvas used for the particles.

See Also

- ParticleSystem
- ParticleSystem:setTexture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getX

ParticleSystem:getX · revision 11254 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0
Use ParticleSystem:getPosition.

Gets the x-coordinate of the particle emitter's position.

Function

Synopsis

```
x = ParticleSystem:getX( )
```

Arguments

None.

Returns

number x
Position along x-axis.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:getY

ParticleSystem:getY · revision 11255 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0
Use ParticleSystem:getPosition.

Gets the y-coordinate of the particle emitter's position.

Function

Synopsis

```
y = ParticleSystem:getY( )
```

Arguments

None.

Returns

number y
Position along y-axis.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:hasRelativeRotation

ParticleSystem:hasRelativeRotation · revision 13056 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets whether particle angles and rotations are relative to their velocities. If enabled, particles are aligned to the angle of their velocities and rotate relative to that angle.

Function

Synopsis

```
relative = ParticleSystem:hasRelativeRotation( )
```

Arguments

None.

Returns

`boolean enable`

True if relative particle rotation is enabled, false if it's disabled.

See Also

- ParticleSystem
- ParticleSystem:setRelativeRotation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:isActive

ParticleSystem:isActive · revision 11729 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Checks whether the particle system is actively emitting particles.

Function

Synopsis

```
active = ParticleSystem:isActive( )
```

Arguments

None.

Returns

`boolean active`
: True if system is active, false otherwise.

See Also

- ParticleSystem
- ParticleSystem:isPaused
- ParticleSystem:isStopped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:isEmpty

ParticleSystem:isEmpty · revision 11772 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removed in LÖVE 0.9.0

Use ParticleSystem:getCount and ParticleSystem:getBufferSize.

Checks whether the particle system is empty of particles.

Function

Synopsis

```
empty = ParticleSystem:isEmpty( )
```

Arguments

None.

Returns

`boolean empty`

True if there are no live particles, false otherwise.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:isFull

ParticleSystem:isFull · revision 11771 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Use ParticleSystem:getCount and ParticleSystem:getBufferSize.

Checks whether the particle system is full of particles.

Function

Synopsis

```
full = ParticleSystem:isFull( )
```

Arguments

None.

Returns

`boolean full`

True if no more particles can be added, false otherwise.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:isPaused

ParticleSystem:isPaused · revision 11727 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Checks whether the particle system is paused.

Function

Synopsis

```
paused = ParticleSystem:isPaused( )
```

Arguments

None.

Returns

`boolean paused`

True if system is paused, false otherwise.

See Also

- ParticleSystem
- ParticleSystem:isActive
- ParticleSystem:isStopped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:isStopped

ParticleSystem:isStopped · revision 11728 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Checks whether the particle system is stopped.

Function

Synopsis

```
stopped = ParticleSystem:isStopped( )
```

Arguments

None.

Returns

`boolean stopped`

True if system is stopped, false otherwise.

See Also

- ParticleSystem
- ParticleSystem:isActive
- ParticleSystem:isPaused

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:moveTo

ParticleSystem:moveTo · revision 12934 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Moves the position of the emitter. This results in smoother particle spawning behaviour than if ParticleSystem:setPosition is used every frame.

Function

Synopsis

```
ParticleSystem:moveTo( x, y )
```

Arguments

number x
: Position along x-axis.

number y
: Position along y-axis.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:pause

ParticleSystem:pause · revision 5829 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Pauses the particle emitter.

Function

Synopsis

```
ParticleSystem:pause( )
```

Arguments

None.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:reset

ParticleSystem:reset · revision 10617 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Resets the particle emitter, removing any existing particles and resetting the lifetime counter.

Function

Synopsis

```
ParticleSystem:reset( )
```

Arguments

None.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setAreaSpread

ParticleSystem:setAreaSpread · revision 22124 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function has been renamed to ParticleSystem:setEmissionArea.

Sets area-based spawn parameters for the particles. Newly created particles will spawn in an area around the emitter based on the parameters to this function.

Function

Synopsis

```
ParticleSystem:setAreaSpread( distribution, dx, dy )
```

Arguments

`AreaSpreadDistribution distribution`
: The type of distribution for new particles.

`number dx`
: The maximum spawn distance from the emitter along the x-axis for uniform distribution, or the standard deviation along the x-axis for normal distribution.

`number dy`
: The maximum spawn distance from the emitter along the y-axis for uniform distribution, or the standard deviation along the y-axis for normal distribution.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:getAreaSpread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setBufferSize

ParticleSystem:setBufferSize · revision 5831 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the size of the buffer (the max allowed amount of particles in the system).

Function

Synopsis

```
ParticleSystem:setBufferSize( size )
```

Arguments

`number size`
: The buffer size.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setColor

ParticleSystem:setColor · revision 10616 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

It has been replaced with ParticleSystem:setColors.

Sets the color of the image.

Function

Synopsis

```
ParticleSystem:setColor( r1, g1, b1, a1, r2, g2, b2, a2 )
```

Arguments

number r1
: Start color, red component (0-255).

number g1
: Start color, green component (0-255).

number b1
: Start color, blue component (0-255).

number a1
: Start color, alpha component (0-255).

number r2 (r1)
: End color, red component (0-255).

number g2 (g1)
: End color, green component (0-255).

number b2 (b1)
: End color, blue component (0-255).

number a2 (a1)
: End color, alpha component (0-255).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setColors

ParticleSystem:setColors · revision 25354 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has replaced ParticleSystem:setColor.

Sets a series of colors to apply to the particle sprite. The particle system will interpolate between each color evenly over the particle's lifetime.

Arguments can be passed in groups of four, representing the components of the desired RGBA value, or as tables of RGBA component values, with a default alpha value of 1 if only three values are given. At least one color must be specified. A maximum of eight may be used.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
ParticleSystem:setColors( r1, g1, b1, a1, r2, g2, b2, a2, ..., r8, g8, b8, a8 )
```

Arguments

number r1
: First color, red component (0-1).

number g1
: First color, green component (0-1).

number b1
: First color, blue component (0-1).

number a1
: First color, alpha component (0-1).

number r2
: Second color, red component (0-1).

number g2
: Second color, green component (0-1).

number b2
: Second color, blue component (0-1).

number a2
: Second color, alpha component (0-1).

number r8
: Eighth color, red component (0-1).

number g8
: Eighth color, green component (0-1).

number b8
: Eighth color, blue component (0-1).

number a8

Eighth color, alpha component (0-1).

Returns

Nothing.

Function

Available since LÖVE 0.9.0
This variant is not supported in earlier versions.

Synopsis

```
ParticleSystem:setColors( rgba1, rgba2, ..., rgba8 )
```

Arguments

`table rgba1`
First color, a numerical indexed table with the red, green, blue and alpha values as numbers (0-1). The alpha is optional and defaults to 1 if it is left out.

`table rgba2`
Second color, a numerical indexed table with the red, green, blue and alpha values as numbers (0-1). The alpha is optional and defaults to 1 if it is left out.

`table rgba8`
Eighth color, a numerical indexed table with the red, green, blue and alpha values as numbers (0-1). The alpha is optional and defaults to 1 if it is left out.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setDirection

ParticleSystem:setDirection · revision 5833 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the direction the particles will be emitted in.

Function

Synopsis

```
ParticleSystem:setDirection( direction )
```

Arguments

`number direction`
: The direction of the particles (in radians).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setEmissionArea

ParticleSystem:setEmissionArea · revision 22657 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from ParticleSystem:setAreaSpread.

Sets area-based spawn parameters for the particles. Newly created particles will spawn in an area around the emitter based on the parameters to this function.

Function

Synopsis

```
ParticleSystem:setEmissionArea( distribution, dx, dy, angle, directionRelativeToCenter )
```

Arguments

`AreaSpreadDistribution distribution`
: The type of distribution for new particles.

`number dx`
: The maximum spawn distance from the emitter along the x-axis for uniform distribution, or the standard deviation along the x-axis for normal distribution.

`number dy`
: The maximum spawn distance from the emitter along the y-axis for uniform distribution, or the standard deviation along the y-axis for normal distribution.

`number angle (0)`
: The angle in radians of the emission area.

`boolean directionRelativeToCenter (false)`
: True if newly spawned particles will be oriented relative to the center of the emission area, false otherwise.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:getEmissionArea

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setEmissionRate

ParticleSystem:setEmissionRate · revision 5834 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the amount of particles emitted per second.

Function

Synopsis

```
ParticleSystem:setEmissionRate( rate )
```

Arguments

`number rate`
: The amount of particles per second.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setEmitterLifetime

ParticleSystem:setEmitterLifetime · revision 10891 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from ParticleSystem:setLifetime.

Sets how long the particle system should emit particles (if -1 then it emits particles forever).

Function

Synopsis

```
ParticleSystem:setEmitterLifetime( life )
```

Arguments

`number life`
: The lifetime of the emitter (in seconds).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setGravity

ParticleSystem:setGravity · revision 10896 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removed in LÖVE 0.9.0

It has been replaced by ParticleSystem:setLinearAcceleration.

Sets the gravity affecting the particles (acceleration along the y-axis). Every particle created will have a gravity between min and max.

Function

Synopsis

```
ParticleSystem:setGravity( min, max )
```

Arguments

`number min`
: The minimum gravity.

`number max (min)`
: The maximum gravity.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setImage

ParticleSystem:setImage · revision 15740 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from ParticleSystem:setSprite.

Removed in LÖVE 0.10.0

Use ParticleSystem:setTexture instead.

Sets the image to be used for the particles.

Function

Synopsis

```
ParticleSystem:setImage( image )
```

Arguments

`Image image`
: An Image to use for the particles.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setInsertMode

ParticleSystem:setInsertMode · revision 11059 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the mode to use when the ParticleSystem adds new particles.

Function

Synopsis

```
ParticleSystem:setInsertMode( mode )
```

Arguments

`ParticleInsertMode mode`

The mode to use when the ParticleSystem adds new particles.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:getInsertMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setLifetime

ParticleSystem:setLifetime · revision 10892 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to ParticleSystem:setEmitterLifetime.

Sets how long the particle system should emit particles (if -1 then it emits particles forever).

Function

Synopsis

```
ParticleSystem:setLifetime( life )
```

Arguments

`number life`
: The lifetime of the emitter (in seconds).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setLinearAcceleration

ParticleSystem:setLinearAcceleration · revision 11740 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has replaced ParticleSystem:setGravity.

Sets the linear acceleration (acceleration along the x and y axes) for particles.

Every particle created will accelerate along the x and y axes between xmin,ymin and xmax,ymax.

Function

Synopsis

```
ParticleSystem:setLinearAcceleration( xmin, ymin, xmax, ymax )
```

Arguments

`number xmin`
: The minimum acceleration along the x axis.

`number ymin`
: The minimum acceleration along the y axis.

`number xmax (xmin)`
: The maximum acceleration along the x axis.

`number ymax (ymin)`
: The maximum acceleration along the y axis.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setLinearDamping

ParticleSystem:setLinearDamping · revision 14279 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Sets the amount of linear damping (constant deceleration) for particles.

Function

Synopsis

```
ParticleSystem:setLinearDamping( min, max )
```

Arguments

`number min`
: The minimum amount of linear damping applied to particles.

`number max (min)`
: The maximum amount of linear damping applied to particles.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:getLinearDamping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setOffset

ParticleSystem:setOffset · revision 5837 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Set the offset position which the particle sprite is rotated around. If this function is not used, the particles rotate around their center.

Function

Synopsis

```
ParticleSystem:setOffset( x, y )
```

Arguments

number x
: The x coordinate of the rotation offset.

number y
: The y coordinate of the rotation offset.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setParticleLife

ParticleSystem:setParticleLife · revision 10894 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to ParticleSystem:setParticleLifetime.

Sets the life of the particles.

Function

Synopsis

```
ParticleSystem:setParticleLife( min, max )
```

Arguments

`number min`
: The minimum life of the particles (seconds).

`number max (min)`
: The maximum life of the particles (seconds).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setParticleLifetime

ParticleSystem:setParticleLifetime · revision 10893 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from ParticleSystem:setParticleLife.

Sets the lifetime of the particles.

Function

Synopsis

```
ParticleSystem:setParticleLifetime( min, max )
```

Arguments

`number min`
: The minimum life of the particles (in seconds).

`number max (min)`
: The maximum life of the particles (in seconds).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setPosition

ParticleSystem:setPosition · revision 5839 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the position of the emitter.

Function

Synopsis

```
ParticleSystem:setPosition( x, y )
```

Arguments

`number x`
: Position along x-axis.

`number y`
: Position along y-axis.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setQuads

ParticleSystem:setQuads · revision 27661 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Sets a series of Quads to use for the particle sprites. Particles will choose a Quad from the list based on the particle's current lifetime, allowing for the use of animated sprite sheets with ParticleSystems.

Function

Synopsis

```
ParticleSystem:setQuads( quad1, quad2, ... )
```

Arguments

`Quad quad1`
: The first Quad to use.

`Quad quad2`
: The second Quad to use.

Returns

Nothing.

Function

Synopsis

```
ParticleSystem:setQuads( quads )
```

Arguments

`table quads`
: A table containing the Quads to use.

Returns

Nothing.

Notes

Prior to 11.4, the particle system offset will be incorrect after this call. Call ParticleSystem:setOffset appropriately to workaround this issue.

See Also

- ParticleSystem

- ParticleSystem:getQuads

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setRadialAcceleration

ParticleSystem:setRadialAcceleration · revision 17793 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Set the radial acceleration (away from the emitter).

Function

Synopsis

```
ParticleSystem:setRadialAcceleration( min, max )
```

Arguments

number min
: The minimum acceleration.

number max (min)
: The maximum acceleration.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:getRadialAcceleration

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setRelativeRotation

ParticleSystem:setRelativeRotation · revision 13055 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Sets whether particle angles and rotations are relative to their velocities. If enabled, particles are aligned to the angle of their velocities and rotate relative to that angle.

Function

Synopsis

```
ParticleSystem:setRelativeRotation( enable )
```

Arguments

`boolean enable`

True to enable relative particle rotation, false to disable it.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:hasRelativeRotation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setRotation

ParticleSystem:setRotation · revision 26020 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sets the rotation of the image upon particle creation (in radians).

Function

Synopsis

```
ParticleSystem:setRotation( min, max )
```

Arguments

number min
: The minimum initial angle (radians).

number max (min)
: The maximum initial angle (radians).

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:getRotation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSize

ParticleSystem:setSize · revision 7964 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

It has been replaced with ParticleSystem:setSizes.

Sets the size of the particle (1.0 being normal size). The particles will grow/shrink from the starting size to the ending size. The variation affects starting size only.

Function

Synopsis

```
ParticleSystem:setSize( min, max, variation )
```

Arguments

`number min`
: The minimum size (1.0 being normal size).

`number max (min)`
: The maximum size (1.0 being normal size).

`number variation (1)`
: The degree of variation (0 meaning no variation and 1 meaning full variation between start and end).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSizes

ParticleSystem:setSizes · revision 12844 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has replaced ParticleSystem:setSize.

Sets a series of sizes by which to scale a particle sprite. 1.0 is normal size. The particle system will interpolate between each size evenly over the particle's lifetime.

At least one size must be specified. A maximum of eight may be used.

Function

Synopsis

```
ParticleSystem:setSizes( size1, size2, ..., size8 )
```

Arguments

`number size1`
: The first size.

`number size2`
: The second size.

`number size8`
: The eighth size.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:setSizeVariation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSizeVariation

ParticleSystem:setSizeVariation · revision 10614 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the amount of size variation (0 meaning no variation and 1 meaning full variation between start and end).

Function

Synopsis

```
ParticleSystem:setSizeVariation( variation )
```

Arguments

`number variation`
: The amount of variation (0 meaning no variation and 1 meaning full variation between start and end).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSpeed

ParticleSystem:setSpeed · revision 5844 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the speed of the particles.

Function

Synopsis

```
ParticleSystem:setSpeed( min, max )
```

Arguments

`number min`
: The minimum linear speed of the particles.

`number max (min)`
: The maximum linear speed of the particles.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSpin

ParticleSystem:setSpin · revision 13386 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Sets the spin of the sprite.

Function

Synopsis

```
ParticleSystem:setSpin( min, max )
```

Arguments

number min
: The minimum spin (radians per second).

number max (min)
: The maximum spin (radians per second).

Returns

Nothing.

Function

Removed in LÖVE 0.9.0
Use ParticleSystem:setSpinVariation instead.

Synopsis

```
ParticleSystem:setSpin( min, max, variation )
```

Arguments

number min
: The minimum spin (radians per second).

number max (min)
: The maximum spin (radians per second).

number variation (0)
: The degree of variation (0 meaning no variation and 1 meaning full variation between start and end).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSpinVariation

ParticleSystem:setSpinVariation · revision 10615 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the amount of spin variation (0 meaning no variation and 1 meaning full variation between start and end).

Function

Synopsis

```
ParticleSystem:setSpinVariation( variation )
```

Arguments

`number variation`
: The amount of variation (0 meaning no variation and 1 meaning full variation between start and end).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSpread

ParticleSystem:setSpread · revision 5847 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the amount of spread for the system.

Function

Synopsis

```
ParticleSystem:setSpread( spread )
```

Arguments

`number spread`
: The amount of spread (radians).

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setSprite

ParticleSystem:setSprite · revision 10752 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to ParticleSystem:setImage.

Sets the image which is to be emitted.

Function

Synopsis

```
ParticleSystem:setSprite( sprite )
```

Arguments

`Image sprite`
: An Image to use for the particle.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setTangentialAcceleration

ParticleSystem:setTangentialAcceleration · revision 5849 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the tangential acceleration (acceleration perpendicular to the particle's direction).

Function

Synopsis

```
ParticleSystem:setTangentialAcceleration( min, max )
```

Arguments

number min
: The minimum acceleration.

number max (min)
: The maximum acceleration.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:setTexture

ParticleSystem:setTexture · revision 12960 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

It has been renamed from ParticleSystem:setImage.

Sets the texture (Image or Canvas) to be used for the particles.

Function

Synopsis

```
ParticleSystem:setTexture( texture )
```

Arguments

`Texture texture`
: An Image or Canvas to use for the particles.

Returns

Nothing.

See Also

- ParticleSystem
- ParticleSystem:getTexture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:start

ParticleSystem:start · revision 5850 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Starts the particle emitter.

Function

Synopsis

```
ParticleSystem:start( )
```

Arguments

None.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:stop

ParticleSystem:stop · revision 5851 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Stops the particle emitter, resetting the lifetime counter.

Function

Synopsis

```
ParticleSystem:stop( )
```

Arguments

None.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ParticleSystem:update

ParticleSystem:update · revision 5852 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Updates the particle system; moving, creating and killing particles.

Function

Synopsis

```
ParticleSystem:update( dt )
```

Arguments

`number dt`
: The time (seconds) since last frame.

Returns

Nothing.

See Also

- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PATH

PATH · revision 29562 · last edited 2025-04-08

On this page

- Windows
  - Windows 11
  - Windows 10, Windows 8
  - Windows 7
  - Windows Vista
  - Windows XP
- Mac OS X
- Linux
- Other Languages

The PATH variable is a list of paths (directories) that get scanned for commands to run without having to enter their full path. This is useful for easier interaction with development executables like `love.exe` (simply `love` on OS X/Linux).

On this page you will learn how to add your LÖVE directory to PATH.

Windows

First, locate your `love.exe`. If it's in `C:\Program Files\LOVE\` then that's the path you need to add.

Windows 11

1. In the start menu, search for 'edit environment variables for your account' (make sure you do not select 'edit the system environment variables')
2. In the top part of the window (User variables for <your user name>), find the Path environment variable and select it. Click Edit. If the Path environment variable does not exist, click New.
3. In the new window (Edit environment variable), click 'New', enter the path, then close both windows you opened by clicking OK (closing without OK will not save your new changes to the path).

Windows 10, Windows 8

1. In Search, search for and then select: System (Control Panel)
2. Click the Advanced system settings link.
3. Click Environment Variables. In the section System Variables, find the PATH environment variable and select it. Click Edit. If the PATH environment variable does not exist, click New.
4. In the Edit System Variable (or New System Variable) window, specify the value of the PATH environment variable. Click OK. Close all remaining windows by clicking OK.

Windows 7

1. From the desktop, right click the Computer icon.
2. Choose Properties from the context menu.
3. Click the Advanced system settings link.
4. Click Environment Variables. In the section System Variables, find the PATH environment variable and select it. Click Edit. If the PATH environment variable does not exist, click New.
5. In the Edit System Variable (or New System Variable) window, specify the value of the PATH environment variable. Click OK. Close all remaining windows by clicking OK.

Windows Vista

1. From the desktop, right click the My Computer icon.
2. Choose Properties from the context menu.
3. Click the Advanced tab (Advanced system settings link in Vista).
4. Click Environment Variables. In the section System Variables, find the PATH environment variable and select it. Click Edit. If the PATH environment variable does not exist, click New.
5. In the Edit System Variable (or New System Variable) window, specify the value of the PATH environment variable. Click OK. Close all remaining windows by clicking OK.

Windows XP

1. Select Start, select Control Panel. double click System, and select the Advanced tab.

2. Click Environment Variables. In the section System Variables, find the PATH environment variable and select it. Click Edit. If the PATH environment variable does not exist, click New.
3. In the Edit System Variable (or New System Variable) window, specify the value of the PATH environment variable. Click OK. Close all remaining windows by clicking OK.

Mac OS X

First, locate your `love.app`. If it's in `/Applications/`, then the path you need to add is `/Applications/love.app/Contents/MacOS/`.

In the Terminal app, run:

```
export PATH=$PATH:/Applications/love.app/Contents/MacOS/
```

To make the change permanent, you will have to add that line to your `~/.profile` file.

Linux

If you installed LÖVE system-wide, `love` most likely is already in PATH. Doesn't work? Consider restarting.

Otherwise, locate your `love` binary and run

```
export PATH=$PATH:/path/to/folder/with/love/binary/
```

and add that your `~/.profile` file to make it permanent.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Physics

Physics · revision 25319 · last edited 2023-07-05

Physics Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
HC	A collision detection system		Any	
bump.lua	Minimal collision detection lib.		Any	
hxdx	Physics library (easier box2d)		Any	
tile-collider	Module for resolving tile collisions (+slopes)		Any	
breezefield	minimal love.physics wrapper		11.2	
Lope2D	A wrap over Love2D physics engine API. It makes dealing with the physics API easier and faster. It also has water pools simulation.		0.8.0+	
newton	Export and load Box2D bodies made in PhysicsEditor		0.8.0+	
LÖVE PhysicsEditor Loader	PhysicsEditor Objects Models Loader		0.10.2	0.9.x

PixelEffect

PixelEffect · revision 15931 · last edited 2026-04-11

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to Shader.

A PixelEffect is used for advanced hardware-accelerated pixel manipulation. These effects are written in a language based on GLSL (OpenGL Shading Language) with a few things simplified for easier coding.

Potential uses for pixel effects include HDR/bloom, motion blur, grayscale/invert/sepia/any kind of color effect, reflection/refraction, distortions, and much more!

Constructors

love.graphics.newPixelEffect	Creates a new PixelEffect.	Added since 0.8.0	Removed in 0.9.0

Functions

PixelEffect:getWarnings	Gets warning messages (if any).	Added since 0.8.0	Removed in 0.9.0
PixelEffect:send	Sends one or more values to the pixel effect.	Added since 0.8.0	Removed in 0.9.0

Supertypes

- Object

See Also

- love.graphics
- love.graphics.setPixelEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

PixelEffect:getWarnings

PixelEffect:getWarnings · revision 22293 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to Shader:getWarnings.

Returns any warning messages from compiling the pixel effect code. This can be used for debugging your pixel effects if there's anything the graphics hardware doesn't like.

Function

Synopsis

```
warnings = PixelEffect:getWarnings( )
```

Arguments

None.

Returns

`string warnings`

Warning messages (if any).

See Also

- PixelEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PixelEffect:send

PixelEffect:send · revision 22294 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to Shader:send.

Sends one or more values to a special (*extern*) variable inside the pixel effect. Extern variables have to be marked using the *extern* keyword, e.g.

```
extern number time;
extern vec2 light_pos;
extern vec4 colors[4];
```

The corresponding send calls would be

```
effect:send("time", t)
effect:send("light_pos", {light_x, light_y})
effect:send("colors", {r1, g1, b1, a1},  {r2, g2, b2, a2},  {r3, g3, b3, a3},  {r4, g4, b4, a4})
```

Function

Synopsis

```
PixelEffect:send( name, number, ... )
```

Arguments

`string name`
: Name of the number to send to the pixel effect.

`number number`
: Number to send to store in the extern.

`number ...`
: Additional numbers to send in case the extern is an array.

Returns

Nothing.

Function

Synopsis

```
PixelEffect:send( name, vector, ... )
```

Arguments

`string name`
: Name of the vector to send to the pixel effect.

`table vector`

Numbers to send to the extern as a vector. The number of elements in the table determines the type of the vector (e.g. two numbers -> vec2). At least two and at most four numbers can be used.

table ...
Additional vectors to send in case the extern is an array. All vectors need to be of the same size (e.g. only vec3's)

Returns

Nothing.

Function

Synopsis

```
PixelEffect:send( name, matrix, ... )
```

Arguments

string name
Name of the matrix to send to the pixel effect.

table matrix
2x2, 3x3, or 4x4 matrix to send to the extern. Using table form: `{{a,b,c,d}, {e,f,g,h}, ... }`

table ...
Additional matrices of the same type as *matrix* to store in the extern array.

Returns

Nothing.

Function

Synopsis

```
PixelEffect:send( name, image, ... )
```

Arguments

string name
Name of the Image to send to the pixel effect.

Image image
Image to send to the extern.

Image ...
Additional images in case the extern is an array.

Returns

Nothing.

Function

Synopsis

```
PixelEffect:send( name, canvas, ... )
```

Arguments

string name
Name of the Canvas to send to the pixel effect.

Canvas canvas
Canvas to send to the extern. The pixel effect type is *Image*.

`Image ...`
: Additional canvases to send to the extern array.

Returns

Nothing.

See Also

- PixelEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PixelFormat

PixelFormat · revision 27354 · last edited 2026-04-10

On this page

- Normal color formats
  - Constants
- Depth / stencil formats
  - Constants
- Compressed formats
  - Constants (desktop GPUs)
  - Constants (mobile GPUs)
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This enum replaces CanvasFormat and CompressedImageFormat.

Pixel formats for Textures, ImageData, and CompressedImageData.

Normal color formats

Constants

Name	Components	Bits per pixel	Range	Usable with Canvases	Usable with ImageData	Note(s)
normal	4	32	[0, 1]	Yes		Alias for `rgba8`, or `srgba8` if gamma-correct rendering is enabled.
r8	1	8	[0, 1]	Yes	11.3	
rg8	2	16	[0, 1]	Yes	11.3	
rgba8	4	32	[0, 1]	Yes	Yes	
srgba8	4	32	[0, 1]	Yes		gamma-correct version of rgba8.
r16	1	16	[0, 1]	11.4	11.3	
rg16	2	32	[0, 1]	11.4	11.3	
rgba16	4	64	[0, 1]	11.4	Yes	
r16f	1	16	[-65504, +65504]*	Yes	11.3	
rg16f	2	32	[-65504, +65504]*	Yes	11.3	
rgba16f	4	64	[-65504, +65504]*	Yes	Yes	
r32f	1	32	[-3.4028235e38, 3.4028235e38]*	Yes	11.3	
rg32f	2	64	[-3.4028235e38, 3.4028235e38]*	Yes	11.3	
rgba32f	4	128	[-3.4028235e38, 3.4028235e38]*	Yes	Yes	
la8	2	16	[0, 1]			Used by GlyphData. Same as `rg8`, but accessed as (L, L, L, A)
rgba4	4	16	[0, 1]	Yes	11.3	
rgb5a1	4	16	[0, 1]	Yes	11.3	
rgb565	3	16	[0, 1]	Yes	11.3	
rgb10a2	4	32	[0, 1]	Yes	11.3	
rg11b10f	3	32	[0, 65024]**	Yes	11.3	

* -infinity and +infinity are also valid values.
** +infinity is also a valid value.

Depth / stencil formats

All depth and stencil pixel formats are only usable in Canvases.

They are non-readable by default, and Canvases with a depth/stencil format created with the readable flag can only access the depth values of their pixels in shaders (stencil values are not readable no matter what).

Constants

Name	Bits per pixel	Has depth	Has stencil	Note(s)
stencil8	8		Yes	
depth16	16	Yes		
depth24	24	Yes		
depth32f	32	Yes		
depth24stencil8	32	Yes	Yes	
depth32fstencil8	40	Yes	Yes	

Compressed formats

All compressed pixel formats are only usable in Images via CompressedImageData (compressed textures).

Unlike regular color formats, these stay compressed in RAM and VRAM. This is good for saving memory space as well as improving performance, since the graphics card will be able to keep more of the image's pixels in its fast-access cache when drawing it.

Constants (desktop GPUs)

Name	Components	Bits per pixel	Range	Note(s)
DXT1	3	4	[0, 1]	Suitable for fully opaque images on desktop systems.
DXT3	4	8	[0, 1]	Smooth variations in opacity do not mix well with this format. DXT1 or DXT5 is generally better in every situation.
DXT5	4	8	[0, 1]	Recommended for images with varying opacity on desktop systems.
BC4	1	4	[0, 1]	Also known as 3Dc+ or ATI1. Stores just the red channel.
BC4s	1	4	[-1, 1]	Less precision than BC4 but allows negative numbers.
BC5	2	8	[0, 1]	Also known as 3Dc or ATI2. Often used for normal maps on desktop systems.
BC5s	2	8	[-1, 1]	Less precision than BC5 but allows negative numbers. Often used for normal maps on desktop systems.
BC6h	3	8	[0, +infinity]	Stores half-precision (https://en.wikipedia.org/wiki/Half-precision_floating-point_format) floating point RGB data. Suitable for HDR images on desktop systems.
BC6hs	3	8	[-infinity, +infinity]	Less precision than BC6h but allows negative numbers.
BC7	4	8	[0, 1]	Very good at representing opaque or transparent images, but requires a DX11 / OpenGL 4-capable GPU.

Constants (mobile GPUs)

Name	Components	Bits per pixel	Range	Note(s)

ETC1	3	4	[0, 1]	Suitable for fully opaque images on older Android devices.
ETC2rgb	3	4	[0, 1]	Suitable for fully opaque images on newer mobile devices
ETC2rgba	4	8	[0, 1]	Recommended for images with varying opacity on newer mobile devices.
ETC2rgba1	4	4	[0, 1]	RGBA variant of the ETC2 format where pixels are either fully transparent or fully opaque.
EACr	1	4	[0, 1]	Stores just the red channel.
EACrs	1	4	[-1, 1]	Less precision than EACr but allows negative numbers.
EACrg	2	8	[0, 1]	Stores red and green channels.
EACrgs	2	8	[-1, 1]	Less precision than EACrg but allows negative numbers.
PVR1rgb2	3	2	[0, 1]	Images using this format must be square and power-of-two sized.
PVR1rgb4	3	4	[0, 1]	Images using this format must be square and power-of-two sized.
PVR1rgba2	4	2	[0, 1]	Images using this format must be square and power-of-two sized.
PVR1rgba4	4	4	[0, 1]	Images using this format must be square and power-of-two sized.
ASTC4x4	4	8	[0, 1]	
ASTC5x4	4	6.4	[0, 1]	
ASTC5x5	4	5.12	[0, 1]	
ASTC6x5	4	4.27	[0, 1]	
ASTC6x6	4	3.56	[0, 1]	
ASTC8x5	4	3.2	[0, 1]	
ASTC8x6	4	2.67	[0, 1]	
ASTC8x8	4	2	[0, 1]	
ASTC10x5	4	2.56	[0, 1]	
ASTC10x6	4	2.13	[0, 1]	
ASTC10x8	4	1.6	[0, 1]	
ASTC10x10	4	1.28	[0, 1]	
ASTC12x10	4	1.07	[0, 1]	
ASTC12x12	4	0.89	[0, 1]	

Notes

Not all formats are supported in love.graphics Images and Canvases on all systems, although the DXT compressed image formats have close to 100% support on desktop operating systems.

The BC4 and BC5 formats are supported on systems with DirectX 10 / OpenGL 3-capable desktop hardware and drivers. The BC6H and BC7 formats are only supported on desktop systems with DirectX 11 / OpenGL 4-capable hardware and very recent drivers. macOS does not support BC6H or BC7 at all currently.

ETC1 is supported by *most* Android devices, as well as the iPhone 5s and newer.

The ETC2 and EAC formats are supported by the iPhone 5s and newer, all OpenGL ES 3-capable Android devices, and OpenGL 4.3-capable desktop GPUs.

The PVR1 formats are supported by all iOS devices, as well as the small number of Android devices which have PowerVR GPUs.

ASTC is supported by new mobile devices (e.g. the iPhone 6 and newer), Android devices which have Adreno 4xx (and later) GPUs, and Skylake (and newer) integrated Intel GPUs. It has a variety of variants to allow for picking the most compressed possible one that doesn't have any noticeable compression artifacts, for a given texture.

Use love.graphics.getCanvasFormats and love.graphics.getImageFormats to check for Canvas and Image support, respectively:

```lua
local supportedformats = love.graphics.getImageFormats()

if not supportedformats["DXT5"] then
    -- Can't load CompressedImageData with the DXT5 format into images!
    -- On some Linux systems with Mesa drivers, the user will need to install a "libtxc-dxtn" package because the DXT (aka S3TC) formats used to be patented.
    -- Support for DXT formats on all other desktop drivers is pretty much guaranteed.
end

if not supportedformats["BC5"] then
    -- Can't load CompressedImageData with the BC5 format into images!
    -- The user likely doesn't have a video card capable of using that format.
end
```

See Also

- love.image
- love.graphics
- love.font
- love.image.newImageData
- love.graphics.getImageFormats
- love.graphics.getCanvasFormats
- love.graphics.newCanvas
- Texture:getFormat
- ImageData:getFormat
- CompressedImageData:getFormat
- GlyphData:getFormat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Pnorar

Pnorar · revision 22551 · last edited 2021-03-14

Forum thread: https://love2d.org/forums/viewtopic.php?f=5&t=76956

Official website: https://rblau.itch.io/pnorar ↗

PointInConcavePolygon

PointInConcavePolygon · revision 19069 · last edited 2021-03-14

A function returning True if the given point is inside the given polygon.

```
function pointInPolygon(pgon, tx, ty)
    if (#pgon < 6) then
        return false
    end

    local x1 = pgon[#pgon - 1]
    local y1 = pgon[#pgon]
    local cur_quad = getQuad(tx,ty,x1,y1)
    local next_quad
    local total = 0
    local i

    for i = 1,#pgon,2 do
        local x2 = pgon[i]
        local y2 = pgon[i+1]
        next_quad = getQuad(tx,ty,x2,y2)
        local diff = next_quad - cur_quad

        if (diff == 2) or (diff == -2) then
            if (x2 - (((y2 - ty) * (x1 - x2)) / (y1 - y2))) < tx then
                diff = -diff
            end
        elseif diff == 3 then
            diff = -1
        elseif diff == -3 then
            diff = 1
        end

        total = total + diff
        cur_quad = next_quad
        x1 = x2
        y1 = y2
    end

    return (math.abs(total)==4)
end

function getQuad(axis_x,axis_y,vert_x,vert_y)
    if vert_x < axis_x then
        if vert_y < axis_y then
            return 1
        else
            return 4
        end
    else
        if vert_y < axis_y then
            return 2
        else
            return 3
        end
    end
end
```

Sample:

```
function love.load()
    poly = {69,119,371,39,724,140,751,491,572,222,420,163,283,232,236,338,399,401,540,446,371,508,195,503,96,327}
    color = {255,255,255}
end

function love.update(dt)
    if pointInPolygon(poly, love.mouse.getX(), love.mouse.getY()) then
        color = {0,255,0}
    else
        color = {255,255,255}
    end
```

```
end

function love.draw()
    if #poly >= 6 then
        love.graphics.setColor(color)
        love.graphics.polygon("line",poly)
    end
end

-- Add functions pointInPolygon and getQuad here
```

PointInConvexPolygon

PointInConvexPolygon · revision 29103 · last edited 2024-02-01

A function returning True if the given point is inside the given convex polygon.

First solution

```lua
local function pointInConvexPolygon(x, y, poly)
    -- poly as {x1,y1, x2,y2, x3,y3, ...}
    local imax = #poly

    local function isVerticesClockwise(poly)
        local sum = 0
        local imax = #poly
        local x1, y1 = poly[imax-1], poly[imax]
        for i = 1, imax - 1, 2 do
            local x2, y2 = poly[i], poly[i + 1]
            sum = sum + (x2 - x1) * (y2 + y1)
            x1, y1 = x2, y2
        end
        local isClockwise = sum < 0
        love.window.setTitle(isClockwise and 'clockwise' or 'counterclockwise')
        return isClockwise
    end

    local sign = isVerticesClockwise(poly) and 1 or -1
    local x1, y1 = poly[imax-1], poly[imax]
    for i = 1, imax - 1, 2 do
        local x2, y2 = poly[i], poly[i + 1]
        local dotProduct = (x - x1) * (y1 - y2) + (y - y1) * (x2 - x1)
        if sign * dotProduct < 0 then
            return false
        end
        x1, y1 = x2, y2
    end
    return true
end
```

Second solution

```lua
 -- from https://stackoverflow.com/questions/77904973/check-if-point-is-in-convex-polygon
local function pointInConvexPolygon (x, y, poly)
-- poly as {x1,y1, x2,y2, x3,y3, ...}
    local function polyEdgeIterator(poly)
        local i, imax = 0, #poly
        local x2, y2 = poly[imax-1], poly[imax]
        return function()
            i = i + 2
            if i <= imax then
                local x1, y1 = x2, y2
                x2, y2 = poly[i-1], poly[i]
                return x1, y1, x2, y2
            end
        end
    end

    local minArea, maxArea = 0, 0
    for x1,y1, x2,y2 in polyEdgeIterator(poly) do
        local area = (x1-x) * (y2-y) - (x2-x) * (y1-y)
        minArea = math.min(minArea, area)
        maxArea = math.max(maxArea, area)
        if minArea * maxArea < 0 then
            return false
        end
```

```
    end
    return true
end
```

Example

```
function love.load()
    poly = {100, 100, 400, 150, 200, 300}
    local x, y = 400, 300
    inside = pointInConvexPolygon (x, y, poly)
end

local function drawArrow(x1, y1, x2, y2)
    local angle = math.atan2(y2 - y1, x2 - x1)
    local arrowSize = 20
    local x3 = x2 - arrowSize * math.cos(angle - math.pi / 16)
    local y3 = y2 - arrowSize * math.sin(angle - math.pi / 16)
    local x4 = x2 - arrowSize * math.cos(angle + math.pi / 16)
    local y4 = y2 - arrowSize * math.sin(angle + math.pi / 16)
    love.graphics.line(x3, y3, x2, y2, x4, y4)
end

function love.draw()
    love.graphics.setLineStyle ('rough')
    love.graphics.setLineWidth (1)
    love.graphics.setLineJoin ('none')
    if inside then
        love.graphics.setColor (1,1,0)
    else
        love.graphics.setColor (0,1,1)
    end
    if #poly > 4 then
        love.graphics.polygon ('line', poly)
        local imax = #poly
        local x1, y1 = poly[imax-1], poly[imax]
        for i = 1, imax - 1, 2 do
            local x2, y2 = poly[i], poly[i + 1]
            drawArrow(x1, y1, x2, y2)
            x1, y1 = x2, y2
        end
    elseif #poly > 2 then
        love.graphics.line (poly)
        drawArrow(poly[1], poly[2], poly[3], poly[4])
    end
    love.graphics.setColor (1,1,1)
    love.graphics.print ('Left mouse botton to add new point', 14, 14)
    love.graphics.print ('Right mouse botton to remove last point', 14, 2*14)
    love.graphics.print ('Is inside: '..tostring(inside), 14, 4*14)
end


function love.mousepressed( x, y, button, istouch, presses )
    if button == 1 then -- Left mouse button
        table.insert (poly, x)
        table.insert (poly, y)
    elseif button == 2 then -- right mouse button
        table.remove (poly)
        table.remove (poly)
    end
    inside = pointInConvexPolygon (x, y, poly)
end

function love.mousemoved( x, y, dx, dy, istouch )
    inside = pointInConvexPolygon (x, y, poly)
end
```

PointStyle

PointStyle · revision 15979 · last edited 2023-07-05

Removed in LÖVE 0.10.0
This enum is not supported in that and later versions.

How points should be drawn.

Constants

rough
: Draw rough points.

smooth
: Draw smooth points.

See Also

- love.graphics
- love.graphics.getPointStyle
- love.graphics.setPointStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PointWithinShape

PointWithinShape · revision 19070 · last edited 2021-09-14

A set of functions for testing if a point lies within an area.

PointWithinShape(shape, x, y)

returns true if (x,y) lays within the bounds of the given shape. Shape defined by an array of 0 to n {x=*x*, y=*y*} values.
It generally defers to the other other functions in the file to do the heavy lifting (*CrossingsMultipyTest(...)*, *PointWithinLine(...)*, *BoundingBox(...)* and *colinear(...)*)

BoundingBox(box, x, y)

returns true if the given point is inside the box defined by two points (in the same format taken by *PointWithinShape(...)*).

colinear(line, x, y, e)

returns true if the given point lies on the infinite line defined by two points (again, in the same format taken by *PointWithinShape(...)*)
e is optional, if given it controls how close (x,y) must be to the line to be considered colinear. it defaults to 0.1

PointWithinLine(line, x, y, e)

returns true if the given point lies on the finite line defined by two points (again, in the same format taken by *PointWithinShape(...)*)
e is optional, if given it controls how close (x,y) must be to the line to be considered within the line. it defaults to 0.66

CrossingsMultiplyTest(polygon, x, y)

returns true if the given point lies within the area of the polygon defined by three or points (again, in the same format taken by *PointWithinShape(...)*)
(it is based on code from Point in Polygon Strategies (http://erich.realtimerendering.com/ptinpoly/))

GetIntersect(points)

returns the x,y coordinates of the intersect, same format taken by *PointWithinShape(...)*
(Added by Bambo - You will need the checkIntersect from the "General math" snippet for it to work)

```
function PointWithinShape(shape, tx, ty)
    if #shape == 0 then
        return false
    elseif #shape == 1 then
        return shape[1].x == tx and shape[1].y == ty
    elseif #shape == 2 then
        return PointWithinLine(shape, tx, ty)
    else
        return CrossingsMultiplyTest(shape, tx, ty)
    end
end

function BoundingBox(box, tx, ty)
    return  (box[2].x >= tx and box[2].y >= ty)
        and (box[1].x <= tx and box[1].y <= ty)
        or  (box[1].x >= tx and box[2].y >= ty)
        and (box[2].x <= tx and box[1].y <= ty)
end

function colinear(line, x, y, e)
    e = e or 0.1
    m = (line[2].y - line[1].y) / (line[2].x - line[1].x)
    local function f(x) return line[1].y + m*(x - line[1].x) end
    return math.abs(y - f(x)) <= e
end

function PointWithinLine(line, tx, ty, e)
    e = e or 0.66
    if BoundingBox(line, tx, ty) then
        return colinear(line, tx, ty, e)
    else
        return false
    end
end

--------------------------------------------------------------------------
```

```lua
-- The following function is based off code from
-- [ http://erich.realtimerendering.com/ptinpoly/ ]
--
--[[
 ======= Crossings Multiply algorithm ===================================
 * This version is usually somewhat faster than the original published in
 * Graphics Gems IV; by turning the division for testing the X axis crossing
 * into a tricky multiplication test this part of the test became faster,
 * which had the additional effect of making the test for "both to left or
 * both to right" a bit slower for triangles than simply computing the
 * intersection each time.  The main increase is in triangle testing speed,
 * which was about 15% faster; all other polygon complexities were pretty much
 * the same as before.  On machines where division is very expensive (not the
 * case on the HP 9000 series on which I tested) this test should be much
 * faster overall than the old code.  Your mileage may (in fact, will) vary,
 * depending on the machine and the test data, but in general I believe this
 * code is both shorter and faster.  This test was inspired by unpublished
 * Graphics Gems submitted by Joseph Samosky and Mark Haigh-Hutchinson.
 * Related work by Samosky is in:
 *
 * Samosky, Joseph, "SectionView: A system for interactively specifying and
 * visualizing sections through three-dimensional medical image data",
 * M.S. Thesis, Department of Electrical Engineering and Computer Science,
 * Massachusetts Institute of Technology, 1993.
 *
 --]]

--[[ Shoot a test ray along +X axis.  The strategy is to compare vertex Y values
 * to the testing point's Y and quickly discard edges which are entirely to one
 * side of the test ray.  Note that CONVEX and WINDING code can be added as
 * for the CrossingsTest() code; it is left out here for clarity.
 *
 * Input 2D polygon _pgon_ with _numverts_ number of vertices and test point
 * _point_, returns 1 if inside, 0 if outside.
 --]]
function CrossingsMultiplyTest(pgon, tx, ty)
    local i, yflag0, yflag1, inside_flag
    local vtx0, vtx1

    local numverts = #pgon

    vtx0 = pgon[numverts]
    vtx1 = pgon[1]

    -- get test bit for above/below X axis
    yflag0 = ( vtx0.y >= ty )
    inside_flag = false

    for i=2,numverts+1 do
        yflag1 = ( vtx1.y >= ty )

        --[[ Check if endpoints straddle (are on opposite sides) of X axis
         * (i.e. the Y's differ); if so, +X ray could intersect this edge.
         * The old test also checked whether the endpoints are both to the
         * right or to the left of the test point.  However, given the faster
         * intersection point computation used below, this test was found to
         * be a break-even proposition for most polygons and a loser for
         * triangles (where 50% or more of the edges which survive this test
         * will cross quadrants and so have to have the X intersection computed
         * anyway).  I credit Joseph Samosky with inspiring me to try dropping
         * the "both left or both right" part of my code.
         --]]
        if ( yflag0 ~= yflag1 ) then
            --[[ Check intersection of pgon segment with +X ray.
             * Note if >= point's X; if so, the ray hits it.
             * The division operation is avoided for the ">=" test by checking
             * the sign of the first vertex wrto the test point; idea inspired
             * by Joseph Samosky's and Mark Haigh-Hutchinson's different
             * polygon inclusion tests.
             --]]
            if ( ((vtx1.y - ty) * (vtx0.x - vtx1.x) >= (vtx1.x - tx) * (vtx0.y - vtx1.y)) == yflag1 ) then
                inside_flag =  not inside_flag
            end
```

```
        end

        -- Move to the next pair of vertices, retaining info as possible.
        yflag0  = yflag1
        vtx0    = vtx1
        vtx1    = pgon[i]
    end

    return  inside_flag
end

function GetIntersect( points )
    local g1 = points[1].x
    local h1 = points[1].y

    local g2 = points[2].x
    local h2 = points[2].y

    local i1 = points[3].x
    local j1 = points[3].y

    local i2 = points[4].x
    local j2 = points[4].y

    local xk = 0
    local yk = 0

    if checkIntersect({x=g1, y=h1}, {x=g2, y=h2}, {x=i1, y=j1}, {x=i2, y=j2}) then
        local a = h2-h1
        local b = (g2-g1)
        local v = ((h2-h1)*g1) - ((g2-g1)*h1)

        local d = i2-i1
        local c = (j2-j1)
        local w = ((j2-j1)*i1) - ((i2-i1)*j1)

        xk = (1/((a*d)-(b*c))) * ((d*v)-(b*w))
        yk = (-1/((a*d)-(b*c))) * ((a*w)-(c*v))
    end
    return xk, yk
end
```

Polybius

Polybius · revision 7851 · last edited 2021-03-14





http://love2d.org/forums/viewtopic.php?f=5&t=2641

PolygonShape

PolygonShape · revision 27409 · last edited 2023-07-05

A PolygonShape is a convex polygon with up to 8 vertices.

Constructors

love.physics.newPolygonShape	Creates a new PolygonShape.
love.physics.newRectangleShape	Shorthand for creating rectangular PolygonShapes.

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
PolygonShape:getPoints	Get the local coordinates of the polygon's vertices.		
PolygonShape:validate	Validates whether the PolygonShape is convex.	Added since 0.9.0	
Shape:computeAABB	Returns the points of the bounding box for the transformed shape.	Added since 0.8.0	
Shape:computeMass	Computes the mass properties for the shape.	Added since 0.8.0	
Shape:destroy	Explicitly destroys the Shape.		Removed in 0.8.0
Shape:getBody	Get the body the shape is attached to.	Added since 0.7.0	Removed in 0.8.0
Shape:getBoundingBox	Gets the bounding box of the shape.		Removed in 0.8.0
Shape:getCategory	Gets the categories this shape is a member of.		Removed in 0.8.0
Shape:getCategoryBits	Gets the categories as a 16-bit integer.		Removed in 0.8.0
Shape:getChildCount	Returns the number of children the shape has.	Added since 0.8.0	
Shape:getData	Get the data set with setData.		Removed in 0.8.0
Shape:getDensity	Gets the density of the Shape.		Removed in 0.8.0
Shape:getFilterData	Gets the filter data of the Shape.		Removed in 0.8.0
Shape:getFriction	Gets the friction of this shape.		Removed in 0.8.0
Shape:getMask	Gets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:getRadius	Gets the radius of the shape.		
Shape:getRestitution	Gets the restitution of this shape.		Removed in 0.8.0
Shape:getType	Gets a string representing the Shape.		
Shape:isSensor	Checks whether a Shape is a sensor or not.		Removed in 0.8.0
Shape:rayCast	Casts a ray against the shape.	Added since 0.8.0	
Shape:setCategory	Sets the categories this shape is a member of.		Removed in 0.8.0
Shape:setData	Set data to be passed to the collision callback.		Removed in 0.8.0
Shape:setDensity	Sets the density of a Shape.		Removed in 0.8.0
Shape:setFilterData	Sets the filter data for a Shape.		Removed in 0.8.0
Shape:setFriction	Sets the friction of the shape.		Removed in 0.8.0
Shape:setMask	Sets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:setRestitution	Sets the restitution of the shape.		Removed in 0.8.0
Shape:setSensor	Sets whether this shape should act as a sensor.		Removed in 0.8.0
Shape:testPoint	Checks whether a point lies inside the shape.		
Shape:testSegment	Checks whether a line segment intersects a shape.		Removed in 0.8.0

Supertypes

- Shape
- Object

See Also

- love.physics
- love.graphics.polygon

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PolygonShape:getPoints

PolygonShape:getPoints · revision 10493 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the local coordinates of the polygon's vertices.

This function has a variable number of return values. It can be used in a nested fashion with love.graphics.polygon.

Function

Synopsis

```
x1, y1, x2, y2, ... x8, y8 = PolygonShape:getPoints( )
```

Arguments

None.

Returns

number x1
: The x-component of the first vertex.

number y1
: The y-component of the first vertex.

number x2
: The x-component of the second vertex.

number y2
: The y-component of the second vertex.

And so on, for up to eight vertices.

This function may have up to 16 return values, since it returns two values for each vertex in the polygon. In other words, it can return the coordinates of up to 8 points.

See Also

- PolygonShape
- Body:getWorldPoints

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PolygonShape:validate

PolygonShape:validate · revision 13360 · last edited 2025-12-23

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Validates whether the PolygonShape is convex. Concave PolygonShapes cannot be used in love.physics.

Function

Synopsis

```
convex = PolygonShape:validate()
```

Arguments

None.

Returns

`boolean convex`

Whether the PolygonShape is convex.

See Also

- PolygonShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Popo

Popo · revision 24448 · last edited 2023-07-05

A character based programmable text module for LÖVE. Simplifies text operations by providing a way for manipulating how each character in a string behaves and is drawn.

Download

github (https://github.com/adonaac/popo)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PowerState

PowerState · revision 11229 · last edited 2026-04-24

Available since LÖVE 0.9.0

This enum is not supported in earlier versions.

The basic state of the system's power supply.

Constants

unknown
: Cannot determine power status.

battery
: Not plugged in, running on a battery.

nobattery
: Plugged in, no battery available.

charging
: Plugged in, charging battery.

charged
: Plugged in, battery is fully charged.

See Also

- love.system
- love.system.getPowerInfo

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint

PrismaticJoint · revision 16457 · last edited 2023-07-05

Restricts relative motion between Bodies to one shared axis.



Constructors

love.physics.newPrismaticJoint	Creates a PrismaticJoint between two bodies.

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
PrismaticJoint:areLimitsEnabled	Checks whether the limits are enabled.	Added since 11.0	
PrismaticJoint:enableLimit	Enables or disables the limits of the joint.	Added since 0.8.0	Removed in 0.9.0
PrismaticJoint:enableMotor	Starts or stops the joint motor.	Added since 0.8.0	Removed in 0.9.0
PrismaticJoint:getAxis	Gets the world-space axis vector of the Prismatic Joint.	Added since 0.10.2	
PrismaticJoint:getJointSpeed	Get the current joint angle speed.		
PrismaticJoint:getJointTranslation	Get the current joint translation.		
PrismaticJoint:getLimits	Gets the joint limits.		
PrismaticJoint:getLowerLimit	Gets the lower limit.		
PrismaticJoint:getMaxMotorForce	Gets the maximum motor force.		
PrismaticJoint:getMotorForce	Returns the current motor force.		
PrismaticJoint:getMotorSpeed	Gets the motor speed.		
PrismaticJoint:getReferenceAngle	Gets the reference angle.	Added since 0.10.2	
PrismaticJoint:getUpperLimit	Gets the upper limit.		
PrismaticJoint:hasLimitsEnabled	Checks whether the limits are enabled.	Added since 0.9.0	
PrismaticJoint:isLimitEnabled	Checks whether the limits are enabled.	Added since 0.8.0	Removed in 0.9.0
PrismaticJoint:isLimitsEnabled	Checks whether limits are enabled.		Removed in 0.8.0
PrismaticJoint:isMotorEnabled	Checks whether the motor is enabled.		
PrismaticJoint:setLimits	Sets the limits.		
PrismaticJoint:setLimitsEnabled	Enables/disables the joint limit.	Added since 0.9.0	Removed in 0.8.0
PrismaticJoint:setLowerLimit	Sets the lower limit.		
PrismaticJoint:setMaxMotorForce	Set the maximum motor force.		
PrismaticJoint:setMotorEnabled	Enables/disables the joint motor.	Added since 0.9.0	Removed in 0.8.0
PrismaticJoint:setMotorSpeed	Sets the motor speed.		
PrismaticJoint:setUpperLimit	Sets the upper limit.		

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:areLimitsEnabled

PrismaticJoint:areLimitsEnabled · revision 22635 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from PrismaticJoint:hasLimitsEnabled.

Checks whether the limits are enabled.

Function

Synopsis

```
enabled = PrismaticJoint:areLimitsEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:enableLimit

PrismaticJoint:enableLimit · revision 13723 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to PrismaticJoint:setLimitsEnabled.

Enables or disables the limits of the joint.

Function

Synopsis

```
PrismaticJoint:enableLimit( enable )
```

Arguments

`boolean enable`
: True to enable, false to disable.

Returns

Nothing.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:enableMotor

PrismaticJoint:enableMotor · revision 13725 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to PrismaticJoint:setMotorEnabled.

Starts or stops the joint motor.

Function

Synopsis

```
PrismaticJoint:enableMotor( enable )
```

Arguments

`boolean enable`
: True to enable, false to disable.

Returns

Nothing.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getAxis

PrismaticJoint:getAxis · revision 18117 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the world-space axis vector of the Prismatic Joint.

Function

Synopsis

```
x, y = PrismaticJoint:getAxis( )
```

Arguments

None.

Returns

number **x**
: The x-axis coordinate of the world-space axis vector.

number **y**
: The y-axis coordinate of the world-space axis vector.

See Also

- PrismaticJoint
- love.physics.newPrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getJointSpeed

PrismaticJoint:getJointSpeed · revision 6085 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the current joint angle speed.

Function

Synopsis

```
s = PrismaticJoint:getJointSpeed( )
```

Arguments

None.

Returns

`number s`
: Joint angle speed in meters/second.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getJointTranslation

PrismaticJoint:getJointTranslation · revision 6086 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Get the current joint translation.

Function

Synopsis

```
t = PrismaticJoint:getJointTranslation( )
```

Arguments

None.

Returns

`number t`
Joint translation, usually in meters..

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getLimits

PrismaticJoint:getLimits · revision 6087 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Gets the joint limits.

Function

Synopsis

```
lower, upper = PrismaticJoint:getLimits( )
```

Arguments

None.

Returns

number lower
: The lower limit, usually in meters.

number upper
: The upper limit, usually in meters.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getLowerLimit

PrismaticJoint:getLowerLimit · revision 6088 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the lower limit.

Function

Synopsis

```
lower = PrismaticJoint:getLowerLimit( )
```

Arguments

None.

Returns

`number lower`
: The lower limit, usually in meters.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getMaxMotorForce

PrismaticJoint:getMaxMotorForce · revision 6089 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the maximum motor force.

Function

Synopsis

```
f = PrismaticJoint:getMaxMotorForce( )
```

Arguments

None.

Returns

number f
: The maximum motor force, usually in N.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getMotorForce

PrismaticJoint:getMotorForce · revision 8532 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Returns the current motor force.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
force = PrismaticJoint:getMotorForce( invdt )
```

Arguments

`number invdt`

How long the force applies. Usually the inverse time step or 1/dt.

Returns

`number force`

The force on the motor in newtons.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
f = PrismaticJoint:getMotorForce( )
```

Arguments

None.

Returns

`number f`

The current motor force, usually in N.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

PrismaticJoint:getMotorSpeed

PrismaticJoint:getMotorSpeed · revision 6091 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the motor speed.

Function

Synopsis

```
s = PrismaticJoint:getMotorSpeed( )
```

Arguments

None.

Returns

number s
: The motor speed, usually in meters per second.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getReferenceAngle

PrismaticJoint:getReferenceAngle · revision 26824 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the reference angle.

Function

Synopsis

```
angle = PrismaticJoint:getReferenceAngle( )
```

Arguments

None.

Returns

`number angle`
: The reference angle in radians.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:getUpperLimit

PrismaticJoint:getUpperLimit · revision 6092 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the upper limit.

Function

Synopsis

```
upper = PrismaticJoint:getUpperLimit( )
```

Arguments

None.

Returns

`number upper`
: The upper limit, usually in meters.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:hasLimitsEnabled

PrismaticJoint:hasLimitsEnabled · revision 24170 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function has been renamed to PrismaticJoint:areLimitsEnabled.

Checks whether the limits are enabled.

Function

Synopsis

```
enabled = PrismaticJoint:hasLimitsEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:isLimitEnabled

PrismaticJoint:isLimitEnabled · revision 13729 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to PrismaticJoint:hasLimitsEnabled.

Checks whether the limits are enabled.

Function

Synopsis

```
enabled = PrismaticJoint:isLimitEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:isLimitsEnabled

PrismaticJoint:isLimitsEnabled · revision 10559 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Checks whether limits are enabled.

Function

Synopsis

```
enabled = PrismaticJoint:isLimitsEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:isMotorEnabled

PrismaticJoint:isMotorEnabled · revision 6094 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Checks whether the motor is enabled.

Function

Synopsis

```
enabled = PrismaticJoint:isMotorEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false if disabled.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:setLimits

PrismaticJoint:setLimits · revision 6095 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the limits.

Function

Synopsis

```
PrismaticJoint:setLimits( lower, upper )
```

Arguments

`number lower`
: The lower limit, usually in meters.

`number upper`
: The upper limit, usually in meters.

Returns

Nothing.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:setLimitsEnabled

PrismaticJoint:setLimitsEnabled · revision 13728 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that version.

Available since LÖVE 0.9.0

It has been renamed again.

Enables/disables the joint limit.

Function

Synopsis

```
enable = PrismaticJoint:setLimitsEnabled( )
```

Arguments

None.

Returns

`boolean enable`

True if enabled, false if disabled.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:setLowerLimit

PrismaticJoint:setLowerLimit · revision 6097 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the lower limit.

Function

Synopsis

```
PrismaticJoint:setLowerLimit( lower )
```

Arguments

`number lower`
: The lower limit, usually in meters.

Returns

Nothing.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:setMaxMotorForce

PrismaticJoint:setMaxMotorForce · revision 6098 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Set the maximum motor force.

Function

Synopsis

```
PrismaticJoint:setMaxMotorForce( f )
```

Arguments

`number f`
: The maximum motor force, usually in N.

Returns

Nothing.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:setMotorEnabled

PrismaticJoint:setMotorEnabled · revision 23931 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that version.

Available since LÖVE 0.9.0

It has been renamed again.

Enables/disables the joint motor.

Function

Synopsis

```
PrismaticJoint:setMotorEnabled( enable )
```

Arguments

`boolean enable`

True to enable, false to disable.

Returns

Nothing.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:setMotorSpeed

PrismaticJoint:setMotorSpeed · revision 6100 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the motor speed.

Function

Synopsis

```
PrismaticJoint:setMotorSpeed( s )
```

Arguments

`number s`
: The motor speed, usually in meters per second.

Returns

Nothing.

See Also

- PrismaticJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PrismaticJoint:setUpperLimit

PrismaticJoint:setUpperLimit · revision 6101 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the upper limit.

Function

Synopsis

```
PrismaticJoint:setUpperLimit( upper )
```

Arguments

`number upper`
: The upper limit, usually in meters.

Returns

Nothing.

See Also

- PrismaticJoint

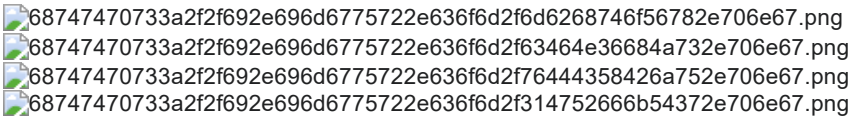
Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Pseudo3D

Pseudo3D · revision 27757 · last edited 2023-07-05

O.png This library seems to be gone (website returns 404)



More information about this engine can be found here.

https://github.com/matiasah/pseudo3d/wiki ↗

Link to the repository.

https://github.com/matiasah/pseudo3d ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint

PulleyJoint · revision 16459 · last edited 2023-07-05

Allows you to simulate bodies connected through pulleys.



Constructors

love.physics.newPulleyJoint	Creates a PulleyJoint to join two bodies to each other and the ground.

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
PulleyJoint:getConstant	Get the total length of the rope.		
PulleyJoint:getGroundAnchors	Get the ground anchor positions in world coordinates.		
PulleyJoint:getLength1	Get the current length of the segment attached to the first body.		Removed in 0.8.0
PulleyJoint:getLength2	Get the current length of the segment attached to the second body.		Removed in 0.8.0
PulleyJoint:getLengthA	Get the current length of the segment attached to the first body.	Added since 0.8.0	
PulleyJoint:getLengthB	Get the current length of the segment attached to the second body.	Added since 0.8.0	
PulleyJoint:getMaxLengths	Get the maximum lengths of the rope segments.		
PulleyJoint:getRatio	Get the pulley ratio.		
PulleyJoint:setConstant	Set the total length of the rope.		
PulleyJoint:setMaxLengths	Set the maximum lengths of the rope segments.		
PulleyJoint:setRatio	Set the pulley ratio.		

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getConstant

PulleyJoint:getConstant · revision 6109 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the total length of the rope.

Function

Synopsis

```
length = PulleyJoint:getConstant( )
```

Arguments

None.

Returns

`number length`
: The length of the rope in the joint.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getGroundAnchors

PulleyJoint:getGroundAnchors · revision 6108 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Get the ground anchor positions in world coordinates.

Function

Synopsis

```
a1x, a1y, a2x, a2y = PulleyJoint:getGroundAnchors( )
```

Arguments

None.

Returns

number a1x
: The x coordinate of the first anchor.

number a1y
: The y coordinate of the first anchor.

number a2x
: The x coordinate of the second anchor.

number a2y
: The y coordinate of the second anchor.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getLength1

PulleyJoint:getLength1 · revision 8525 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the current length of the rope segment attached to the first body.

Function

Synopsis

```
length = PulleyJoint:getLength1( )
```

Arguments

None.

Returns

`number length`

The length of the rope segment.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getLength2

PulleyJoint:getLength2 · revision 8526 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the current length of the rope segment attached to the second body.

Function

Synopsis

```
length = PulleyJoint:getLength2( )
```

Arguments

None.

Returns

`number length`

The length of the rope segment.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getLengthA

PulleyJoint:getLengthA · revision 8567 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Get the current length of the rope segment attached to the first body.

Function

Synopsis

```
length = PulleyJoint:getLengthA( )
```

Arguments

None.

Returns

`number length`

The length of the rope segment.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getLengthB

PulleyJoint:getLengthB · revision 8566 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Get the current length of the rope segment attached to the second body.

Function

Synopsis

```
length = PulleyJoint:getLengthB( )
```

Arguments

None.

Returns

`number length`

The length of the rope segment.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getMaxLengths

PulleyJoint:getMaxLengths · revision 6112 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the maximum lengths of the rope segments.

Function

Synopsis

```
len1, len2 = PulleyJoint:getMaxLengths( )
```

Arguments

None.

Returns

number len1
: The maximum length of the first rope segment.

number len2
: The maximum length of the second rope segment.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:getRatio

PulleyJoint:getRatio · revision 6113 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the pulley ratio.

Function

Synopsis

```
ratio = PulleyJoint:getRatio( )
```

Arguments

None.

Returns

`number ratio`
: The pulley ratio of the joint.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:setConstant

PulleyJoint:setConstant · revision 6114 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set the total length of the rope.

Setting a new length for the rope updates the maximum length values of the joint.

Function

Synopsis

```
PulleyJoint:setConstant( length )
```

Arguments

`number length`
: The new length of the rope in the joint.

Returns

Nothing.

See Also

- PulleyJoint
- PulleyJoint:getMaxLengths

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:setMaxLengths

PulleyJoint:setMaxLengths · revision 6115 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Set the maximum lengths of the rope segments.

The physics module also imposes maximum values for the rope segments. If the parameters exceed these values, the maximum values are set instead of the requested values.

Function

Synopsis

```
PulleyJoint:setMaxLengths( max1, max2 )
```

Arguments

number max1
: The new maximum length of the first segment.

number max2
: The new maximum length of the second segment.

Returns

Nothing.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PulleyJoint:setRatio

PulleyJoint:setRatio · revision 6116 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set the pulley ratio.

Function

Synopsis

```
PulleyJoint:setRatio( ratio )
```

Arguments

`number ratio`
: The new pulley ratio of the joint.

Returns

Nothing.

See Also

- PulleyJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

PyCharm

PyCharm · revision 17373 · last edited 2021-03-14

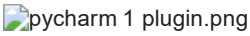
PyCharm is an awesome IDE originally designed for Python development. However, it can also work with many other languages, including Lua.

Download here (https://w w w.jetbrains.com/pycharm/).

This is a short tutorial on setting up PyCharm on OSX Lion for LÖVE projects.

I assume you already have PyCharm installed, and love is in your Applications directory.

1. Open PyCharm Preferences and go to the Plugins page. Search the available plugins for lua. Right-click on it and select Download and Install.



2. Go to the External Tools page in PyCharm Preferences. Add a new tool, give it a name. Fill in the following options:

- Program: open
- Parameters: -n -a love "$ProjectFileDir$"

$ProjectFileDir$ is a macro that inserts the project directory. You can find more macros in the "Insert Macro" button.



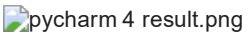
At this point, you can launch LÖVE projects by running your new tool from the Tools menu. I like to have a shortcut key, but the next step is optional.

3. Go to the Keymap page in the PyCharm Preferences. Search for the name of your tool, and click Add Keyboard Shortcut...



I used the shortcut Ctrl-Command-A.

That's it! You can now make a new project, add a new file called "main.lua" and fill it with all your lovey goodness. Run it simply by hitting your shortcut key or running LÖVE from the Tools menu.

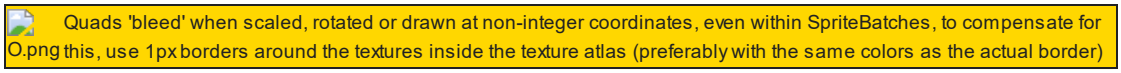


Quad

Quad · revision 25625 · last edited 2023-07-05

A quadrilateral (a polygon with four sides and four corners) with Texture coordinate information.

Quads can be used to select part of a texture to draw. In this way, one large texture atlas (http://en.wikipedia.org/wiki/Texture_atlas) can be loaded, and then split up into sub-images.



Constructors

love.graphics.newQuad	Creates a new Quad.

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Quad:flip	Flips this quad horizontally, vertically, or both.		Removed in 0.9.0
Quad:getLayer	Gets the layer specified by Quad:setLayer.	Added since 11.0	
Quad:getTextureDimensions	Gets reference texture dimensions initially specified in love.graphics.newQuad.	Added since 0.10.2	
Quad:getViewport	Gets the current viewport of this Quad.		
Quad:setLayer	Sets the layer to use in Array Textures.	Added since 11.0	
Quad:setViewport	Sets the texture coordinates according to a viewport.		

Supertypes

- Object

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Quad:flip

Quad:flip · revision 11833 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

This function is not supported in that and later versions.

Flips this quad horizontally, vertically, or both.

Function

Synopsis

```
Quad:flip( x, y )
```

Arguments

`boolean x`
: True to flip horizontally, false to leave as-is.

`boolean y`
: True to flip vertically, false to leave as-is.

Returns

Nothing.

See Also

- Quad

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Quad:getLayer

Quad:getLayer · revision 28019 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the layer specified by Quad:setLayer.

Function

Synopsis

```
layerindex = Quad:getLayer( )
```

Arguments

None.

Returns

`number layerindex`

Layer index used by the Quad for Array Textures.

See Also

- Quad
- Quad:setLayer
- love.graphics.drawLayer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Quad:getTextureDimensions

Quad:getTextureDimensions · revision 18090 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets reference texture dimensions initially specified in love.graphics.newQuad.

Function

Synopsis

```
sw, sh = Quad:getTextureDimensions( )
```

Arguments

None.

Returns

number sw
: The Texture width used by the Quad.

number sh
: The Texture height used by the Quad.

See Also

- Quad
- Quad:getViewport

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Quad:getViewport

Quad:getViewport · revision 23126 · last edited 2025-10-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Gets the current viewport of this Quad.

Function

Synopsis

```
x, y, w, h = Quad:getViewport( )
```

Arguments

None.

Returns

number x
: The top-left corner along the x-axis.

number y
: The top-left corner along the y-axis.

number w
: The width of the viewport.

number h
: The height of the viewport.

See Also

- Quad
- Quad:setViewport

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Quad:setLayer

Quad:setLayer · revision 28021 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets the layer to use in Array Textures.

Function

Synopsis

```
Quad:setLayer( layerindex )
```

Arguments

`number layerindex`
: Layer index to be used by the Quad for Array Textures.

Returns

Nothing.

See Also

- Quad
- Quad:getLayer
- love.graphics.drawLayer
- SpriteBatch:setLayer
- SpriteBatch:addLayer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Quad:setViewport

Quad:setViewport · revision 29298 · last edited 2024-08-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sets the texture coordinates according to a viewport.

Function

Synopsis

```
Quad:setViewport( x, y, w, h, sw, sh )
```

Arguments

number x
: The top-left corner along the x-axis.

number y
: The top-left corner along the y-axis.

number w
: The width of the viewport.

number h
: The height of the viewport.

Available since LÖVE 0.9.0

number sw (nil)
: Optional new reference width, the width of the Texture. Must be greater than 0 if set.

number sh (nil)
: Optional new reference height, the height of the Texture. Must be greater than 0 if set.

Returns

Nothing.

See Also

- Quad
- Quad:getViewport

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Quasimofo

Quasimofo · revision 28191 · last edited 2023-01-25

Quasimofo - a remake of the classic Hunchback game, my first attempt at using Love2d. Mainly a learning exercise to explore how easy it would be to quickly write a full game using it, and what I'd need to learn to do it better.

Keys to use are available from the main menu.

Gameplay



Download

- Source code and assets available on GitHub (https://github.com/Krunkathos/Quasimofo)

Other Credits

- Tserial Lua library - written by Taehl (https://love2d.org/wiki/User:Taehl)
- Tile images - modified from tileset by Ayene Chan (http://ayene-chan.deviantart.com/art/RPG-Maker-Tiles-255998841), provided under CC-BY-3.0 license (http://creativecommons.org/licenses/by/3.0/)
- Title image - modified from image by nahim89 (http://nahim89.deviantart.com/), provided under CC-BY-3.0 license (http://creativecommons.org/licenses/by/3.0/)
- Music - Automatically generated by cgMusic (http://codeminion.com/blogs/maciek/2008/05/cgmusic-computers-create-music/)
- gfx/fonts/PressStart2P/PressStart2P-Regular.ttf - created by Cody "CodeMan38" Boisclair (mailto:cody@zone38.net), provided under SIL Open Font License v1.1

Quickie

Quickie · revision 24522 · last edited 2023-07-05

Quickie is a immediate mode GUI (http://www.mollyrocket.com/forums/viewforum.php?f=10) implementation for LÖVE.


In the recent versions, Quickie has been renamed to SUIT

Immediate mode GUIs

In immediate mode GUIs you don't have button-*objects*, slider-*objects* and other-*objects* that are created in one place and used in another place. Rather, all widgets are just functions that put stuff where you want it. In addition, widgets don't keep track of their state (e.g. the slider value) - you have to do it. This approach makes a lot of things easier:

- Want to hide the widget? Simply don't call the function!
- Want to place the widget at a different position, for example above the player's head? Just call the function with different arguments!
- Want to get the value out of a widget? You already have it!

Widgets

Quickie defines several widgets often found in games:

- Buttons,
- Input boxes,
- Sliders,
- Checkbockes, and
- Labels.

If you need more, you can always create new widgets. It's super easy.

Quickie also includes flexible group layout tool.

Links

- Repository (https://github.com/vrld/Quickie)
- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=7811)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

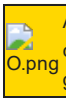
RandomGenerator

RandomGenerator · revision 29677 · last edited 2025-09-11

Available since LÖVE 0.9.0

This type is not supported in earlier versions.

A random number generation object which has its own random state.



As with love.math.random and math.random (https://www.lua.org/manual/5.1/manual.html#pdf-math.random), this class is not a cryptographically secure (https://en.wikipedia.org/wiki/Cryptographically_secure_pseudorandom_number_generator) random number generator, and should not be used for cryptography!

Constructors

love.math.newRandomGenerator	Creates a new RandomGenerator object.	Added since 0.9.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
RandomGenerator:getSeed	Gets the seed of the random number generator.	Added since 0.9.0
RandomGenerator:getState	Gets the current state of the random number generator.	Added since 0.9.1
RandomGenerator:random	Generates a pseudo-random number in a platform independent manner.	Added since 0.9.0
RandomGenerator:randomNormal	Get a normally distributed pseudo random number.	Added since 0.9.0
RandomGenerator:setSeed	Sets the seed of the random number generator.	Added since 0.9.0
RandomGenerator:setState	Sets the current state of the random number generator.	Added since 0.9.1

Supertypes

- Object

See Also

- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RandomGenerator:getSeek

RandomGenerator:getSeed · revision 11986 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the seed of the random number generator object.

The seed is split into two numbers due to Lua's use of doubles (http://en.w ikipedia.org/w iki/Double-precision_floating-point_format) for all number values - doubles can't accurately represent integer values above 2^53, but the seed value is an integer number in the range of [0, 2^64 - 1].

Function

Synopsis

```
low, high = RandomGenerator:getSeed( )
```

Arguments

None.

Returns

`number low`
: Integer number representing the lower 32 bits of the RandomGenerator's 64 bit seed value.

`number high`
: Integer number representing the higher 32 bits of the RandomGenerator's 64 bit seed value.

See Also

- RandomGenerator
- RandomGenerator:setSeed
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RandomGenerator:getState

RandomGenerator:getState · revision 22854 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets the current state of the random number generator. This returns an opaque string which is only useful for later use with RandomGenerator:setState in the same major version of LÖVE.

This is different from RandomGenerator:getSeed in that getState gets the RandomGenerator's current state, whereas getSeed gets the previously set seed number.

Function

Synopsis

```
state = RandomGenerator:getState( )
```

Arguments

None.

Returns

`string state`

The current state of the RandomGenerator object, represented as a string.

Notes

The value of the state string does not depend on the current operating system.

Examples

```
rng = love.math.newRandomGenerator(os.time())

for i=1, 100 do
    -- Use some random numbers.
    rng:random()
end

-- Make a new RandomGenerator and set its state to the current state of the first one.
rng2 = love.math.newRandomGenerator()
rng2:setState(rng:getState())

-- Both 'rng' and 'rng2' will now give the same results.
assert(rng:random() == rng2:random())
```

See Also

- RandomGenerator

- RandomGenerator:setState

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RandomGenerator:random

RandomGenerator:random · revision 26565 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Generates a pseudo-random number in a platform independent manner.

Function

Get uniformly distributed pseudo-random real number within [0, 1).

Synopsis

```
number = RandomGenerator:random( )
```

Arguments

None.

Returns

`number number`

The pseudo-random number.

Function

Get uniformly distributed pseudo-random **integer** number within [1, max].

Synopsis

```
number = RandomGenerator:random( max )
```

Arguments

`number max`

The maximum possible value it should return.

Returns

`number number`

The pseudo-random integer number.

Function

Get uniformly distributed pseudo-random **integer** number within [min, max].

Synopsis

```
number = RandomGenerator:random( min, max )
```

Arguments

number min
The minimum possible value it should return.

number max
The maximum possible value it should return.

Returns

number number
The pseudo-random integer number.

Notes

When using the 2nd and 3rd variant, numbers passed will be rounded, thus, `RandomGenerator:random(0, 76.767)` may return 77

See Also

- RandomGenerator
- love.math.random
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RandomGenerator:randomNormal

RandomGenerator:randomNormal · revision 27915 · last edited 2023-07-05

On this page

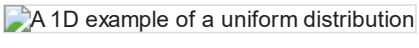
- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

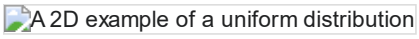
Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Generates a normally-distributed (https://en.wikipedia.org/wiki/Normal_distribution) pseudo-random number.

While a typical uniform distribution looks like this:





A typical normal distribution looks like this (note the values aggregating at the center, and the shape of a bell curve):





Function

Synopsis

```
number = RandomGenerator:randomNormal( stddev, mean )
```

Arguments

`number stddev (1)`
: Standard deviation of the distribution.

`number mean (0)`
: The mean of the distribution.

Returns

`number number`
: Normally distributed random number with variance (stddev)² and the specified mean.

See Also

- RandomGenerator
- love.math.randomNormal
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RandomGenerator:setSeed

RandomGenerator:setSeed · revision 12178 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
- Synopsis
- Arguments
- Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the seed of the random number generator using the specified integer number.

Function

Synopsis

```
RandomGenerator:setSeed( seed )
```

Arguments

number seed

The integer number with which you want to seed the randomization. Must be within the range of [1, 2^53].

Returns

Nothing.

Notes

Due to Lua's use of double-precision floating point numbers, values above 2^53 cannot be accurately represented. Use the other variant of this function if your seed will have a larger value.

Function

Combines two 32-bit integer numbers into a 64-bit integer value and sets the seed of the random number generator using the value.

Synopsis

```
RandomGenerator:setSeed( low, high )
```

Arguments

number low

The lower 32 bits of the seed value. Must be within the range of [0, 2^32 - 1].

number high

The higher 32 bits of the seed value. Must be within the range of [0, 2^32 - 1].

Returns

Nothing.

Examples

Creates a new RandomGenerator object, sets the seed to the system clock's time, then generates a number between 1 and 100 inclusive. Note that the seed can be any number within the range of [0, 2^53 - 1].

```
function love.load()
    rng = love.math.newRandomGenerator()
```

```
    rng:setSeed(os.time())
    randomNumber = rng:random(1,100)
end
```

See Also

- RandomGenerator
- RandomGenerator:getSeed
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RandomGenerator:setState

RandomGenerator:setState · revision 22855 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Sets the current state of the random number generator. The value used as an argument for this function is an opaque string and should only originate from a previous call to RandomGenerator:getState in the same major version of LÖVE.

This is different from RandomGenerator:setSeed in that setState directly sets the RandomGenerator's current implementation-dependent state, whereas setSeed gives it a new seed value.

Function

Synopsis

```
RandomGenerator:setState( state )
```

Arguments

`string state`

The new state of the RandomGenerator object, represented as a string. This should originate from a previous call to RandomGenerator:getState.

Returns

Nothing.

Notes

The effect of the state string does not depend on the current operating system.

Examples

```
rng = love.math.newRandomGenerator(os.time())

for i=1, 100 do
    -- Use some random numbers.
    rng:random()
end

-- Make a new RandomGenerator and set its state to the current state of the first one.
rng2 = love.math.newRandomGenerator()
rng2:setState(rng:getState())

-- Both 'rng' and 'rng2' will now give the same results.
assert(rng:random() == rng2:random())
```

See Also

- RandomGenerator

- RandomGenerator:getState

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RandomLua

RandomLua · revision 24476 · last edited 2023-07-05

math.random used to give different numbers on different platforms. Since I didn't intend to compile C code for every platform, I decided to create a Pure Lua Random Generator (probably another one...).

This is not true anymore, but the library still remains for the curious and those who want to try different pseudo-random generators.

3 algorithms:

- Mersenne twister [1] (http://en.w ikipedia.org/w iki/Mersenne_tw ister), very good and slow.
- Multiply-with-carry [2] (http://en.w ikipedia.org/w iki/Multiply-w ith-carry), probably the best option.
- Linear congruential generator [3] (http://en.w ikipedia.org/w iki/Linear_congruential_generator), fast and, well, it works... kind of.

MWC and LCG come with 3 pre-defined parameters.

Original Thread (http://love2d.org/forums/viewtopic.php?f=5&t=3424) Download (https://github.com/linux-man/randomlua)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rasterizer

Rasterizer · revision 10245 · last edited 2023-07-05

Available since LÖVE 0.7.0

This type is not supported in earlier versions.

A Rasterizer handles font rendering, containing the font data (image or TrueType font) and drawable glyphs.

Constructors

love.font.newBMFontRasterizer	Creates a new BMFont Rasterizer.	Added since 0.10.0
love.font.newImageRasterizer	Creates a new Image Rasterizer.	Added since 0.10.0
love.font.newRasterizer	Creates a new Rasterizer.	Added since 0.7.0
love.font.newTrueTypeRasterizer	Creates a new TrueType Rasterizer.	Added since 0.10.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
Rasterizer:getAdvance	Gets font advance.	Added since 0.7.0
Rasterizer:getAscent	Gets ascent height.	Added since 0.7.0
Rasterizer:getDescent	Gets descent height.	Added since 0.7.0
Rasterizer:getGlyphCount	Gets number of glyphs in font.	Added since 0.7.0
Rasterizer:getGlyphData	Gets glyph data of a specified glyph.	Added since 0.7.0
Rasterizer:getHeight	Gets font height.	Added since 0.7.0
Rasterizer:getLineHeight	Gets line height of a font.	Added since 0.7.0
Rasterizer:hasGlyphs	Checks if font contains specified glyphs.	Added since 0.7.0

Supertypes

- Object

See Also

- love.font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Rasterizer:getAdvance

Rasterizer:getAdvance · revision 25253 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets font advance.

Function

Synopsis

```
advance = Rasterizer:getAdvance()
```

Arguments

None.

Returns

`number advance`

Font advance.

See Also

- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rasterizer:getAscent

Rasterizer:getAscent · revision 25251 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets ascent height.

Function

Synopsis

```
height = Rasterizer:getAscent()
```

Arguments

None.

Returns

number height

Ascent height.

See Also

- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rasterizer:getDescent

Rasterizer:getDescent · revision 25252 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets descent height.

Function

Synopsis

```
height = Rasterizer:getDescent()
```

Arguments

None.

Returns

number height

Descent height.

See Also

- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rasterizer:getGlyphCount

Rasterizer:getGlyphCount · revision 25247 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets number of glyphs in font.

Function

Synopsis

```
count = Rasterizer:getGlyphCount()
```

Arguments

None.

Returns

`number count`
: Glyphs count.

See Also

- Rasterizer
- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rasterizer:getGlyphData

Rasterizer:getGlyphData · revision 25254 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph data of a specified glyph.

Function

Synopsis

```
glyphData = Rasterizer:getGlyphData( glyph )
```

Arguments

`string glyph`
: Glyph

Returns

`GlyphData glyphData`
: Glyph data

Function

Synopsis

```
glyphData = Rasterizer:getGlyphData( glyphNumber )
```

Arguments

`number glyphNumber`
: Glyph number

Returns

`GlyphData glyphData`
: Glyph data

See Also

- Rasterizer
- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Rasterizer:getHeight

Rasterizer:getHeight · revision 25246 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets font height.

Function

Synopsis

```
height = Rasterizer:getHeight()
```

Arguments

None.

Returns

number height

Font height

See Also

- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rasterizer:getLineHeight

Rasterizer:getLineHeight · revision 26769 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets line height of a font.

Function

Synopsis

```
height = Rasterizer:getLineHeight()
```

Arguments

None.

Returns

`number height`

Line height of a font.

See Also

- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rasterizer:hasGlyphs

Rasterizer:hasGlyphs · revision 25250 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Checks if font contains specified glyphs.

Function

Synopsis

```
hasGlyphs = Rasterizer:hasGlyphs( glyph1, glyph2, ... )
```

Arguments

`string or number glyph1`
: Glyph

`string or number glyph2`
: Glyph

`string or number ...`
: Additional glyphs

Returns

`boolean hasGlyphs`
: Whatever font contains specified glyphs.

See Also

- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Rebound

Rebound · revision 4501 · last edited 2021-03-14

Developed by TechnoCat
LÖVE: 0.7.0

Rebound is a Pong (http://en.w ikipedia.org/w iki/Pong) inspired game. *Rebound* adds in some new features such as gravity wells and power-ups.

Download: Forum post (http://love2d.org/forum/viewtopic.php?f=5&t=1324)
BitBucket Page (http://bitbucket.org/dannyfritz/rebound)



"Blue is sucking,"

RecordingDevice

RecordingDevice · revision 21303 · last edited 2026-05-07

Available since LÖVE 11.0
This type is not supported in earlier versions.

Represents an audio input device capable of recording sounds.

Constructors

love.audio.getRecordingDevices	Gets a list of RecordingDevices on the system.	Added since 11.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
RecordingDevice:getBitDepth	Gets the number of bits per sample in the data currently being recorded.	Added since 11.0
RecordingDevice:getChannelCount	Gets the number of channels currently being recorded (mono or stereo).	Added since 11.0
RecordingDevice:getData	Gets all recorded audio SoundData stored in the device's internal ring buffer.	Added since 11.0
RecordingDevice:getName	Gets the name of the recording device.	Added since 11.0
RecordingDevice:getSampleCount	Gets the number of currently recorded samples.	Added since 11.0
RecordingDevice:getSampleRate	Gets the number of samples per second currently being recorded.	Added since 11.0
RecordingDevice:isRecording	Gets whether the device is currently recording.	Added since 11.0
RecordingDevice:start	Begins recording audio using this device.	Added since 11.0
RecordingDevice:stop	Stops recording audio from this device.	Added since 11.0

Supertypes

- Object

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:getBitDepth

RecordingDevice:getBitDepth · revision 21305 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the number of bits per sample in the data currently being recorded.

Function

Synopsis

```
bits = RecordingDevice:getBitDepth( )
```

Arguments

None.

Returns

`number bits`

The number of bits per sample in the data that's currently being recorded.

See Also

- RecordingDevice
- RecordingDevice:start

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:getChannelCount

RecordingDevice:getChannelCount · revision 21306 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the number of channels currently being recorded (mono or stereo).

Function

Synopsis

```
channels = RecordingDevice:getChannelCount( )
```

Arguments

None.

Returns

`number channels`

The number of channels being recorded (1 for mono, 2 for stereo).

See Also

- RecordingDevice
- RecordingDevice:start

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:getData

RecordingDevice:getData · revision 24213 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets all recorded audio SoundData stored in the device's internal ring buffer.

The internal ring buffer is cleared when this function is called, so calling it again will only get audio recorded after the previous call. If the device's internal ring buffer completely fills up before this function is called, the oldest data that doesn't fit into the buffer will be lost.

Function

Synopsis

```
data = RecordingDevice:getData( )
```

Arguments

None.

Returns

`SoundData data (nil)`

The recorded audio data, or nil if the device isn't recording.

See Also

- RecordingDevice
- RecordingDevice:start
- RecordingDevice:stop
- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:getName

RecordingDevice:getName · revision 21308 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the name of the recording device.

Function

Synopsis

```
name = RecordingDevice:getName( )
```

Arguments

None.

Returns

`string name`

The name of the device.

See Also

- RecordingDevice

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:getSampleCount

RecordingDevice:getSampleCount · revision 21309 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the number of currently recorded samples.

Function

Synopsis

```
samples = RecordingDevice:getSampleCount( )
```

Arguments

None.

Returns

`number samples`

The number of samples that have been recorded so far.

See Also

- RecordingDevice
- RecordingDevice:getData
- RecordingDevice:start
- RecordingDevice:stop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:getSampleRate

RecordingDevice:getSampleRate · revision 21310 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the number of samples per second currently being recorded.

Function

Synopsis

```
rate = RecordingDevice:getSampleRate( )
```

Arguments

None.

Returns

`number rate`

The number of samples being recorded per second (sample rate).

See Also

- RecordingDevice
- RecordingDevice:start

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:isRecording

RecordingDevice:isRecording · revision 21311 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether the device is currently recording.

Function

Synopsis

```
recording = RecordingDevice:isRecording( )
```

Arguments

None.

Returns

`boolean recording`

True if the RecordingDevice is recording, false otherwise.

See Also

- RecordingDevice
- RecordingDevice:start
- RecordingDevice:stop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:start

RecordingDevice:start · revision 24209 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Begins recording audio using this device.

Function

Synopsis

```
success = RecordingDevice:start( samplecount, samplerate, bitdepth, channels )
```

Arguments

`number samplecount`
: The maximum number of samples to store in an internal ring buffer when recording. RecordingDevice:getData clears the internal buffer when called.

`number samplerate (8000)`
: The number of samples per second to store when recording.

`number bitdepth (16)`
: The number of bits per sample.

`number channels (1)`
: Whether to record in mono or stereo. Most microphones don't support more than 1 channel.

Returns

`boolean success`
: True if the device successfully began recording using the specified parameters, false if not.

Notes

A ring buffer is used internally to store recorded data until RecordingDevice:getData or RecordingDevice:stop are called – the former clears the buffer. If the buffer completely fills up before getData or stop are called, the oldest data that doesn't fit into the buffer will be lost.

See Also

- RecordingDevice
- RecordingDevice:getData
- RecordingDevice:stop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RecordingDevice:stop

RecordingDevice:stop · revision 21313 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Stops recording audio from this device. Any sound data currently in the device's buffer will be returned.

Function

Synopsis

```
data = RecordingDevice:stop( )
```

Arguments

None.

Returns

`SoundData data (nil)`

The sound data currently in the device's buffer, or nil if the device wasn't recording.

See Also

- RecordingDevice
- RecordingDevice:start
- RecordingDevice:getData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint

RevoluteJoint · revision 25622 · last edited 2023-07-05

Allow two Bodies to revolve around a shared point.



Constructors

love.physics.newRevoluteJoint	Creates a pivot joint between two bodies.

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
RevoluteJoint:areLimitsEnabled	Checks whether limits are enabled.	Added since 11.0	
RevoluteJoint:enableLimit	Enables or disables the joint limits.	Added since 0.8.0	Removed in 0.9.0
RevoluteJoint:enableMotor	Starts or stops the joint motor.	Added since 0.8.0	Removed in 0.9.0
RevoluteJoint:getJointAngle	Get the current joint angle.		
RevoluteJoint:getJointSpeed	Get the current joint angle speed.		
RevoluteJoint:getLimits	Gets the joint limits.		
RevoluteJoint:getLowerLimit	Gets the lower limit.		
RevoluteJoint:getMaxMotorTorque	Gets the maximum motor force.		
RevoluteJoint:getMotorSpeed	Gets the motor speed.		
RevoluteJoint:getMotorTorque	Get the current motor force.		
RevoluteJoint:getReferenceAngle	Gets the reference angle.	Added since 0.10.2	
RevoluteJoint:getUpperLimit	Gets the upper limit.		
RevoluteJoint:hasLimitsEnabled	Checks whether limits are enabled.	Added since 0.9.0	Deprecated in 11.0
RevoluteJoint:isLimitsEnabled	Checks whether limits are enabled.		Removed in 0.9.0
RevoluteJoint:isMotorEnabled	Checks whether the motor is enabled.		
RevoluteJoint:setLimits	Sets the limits.		
RevoluteJoint:setLimitsEnabled	Enables/disables the joint limit.	Added since 0.9.0	
RevoluteJoint:setLowerLimit	Sets the lower limit.		
RevoluteJoint:setMaxMotorTorque	Set the maximum motor force.		
RevoluteJoint:setMotorEnabled	Enables/disables the joint motor.	Added since 0.9.0	
RevoluteJoint:setMotorSpeed	Sets the motor speed.		

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:areLimitsEnabled

RevoluteJoint:areLimitsEnabled · revision 22636 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from RevoluteJoint:hasLimitsEnabled.

Checks whether limits are enabled.

Function

Synopsis

```
enabled = RevoluteJoint:areLimitsEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:enableLimit

RevoluteJoint:enableLimit · revision 13575 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to RevoluteJoint:setLimitsEnabled.

Enables or disables the joint limits.

Function

Synopsis

```
RevoluteJoint:enableLimit( enable )
```

Arguments

`boolean enable`
: True to enable, false to disable.

Returns

Nothing.

See Also

- RevoluteJoint
- RevoluteJoint:isLimitsEnabled
- RevoluteJoint:setLimits

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:enableMotor

RevoluteJoint:enableMotor · revision 13730 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to RevoluteJoint:setMotorEnabled.

Starts or stops the joint motor.

Function

Synopsis

```
RevoluteJoint:enableMotor( enable )
```

Arguments

`boolean enable`
: True to enable, false to disable.

Returns

Nothing.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getJointAngle

RevoluteJoint:getJointAngle · revision 6119 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Get the current joint angle.

Function

Synopsis

```
angle = RevoluteJoint:getJointAngle( )
```

Arguments

None.

Returns

`number angle`
: The joint angle in radians.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getJointSpeed

RevoluteJoint:getJointSpeed · revision 6120 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Get the current joint angle speed.

Function

Synopsis

```
s = RevoluteJoint:getJointSpeed( )
```

Arguments

None.

Returns

`number s`
: Joint angle speed in radians/second.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getLimits

RevoluteJoint:getLimits · revision 6121 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the joint limits.

Function

Synopsis

```
lower, upper = RevoluteJoint:getLimits( )
```

Arguments

None.

Returns

`number lower`
: The lower limit, in radians.

`number upper`
: The upper limit, in radians.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getLowerLimit

RevoluteJoint:getLowerLimit · revision 6122 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the lower limit.

Function

Synopsis

```
lower = RevoluteJoint:getLowerLimit( )
```

Arguments

None.

Returns

number lower
: The lower limit, in radians.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getMaxMotorTorque

RevoluteJoint:getMaxMotorTorque · revision 6123 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the maximum motor force.

Function

Synopsis

```
f = RevoluteJoint:getMaxMotorTorque( )
```

Arguments

None.

Returns

number f
: The maximum motor force, in Nm.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getMotorSpeed

RevoluteJoint:getMotorSpeed · revision 6124 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the motor speed.

Function

Synopsis

```
s = RevoluteJoint:getMotorSpeed( )
```

Arguments

None.

Returns

number s
: The motor speed, radians per second.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getMotorTorque

RevoluteJoint:getMotorTorque · revision 27211 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the current motor force.

Function

Synopsis

```
f = RevoluteJoint:getMotorTorque(invdt)
```

Arguments

`number invdt`
: Inverse delta time since last frame (1 / dt)

Returns

`number f`
: The current motor force, in Nm.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getReferenceAngle

RevoluteJoint:getReferenceAngle · revision 26823 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the reference angle.

Function

Synopsis

```
angle = RevoluteJoint:getReferenceAngle( )
```

Arguments

None.

Returns

`number angle`

The reference angle in radians.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:getUpperLimit

RevoluteJoint:getUpperLimit · revision 6126 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the upper limit.

Function

Synopsis

```
upper = RevoluteJoint:getUpperLimit( )
```

Arguments

None.

Returns

number upper
: The upper limit, in radians.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:hasLimitsEnabled

RevoluteJoint:hasLimitsEnabled · revision 22625 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from RevoluteJoint:isLimitsEnabled.

Deprecated in LÖVE 11.0

This function has been renamed to RevoluteJoint:areLimitsEnabled.

Checks whether limits are enabled.

Function

Synopsis

```
enabled = RevoluteJoint:hasLimitsEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:isLimitsEnabled

RevoluteJoint:isLimitsEnabled · revision 13576 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to RevoluteJoint:hasLimitsEnabled.

Checks whether limits are enabled.

Function

Synopsis

```
enabled = RevoluteJoint:isLimitsEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

RevoluteJoint:isMotorEnabled

RevoluteJoint:isMotorEnabled · revision 6128 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Checks whether the motor is enabled.

Function

Synopsis

```
enabled = RevoluteJoint:isMotorEnabled( )
```

Arguments

None.

Returns

`boolean enabled`
: True if enabled, false if disabled.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:setLimits

RevoluteJoint:setLimits · revision 6129 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sets the limits.

Function

Synopsis

```
RevoluteJoint:setLimits( lower, upper )
```

Arguments

`number lower`
: The lower limit, in radians.

`number upper`
: The upper limit, in radians.

Returns

Nothing.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:setLimitsEnabled

RevoluteJoint:setLimitsEnabled · revision 13578 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from RevoluteJoint:enableLimits.

Enables/disables the joint limit.

Function

Synopsis

```
RevoluteJoint:setLimitsEnabled( enable )
```

Arguments

`boolean enable`
: True to enable, false to disable.

Returns

Nothing.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:setLowerLimit

RevoluteJoint:setLowerLimit · revision 6131 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the lower limit.

Function

Synopsis

```
RevoluteJoint:setLowerLimit( lower )
```

Arguments

`number lower`
: The lower limit, in radians.

Returns

Nothing.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:setMaxMotorTorque

RevoluteJoint:setMaxMotorTorque · revision 6132 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Set the maximum motor force.

Function

Synopsis

```
RevoluteJoint:setMaxMotorTorque( f )
```

Arguments

`number f`
: The maximum motor force, in Nm.

Returns

Nothing.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:setMotorEnabled

RevoluteJoint:setMotorEnabled · revision 14415 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from RevoluteJoint:enableMotor.

Enables/disables the joint motor.

Function

Synopsis

```
RevoluteJoint:setMotorEnabled( enable )
```

Arguments

`boolean enable`
: True to enable, false to disable.

Returns

Nothing.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:setMotorSpeed

RevoluteJoint:setMotorSpeed · revision 6134 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the motor speed.

Function

Synopsis

```
RevoluteJoint:setMotorSpeed( s )
```

Arguments

`number s`
: The motor speed, radians per second.

Returns

Nothing.

See Also

- RevoluteJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RevoluteJoint:setUpperLimit

RevoluteJoint:setUpperLimit · revision 9931 · last edited 2023-07-05

Sets the upper limit.

Function

Synopsis

```
RevoluteJoint:setUpperLimit( upper )
```

Arguments

number upper
: The upper limit, in radians.

Returns

Nothing.

See Also

- RevoluteJoint

RichText

RichText · revision 8898 · last edited 2023-07-05

Rich. Text.

GitHub (https://github.com/gvx/richtext)

RopeJoint

RopeJoint · revision 10247 · last edited 2023-07-05

Available since LÖVE 0.8.0
This type is not supported in earlier versions.

The RopeJoint enforces a maximum distance between two points on two bodies. It has no other effect.

Constructors

love.physics.newRopeJoint	Creates a joint between two bodies that enforces a max distance between them.	Added since 0.8.0

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
RopeJoint:getMaxLength	Gets the maximum length of a RopeJoint.	Added since 0.8.0	
RopeJoint:setMaxLength	Sets the maximum length of a RopeJoint.	Added since 11.0	

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RopeJoint:getMaxLength

RopeJoint:getMaxLength · revision 20867 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Gets the maximum length of a RopeJoint.

Function

Synopsis

```
maxLength = RopeJoint:getMaxLength( )
```

Arguments

None.

Returns

`number maxLength`

The maximum length of the RopeJoint.

See Also

- RopeJoint
- RopeJoint:setMaxLength

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RopeJoint:setMaxLength

RopeJoint:setMaxLength · revision 22637 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets the maximum length of a RopeJoint.

Function

Synopsis

```
RopeJoint:setMaxLength( maxlength )
```

Arguments

`number maxLength`
: The new maximum length of the RopeJoint.

Returns

Nothing.

See Also

- RopeJoint
- RopeJoint:getMaxLength

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

RotLove

RotLove · revision 20400 · last edited 2023-07-05

RogueLike Toolkit in Love

Bringing rot.js (http://ondras.github.io/rot.js/hp/) functionality to Love2D

See this (http://paulofmandow n.github.io/rotLove/) for a quick and dirty run down of all the functionalit provided.

- Display - via rlLove (https://github.com/paulofmandow n/rlLove), only supports cp437 emulation rather than full font support.
- rng - via RandmLua (http://love2d.org/forums/viewtopic.php?f=5&t=3424). Multiply With Carry, Linear congruential generator, and Mersenne Twister. Extended with set/getState methods.
- String Generator - Direct Port from rot.js (http://ondras.github.io/rot.js/hp/)
- Map - Arena, Divided/Icey/Eller Maze, Digger/Uniform/Rogue* Dungeons. Ported from rot.js (http://ondras.github.io/rot.js/hp/).
- Noise Generator - Simplex Noise
- FOV - Bresenham Line based Ray Casting, Precise Shadow Casting
- Color - 147 Predefined colors; generate valid colors from string; add, multiply, or interpolate colors; generate a random color from a reference and set of standard deviations (straight port from rot.js (http://ondras.github.io/rot.js/hp/))
- Path Finding - Dijkstra and AStar pathfinding ported from rot.js (http://ondras.github.io/rot.js/hp/).
- Lighting - compute light emission and blending, ported from rot.js (http://ondras.github.io/rot.js/hp/).

Links

- Source - GitHub (https://github.com/paulofmandow n/rotLove)
- Forums - rotLove - A port of rot.js to lua/Love2D (https://love2d.org/forums/viewtopic.php?t=33506)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Ryomaechizen

Ryomaechizen · revision 17445 · last edited 2021-03-14

Scancode

Scancode · revision 28094 · last edited 2026-04-08

On this page

- Constants
  - Keypad scancodes
  - International / language scancodes
  - Media scancodes
  - Hardware setting scancodes
  - Misc.
- See Also
- Other Languages

Available since LÖVE 0.9.2

This enum is not supported in earlier versions.

Keyboard scancodes.

Scancodes are keyboard layout-independent, so the scancode string "w" will be generated if the key in the same place as the "w" key on an American QWERTY keyboard (https://en.wikipedia.org/wiki/Keyboard_layout#/media/File:ISO_keyboard_(105)_QWERTY_UK.svg) is pressed, no matter what the key is labelled or what the user's operating system settings are.

Using scancodes, rather than keycodes, is useful because keyboards with layouts differing from the US/UK layout(s) might have keys that generate 'unknown' keycodes, but the scancodes will still be detected. This however would necessitate having a list for each keyboard layout one would choose to support.

One could use textinput or textedited instead, but those only give back the end result of keys used, i.e. you can't get modifiers on their own from it, only the final symbols that were generated.

Constants

a
: The 'A' key on an American layout.

b
: The 'B' key on an American layout.

c
: The 'C' key on an American layout.

d
: The 'D' key on an American layout.

e
: The 'E' key on an American layout.

f
: The 'F' key on an American layout.

g
: The 'G' key on an American layout.

h
: The 'H' key on an American layout.

i
: The 'I' key on an American layout.

j
: The 'J' key on an American layout.

k
: The 'K' key on an American layout.

l
: The 'L' key on an American layout.

m
: The 'M' key on an American layout.

n
: The 'N' key on an American layout.

o
: The 'O' key on an American layout.

p
: The 'P' key on an American layout.

q
: The 'Q' key on an American layout.

r

The 'R' key on an American layout.

s
The 'S' key on an American layout.

t
The 'T' key on an American layout.

u
The 'U' key on an American layout.

v
The 'V' key on an American layout.

w
The 'W' key on an American layout.

x
The 'X' key on an American layout.

y
The 'Y' key on an American layout.

z
The 'Z' key on an American layout.

1
The '1' key on an American layout.

2
The '2' key on an American layout.

3
The '3' key on an American layout.

4
The '4' key on an American layout.

5
The '5' key on an American layout.

6
The '6' key on an American layout.

7
The '7' key on an American layout.

8
The '8' key on an American layout.

9
The '9' key on an American layout.

0
The '0' key on an American layout.

return
The 'return' / 'enter' key on an American layout.

escape
The 'escape' key on an American layout.

backspace
The 'backspace' key on an American layout.

tab
The 'tab' key on an American layout.

space
The spacebar on an American layout.

-
The minus key on an American layout.

=
The equals key on an American layout.

[
The left-bracket key on an American layout.

]
The right-bracket key on an American layout.

****
The backslash key on an American layout.

nonus#
: The non-U.S. hash scancode.

;
: The semicolon key on an American layout.

'
: The apostrophe key on an American layout.

`
: The back-tick / grave key on an American layout.

,
: The comma key on an American layout.

.
: The period key on an American layout.

/
: The forward-slash key on an American layout.

capslock
: The capslock key on an American layout.

f1
: The F1 key on an American layout.

f2
: The F2 key on an American layout.

f3
: The F3 key on an American layout.

f4
: The F4 key on an American layout.

f5
: The F5 key on an American layout.

f6
: The F6 key on an American layout.

f7
: The F7 key on an American layout.

f8
: The F8 key on an American layout.

f9
: The F9 key on an American layout.

f10
: The F10 key on an American layout.

f11
: The F11 key on an American layout.

f12
: The F12 key on an American layout.

f13
: The F13 key on an American layout.

f14
: The F14 key on an American layout.

f15
: The F15 key on an American layout.

f16
: The F16 key on an American layout.

f17
: The F17 key on an American layout.

f18
: The F18 key on an American layout.

f19
: The F19 key on an American layout.

f20
: The F20 key on an American layout.

f21

The F21 key on an American layout.

f22

The F22 key on an American layout.

f23

The F23 key on an American layout.

f24

The F24 key on an American layout.

lctrl

The left control key on an American layout.

lshift

The left shift key on an American layout.

lalt

The left alt / option key on an American layout.

lgui

The left GUI (command / windows / super) key on an American layout.

rctrl

The right control key on an American layout.

rshift

The right shift key on an American layout.

ralt

The right alt / option key on an American layout.

rgui

The right GUI (command / windows / super) key on an American layout.

printscreen

The printscreen key on an American layout.

scrolllock

The scroll-lock key on an American layout.

pause

The pause key on an American layout.

insert

The insert key on an American layout.

home

The home key on an American layout.

numlock

The numlock / clear key on an American layout.

pageup

The page-up key on an American layout.

delete

The forward-delete key on an American layout.

end

The end key on an American layout.

pagedown

The page-down key on an American layout.

right

The right-arrow key on an American layout.

left

The left-arrow key on an American layout.

down

The down-arrow key on an American layout.

up

The up-arrow key on an American layout.

nonusbackslash

The non-U.S. backslash scancode.

application

The application key on an American layout. Windows contextual menu, compose key.

execute

The 'execute' key on an American layout.

help
: The 'help' key on an American layout.

menu
: The 'menu' key on an American layout.

select
: The 'select' key on an American layout.

stop
: The 'stop' key on an American layout.

again
: The 'again' key on an American layout.

undo
: The 'undo' key on an American layout.

cut
: The 'cut' key on an American layout.

copy
: The 'copy' key on an American layout.

paste
: The 'paste' key on an American layout.

find
: The 'find' key on an American layout.

Keypad scancodes

kp/
: The keypad forward-slash key on an American layout.

kp*
: The keypad '*' key on an American layout.

kp-
: The keypad minus key on an American layout.

kp+
: The keypad plus key on an American layout.

kp=
: The keypad equals key on an American layout.

kpenter
: The keypad enter key on an American layout.

kp1
: The keypad '1' key on an American layout.

kp2
: The keypad '2' key on an American layout.

kp3
: The keypad '3' key on an American layout.

kp4
: The keypad '4' key on an American layout.

kp5
: The keypad '5' key on an American layout.

kp6
: The keypad '6' key on an American layout.

kp7
: The keypad '7' key on an American layout.

kp8
: The keypad '8' key on an American layout.

kp9
: The keypad '9' key on an American layout.

kp0
: The keypad '0' key on an American layout.

kp.

The keypad period key on an American layout.

International / language scancodes

international1
The 1st international key on an American layout. Used on Asian keyboards.
international2
The 2nd international key on an American layout.
international3
The 3rd international key on an American layout. Yen.
international4
The 4th international key on an American layout.
international5
The 5th international key on an American layout.
international6
The 6th international key on an American layout.
international7
The 7th international key on an American layout.
international8
The 8th international key on an American layout.
international9
The 9th international key on an American layout.
lang1
Hangul/English toggle scancode.
lang2
Hanja conversion scancode.
lang3
Katakana scancode.
lang4
Hiragana scancode.
lang5
Zenkaku/Hankaku scancode.

Media scancodes

mute
The mute key on an American layout.
volumeup
The volume up key on an American layout.
volumedown
The volume down key on an American layout.
audionext
The audio next track key on an American layout.
audioprev
The audio previous track key on an American layout.
audiostop
The audio stop key on an American layout.
audioplay
The audio play key on an American layout.
audiomute
The audio mute key on an American layout.
mediaselect
The media select key on an American layout.
www
The 'WWW' key on an American layout.
mail
The Mail key on an American layout.
calculator
The calculator key on an American layout.

computer
The 'computer' key on an American layout.
acsearch
The AC Search key on an American layout.
achome
The AC Home key on an American layout.
acback
The AC Back key on an American layout.
acforward
The AC Forward key on an American layout.
acstop
Th AC Stop key on an American layout.
acrefresh
The AC Refresh key on an American layout.
acbookmarks
The AC Bookmarks key on an American layout.

Hardware setting scancodes

power
The system power scancode.
brightnessdown
The brightness-down scancode.
brightnessup
The brightness-up scancode.
displayswitch
The display switch scancode.
kbdillumtoggle
The keyboard illumination toggle scancode.
kbdillumdown
The keyboard illumination down scancode.
kbdillumup
The keyboard illumination up scancode.
eject
The eject scancode.
sleep
The system sleep scancode.

Misc.

alterase
The alt-erase key on an American layout.
sysreq
The sysreq key on an American layout.
cancel
The 'cancel' key on an American layout.
clear
The 'clear' key on an American layout.
prior
The 'prior' key on an American layout.
return2
The 'return2' key on an American layout.
separator
The 'separator' key on an American layout.
out
The 'out' key on an American layout.
oper
The 'oper' key on an American layout.
clearagain

The 'clearagain' key on an American layout.

crsel

The 'crsel' key on an American layout.

exsel

The 'exsel' key on an American layout.

kp00

The keypad 00 key on an American layout.

kp000

The keypad 000 key on an American layout.

thsousandsseparator

The thousands-separator key on an American layout.

decimalseparator

The decimal separator key on an American layout.

currencyunit

The currency unit key on an American layout.

currencysubunit

The currency sub-unit key on an American layout.

app1

The 'app1' scancode.

app2

The 'app2' scancode.

unknown

An unknown key.

See Also

- Scancodes in plain text (https://pastebin.com/kpxCK6qU)
- love.keyboard
- love.keypressed
- love.keyreleased
- love.keyboard.isScancodeDown
- love.keyboard.getScancodeFromKey
- love.keyboard.getKeyFromScancode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SciTE

SciTE · revision 15236 · last edited 2021-03-14

SciTE is a Text Editor with facilities for building and running programs.

Running love projects from SciTE



You can use SciTE to launch your LÖVE game by adding the following to your lua.properties:

```
command.go.main.lua="C:\Path\to\love.exe" "$(FileDir)"
```

Just make sure you put it in front of the `command.go.*.lua` entries.

See also

- "Using SciTE with LÖVE" forum thread (http://love2d.org/forums/viewtopic.php?f=3&t=1037)
- SciTE Website (http://www.scintilla.org/SciTE.html)

Scite on Windows

SensorType

SensorType · revision 28140 · last edited 2023-07-05

Available since LÖVE 12.0

This enum is not supported in earlier versions.

Kinds of sensors.

Constants

accelerometer

Accelerometer sensor.

gyroscope

Gyroscope sensor.

See Also

- love.sensor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Ser

Ser · revision 24509 · last edited 2023-07-05

Ser is a fast, robust, richly-featured table serialization library for Lua. It was specifically written to store configuration and save files for LÖVE games, but can be used anywhere.



Simple

Ser is very simple and easy to use:

```lua
local serialize = require 'ser'

print(serialize({"Hello", world = true}))
-- prints:
-- return {"Hello", world = true}
```

Fast

Using Serpent's benchmark code, Ser is 33% faster than Serpent.

Robust

Sometimes you have strange, non-euclidean geometries in your table constructions. It happens, I don't judge. Ser can deal with that, where some other serialization libraries cry "lä! lä! Cthulhu fhtagn!" and give up — or worse, silently produce incorrect data.

Download

Get Ser from its GitHub repositiory (http://github.com/gvx/Ser).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Serial

Serial · revision 24489 · last edited 2023-07-05

Serial

Converts tables into Lua script that, when run, recreates the table.

Features

As keys: Strings, numbers, and booleans

As values: Strings, numbers, booleans, functions without upvalues, tables, recursive tables, tables that are seen in other tables, and objects with the proper support.

To make an object supported, make a function __serialize in the metatable that returns a string that recreates the object.

Download

Serial (http://github.com/Whitew ater/Serial) on Github

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Serialization

Serialization · revision 4586 · last edited 2021-03-14

Serialization is the process of turning a table and all its contents into a string. Deserialization is its inverse. This is generally used when a table of data needs to be stored to a file or sent across a network.

Serializing

Serializing · revision 25315 · last edited 2023-07-05

Serializing Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
Blob	A serialization and parser library for binary file formats		Any	
LuaTable	A library that converts tables to Lua code that can be saved.		Any	
Serial	A serialization library optimized for very large tables.		Any	
Tserial	Converts tables into strings and back. Good for saving games, multiplayer, etc.		Any	
binser	Customizable Lua Serializer - Great for Saving Games		Any	
bitser	Serializes and deserializes Lua values with LuaJIT		Any	
libSaveTableToFile	Load and save tables as text files.		Any	
libxml	Lua XML Library		Any	
love.bundle	Data saving lib for Love.		Any	
Jupiter	Table serialisation and file I/O.		0.8.0	

ServerObject

ServerObject · revision 3685 · last edited 2023-07-05

A Server object

Functions

ServerObject.tcp:acceptAll	Connect to a server.
ServerObject:Init	Initializes the server.
ServerObject:checkPing	Check for ping messages.
ServerObject:receive	Receive data from client(s).
ServerObject:respondToServBrowser	Dis/Allow Server Browser responses.
ServerObject:send	Send data to client(s).
ServerObject:setCallback	Sets the callbacks for the server.
ServerObject:setHandshake	Sets a handshake for the server.
ServerObject:setPing	Sets the ping for the server.
ServerObject:startserver	Starts and sets the server.
ServerObject:update	Update the server.

See Also

- LUBE
- LUBE.server

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject.tcp:acceptAll

ServerObject.tcp:acceptAll · revision 3687 · last edited 2023-07-05

Accepts connecting clients (TCP Only)

Function

Synopsis

```
server.tcp:acceptAll()
```

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:checkPing

ServerObject:checkPing · revision 3704 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Check for ping messages. If clients are timed out, disconnect them.

Function

Synopsis

```
server:checkPing( delta )
```

Arguments

`number delta`
: Some description goes here.

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:Init

ServerObject:Init · revision 3703 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Initializes the server.

Function

Synopsis

```
server:init(port, sockettype)
```

Arguments

number port
: any.

string sockettype
: "tcp", "udp" or user-defined.

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:receive

ServerObject:receive · revision 3701 · last edited 2023-07-05

Receive data from client(s).

Function

Synopsis

```
data = server:receive()
```

Returns

Data from clients

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:respondToServBrowser

ServerObject:respondToServBrowser · revision 3699 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets if the server should respond to Server Browser requests.

Function

Synopsis

```
server:respondToServBrowser(active, info)
```

Arguments

`boolean active`
: Should server respond

`table serverName`
: A table containing the server name and version as named keys

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:send

ServerObject:send · revision 9975 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Send data to client(s).

Function

Synopsis

```
server:send(data, recipient)
```

Arguments

`userdata data`
: The data to send to the client(s).

`recipient recipient`
: The client who shall receive the data, If nil then sends to all client(s).

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:setCallback

ServerObject:setCallback · revision 6639 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the callbacks for the server.

Function

Synopsis

```
server:setCallback(receive, connect, disconnect)
```

Arguments

`function receive`
: Callback called on incoming data(filtered): function(data, id).

`function connect`
: Callback called when client(s) connect: function(id).

`function disconnect`
: Callback called when client(s) disconnect (also called by ping): function(id).

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:setHandshake

ServerObject:setHandshake · revision 3695 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets a handshake for the server.

Function

Synopsis

```
server:setHandshake(handshake)
```

Arguments

`string handshake`
: A unique string, share by client and server to identify connecting and disconnecting client(s).

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:setPing

ServerObject:setPing · revision 5504 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sets the ping for the server.

Function

Synopsis

```
server:setPing(enabled, time, msg)
```

Arguments

`boolean enabled`
: Is enabled (True/False).

`number time`
: How many seconds between pings.

`string msg`
: Ping package content.

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:startserver

ServerObject:startserver · revision 3706 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Starts and sets the server.

This is usually and normally called by server:Init

Function

Synopsis

```
server:startserver(port)
```

Arguments

`number port`
: The port the server should start on.

Returns

Nothing.

See Also

- ServerObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ServerObject:update

ServerObject:update · revision 3707 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Update the server. Checks for incoming messages, and classifies as ping, connect, disconnect and raw data, Also checks ping (since 0.6)

Function

Synopsis

```
server:update(dt)
```

Arguments

`number dt`
: Delta-time.

Returns

Nothing.

See Also

- ServerObject

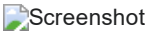
Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

sfxr.lua

sfxr.lua · revision 24455 · last edited 2023-07-05

sfxr.lua is a pure Lua port of the sfxr game sound generator. It performs fast enough for short sounds or pregenerated longer ones.


Screenshot

This single-file library allows you to dynamically create retro sound effects based on game-defined parameters. It also comes with a demo application as shown above based on Löve Frames.

Usage example (plays a random sound on every keypress):

```
local sfxr = require("sfxr")

function love.keypressed(key, rep)
    local sound = sfxr.newSound()
    sound:randomize()
    sound:play()
end
```

It supports all 7 preset generators for Pickup/Coin, Laser/Shoot, Explosion, Power-Up, Hit/Hurt, Jump and Blip/Select sounds and also both a randomizing and a mutating function. It also has all parameters from the original synthesizer, including basic envelopes, frequency slides, vibrato, change, square duty sweeping, phaser effect, low and high pass filters and sweep repeating.

You can also design sounds with the demo application and safe them either in a Lua-based format or the binary format of the original sfxr. Both formats can be loaded too, of course.

- **Get the source from GitHub and collaborate** (https://github.com/nucular/sfxrlua)
- Direct ZIP download (https://github.com/nucular/sfxrlua/archive/master.zip)
- Documentation Wiki (https://github.com/nucular/sfxrlua/wiki)
- Forum post (http://love2d.org/forums/viewtopic.php?f=5&t=78053)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader

Shader · revision 28299 · last edited 2025-10-21

Available since LÖVE 0.9.0
This type is not supported in earlier versions.

A Shader is used for advanced hardware-accelerated pixel or vertex manipulation. These effects are written in a language based on GLSL (OpenGL Shading Language) with a few things simplified for easier coding.

Potential uses for shaders include HDR/bloom, motion blur, grayscale/invert/sepia/any kind of color effect, reflection/refraction, distortions, bump mapping, and much more! Here is a collection of basic shaders and good starting point to learn: https://github.com/vrld/moonshine ↗

To use the most recent version of GLSL, you must put `#pragma language glsl3` at the top of your shader files. See love.graphics.newShader for details.

Constructors

love.graphics.newShader	Creates a new Shader.	Added since 0.9.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Shader:getExternVariable	Gets information about an 'extern' ('uniform') variable in the Shader.	Added since 0.9.2	Removed in 11.0
Shader:getWarnings	Gets warning and error messages (if any).	Added since 0.9.0	
Shader:hasUniform	Gets whether a uniform / extern variable exists in the Shader.	Added since 11.0	
Shader:send	Sends one or more values to the shader.	Added since 0.9.0	
Shader:sendColor	Sends one or more colors to the shader.	Added since 0.10.0	

Supertypes

- Object

See Also

- love.graphics
- love.graphics.setShader
- Shader Variables
- A Beginner's Guide to Shaders (https://blogs.love2d.org/content/beginners-guide-shaders)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader:getDebugName

Shader:getDebugName · revision 30570 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Gets the debug name that was set in love.graphics.newShader.

Graphics debugging tools such as RenderDoc will recognize and display the debug name inside the tool's interface, if it's set.

Function

Synopsis

```
name = Shader:getDebugName( )
```

Arguments

None.

Returns

`string name`
: The previously set debug name for this Shader.

See Also

- Shader
- love.graphics.newShader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader:getExternVariable

Shader:getExternVariable · revision 21257 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2 and removed in LÖVE 11.0

It has been replaced by Shader:hasUniform.

Gets information about an 'extern' ('uniform') variable in the shader.

Returns nil if the variable name doesn't exist in the shader, or if the video driver's shader compiler has determined that the variable doesn't affect the final output of the shader.

Function

Synopsis

```
type, components, arrayelements = Shader:getExternVariable( name )
```

Arguments

`string name`
: The name of the extern variable.

Returns

`ShaderVariableType type (nil)`
: The base type of the variable.

`number components (nil)`
: The number of components in the variable (e.g. 2 for a vec2 or mat2.)

`number arrayelements (nil)`
: The number of elements in the array if the variable is an array, or 1 if not.

See Also

- Shader
- Shader:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader:getWarnings

Shader:getWarnings · revision 10776 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from PixelEffect:getWarnings.

Returns any warning and error messages from compiling the shader code. This can be used for debugging your shaders if there's anything the graphics hardware doesn't like.

Function

Synopsis

```
warnings = Shader:getWarnings( )
```

Arguments

None.

Returns

`string warnings`

Warning and error messages (if any).

See Also

- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader:hasStage

Shader:hasStage · revision 29590 · last edited 2026-08-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets whether the Shader was created with the given shader stage.

This can be used to query whether the Shader is a compute shader or not.

Function

Synopsis

```
hasstage = Shader:hasStage( stage )
```

Arguments

`ShaderStage stage`
: The shader stage to check for.

Returns

`boolean hasstage`
: Whether the Shader has the shader stage.

See Also

- Shader
- love.graphics.newShader
- love.graphics.newComputeShader.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader:hasUniform

Shader:hasUniform · revision 22629 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has replaced Shader:getExternVariable.

Gets whether a uniform / extern variable exists in the Shader.

If a graphics driver's shader compiler determines that a uniform / extern variable doesn't affect the final output of the shader, it may optimize the variable out. This function will return false in that case.

Function

Synopsis

```
hasuniform = Shader:hasUniform( name )
```

Arguments

`string name`
: The name of the uniform variable.

Returns

`boolean hasuniform`
: Whether the uniform exists in the shader and affects its final output.

See Also

- Shader
- Shader:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader:send

Shader:send · revision 21369 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from PixelEffect:send.

Sends one or more values to a special (*uniform*) variable inside the shader. Uniform variables have to be marked using the *uniform* or *extern* keyword, e.g.

```
uniform float time;  // "float" is the typical number type used in GLSL shaders.
uniform float vars[2];
uniform vec2 light_pos;
uniform vec4 colors[4];
```

The corresponding send calls would be

```
shader:send("time", t)
shader:send("vars",a,b)
shader:send("light_pos", {light_x, light_y})
shader:send("colors", {r1, g1, b1, a1},  {r2, g2, b2, a2},  {r3, g3, b3, a3},  {r4, g4, b4, a4})
```

Uniform / extern variables are read-only in the shader code and remain constant until modified by a Shader:send call. Uniform variables can be accessed in both the Vertex and Pixel components of a shader, as long as the variable is declared in each.



There is a bug in version 0.10.2 which causes Shader:send to ignore the last argument when sending arrays. A simple workaround is to add an extra dummy argument when sending multiple values to a uniform array.

Function

Synopsis

```
Shader:send( name, number, ... )
```

Arguments

`string name`

Name of the number to send to the shader.

`number number`

Number to send to store in the uniform variable.

`number ...`

Additional numbers to send if the uniform variable is an array.

Returns

Nothing.

Notes

Because all numbers in Lua are floating point, in versions prior to 0.10.2 you must use the function Shader:sendInt to send values to `uniform int` variables in the shader's code.

Function

Synopsis

```
Shader:send( name, vector, ... )
```

Arguments

`string name`
: Name of the vector to send to the shader.

`table vector`
: Numbers to send to the uniform variable as a vector. The number of elements in the table determines the type of the vector (e.g. two numbers -> vec2). At least two and at most four numbers can be used.

`table ...`
: Additional vectors to send if the uniform variable is an array. All vectors need to be of the same size (e.g. only vec3's).

Returns

Nothing.

Function

Synopsis

```
Shader:send( name, matrix, ... )
```

Arguments

`string name`
: Name of the matrix to send to the shader.

`table matrix`
: 2x2, 3x3, or 4x4 matrix to send to the uniform variable. Using table form: `{{a,b,c,d}, {e,f,g,h}, ... }` or (since version 0.10.2) `{a,b,c,d, e,f,g,h, ...}`. The order in 0.10.2 is column-major; starting in 11.0 it's row-major instead.

`table ...`
: Additional matrices of the same type as *matrix* to store in a uniform array.

Returns

Nothing.

Function

Synopsis

```
Shader:send( name, texture )
```

Arguments

`string name`

Name of the Texture to send to the shader.

`Texture texture`

Texture (Image or Canvas) to send to the uniform variable.

Returns

Nothing.

Function

Synopsis

```
Shader:send( name, boolean, ... )
```

Arguments

`string name`

Name of the boolean to send to the shader.

`boolean boolean`

Boolean to send to store in the uniform variable.

`boolean ...`

Additional booleans to send if the uniform variable is an array.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
Shader:send( name, matrixlayout, matrix, ... )
```

Arguments

`string name`

Name of the matrix to send to the shader.

`MatrixLayout matrixlayout`

The layout (row- or column-major) of the matrix.

`table matrix`

2x2, 3x3, or 4x4 matrix to send to the uniform variable. Using table form: `{{a,b,c,d}, {e,f,g,h}, ... }` or `{a,b,c,d, e,f,g,h, ...}`.

`table ...`

Additional matrices of the same type as *matrix* to store in a uniform array.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Sends uniform values to the Shader sourced from the contents of a Data object. This directly copies the bytes of the data.

Synopsis

```
Shader:send( name, data, offset, size )
```

Arguments

`string name`
: Name of the uniform to send to the shader.

`Data data`
: Data object containing the values to send.

`number offset (0)`
: Offset in bytes from the start of the Data object.

`number size (all)`
: Size in bytes of the data to send. If nil, as many bytes as the specified uniform uses will be copied.

Returns

Nothing.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Sends uniform matrices to the Shader sourced from the contents of a Data object. This directly copies the bytes of the data.

Synopsis

```
Shader:send( name, data, matrixlayout, offset, size )
```

Arguments

`string name`
: Name of the uniform matrix to send to the shader.

`Data data`
: Data object containing the values to send.

`MatrixLayout matrixlayout`
: The layout (row- or column-major) of the matrix in memory.

`number offset (0)`
: Offset in bytes from the start of the Data object.

`number size (all)`
: Size in bytes of the data to send. If nil, as many bytes as the specified uniform uses will be copied.

Returns

Nothing.

See Also

- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shader:sendColor

Shader:sendColor · revision 29308 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sends one or more colors to a special (*uniform*) vec3 or vec4 variable inside the shader. The color components must be in the range of [0, 1]. The colors are gamma-corrected if global gamma-correction is enabled.

Uniforms must be marked using the *uniform* keyword, e.g.

```
uniform vec4 Color;
```

The corresponding sendColor call would be

```
shader:sendColor("Color", {r, g, b, a})
```

Uniforms can be accessed in both the Vertex and Pixel stages of a shader, as long as the variable is declared in each.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
Shader:sendColor( name, color, ... )
```

Arguments

`string name`
: The name of the color uniform to send to in the shader.

`table color`
: A table with red, green, blue, and optional alpha color components in the range of [0, 1] to send to the uniform as a vector.

`table ...`
: Additional colors to send in case the uniform is an array. All colors need to be of the same size (e.g. only vec3's).

Returns

Nothing.

See Also

- Shader
- Shader:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ShaderStage

ShaderStage · revision 31695 · last edited 2026-08-06

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Spots within the graphics pipeline where a Shader runs its code.

Constants

vertex
: Vertex shader.

pixel
: Pixel shader.

compute
: Compute shader.

See Also

- love.graphics
- Shader:hasStage
- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape

Shape · revision 25922 · last edited 2025-06-28

On this page

- Constructors
- Functions
- Enums
- Supertypes
- Subtypes
- See Also
- Other Languages

Shapes are solid 2d geometrical objects which handle the mass and collision of a Body in love.physics.

Shapes are attached to a Body via a Fixture. The Shape object is copied when this happens.

The Shape's position is relative to the position of the Body it has been attached to.

Constructors

love.physics.newChainShape	Creates a new ChainShape.	Added since 0.8.0
love.physics.newCircleShape	Creates a new CircleShape.	
love.physics.newEdgeShape	Creates a new EdgeShape.	Added since 0.8.0
love.physics.newPolygonShape	Creates a new PolygonShape.	
love.physics.newRectangleShape	Shorthand for creating rectangular PolygonShapes.	

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Shape:computeAABB	Returns the points of the bounding box for the transformed shape.	Added since 0.8.0	
Shape:computeMass	Computes the mass properties for the shape.	Added since 0.8.0	
Shape:destroy	Explicitly destroys the Shape.		Removed in 0.8.0
Shape:getBody	Get the body the shape is attached to.	Added since 0.7.0	Removed in 0.8.0
Shape:getBoundingBox	Gets the bounding box of the shape.		Removed in 0.8.0
Shape:getCategory	Gets the categories this shape is a member of.		Removed in 0.8.0
Shape:getCategoryBits	Gets the categories as a 16-bit integer.		Removed in 0.8.0
Shape:getChildCount	Returns the number of children the shape has.	Added since 0.8.0	
Shape:getData	Get the data set with setData.		Removed in 0.8.0
Shape:getDensity	Gets the density of the Shape.		Removed in 0.8.0
Shape:getFilterData	Gets the filter data of the Shape.		Removed in 0.8.0
Shape:getFriction	Gets the friction of this shape.		Removed in 0.8.0
Shape:getMask	Gets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:getRadius	Gets the radius of the shape.		
Shape:getRestitution	Gets the restitution of this shape.		Removed in 0.8.0
Shape:getType	Gets a string representing the Shape.		
Shape:isSensor	Checks whether a Shape is a sensor or not.		Removed in 0.8.0
Shape:rayCast	Casts a ray against the shape.	Added since 0.8.0	
Shape:setCategory	Sets the categories this shape is a member of.		Removed in 0.8.0
Shape:setData	Set data to be passed to the collision callback.		Removed in 0.8.0
Shape:setDensity	Sets the density of a Shape.		Removed in 0.8.0
Shape:setFilterData	Sets the filter data for a Shape.		Removed in 0.8.0
Shape:setFriction	Sets the friction of the shape.		Removed in 0.8.0
Shape:setMask	Sets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:setRestitution	Sets the restitution of the shape.		Removed in 0.8.0
Shape:setSensor	Sets whether this shape should act as a sensor.		Removed in 0.8.0
Shape:testPoint	Checks whether a point lies inside the shape.		

Shape:testSegment	Checks whether a line segment intersects a shape.	Removed in 0.8.0

Enums

ShapeType	The different types of Shapes, as returned by Shape:getType.

Supertypes

- Object

Subtypes

ChainShape	A ChainShape consists of multiple line segments.
CircleShape	Circle extends Shape and adds a radius and a local position.
EdgeShape	EdgeShape is a line segment.
PolygonShape	Polygon is a convex polygon with up to 8 sides.

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:computeAABB

Shape:computeAABB · revision 10567 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the points of the bounding box for the transformed shape.

Function

Synopsis

```
topLeftX, topLeftY, bottomRightX, bottomRightY = Shape:computeAABB( tx, ty, tr, childIndex )
```

Arguments

number tx
: The translation of the shape on the x-axis.

number ty
: The translation of the shape on the y-axis.

number tr
: The shape rotation.

number childIndex (1)
: The index of the child to compute the bounding box of.

Returns

number topLeftX
: The x position of the top-left point.

number topLeftY
: The y position of the top-left point.

number bottomRightX
: The x position of the bottom-right point.

number bottomRightY
: The y position of the bottom-right point.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:computeMass

Shape:computeMass · revision 10569 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Computes the mass properties for the shape with the specified density.

Function

Synopsis

```
x, y, mass, inertia = Shape:computeMass( density )
```

Arguments

number density
: The shape density.

Returns

number x
: The x postition of the center of mass.

number y
: The y postition of the center of mass.

number mass
: The mass of the shape.

number inertia
: The rotational inertia.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:destroy

Shape:destroy · revision 10568 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Explicitly destroys the Shape. When you don't have time to wait for garbage collection, this function may be used to free the object immediately, but note that an error will occur if you attempt to use the object after calling this function.

Note that Box2D doesn't allow destroying or creating shapes during collision callbacks.

Function

Synopsis

```
Shape:destroy( )
```

Arguments

None.

Returns

Nothing.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getBody

Shape:getBody · revision 12526 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Get the Body the shape is attached to.

Function

Synopsis

```
body = Shape:getBody( )
```

Arguments

None.

Returns

Body **body**

The body the shape is attached to.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getBoundingBox

Shape:getBoundingBox · revision 10585 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Gets the bounding box of the shape. This function can be used in a nested fashion with love.graphics.polygon.

Function

Synopsis

A bounding box is the smallest rectangle that encapsulates the entire polygon.

```
x1, y1, x2, y2, x3, y3, x4, y4 = Shape:getBoundingBox( )
```

Vertexes are returned starting from the bottom-left in a clockwise fashion (bottom-left, top-left, top-right, bottom-right).

Arguments

None.

Returns

number x1
: The x-component of the first vertex, bottom-left.

number y1
: The y-component of the first vertex, bottom-left.

number x2
: The x-component of the second vertex, top-left.

number y2
: The y-component of the second vertex, top-left.

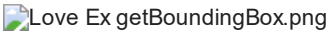
number x3
: The x-component of the third vertex, top-right.

number y3
: The y-component of the third vertex, top-right.

number x4
: The x-component of the fourth vertex, bottom-right.

number y4
: The y-component of the fourth vertex, bottom-right.

Example



And here's the source code if you want to try/test/see it yourself.

```
function love.load()

    world = love.physics.newWorld(650, 650)
        world:setGravity(0,700)
        world:setMeter(64)

    body = love.physics.newBody(world, 650/2, 650/2, 0, 0)
    shape = love.physics.newRectangleShape(body, 0, 0, 300, 300, 0)
```

```
    love.graphics.setFont(18)
    love.graphics.setBackgroundColor(255, 255, 255, 255)
    love.graphics.setMode(650, 650, false, true, 0)

end

function love.draw()

    X1, Y1, X2, Y2, X3, Y3, X4, Y4 = shape:getBoundingBox()

    love.graphics.setColor(0,0,0,255)

    love.graphics.print("X1, Y1", X1, Y1)
    love.graphics.print("X2, Y2", X2, Y2)
    love.graphics.print("X3, Y3", X3, Y3)
    love.graphics.print("X4, Y4", X4, Y4)

end
```

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getCategory

Shape:getCategory · revision 10564 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
Use Fixture:setCategory instead.

Gets the categories this shape is a member of. The number of categories the shape is a member of is the number of return values. Categories are used for allowing/disallowing certain collision.

Function

Synopsis

```
... = Shape:getCategory( )
```

Arguments

None.

Returns

numbers ...
Numbers from 1-16.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getCategoryBits

Shape:getCategoryBits · revision 10565 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
Use Fixture:getCategory instead.

Gets the categories as a 16-bit integer. A set bit indicates membership of that category.

Function

Synopsis

```
An = Shape:getCategoryBits( )
```

Arguments

None.

Returns

number An
Integer value representing the categories.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getChildCount

Shape:getChildCount · revision 10566 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the number of children the shape has.

Function

Synopsis

```
count = Shape:getChildCount( )
```

Arguments

None.

Returns

`number count`

The number of children.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getData

Shape:getData · revision 10584 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removed in LÖVE 0.8.0
Use Fixture:getUserData instead.

Get the data set with setData.

Function

Synopsis

```
v = Shape:getData( )
```

Arguments

None.

Returns

any v
The data previously set, or nil if none.

See Also

- Shape
- Shape:setData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getDensity

Shape:getDensity · revision 10563 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Gets the density of the Shape.

Function

Synopsis

```
density = Shape:getDensity( )
```

Arguments

None.

Returns

`number density`

The density of the Shape.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getFilterData

Shape:getFilterData · revision 10562 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Gets the filter data of the Shape.

Function

Synopsis

```
categoryBits, maskBits, groupIndex = Shape:getFilterData( )
```

Arguments

None.

Returns

number categoryBits
: A 16-bit integer representing category membership.

number maskBits
: A 16-bit integer representing masked categories.

number groupIndex
: An integer representing the group index.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getFriction

Shape:getFriction · revision 10586 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
This function is not supported in that and later versions.

Gets the friction of this shape.

Function

Synopsis

```
friction = Shape:getFriction( )
```

Arguments

None.

Returns

`number friction`
The friction of this Shape.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getMask

Shape:getMask · revision 10587 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - With how many categories does this shape collide?
- See Also
- Other Languages

Removed in LÖVE 0.8.0
Use Fixture:getMask instead.

Gets which categories this shape should **NOT** collide with. The number of masked categories is the number of return values.

Function

Synopsis

```
... = Shape:getMask( )
```

Arguments

None.

Returns

number ...
: Numbers from 1-16

Examples

With how many categories does this shape collide?

```
print('shape collides with', select('#', shape:getMask()), 'categories')
```

See Also

- Shape
- Shape:setMask

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getMaskBits

Shape:getMaskBits · revision 6038 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.6.1

This function is not supported in that and later versions.

Returns a 16-bit integer representing the masked categories. A set bit means that a collision occurs with members of that category.

Function

Synopsis

```
mask = Shape:getMaskBits( )
```

Arguments

None.

Returns

`number mask`

An integer value representing the mask.

See Also

- Shape
- Shape:getMask
- Shape:setMask

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Shape:getRadius

Shape:getRadius · revision 17957 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Gets the radius of the shape.

Function

Synopsis

```
radius = Shape:getRadius()
```

From the Box2D manual:

> Polygons inherit a radius from Shape. The radius creates a skin around the polygon. The skin is used in stacking scenarios to keep polygons slightly separated. This allows continuous collision to work against the core polygon.

Arguments

None.

Returns

`number radius`
: The radius of the shape.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getRestitution

Shape:getRestitution · revision 10588 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
This function is not supported in that and later versions.

Gets the restitution of this shape.

Function

Synopsis

```
restitution = Shape:getRestitution( )
```

Arguments

None.

Returns

`number restitution`
The restitution of this Shape.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:getType

Shape:getType · revision 6040 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Printing the type of a shape
- See Also
- Other Languages

Gets a string representing the Shape. This function can be useful for conditional debug drawing.

Function

Synopsis

```
type = Shape:getType( )
```

Arguments

None.

Returns

`ShapeType type`
: The type of the Shape.

Examples

Printing the type of a shape

```
shape1 = love.physics.newCircleShape( my_body, 0, 0, 20 )
print(shape1:getType()) -- outputs: 'circle'

shape2 = love.physics.newPolygonShape( my_body, ... )
print(shape2:getType()) -- outputs: 'polygon'

shape3 = love.physics.newRectangleShape( my_body, x, y, w, h, angle )
print(shape3:getType()) -- outputs: 'polygon'
```

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:isSensor

Shape:isSensor · revision 10442 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
This function is not supported in that and later versions.

Checks whether a Shape is a sensor or not.

Function

Synopsis

```
s = Shape:isSensor( )
```

Arguments

None.

Returns

`boolean s`
True if sensor, false otherwise.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:rayCast

Shape:rayCast · revision 16745 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Casting 2 different rays against a box.
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Casts a ray against the shape and returns the surface normal vector and the line position where the ray hit. If the ray missed the shape, nil will be returned. The Shape can be transformed to get it into the desired position.

The ray starts on the first point of the input line and goes towards the second point of the line. The fourth argument is the maximum distance the ray is going to travel as a scale factor of the input line length.

The childIndex parameter is used to specify which child of a parent shape, such as a ChainShape, will be ray casted. For ChainShapes, the index of 1 is the first edge on the chain. Ray casting a parent shape will only test the child specified so if you want to test every shape of the parent, you must loop through all of its children.

The world position of the impact can be calculated by multiplying the line vector with the third return value and adding it to the line starting point.

```
hitx, hity = x1 + (x2 - x1) * fraction, y1 + (y2 - y1) * fraction
```



There is a bug in 0.8.0 where the normal vector returned by this function gets scaled by love.physics.getMeter.

Function

Synopsis

```
xn, yn, fraction = Shape:rayCast( x1, y1, x2, y2, maxFraction, tx, ty, tr, childIndex )
```

Arguments

number x1
: The x position of the input line starting point.

number y1
: The y position of the input line starting point.

number x2
: The x position of the input line end point.

number y2
: The y position of the input line end point.

number maxFraction
: Ray length parameter.

number tx
: The translation of the shape on the x-axis.

number ty
: The translation of the shape on the y-axis.

number tr
: The shape rotation.

number childIndex (1)
: The index of the child the ray gets cast against.

Returns

number xn

The x component of the normal vector of the edge where the ray hit the shape.

number yn

The y component of the normal vector of the edge where the ray hit the shape.

number fraction

The position on the input line where the intersection happened as a factor of the line length.

Examples

Casting 2 different rays against a box.

```
function love.load()
    -- Setting this to 1 to avoid all current scaling bugs.
    love.physics.setMeter(1)

    Box = {}
    Box.x, Box.y, Box.r = 400, 300, 3.3
    Box.Shape = love.physics.newRectangleShape(150, 150)


    Ray1 = {
        point1 = {},
        point2 = {},
    }
    Ray1.point1.x, Ray1.point1.y = 50, 50
    Ray1.point2.x, Ray1.point2.y = 400, 300
    Ray1.scale = 1

    Ray2 = {
        point1 = {},
        point2 = {},
    }
    Ray2.point1.x, Ray2.point1.y = 500, 400
    Ray2.point2.x, Ray2.point2.y = Ray2.point1.x + math.cos(math.pi*1.45), Ray2.point1.y + math.sin(math.pi*1.45)
    Ray2.scale = 150
end

function love.draw()
    -- Drawing the box.
    love.graphics.setColor(255, 255, 255)
    love.graphics.push()
    -- For great point rotation!
    love.graphics.translate(Box.x, Box.y)
    love.graphics.rotate(Box.r)
    love.graphics.polygon("line", Box.Shape:getPoints())
    love.graphics.pop()

    -- Drawing the input lines of the rays and the reach of ray 2 in gray.
    love.graphics.setLineWidth(3)
    love.graphics.setColor(50, 50, 50)
    love.graphics.line(Ray2.point1.x, Ray2.point1.y, Ray2.point1.x + (Ray2.point2.x - Ray2.point1.x) * Ray2.scale, Ray2.point1.y +
(Ray2.point2.y - Ray2.point1.y) * Ray2.scale)
    love.graphics.setColor(255, 255, 255)
    love.graphics.line(Ray1.point1.x, Ray1.point1.y, Ray1.point2.x, Ray1.point2.y)
    love.graphics.line(Ray2.point1.x, Ray2.point1.y, Ray2.point2.x, Ray2.point2.y)
    love.graphics.setLineWidth(1)

    -- Remember that the ray cast can return nil if it hits nothing.
    local r1nx, r1ny, r1f = Box.Shape:rayCast(Ray1.point1.x, Ray1.point1.y, Ray1.point2.x, Ray1.point2.y, Ray1.scale, Box.x, Box.y, Box.r)
    local r2nx, r2ny, r2f = Box.Shape:rayCast(Ray2.point1.x, Ray2.point1.y, Ray2.point2.x, Ray2.point2.y, Ray2.scale, Box.x, Box.y, Box.r)

    if r1nx then
        -- Calculating the world position where the ray hit.
        local r1HitX = Ray1.point1.x + (Ray1.point2.x - Ray1.point1.x) * r1f
        local r1HitY = Ray1.point1.y + (Ray1.point2.y - Ray1.point1.y) * r1f

        -- Drawing the ray from the starting point to the position on the shape.
        love.graphics.setColor(255, 0, 0)
        love.graphics.line(Ray1.point1.x, Ray1.point1.y, r1HitX, r1HitY)

        -- We also get the surface normal of the edge the ray hit. Here drawn in green
```

```
        love.graphics.setColor(0, 255, 0)
        love.graphics.line(r1HitX, r1HitY, r1HitX + r1nx * 25, r1HitY + r1ny * 25)
    end

    if r2nx then
        -- Calculating the world position where the ray hit.
        local r2HitX = Ray2.point1.x + (Ray2.point2.x - Ray2.point1.x) * r2f
        local r2HitY = Ray2.point1.y + (Ray2.point2.y - Ray2.point1.y) * r2f

        -- Drawing the ray from the starting point to the position on the shape.
        love.graphics.setColor(255, 0, 0)
        love.graphics.line(Ray2.point1.x, Ray2.point1.y, r2HitX, r2HitY)

        -- We also get the surface normal of the edge the ray hit. Here drawn in green
        love.graphics.setColor(0, 255, 0)
        love.graphics.line(r2HitX, r2HitY, r2HitX + r2nx * 25, r2HitY + r2ny * 25)
    end
end
```



Screenshot of how the example could look like.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:setCategory

Shape:setCategory · revision 10589 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
Use Fixture:setCategory instead.

Sets the categories this shape is a member of. Sets the categories of this shape by specifying numbers from 1-16 as parameters. Categories can be used to prevent certain shapes from colliding.

Function

Synopsis

```
Shape:setCategory( ... )
```

Arguments

numbers ...
Numbers from 1-16

Returns

Nothing.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:setData

Shape:setData · revision 10576 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removed in LÖVE 0.8.0

Use Fixture:setUserData instead.

Set data to be passed to the collision callback.

When a shape collides, the value set here will be passed to the collision callback as one of the parameters. Typically, you would want to store a table reference here, but any value can be used.

Function

Synopsis

```
Shape:setData( v )
```

Arguments

any v

Any Lua value.

Returns

Nothing.

See Also

- Shape
- Shape:getData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Shape:setDensity

Shape:setDensity · revision 10577 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
This function is not supported in that and later versions.

Sets the density of a Shape. Do this before calling Body:setMassFromShapes.

Function

Synopsis

```
Shape:setDensity( density )
```

Arguments

`number density`
The new density of the Shape.

Returns

Nothing.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:setFilterData

Shape:setFilterData · revision 10578 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Sets the filter data for a Shape. Info stolen from box2d.org:

Collision filtering is a system for preventing collision between shapes. For example, say you make a character that rides a bicycle. You want the bicycle to collide with the terrain and the character to collide with the terrain, but you don't want the character to collide with the bicycle (because they must overlap). Box2D supports such collision filtering using categories and groups.

Read more about this in the Box2D documentation (http://box2d.org/manual.html#d0e845).

Function

Synopsis

```
Shape:setFilterData( categoryBits, maskBits, groupIndex )
```

Arguments

number categoryBits
: A 16-bit integer representing category membership.

number maskBits
: A 16-bit integer representing masked categories.

number groupIndex
: An integer representing the group index.

Returns

Nothing.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:setFriction

Shape:setFriction · revision 19121 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
Has been replaced by Fixture:setFriction.

Sets the friction of the shape. Friction determines how shapes react when they "slide" along other shapes. Low friction indicates a slippery surface, like ice, while high friction indicates a rough surface, like concrete. Range: 0.0 - 1.0.

Function

Synopsis

```
Shape:setFriction( friction )
```

Arguments

`number friction`
The friction of the shape.

Returns

Nothing.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:setMask

Shape:setMask · revision 10580 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Only collide with category 6
- See Also
- Others Languages

Removed in LÖVE 0.8.0
Use Fixture:setMask instead.

Sets which categories this shape should **NOT** collide with. With this function, you can exclude certain shape categories from collisions with this shape. The categories passed as parameters will be excluded from collisions - all others included.

Function

Synopsis

```
Shape:setMask( ... )
```

Arguments

`number ...`
Numbers from 1-16.

Returns

Nothing.

Examples

Only collide with category 6

```
shape:setMask(1, 2, 3, 4, 5, 7, 8, 9, 10, 12, 13, 14, 15, 16)
```

See Also

- Shape
- Shape:getMask

Others Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:setRestitution

Shape:setRestitution · revision 10581 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Removed in LÖVE 0.8.0
This function is not supported in that and later versions.

Sets the restitution of the shape. Restitution indicates the "bounciness" of the shape. High restitution can be used to model stuff like a rubber ball, while low restitution can be used for "dull" objects, like a bag of sand.

Function

Synopsis

```
Shape:setRestitution( restitution )
```

Arguments

`number restitution`
The restitution of the shape.

Returns

Nothing.

Notes

A shape with a restitution of 0 will still bounce a little if the shape it collides with has a restitution higher than 0.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:setSensor

Shape:setSensor · revision 10582 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
This function is not supported in that and later versions.

Sets whether this shape should act as a sensor. Set the shape as a sensor if you want to be notified when collision between shapes occur, but don't want a physical response (for instance, maybe you want enemies to appear when the player "touches" a certain point).

Function

Synopsis

```
Shape:setSensor( sensor )
```

Arguments

`boolean sensor`
True for sensor, false otherwise.

Returns

Nothing.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:testPoint

Shape:testPoint · revision 8518 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

This is particularly useful for mouse interaction with the shapes. By looping through all shapes and testing the mouse position with this function, we can find which shapes the mouse touches.



There's a bug in 0.8.0 preventing this function from working.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
hit = Shape:testPoint( tx, ty, tr, x, y )
```

Arguments

number tx
: Translates the shape along the x-axis.

number ty
: Translates the shape along the y-axis.

number tr
: Rotates the shape.

number x
: The x-component of the point.

number y
: The y-component of the point.

Returns

boolean hit
: True if inside, false if outside

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
hit = Shape:testPoint( x, y )
```

Arguments

number x
: The x-component of the point.

number y

The y-component of the point.

Returns

boolean hit
True if inside, false if outside

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shape:testSegment

Shape:testSegment · revision 10583 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Checks whether a line segment intersects a shape. This function will either return the "time" of impact and the surface normal at the point of collision, or nil if the line does not intersect the shape. The "time" is a value between 0.0 and 1.0 and can be used to calculate where the collision occured.

Function

Synopsis

```
t, xn, yn = Shape:testSegment( x1, y1, x2, y2 )
```

Arguments

`number x1`
: The x-component of the first endpoint.

`number y1`
: The y-component of the first endpoint.

`number x2`
: The x-component of the second endpoint.

`number y2`
: The y-component of the second endpoint.

Returns

`number t`
: The time of impact, or nil if no impact.

`number xn`
: The x-component of the surface normal.

`number yn`
: The y-component of the surface normal.

See Also

- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ShapeType

ShapeType · revision 18922 · last edited 2023-07-05

The different types of Shapes, as returned by Shape:getType.

Constants

circle
: The Shape is a CircleShape.

polygon
: The Shape is a PolygonShape.

Available since LÖVE 0.8.0

edge
: The Shape is a EdgeShape.

chain
: The Shape is a ChainShape.

See Also

- love.physics
- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Shear

Shear · revision 19072 · last edited 2023-07-05

This function was borrowed from way's 3D dice roller demo (https://love2d.org/forums/viewtopic.php?f=5&t=1692&p=16988). It performs a graphics shear (https://en.w ikipedia.org/w iki/Shear_mapping) (AKA "skew") coordinate transformation (which affects all following draws). It's recommended to perform a love.graphics.push before shearing.

```
function shear(ox, oy, xx, xy, yx, yy)
    local atan2, acos, tan = math.atan2, math.acos, math.tan
    local ex, ey, fx,fy = xx-ox, xy-oy, yx-ox, yy-oy
    if ex*fy<ey*fx then ex,ey,fx,fy=fx,fy,ex,ey end
    local e,f = (ex*ex+ey*ey)^.5, (fx*fx+fy*fy)^.5
    ex,ey = ex/e, ey/e
    fx,fy = fx/f, fy/f
    local desiredOrientation=atan2(ey+fy,ex+fx)
    local desiredAngle=acos(ex*fx+ey*fy)/2
    local z=tan(desiredAngle)
    local distortion=((1+z*z)/2)^.5
    love.graphics.translate(ox, oy)
    love.graphics.rotate(desiredOrientation)
    love.graphics.scale(1, z)
    love.graphics.rotate(-pi/4)
    love.graphics.scale(e/distortion,f/distortion)
end
```

Available since LÖVE 0.8.0

This code has become obsolete as of version 0.8.0.

You can use love.graphics.shear to shear/skew the whole scene or the shear parameters in love.graphics.draw to shear/skew a single drawable.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SICK

SICK · revision 24401 · last edited 2023-07-05

SICK stands for Simple Indicative of Competitive sKill and is a high-score library, loosly based on Highscöre. It saves player name and score separated by a tab rather than a newline, which makes them incompatible.

The authors have dedicated the library to the public domain, as per the conditions of the Creative Commons Zero dedication (http://creativecommons.org/publicdomain/zero/1.0/)

You can download it from https://gist.github.com/Kyrremann/b29397159e939cff2896ed53f1e7c10f ↗

API functions

```
highscore.set(filename, places, name, score)
```

Called at the loading of the game, it sets the filename to load and save high-scores from and to, as well as the number of slots to be used. The last two arguments are dummy values, used if the high-score file does not exist yet.

```
highscore.add(name, score)
```

Adds a record to the table. Don't bother checking whether it was high enough to count: superfluous entries will be discarded on saving.

```
highscore.save()
```

Just make sure this is called before closing the game, and you're set.

```
highscore()
```

Returns an iterator, useful to draw high-score tables. Used as in:

```
for i, score, name in highscore() do
    love.graphics.print(name, 400, i * 40)
    love.graphics.print(score, 500, i * 40)
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SLAM

SLAM · revision 24457 · last edited 2023-07-05

Simple LÖVE Audio Manager

A simple alternative sound manager similar to how audio works in Flash/AS3.

love.audio.newSource() creates a new source as usual which can be fed into love.audio.play(). Doing so returns a *handle* to a playing object. This way, one source can be played multiple times.

Repository

https://github.com/vrld/slam ↗

Direct Link

https://raw.github.com/vrld/slam/master/slam.lua ↗

Example and Documentation

https://github.com/vrld/slam/blob/master/README.md ↗

Forum Post

http://love2d.org/forums/viewtopic.php?t=3804

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData

SoundData · revision 22730 · last edited 2025-09-29

Contains raw audio samples. You can not play SoundData back directly. You must wrap a Source object around it.

Constructors

Decoder:decode	Decodes a chunk of audio data to a SoundData.	Added since 11.0
RecordingDevice:getData	Gets all recorded audio SoundData stored in the device's internal ring buffer.	Added since 11.0
love.sound.newSoundData	Creates a new SoundData.	

Functions

Data:clone	Creates a new copy of the Data object.	Added since 11.0	
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3	
Data:getPointer	Gets a pointer to the Data.		
Data:getSize	Gets the Data's size in bytes.		
Data:getString	Gets the full Data as a string.	Added since 0.9.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
SoundData:getBitDepth	Returns the number of bits per sample.	Added since 0.9.0	
SoundData:getBits	Returns the number of bits per sample.		Removed in 0.9.0
SoundData:getChannelCount	Returns the number of channels in the SoundData.	Added since 11.0	
SoundData:getChannels	Returns the number of channels in the stream.		Deprecated in 11.0
SoundData:getDuration	Gets the duration of the sound data.	Added since 0.9.0	
SoundData:getSample	Gets the value of the samplepoint at the specified position.		
SoundData:getSampleCount	Returns the sample count of the SoundData.	Added since 0.9.0	
SoundData:getSampleRate	Returns the sample rate of the SoundData.		
SoundData:setSample	Sets the sample at the specified position.		

Supertypes

- Data
- Object

See Also

- love.sound
- love.audio
- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:copyFrom

SoundData:copyFrom · revision 29447 · last edited 2025-01-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Copies the specified section of the given SoundData into this one.

Function

Synopsis

```
SoundData:copyFrom( sourcedata, sourcestart, samplecount, deststart )
```

Arguments

`SoundData sourcedata`
: The SoundData to copy from.

`number sourcestart`
: The first sample in the source SoundData to copy from.

`number samplecount`
: The total number of samples to copy.

`number deststart`
: The first sample in the destination SoundData to copy to.

Returns

Nothing.

Notes

Samples start from zero. If a SoundData has multiple channels, each sample contains data for all channels.

This function does not perform sample rate conversion. If the SoundDatas have different sample rate, the resulting audio may played slower or faster depending on the differences.

See Also

- SoundData
- SoundData:slice

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:getBitDepth

SoundData:getBitDepth · revision 12460 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from SoundData:getBits.

Returns the number of bits per sample.

Function

Synopsis

```
bitdepth = SoundData:getBitDepth( )
```

Arguments

None.

Returns

`number bitdepth`

Either 8, or 16.

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

SoundData:getBits

SoundData:getBits · revision 10883 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to SoundData:getBitDepth.

Returns the number of bits per sample.

Function

Synopsis

```
bits = SoundData:getBits( )
```

Arguments

None.

Returns

`number bits`

Either 8, or 16.

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

SoundData:getChannelCount

SoundData:getChannelCount · revision 22633 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from SoundData:getChannels.

Returns the number of channels in the SoundData.

Function

Synopsis

```
channels = SoundData:getChannelCount( )
```

Arguments

None.

Returns

`number channels`

1 for mono, 2 for stereo.

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:getChannels

SoundData:getChannels · revision 22109 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function has been renamed to SoundData:getChannelCount.

Returns the number of channels in the stream.

Function

Synopsis

```
channels = SoundData:getChannels( )
```

Arguments

None.

Returns

`number channels`

1 for mono, 2 for stereo.

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:getDuration

SoundData:getDuration · revision 10307 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the duration of the sound data.

Function

Synopsis

```
duration = SoundData:getDuration( )
```

Arguments

None.

Returns

`number duration`

The duration of the sound data in seconds.

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:getSample

SoundData:getSample · revision 22110 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Gets the value of the sample-point at the specified position. For stereo SoundData objects, the data from the left and right channels are interleaved in that order.

Function

Synopsis

```
sample = SoundData:getSample( i )
```

Arguments

number i
: An integer value specifying the position of the sample (starting at 0).

Returns

number sample
: The normalized samplepoint (range -1.0 to 1.0).

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Gets the value of a sample using an explicit sample index instead of interleaving them in the sample position parameter.

Synopsis

```
sample = SoundData:getSample( i, channel )
```

Arguments

number i
: An integer value specifying the position of the sample (starting at 0).

number channel
: The index of the channel to get within the given sample.

Returns

number sample
: The normalized samplepoint (range -1.0 to 1.0).

See Also

- SoundData
- SoundData:setSample
- SoundData:getSampleCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:getSampleCount

SoundData:getSampleCount · revision 16561 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Returns the number of samples per channel of the SoundData.

Function

Synopsis

```
count = SoundData:getSampleCount( )
```

Arguments

None.

Returns

`number count`

Total number of samples.

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:getSampleRate

SoundData:getSampleRate · revision 6187 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the sample rate of the SoundData.

Function

Synopsis

```
rate = SoundData:getSampleRate( )
```

Arguments

None.

Returns

`number rate`
: Number of samples per second.

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:setSample

SoundData:setSample · revision 27726 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Example: Generate notes with a sine wave
- See Also
- Other Languages

Sets the value of the sample-point at the specified position. For stereo SoundData objects, the data from the left and right channels are interleaved in that order.

Function

Synopsis

```
SoundData:setSample( i, sample )
```

Arguments

number i
: An integer value specifying the position of the sample (starting at 0).

number sample
: The normalized samplepoint (range -1.0 to 1.0).

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Sets the value of a sample using an explicit sample index instead of interleaving them in the sample position parameter.

Synopsis

```
SoundData:setSample( i, channel, sample )
```

Arguments

number i
: An integer value specifying the position of the sample (starting at 0).

number channel
: The index of the channel to set within the given sample (starting at 1). For stereo, 1 is left channel and 2 is right channel.

number sample
: The normalized samplepoint (range -1.0 to 1.0).

Returns

Nothing.

Example: Generate notes with a sine wave

```
local tau          =   math.pi * 2
local samplerate   =        44100 -- Hz
local bits         =           16 -- 8 bits results in very low quality sound
local channels     =            2 -- löve only supports mono or stereo
```

```lua
local buffercount  =             2 -- don't need too many buffers
local qsource      = love.audio.newQueueableSource(samplerate, bits, channels, buffercount)
local samplepoints = samplerate/10 -- how many numbers does one buffer hold
local buffer       = love.sound.newSoundData(samplepoints, samplerate, bits, channels)
local phase        =           0.0
local frequency    =        220.00 -- Hz
local amplitude    =           0.5 -- 50%


local function synth()
    phase = phase + (tau * frequency / samplerate)
    return math.sin(phase) * amplitude
end

function love.update(dt)
    while qsource:getFreeBufferCount() > 0 do
        for i = 0, samplepoints - 1 do
            buffer:setSample(i, synth())
        end
        qsource:queue(buffer)
        qsource:play()
    end
end

function love.keypressed(key)
    if key == 'k' then
        frequency = frequency * 2^(1/12)
    elseif key == 'j' then
        frequency = frequency * 2^(-1/12)
    end
end
```

See Also

- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundData:slice

SoundData:slice · revision 29269 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Creates a new copy of a section of this SoundData.

Function

Synopsis

```
data = SoundData:slice( start, samplecount )
```

Arguments

`number start`
: The first sample the given SoundData to copy from.

`number samplecount (-1)`
: The total number of samples to copy to the new SoundData. The default value of -1 copies all samples.

Returns

`SoundData data`
: The new SoundData containing a copy of the specified section of the original SoundData.

Notes

Samples start from zero. If a SoundData has multiple channels, each sample contains data for all channels.

See Also

- SoundData
- SoundData:copyFrom

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SoundManager

SoundManager · revision 19075 · last edited 2021-03-14

This is the simple sound manager that's used in bartbes and Nevon's game Volley Brawl.

```
Copyright (c) 2011, Bart van Strien and Tommy Brunn
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:
    * Redistributions of source code must retain the above copyright
      notice, this list of conditions and the following disclaimer.
    * Redistributions in binary form must reproduce the above copyright
      notice, this list of conditions and the following disclaimer in the
      documentation and/or other materials provided with the distribution.
    * Neither the name of the <organization> nor the
      names of its contributors may be used to endorse or promote products
      derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL <COPYRIGHT HOLDER> BE LIABLE FOR ANY
DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND
ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

soundmanager = {}
soundmanager.queue = {}
soundmanager.playlist = {}
soundmanager.currentsong = -1

local function shuffle(a, b)
   return math.random(1, 2) == 1
end

--do the magic
function soundmanager:play(sndData)
   --make a source out of the sound data
   local src = love.audio.newSource(sndData, "static")
   --put it in the queue
   table.insert(self.queue, src)
   --and play it
   love.audio.play(src)
end

--do the music magic
function soundmanager:playMusic(first, ...)
   --stop all currently playing music
   for i, v in ipairs(self.playlist) do
      love.audio.stop(v)
   end
   --decide if we were passed a table or a vararg,
   --and assemble the playlist
   if type(first) == "table" then
      self.playlist = first
   else
      self.playlist = {first, ...}
   end
   self.currentsong = 1
   --play
   love.audio.play(self.playlist[1])
end

--do some shufflin'
function soundmanager:shuffle(first, ...)
   local playlist
   if type(first) == "table" then
```

```
      playlist = first
   else
      playlist = {first, ...}
   end
   table.sort(playlist, shuffle)
   return unpack(playlist)
end

--update
function soundmanager:update(dt)
   --check which sounds in the queue have finished, and remove them
   local removelist = {}
   for i, v in ipairs(self.queue) do
      if v:isStopped() then
         table.insert(removelist, i)
      end
   end
   --we can't remove them in the loop, so use another loop
   for i, v in ipairs(removelist) do
      table.remove(self.queue, v-i+1)
   end
   --advance the playlist if necessary
   if self.currentsong ~= -1 and self.playlist and self.playlist[self.currentsong]:isStopped() then
      self.currentsong = self.currentsong + 1
      if self.currentsong > #self.playlist then
         self.currentsong = 1
      end
      love.audio.play(self.playlist[self.currentsong])
   end
end
```

Source

Source · revision 25623 · last edited 2025-09-28

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

A Source represents audio you can play back. You can do interesting things with Sources, like set the volume, pitch, and its position relative to the listener. Please note that positional audio only works for mono (i.e. non-stereo) sources.

The source is internally referenced as long as it is playing.

The Source controls (play/pause/stop) act according to the following state table.

	Playing	Paused
play()	No change	Play
stop()	Pause + Rewind	Rewind
pause()	Pause	No change

And for fans of flowcharts (note: omitted calls have no effect, stopping always rewinds).



Constructors

love.audio.newQueueableSource	Creates a new Source usable for real-time generated sound playback with Source:queue.	Added since 11.0
love.audio.newSource	Creates a new Source from a file, SoundData, or Decoder.	

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Source:clone	Creates an identical copy of the Source in the stopped state.	Added since 0.9.1	
Source:getActiveEffects	Gets a list of the Source's active effect names.	Added since 11.0	
Source:getAirAbsorption	Gets the amount of air absorption applied to the Source.	Added since 11.2	
Source:getAttenuationDistances	Gets the reference and maximum attenuation distances of the Source.	Added since 0.9.0	
Source:getChannelCount	Gets the number of channels in the Source.	Added since 11.0	
Source:getChannels	Gets the number of channels in the Source.	Added since 0.9.0	
Source:getCone	Gets the Source's directional volume cones.	Added since 0.9.0	
Source:getDirection	Gets the direction of the Source.	Added since 0.7.0	
Source:getDistance	Returns the reference and maximum distance of the source.	Added since 0.8.0	Removed in 0.9.0
Source:getDuration	Gets the duration of the Source.	Added since 0.10.0	
Source:getEffect	Retrieve filter settings associated to a specific effect.	Added since 11.0	
Source:getFilter	Gets the filter settings currently applied to the Source.	Added since 11.0	
Source:getFreeBufferCount	Gets the number of free buffer slots of a queueable Source.	Added since 11.0	
Source:getPitch	Gets the current pitch of the Source.		
Source:getPosition	Gets the position of the Source.	Added since 0.7.0	
Source:getRolloff	Returns the rolloff factor of the source.	Added since 0.8.0	
Source:getType	Gets the type of the Source.	Added since 0.10.0	
Source:getVelocity	Gets the velocity of the Source.	Added since 0.7.0	
Source:getVolume	Gets the current volume of the Source.		
Source:getVolumeLimits	Returns the volume limits of the source.	Added since 0.8.0	
Source:isLooping	Returns whether the Source will loop.		
Source:isPaused	Returns whether the Source is paused.	Added since 0.7.1	Removed in 11.0
Source:isPlaying	Returns whether the Source is playing.	Added since 0.9.0	
Source:isRelative	Gets whether the Source's position and direction are relative to the listener.	Added since 0.9.0	

Source:isStatic	Returns whether the Source is static.	Added since 0.7.0	Removed in 0.10.0
Source:isStopped	Returns whether the Source is stopped.		Removed in 11.0
Source:pause	Pauses a source.	Added since 0.7.0	
Source:play	Plays a source.	Added since 0.7.0	
Source:queue	Queues SoundData for playback in a queueable Source.	Added since 11.0	
Source:resume	Resumes a paused source.	Added since 0.7.0	Removed in 11.0
Source:rewind	Rewinds a source.	Added since 0.7.0	Removed in 11.0
Source:seek	Sets the currently playing position of the Source.	Added since 0.8.0	
Source:setAirAbsorption	Sets the amount of air absorption applied to the Source.	Added since 11.2	
Source:setAttenuationDistances	Sets the reference and maximum attenuation distances of the Source.	Added since 0.9.0	
Source:setCone	Sets the Source's directional volume cones.	Added since 0.9.0	
Source:setDirection	Sets the direction of the Source.	Added since 0.7.0	
Source:setDistance	Sets the reference and maximum distance of the source.	Added since 0.8.0	Removed in 0.9.0
Source:setEffect	Applies an audio effect to the Source.	Added since 11.0	
Source:setFilter	Sets a low-pass, high-pass, or band-pass filter to apply when playing the Source.	Added since 11.0	
Source:setLooping	Sets whether the Source should loop.		
Source:setPitch	Sets the pitch of the Source.		
Source:setPosition	Sets the position of the Source.	Added since 0.7.0	
Source:setRelative	Sets whether the Source's position and direction are relative to the listener.	Added since 0.9.0	
Source:setRolloff	Sets the rolloff factor.	Added since 0.8.0	
Source:setVelocity	Sets the velocity of the Source.	Added since 0.7.0	
Source:setVolume	Sets the current volume of the Source.		
Source:setVolumeLimits	Sets the volume limits of the source.	Added since 0.8.0	
Source:stop	Stops a source.	Added since 0.7.0	
Source:tell	Gets the currently playing position of the Source.	Added since 0.8.0	

Enums

FilterType	Types of filters for Sources.	Added since 11.0
SourceType	Types of audio sources.	

Supertypes

- Object

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:clone

Source:clone · revision 12783 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Creates an identical copy of the Source in the stopped state.

Static Sources will use significantly less memory and take much less time to be created if Source:clone is used to create them instead of love.audio.newSource, so this method should be preferred when making multiple Sources which play the same sound.

Function

Synopsis

```
source = Source:clone( )
```

Arguments

None.

Returns

`Source source`
: The new identical copy of this Source.

Notes

Cloned Sources inherit all the set-able state of the original Source, but they are initialized stopped.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getActiveEffects

Source:getActiveEffects · revision 24389 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets a list of the Source's active effect names.

Function

Synopsis

```
effects = Source:getActiveEffects()
```

Arguments

None.

Returns

`table effects`

A list of the source's active effect names.

See Also

- Source
- Source:setEffect
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Source:getAirAbsorption

Source:getAirAbsorption · revision 21881 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.2

This function is not supported in earlier versions.

Gets the amount of air absorption applied to the Source.

By default the value is set to 0 which means that air absorption effects are disabled. A value of 1 will apply high frequency attenuation to the Source at a rate of 0.05 dB per meter.

Function

Synopsis

```
amount = Source:getAirAbsorption( )
```

Arguments

None.

Returns

`number amount`

The amount of air absorption applied to the Source.

Notes

Audio air absorption functionality is not supported on iOS.

See Also

- Source
- Source:setAirAbsorption

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getAttenuationDistances

Source:getAttenuationDistances · revision 11888 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from Source:getDistance.

Gets the reference and maximum attenuation distances of the Source. The values, combined with the current DistanceModel, affect how the Source's volume attenuates based on distance from the listener.

Function

Synopsis

```
ref, max = Source:getAttenuationDistances( )
```

Arguments

None.

Returns

`number ref`

The current reference attenuation distance. If the current DistanceModel is clamped, this is the minimum distance before the Source is no longer attenuated.

`number max`

The current maximum attenuation distance.

See Also

- Source
- Source:setAttenuationDistances
- love.audio.setDistanceModel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getChannelCount

Source:getChannelCount · revision 22632 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from Source:getChannels.

Gets the number of channels in the Source. Only 1-channel (mono) Sources can use directional and positional effects.

Function

Synopsis

```
channels = Source:getChannelCount( )
```

Arguments

None.

Returns

`number channels`

1 for mono, 2 for stereo.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getChannels

Source:getChannels · revision 22123 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function has been renamed to Source:getChannelCount.

Gets the number of channels in the Source. Only 1-channel (mono) Sources can use directional and positional effects.

Function

Synopsis

```
channels = Source:getChannels( )
```

Arguments

None.

Returns

`number channels`

1 for mono, 2 for stereo.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getCone

Source:getCone · revision 11885 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the Source's directional volume cones. Together with Source:setDirection, the cone angles allow for the Source's volume to vary depending on its direction.

Function

Synopsis

```
innerAngle, outerAngle, outerVolume = Source:getCone( )
```

Arguments

None.

Returns

`number innerAngle`
: The inner angle from the Source's direction, in radians. The Source will play at normal volume if the listener is inside the cone defined by this angle.

`number outerAngle`
: The outer angle from the Source's direction, in radians. The Source will play at a volume between the normal and outer volumes, if the listener is in between the cones defined by the inner and outer angles.

`number outerVolume`
: The Source's volume when the listener is outside both the inner and outer cone angles.

See Also

- Source
- Source:setCone
- Source:setDirection

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getDirection

Source:getDirection · revision 5707 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets the direction of the Source.

Function

Synopsis

```
x, y, z = Source:getDirection( )
```

Arguments

None.

Returns

number x
: The X part of the direction vector.

number y
: The Y part of the direction vector.

number z
: The Z part of the direction vector.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getDistance

Source:getDistance · revision 11820 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to Source:getAttenuationDistances.

Returns the reference and maximum distance of the source.


There's a bug in 0.8.0 preventing this function from working

Function

Synopsis

```
ref, max = Source:getDistance( )
```

Arguments

None.

Returns

`number ref`
: The reference distance.

`number max`
: The maximum distance.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getDuration

Source:getDuration · revision 21009 · last edited 2023-09-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the duration of the Source. For streaming Sources it may not always be sample-accurate, and may return -1 if the duration cannot be determined at all.

Function

Synopsis

```
duration = Source:getDuration( unit )
```

Arguments

`TimeUnit unit ("seconds")`

The time unit for the return value.

Returns

`number duration`

The duration of the Source, or -1 if it cannot be determined.

See Also

- Source:tell
- Source:seek
- Source
- Decoder:getDuration
- SoundData:getDuration

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getEffect

Source:getEffect · revision 21918 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the filter settings associated to a specific effect.

This function returns nil if the effect was applied with no filter settings associated to it.

Function

Synopsis

```
filtersettings = Source:getEffect( name, filtersettings )
```

Arguments

`string name`
: The name of the effect.

`table filtersettings ({})`
: An optional empty table that will be filled with the filter settings.

Returns

`table filtersettings`
: The settings for the filter associated to this effect, or nil if the effect is not present in this Source or has no filter associated. The table has the following fields:

  `number volume`
  : The overall volume of the audio.

  `number highgain`
  : Volume of high-frequency audio. Only applies to low-pass and band-pass filters.

  `number lowgain`
  : Volume of low-frequency audio. Only applies to high-pass and band-pass filters.

See Also

- Source
- Source:setEffect
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getFilter

Source:getFilter · revision 21314 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the filter settings currently applied to the Source.

Function

Synopsis

```
settings = Source:getFilter( )
```

Arguments

None.

Returns

`table settings (nil)`

The filter settings to use for this Source, or nil if the Source has no active filter. The table has the following fields:

`FilterType type`

The type of filter to use.

`number volume`

The overall volume of the audio.

`number highgain`

Volume of high-frequency audio. Only applies to low-pass and band-pass filters.

`number lowgain`

Volume of low-frequency audio. Only applies to high-pass and band-pass filters.

See Also

- Source
- Source:setFilter
- Source:setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getFreeBufferCount

Source:getFreeBufferCount · revision 27358 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the number of free buffer slots in a queueable Source. If the queueable Source is playing, this value will increase up to the amount the Source was created with. If the queueable Source is stopped, it will process all of its internal buffers first, in which case this function will always return the amount it was created with.

Function

Synopsis

```
buffers = Source:getFreeBufferCount( )
```

Arguments

None.

Returns

`number buffers`

How many more SoundData objects can be queued up.

See Also

- Source
- love.audio.newQueueableSource

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getPitch

Source:getPitch · revision 5708 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the current pitch of the Source.

Function

Synopsis

```
pitch = Source:getPitch( )
```

Arguments

None.

Returns

`number pitch`
: The pitch, where 1.0 is normal.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getPosition

Source:getPosition · revision 5709 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets the position of the Source.

Function

Synopsis

```
x, y, z = Source:getPosition( )
```

Arguments

None.

Returns

number x
: The X position of the Source.

number y
: The Y position of the Source.

number z
: The Z position of the Source.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getRolloff

Source:getRolloff · revision 8238 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the rolloff factor of the source.

Function

Synopsis

```
rolloff = Source:getRolloff( )
```

Arguments

None.

Returns

`number rolloff`

The rolloff factor.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getType

Source:getType · revision 24224 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0
It has replaced Source:isStatic.

Gets the type of the Source.

Function

Synopsis

```
sourcetype = Source:getType( )
```

Arguments

None.

Returns

`SourceType sourcetype`
The type of the source.

See Also

- Source
- love.audio.newSource
- SourceType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getVelocity

Source:getVelocity · revision 5710 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets the velocity of the Source.

Function

Synopsis

```
x, y, z = Source:getVelocity( )
```

Arguments

None.

Returns

number x
: The X part of the velocity vector.

number y
: The Y part of the velocity vector.

number z
: The Z part of the velocity vector.

See Also

- Source
- Source:setVelocity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getVolume

Source:getVolume · revision 5711 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the current volume of the Source.

Function

Synopsis

```
volume = Source:getVolume( )
```

Arguments

None.

Returns

`number volume`
: The volume of the Source, where 1.0 is normal volume.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:getVolumeLimits

Source:getVolumeLimits · revision 8239 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the volume limits of the source.

Function

Synopsis

```
min, max = Source:getVolumeLimits( )
```

Arguments

None.

Returns

number min
: The minimum volume.

number max
: The maximum volume.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:isLooping

Source:isLooping · revision 5713 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns whether the Source will loop.

Function

Synopsis

```
loop = Source:isLooping( )
```

Arguments

None.

Returns

`boolean loop`

True if the Source will loop, false otherwise.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:isPaused

Source:isPaused · revision 28090 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.1 and removed in LÖVE 11.0
It has been replaced by Source:isPlaying.

Returns whether the Source is paused.

Function

Synopsis

```
paused = Source:isPaused( )
```

Arguments

None.

Returns

`boolean paused`
True if the Source is paused, false otherwise.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:isPlaying

Source:isPlaying · revision 13104 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Returns whether the Source is playing.

Function

Synopsis

```
playing = Source:isPlaying( )
```

Arguments

None.

Returns

`boolean playing`

True if the Source is playing, false otherwise.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:isRelative

Source:isRelative · revision 11847 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the Source's position, velocity, direction, and cone angles are relative to the listener.

Function

Synopsis

```
relative = Source:isRelative( )
```

Arguments

None.

Returns

`boolean relative`

True if the position, velocity, direction and cone angles are relative to the listener, false if they're absolute.

See Also

- Source
- Source:setRelative

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:isStatic

Source:isStatic · revision 15924 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.10.0

It has been replaced by Source:getType.

Returns whether the Source is static.

Function

Synopsis

```
static = Source:isStatic( )
```

Arguments

None.

Returns

`boolean static`

True if the Source is static, false otherwise.

See Also

- Source
- SourceType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:isStopped

Source:isStopped · revision 22100 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 11.0
It has been replaced by Source:isPlaying.

Returns whether the Source is stopped.

Function

Synopsis

```
stopped = Source:isStopped( )
```

Arguments

None.

Returns

`boolean stopped`
True if the Source is stopped, false otherwise.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:pause

Source:pause · revision 5717 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Pauses the Source.

Function

Synopsis

```
Source:pause()
```

Arguments

None.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:play

Source:play · revision 14405 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Starts playing the Source.

Function

Available since LÖVE 0.9.1

This variant is not supported in earlier versions.

Synopsis

```
success = Source:play()
```

Arguments

None.

Returns

`boolean success`

Whether the Source was able to successfully start playing.

Function

Removed in LÖVE 0.9.1

This variant is not supported in that and later versions.

Synopsis

```
Source:play()
```

Arguments

None.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:queue

Source:queue · revision 27442 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Queues SoundData for playback in a queueable Source.

This method requires the Source to be created via love.audio.newQueueableSource.

Function

Synopsis

```
success = Source:queue( soundData, length )
```

Arguments

SoundData soundData

The data to queue. The SoundData's sample rate, bit depth, and channel count must match the Source's.

number length (sounddata:getSize())

Length in bytes to queue.

Returns

boolean success

True if the data was successfully queued for playback, false if there were no available buffers to use for queueing.

Function

Synopsis

```
success = Source:queue( soundData, offset, length )
```

Arguments

SoundData soundData

The data to queue. The SoundData's sample rate, bit depth, and channel count must match the Source's.

number offset

Starting position in bytes to queue.

number length

Length in bytes to queue starting from specified offset.

Returns

boolean success

True if the data was successfully queued for playback, false if there were no available buffers to use for queueing.

Function

Synopsis

```
success = Source:queue( rawData, offset, length, sampleRate, bitDepth, channels )
```

Arguments

light userdata rawData
: A pointer returns from Data:getPointer

number offset
: Starting position in bytes to queue.

number length
: Length in bytes to queue starting from specified offset.

number sampleRate
: Sample rate.

number bitDepth
: Bit depth.

number channels
: Channel count.

Returns

boolean success
: True if the data was successfully queued for playback, false if there were no available buffers to use for queueing.

Notes

To convert sample unit to bytes, multiply the sample unit by `SoundData:getChannelCount() * SoundData:getBitDepth() / 8`.

See Also

- Source
- love.audio.newQueueableSource
- SourceType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:resume

Source:resume · revision 21362 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 11.0

Use Source:play instead.

Resumes a paused Source.

Function

Synopsis

```
Source:resume()
```

Arguments

None.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:rewind

Source:rewind · revision 21251 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 11.0

It has been replaced by Source:stop or Source:seek(0).

Rewinds a Source.

Function

Synopsis

```
Source:rewind()
```

Arguments

None.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:seek

Source:seek · revision 29126 · last edited 2024-03-16

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the currently playing position of the Source. With the queueable source type, this method can only work on the currently playing buffer.

Function

Synopsis

```
Source:seek( offset, unit )
```

Arguments

`number offset`
: The position to seek to.

`TimeUnit unit ("seconds")`
: The unit of the position value.

Returns

Nothing.

See Also

- Source:tell
- Source:getDuration
- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setAirAbsorption

Source:setAirAbsorption · revision 21880 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.2

This function is not supported in earlier versions.

Sets the amount of air absorption applied to the Source.

By default the value is set to 0 which means that air absorption effects are disabled. A value of 1 will apply high frequency attenuation to the Source at a rate of 0.05 dB per meter.

Air absorption can simulate sound transmission through foggy air, dry air, smoky atmosphere, etc. It can be used to simulate different atmospheric conditions within different locations in an area.

Function

Synopsis

```
Source:setAirAbsorption( amount )
```

Arguments

`number amount`

The amount of air absorption applied to the Source. Must be between 0 and 10.

Returns

Nothing.

Notes

Audio air absorption functionality is not supported on iOS.

See Also

- Source
- Source:getAirAbsorption

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setAttenuationDistances

Source:setAttenuationDistances · revision 13610 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from Source:setDistance.

Sets the reference and maximum attenuation distances of the Source. The parameters, combined with the current DistanceModel, affect how the Source's volume attenuates based on distance. Distance attenuation is only applicable to Sources based on mono (rather than stereo) audio.

Function

Synopsis

```
Source:setAttenuationDistances( ref, max )
```

Arguments

`number ref`
: The new reference attenuation distance. If the current DistanceModel is clamped, this is the minimum attenuation distance.

`number max`
: The new maximum attenuation distance.

Returns

Nothing.

See Also

- Source
- Source:getAttenuationDistances
- love.audio.setDistanceModel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setCone

Source:setCone · revision 11884 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the Source's directional volume cones. Together with Source:setDirection, the cone angles allow for the Source's volume to vary depending on its direction.

Function

Synopsis

```
Source:setCone( innerAngle, outerAngle, outerVolume )
```

Arguments

`number innerAngle`
: The inner angle from the Source's direction, in radians. The Source will play at normal volume if the listener is inside the cone defined by this angle.

`number outerAngle`
: The outer angle from the Source's direction, in radians. The Source will play at a volume between the normal and outer volumes, if the listener is in between the cones defined by the inner and outer angles.

`number outerVolume (0)`
: The Source's volume when the listener is outside both the inner and outer cone angles.

Returns

Nothing.

See Also

- Source
- Source:getCone
- Source:setDirection

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setDirection

Source:setDirection · revision 5724 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Sets the direction vector of the Source. A zero vector makes the source non-directional.

Function

Synopsis

```
Source:setDirection( x, y, z )
```

Arguments

number x
: The X part of the direction vector.

number y
: The Y part of the direction vector.

number z
: The Z part of the direction vector.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setDistance

Source:setDistance · revision 11821 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to Source:setAttenuationDistances.

Sets the reference and maximum distance of the source.

Function

Synopsis

```
Source:setDistance( ref, max )
```

Arguments

`number ref`
: The new reference distance.

`number max`
: The new maximum distance.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setEffect

Source:setEffect · revision 27970 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies an audio effect to the Source.

The effect must have been previously defined using love.audio.setEffect.

Function

Applies the given previously defined effect to this Source.

Synopsis

```
success = Source:setEffect( name, enable )
```

Arguments

`string name`
: The name of the effect previously set up with love.audio.setEffect.

`boolean enable (true)`
: If false and the given effect name was previously enabled on this Source, disables the effect.

Returns

`boolean success`
: Whether the effect was successfully applied to this Source.

Function

Applies the given previously defined effect to this Source, and applies a filter to the Source which affects the sound fed into the effect.

Synopsis

```
success = Source:setEffect( name, filtersettings )
```

Arguments

`string name`
: The name of the effect previously set up with love.audio.setEffect.

`table filtersettings`
: The filter settings to apply prior to the effect, with the following fields:

  `FilterType type`
  : The type of filter to use.

  `number volume`
  : The overall audio input volume for the effect. Must be between 0 and 1. (Does not affect the dry audio output for the source.)

  `number highgain`
  : Volume of high-frequency audio. Only applies to `lowpass` and `bandpass` filters. Must be between 0 and 1.

  `number lowgain`

Volume of low-frequency audio. Only applies to `highpass` and `bandpass` filters. Must be between 0 and 1.

Returns

`boolean success`

Whether the effect and filter were successfully applied to this Source.

Notes

Audio effect functionality is not supported on iOS.

Audio produced by effects are added on top of the normal dry sound from Sources. Source:setFilter can be used to change the volume of, or mute, the Source without affecting the effects.

While the cutoff frequency for the filter cannot be set directly, changing high/lowgain has the effect of altering the cutoff. See it explained in this thread (https://github.com/kcat/openal-soft/issues/164).

See Also

- Source
- Source:getEffect
- Source:setFilter
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setFilter

Source:setFilter · revision 27969 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Play music at half volume with a low-pass filter applied
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets a low-pass, high-pass, or band-pass filter to apply when playing the Source.

Function

Synopsis

```
success = Source:setFilter( settings )
```

Arguments

table settings

The filter settings to use for this Source, with the following fields:

FilterType type

The type of filter to use.

number volume

The overall volume of the audio. Must be between 0 and 1.

number highgain

Volume of high-frequency audio. Only applies to `lowpass` and `bandpass` filters. Must be between 0 and 1.

number lowgain

Volume of low-frequency audio. Only applies to `highpass` and `bandpass` filters. Must be between 0 and 1.

Returns

boolean success

Whether the filter was successfully applied to the Source.

Function

Disables filtering on this Source.

Synopsis

```
Source:setFilter( )
```

Arguments

None.

Returns

Nothing.

Examples

Play music at half volume with a low-pass filter applied

```
function love.load()
    local source = love.audio.newSource("music.ogg", "stream")
    source:setFilter{
        type     = "lowpass",
        volume   = .5,
        highgain = .4,
    }
    source:play()
end
```

Notes

Audio filter functionality is not supported on iOS.

While the cutoff frequency cannot be set directly, changing high/lowgain has the effect of altering the cutoff. See it explained in this thread (https://github.com/kcat/openal-soft/issues/164).

The filter does not affect any effects for the Source.

See Also

- Source
- Source:getFilter
- Source:setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setLooping

Source:setLooping · revision 30711 · last edited 2026-04-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Sets whether the Source should loop.

Function

Synopsis

```
Source:setLooping( loop )
```

Arguments

`boolean loop`
: True if the source should loop, false otherwise.

Returns

Nothing.

Examples

```
function love.load()
    music = love.audio.newSource("music_loop.wav", "stream")

    music:setLooping(true)
    music:play()
    -- Music will now play forever, until stopped or paused.
end
```

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setPitch

Source:setPitch · revision 18474 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Sets the pitch of the Source.

Function

Synopsis

```
Source:setPitch( pitch )
```

Arguments

number pitch
: Calculated with regard to 1 being the base pitch. Each reduction by 50 percent equals a pitch shift of -12 semitones (one octave reduction). Each doubling equals a pitch shift of 12 semitones (one octave increase). Zero is not a legal value.

Returns

Nothing.

Examples

```
function love.load()
    sound = love.audio.newSource("sound.wav")

    -- Note that this code, as-is, will set the pitch to 1.0, as per the last line, and that's how sound:play() will play it back.
    sound:setPitch(0.5) -- One octave lower
    sound:setPitch(2) -- One octave higher
    sound:setPitch(1) -- Reset to normal pitch
end
```

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setPosition

Source:setPosition · revision 13721 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Sets the position of the Source. Please note that this only works for mono (i.e. non-stereo) sound files!

Function

Synopsis

```
Source:setPosition( x, y, z )
```

Arguments

number x
: The X position of the Source.

number y
: The Y position of the Source.

number z
: The Z position of the Source.

Returns

Nothing.

See Also

- Source
- love.audio.setPosition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setRelative

Source:setRelative · revision 25256 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Absolute Source Demonstration
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets whether the Source's position, velocity, direction, and cone angles are relative to the listener, or absolute.

By default, all sources are absolute and therefore relative to the origin of love's coordinate system [0, 0, 0]. Only absolute sources are affected by the position of the listener. Please note that positional audio only works for mono (i.e. non-stereo) sources.

Function

Synopsis

```
Source:setRelative( enable )
```

Arguments

boolean enable (false)

True to make the position, velocity, direction and cone angles relative to the listener, false to make them absolute.

Returns

Nothing.

Absolute Source Demonstration

This example demonstrates how an absolute source is affected by the position of the listener.

```
local x, y, z = 0, 0, 0;
local snd;
local relative;

function love.load()
    snd = love.audio.newSource('beep.wav', 'static')
    snd:setLooping(true);
    snd:play();

    -- By default the sound is not relative.
    relative = snd:isRelative();
end

function love.keypressed(key)
    -- Move the listener via WASD.
    if key == 'a' then
        x = x - 1;
    elseif key == 'd' then
        x = x + 1;
    elseif key == 'w' then
        y = y - 1;
    elseif key == 's' then
        y = y + 1;
    end
    love.audio.setPosition(x, y, z);

    -- Toggle between a relative and absolute source.
    if key == 'r' then
        relative = snd:isRelative();
        snd:setRelative(not relative);
    end
```

```
end

function love.draw()
    love.graphics.print('Relative: ' .. tostring(relative), 20, 20);
    love.graphics.print('Listener: (x = ' .. x .. ', y = ' .. y .. ')', 20, 40);
end
```

See Also

- Source
- Source:isRelative

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setRolloff

Source:setRolloff · revision 16477 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the rolloff factor which affects the strength of the used distance attenuation.

Extended information and detailed formulas can be found in the chapter "3.4. Attenuation By Distance" of OpenAL 1.1 specification (https://www.openal.org/documentation/openal-1.1-specification.pdf).

Function

Synopsis

```
Source:setRolloff( rolloff )
```

Arguments

`number rolloff`
: The new rolloff factor.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setVelocity

Source:setVelocity · revision 7104 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Sets the velocity of the Source.

This does **not** change the position of the Source, but lets the application know how it has to calculate the doppler effect.

Function

Synopsis

```
Source:setVelocity( x, y, z )
```

Arguments

`number x`
: The X part of the velocity vector.

`number y`
: The Y part of the velocity vector.

`number z`
: The Z part of the velocity vector.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setVolume

Source:setVolume · revision 21456 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Sets the current volume of the Source.

Function

Synopsis

```
Source:setVolume( volume )
```

Arguments

number volume
: The volume for a Source, where 1.0 is normal volume. Volume cannot be raised above 1.0.

Returns

Nothing.

Examples

Make a sound quieter or completely silent.

```
function love.load()
    sound = love.audio.newSource("sound.wav")

    -- Note that this code, as-is, will set the volume to 1.0, as per the last line, and that's how sound:play() will play it back.
    sound:setVolume(0.5) -- 50% volume
    sound:setVolume(0) -- No sound
    sound:setVolume(1) -- Reset to maximum volume.
end
```

Set different volumes depending on the sound type.

```
function love.load()
    effect = love.audio.newSource("soundeffect.wav")
    music = love.audio.newSource("music.mp3")

    masterVolume = 0.5 -- Maximum volume for all sounds
    effectVolume = 0.75
    musicVolume = 1

    effect:setVolume(masterVolume * effectVolume)
    music:setVolume(masterVolume * musicVolume)
end
```

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:setVolumeLimits

Source:setVolumeLimits · revision 8242 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the volume limits of the source. The limits have to be numbers from 0 to 1.

Function

Synopsis

```
Source:setVolumeLimits( min, max )
```

Arguments

`number min`
: The minimum volume.

`number max`
: The maximum volume.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:stop

Source:stop · revision 12981 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Stops a Source.

Function

Synopsis

```
Source:stop()
```

Arguments

None.

Returns

Nothing.

See Also

- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Source:tell

Source:tell · revision 29127 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Gets the currently playing position of the Source. With the queueable source type, this method returns a value based only on the currently playing buffer.

Function

Synopsis

```
position = Source:tell( unit )
```

Arguments

`TimeUnit unit ("seconds")`

The type of unit for the return value.

Returns

`number position`

The currently playing position of the Source.

See Also

- Source:seek
- Source:getDuration
- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SourceTree

SourceTree · revision 11673 · last edited 2021-03-14

Use the full capability of Git and Mercurial in the SourceTree desktop app. Manage all your repositories, hosted or local, through SourceTree's simple interface.

Get SourceTree (http://w w w.sourcetreeapp.com/)

SourceType

SourceType · revision 26812 · last edited 2023-09-22

Types of audio sources.

A good rule of thumb is to use `stream` for music files and `static` for all short sound effects. Basically, you want to avoid loading large files into memory at once.

Constants

static
: The whole audio is decoded.

stream
: The audio is decoded in chunks when needed.

Available since LÖVE 11.0

queue
: The audio must be manually queued by the user.

See Also

- love.audio
- love.audio.play
- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Space

Space · revision 2798 · last edited 2021-03-14

Space is a small space shooter, risen from the ashes of SpaceFlight2D. Robin is the author.

Its development is mostly driven by polls (http://answ ers.polldaddy.com/robin-gvx) for the community.

Spookyard

Spookyard · revision 29483 · last edited 2025-02-26

Spookyard is a local 2D multiplayer computer game that has different and competitive dynamics for each arena, offering game modes that can vary within the topdown and sidescroll camera styles.

Available for free at steam: steam page (https://store.steampow ered.com/app/1222380/Spookyard/)

SpriteBatch

SpriteBatch · revision 23367 · last edited 2024-06-15

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Using a single image, draw any number of identical copies of the image using a single call to love.graphics.draw(). This can be used, for example, to draw repeating copies of a single background image with high performance.

A SpriteBatch can be even more useful when the underlying image is a texture atlas (http://en.w ikipedia.org/w iki/Texture_atlas) (a single image file containing many independent images); by adding Quads to the batch, different sub-images from within the atlas can be drawn.

Constructors

love.graphics.newSpriteBatch	Creates a new SpriteBatch.

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
SpriteBatch:add	Adds a sprite to the batch.		
SpriteBatch:addLayer	Adds a sprite to a batch created with an Array Texture.	Added since 11.0	
SpriteBatch:addq	Adds a Quad to the batch.		Removed in 0.9.0
SpriteBatch:attachAttribute	Attaches a per-vertex attribute from a Mesh onto this SpriteBatch, for use when drawing.	Added since 0.10.0	
SpriteBatch:bind	Binds the SpriteBatch to memory for more efficient updating.	Added since 0.8.0	Removed in 0.10.0
SpriteBatch:clear	Removes all sprites from the buffer.		
SpriteBatch:flush	Immediately sends all new and modified sprite data to the graphics card.	Added since 0.9.2	
SpriteBatch:getBufferSize	Gets the maximum number of sprites the SpriteBatch can hold.	Added since 0.9.0	
SpriteBatch:getColor	Gets the color that will be used for the next add and set operations.	Added since 0.9.0	
SpriteBatch:getCount	Gets the number of sprites currently in the SpriteBatch.	Added since 0.9.0	
SpriteBatch:getImage	Returns the image used by the SpriteBatch.	Added since 0.8.0	Removed in 0.10.0
SpriteBatch:getTexture	Gets the texture (Image or Canvas) used by the SpriteBatch.	Added since 0.9.1	
SpriteBatch:set	Changes a sprite in the batch.	Added since 0.8.0	
SpriteBatch:setBufferSize	Sets the maximum number of sprites the SpriteBatch can hold.	Added since 0.9.0	Removed in 11.0
SpriteBatch:setColor	Sets the color that will be used for the next add or set operations.	Added since 0.8.0	
SpriteBatch:setDrawRange	Restricts the drawn sprites in the SpriteBatch to a subset of the total.	Added since 11.0	
SpriteBatch:setImage	Replaces the image used for the sprites.	Added since 0.7.2	Removed in 0.10.0
SpriteBatch:setLayer	Changes a sprite previously added with add or addLayer, in a batch created with an Array Texture.	Added since 11.0	
SpriteBatch:setTexture	Sets the texture (Image or Canvas) used for the sprites in the batch.	Added since 0.9.1	
SpriteBatch:setq	Changes a sprite with a quad in the batch.	Added since 0.8.0	Removed in 0.9.0

SpriteBatch:unbind	Unbinds the SpriteBatch.	Added since 0.8.0	Removed in 0.10.0

Enums

BufferDataUsage	Usage hints for SpriteBatches, Meshes, and GraphicsBuffers to optimize data storage and access.	Added since 0.8.0

Supertypes

- Drawable
- Object

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:add

SpriteBatch:add · revision 31694 · last edited 2026-08-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Adds a sprite to the batch. Sprites are drawn in the order they are added.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
id = SpriteBatch:add( x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`number x`
: The position to draw the object (x-axis).

`number y`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shear factor (x-axis).

`number ky (0)`
: Shear factor (y-axis).

Returns

`number id`
: An identifier for the added sprite.

Function

Available since LÖVE 0.9.0

This variant has replaced SpriteBatch:addq.

Adds a Quad to the batch.

Synopsis

```
id = SpriteBatch:add( quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Quad quad`
: The Quad to add.

`number x`
: The position to draw the object (x-axis).

`number y`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shear factor (x-axis).

`number ky (0)`
: Shear factor (y-axis).

Returns

`number id`
: An identifier for the added sprite.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
id = SpriteBatch:add( transform )
```

Arguments

`Transform transform`
: Transformation object.

Returns

`number id`
: An identifier for the added sprite.

Function

Available since LÖVE 0.9.0

This variant has replaced SpriteBatch:addq.

Adds a Quad to the batch.

Synopsis

```
id = SpriteBatch:add( quad, transform )
```

Arguments

`Quad quad`
: The Quad to add.

`Transform transform`
: Transformation object.

Returns

`number id`
: An identifier for the added sprite.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
SpriteBatch:add( x, y, r, sx, sy, ox, oy )
```

Arguments

`number x`
: The position to draw the object (x-axis).

`number y`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

Returns

Nothing.

Notes

The returned `id`s range from 1 to the set buffer size. If the buffer size is exceeded the returned `id` will default to 0 and the sprite won't be drawn.

See Also

- SpriteBatch
- SpriteBatch:set

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:addLayer

SpriteBatch:addLayer · revision 22655 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Notes
- Examples
  - Draw multiple layers of an Array Image in a SpriteBatch
  - Use a custom shader
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Adds a sprite to a batch created with an Array Texture.

Function

Adds a layer of the SpriteBatch's Array Texture.

Synopsis

```
spriteindex = SpriteBatch:addLayer( layerindex, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

number layerindex
: The index of the layer to use for this sprite.

number x (0)
: The position to draw the sprite (x-axis).

number y (0)
: The position to draw the sprite (y-axis).

number r (0)
: Orientation (radians).

number sx (1)
: Scale factor (x-axis).

number sy (sx)
: Scale factor (y-axis).

number ox (0)
: Origin offset (x-axis).

number oy (0)
: Origin offset (y-axis).

number kx (0)
: Shearing factor (x-axis).

number ky (0)
: Shearing factor (y-axis).

Returns

number spriteindex

The index of the added sprite, for use with SpriteBatch:set or SpriteBatch:setLayer.

Function

Adds a layer of the SpriteBatch's Array Texture using the specified Quad.

Synopsis

```
spriteindex = SpriteBatch:addLayer( layerindex, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

number layerindex
The index of the layer to use for this sprite.

Quad quad
The subsection of the texture's layer to use when drawing the sprite.

number x (0)
The position to draw the sprite (x-axis).

number y (0)
The position to draw the sprite (y-axis).

number r (0)
Orientation (radians).

number sx (1)
Scale factor (x-axis).

number sy (sx)
Scale factor (y-axis).

number ox (0)
Origin offset (x-axis).

number oy (0)
Origin offset (y-axis).

number kx (0)
Shearing factor (x-axis).

number ky (0)
Shearing factor (y-axis).

Returns

number spriteindex
The index of the added sprite, for use with SpriteBatch:set or SpriteBatch:setLayer.

Notes

The specified layer index overrides any layer index set on the Quad via Quad:setLayer.

Function

Adds a layer of the SpriteBatch's Array Texture using the specified Transform.

Synopsis

```
spriteindex = SpriteBatch:addLayer( layerindex, transform )
```

Arguments

number layerindex
The index of the layer to use for this sprite.

Transform transform

A transform object.

Returns

number spriteindex
The index of the added sprite, for use with SpriteBatch:set or SpriteBatch:setLayer.

Function

Adds a layer of the SpriteBatch's Array Texture using the specified Quad and Transform.

Synopsis

```
spriteindex = SpriteBatch:addLayer( layerindex, quad, transform )
```

Arguments

number layerindex
The index of the layer to use for this sprite.

Quad quad
The subsection of the texture's layer to use when drawing the sprite.

Transform transform
A transform object.

Returns

number spriteindex
The index of the added sprite, for use with SpriteBatch:set or SpriteBatch:setLayer.

Notes

The specified layer index overrides any layer index set on the Quad via Quad:setLayer.

Notes

In order to use an Array Texture or other non-2D texture types as the main texture in a custom Shader, the void effect() variant must be used in the pixel shader, and MainTex must be declared as an ArrayImage or sampler2DArray like so: `uniform ArrayImage MainTex;`.

Examples

Draw multiple layers of an Array Image in a SpriteBatch

```
function love.load()
    local sprites = {"sprite1.png", "sprite2.png"}
    image = love.graphics.newArrayImage(sprites)

    batch = love.graphics.newSpriteBatch(image)
    batch:addLayer(1, 50, 50)
    batch:addLayer(2, 250, 50)
end

function love.draw()
    love.graphics.draw(batch)
end
```

Use a custom shader

```
shader = love.graphics.newShader[[
uniform ArrayImage MainTex;

void effect() {
    // Texel uses a third component of the texture coordinate for the layer index, when an Array Texture is passed in.
    // love sets up the texture coordinates to contain the layer index specified in love.graphics.drawLayer, when
```

```
    // rendering the Array Texture.
    love_PixelColor = Texel(MainTex, VaryingTexCoord.xyz) * VaryingColor;
}
]]

function love.load()
    local sprites = {"sprite1.png", "sprite2.png"}
    image = love.graphics.newArrayImage(sprites)

    batch = love.graphics.newSpriteBatch(image)
    batch:addLayer(1, 50, 50)
    batch:addLayer(2, 250, 50)
end

function love.draw()
    love.graphics.setShader(shader)
    love.graphics.draw(batch)
end
```

See Also

- SpriteBatch
- SpriteBatch:setLayer
- love.graphics.newArrayImage
- love.graphics.newCanvas
- love.graphics.newShader
- TextureType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:addq

SpriteBatch:addq · revision 11006 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0
It has been merged into SpriteBatch:add.

Adds a Quad to the batch.

Function

Available since LÖVE 0.8.0
This variant is not supported in earlier versions.

Synopsis

```
id = SpriteBatch:addq( quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Quad quad`
: The Quad to add.

`number x`
: The position to draw the object (x-axis).

`number y`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shear factor (x-axis).

`number ky (0)`
: Shear factor (y-axis).

Returns

`number id`
: An identifier for the added sprite.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
SpriteBatch:addq( quad, x, y, r, sx, sy, ox, oy )
```

Arguments

`Quad quad`
: The Quad to add.

`number x`
: The position to draw the object (x-axis).

`number y`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:add
- SpriteBatch:set
- SpriteBatch:setq

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:attachAttribute

SpriteBatch:attachAttribute · revision 20794 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Attaches a per-vertex attribute from a Mesh onto this SpriteBatch, for use when drawing. This can be combined with a Shader to augment a SpriteBatch with per-vertex or additional per-sprite information instead of just having per-sprite colors.

Each sprite in a SpriteBatch has 4 vertices in the following order: top-left, bottom-left, top-right, bottom-right. The index returned by SpriteBatch:add (and used by SpriteBatch:set) can used to determine the first vertex of a specific sprite with the formula `1 + 4 * ( id - 1 )`.

Function

Synopsis

```
SpriteBatch:attachAttribute( name, mesh )
```

Arguments

`string name`
: The name of the vertex attribute to attach.

`Mesh mesh`
: The Mesh to get the vertex attribute from.

Returns

Nothing.

Notes

If a Mesh wasn't created with a custom vertex format, it will have 3 vertex attributes named `VertexPosition`, `VertexTexCoord`, and `VertexColor`. If vertex attributes with those names are attached to the SpriteBatch, it will override the SpriteBatch's sprite positions, texture coordinates, and sprite colors, respectively.

Custom named attributes can be accessed in a vertex shader by declaring them as `attribute vec4 MyCustomAttributeName;` at the top-level of the vertex shader code. The name must match what was specified in the Mesh's vertex format and in the `name` argument of SpriteBatch:attachAttribute.

A Mesh must have at least 4 * SpriteBatch:getBufferSize vertices in order to be attachable to a SpriteBatch.

See Also

- SpriteBatch
- Mesh
- Mesh:getVertexFormat
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:bind

SpriteBatch:bind · revision 29024 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Updating a SpriteBatch with binding
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from SpriteBatch:lock.

Deprecated in LÖVE 0.9.2

It happens automatically since this version.

Removed in LÖVE 0.10.0

Use SpriteBatch:flush if absolutely necessary.

Binds the SpriteBatch to memory for more efficient updating.

Binding a SpriteBatch before updating its content can improve the performance as it doesn't push each update to the graphics card separately. Don't forget to unbind the SpriteBatch or the updates won't show up.

Function

Synopsis

```
SpriteBatch:bind( )
```

Arguments

None.

Returns

Nothing.

Examples

Updating a SpriteBatch with binding

```
function update_spritebatch(spritebatch)
    spritebatch:bind()

    for i = 1, 100 do
        add_tile(spritebatch)
    end

    spritebatch:unbind()
end
```

See Also

- SpriteBatch
- SpriteBatch:unbind

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:clear

SpriteBatch:clear · revision 5861 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removes all sprites from the buffer.

Function

Synopsis

```
SpriteBatch:clear( )
```

Arguments

None.

Returns

Nothing.

See Also

- SpriteBatch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:flush

SpriteBatch:flush · revision 13768 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Initialize a static spritebatch with sprites and immediately send the data to the graphics card
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Immediately sends all new and modified sprite data in the batch to the graphics card.

Normally it isn't necessary to call this method as love.graphics.draw(spritebatch, ...) will do it automatically if needed, but explicitly using SpriteBatch:flush gives more control over when the work happens.

If this method is used, it generally shouldn't be called more than once (at most) between love.graphics.draw(spritebatch, ...) calls.

Function

Synopsis

```
SpriteBatch:flush( )
```

Arguments

None.

Returns

Nothing.

Examples

Initialize a static spritebatch with sprites and immediately send the data to the graphics card

```
function love.load()
    image = love.graphics.newImage("tile.png")
    spritebatch = love.graphics.newSpriteBatch(image, 20 * 20, "static")

    for y = 1, 20 do
        for x = 1, 20 do
            spritebatch:add((x - 1) * 64, (y - 1) * 64)
        end
    end

    -- If we call SpriteBatch:flush now then it won't be called internally when the SpriteBatch is drawn for the first time.
    -- In other words, we're choosing to do the work in love.load instead of the first love.draw.
    spritebatch:flush()
end

function love.draw()
```

```
    love.graphics.draw(spritebatch, 0, 0)
end
```

See Also

- SpriteBatch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:getBufferSize

SpriteBatch:getBufferSize · revision 11470 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the maximum number of sprites the SpriteBatch can hold.

Function

Synopsis

```
size = SpriteBatch:getBufferSize( )
```

Arguments

None.

Returns

`number size`

The maximum number of sprites the batch can hold.

See Also

- SpriteBatch
- SpriteBatch:setBufferSize
- SpriteBatch:getCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:getColor

SpriteBatch:getColor · revision 21264 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This method is not supported in earlier versions.

Gets the color that will be used for the next add and set operations.

If no color has been set with SpriteBatch:setColor or the current SpriteBatch color has been cleared, this method will return nil.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
r, g, b, a = SpriteBatch:getColor( )
```

Arguments

None.

Returns

number r
: The red component (0-1).

number g
: The green component (0-1).

number b
: The blue component (0-1).

number a
: The alpha component (0-1).

See Also

- SpriteBatch
- SpriteBatch:setColor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:getCount

SpriteBatch:getCount · revision 11663 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the number of sprites currently in the SpriteBatch.

Function

Synopsis

```
count = SpriteBatch:getCount( )
```

Arguments

None.

Returns

`number count`

The number of sprites currently in the batch.

See Also

- SpriteBatch
- SpriteBatch:getBufferSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:getImage

SpriteBatch:getImage · revision 15739 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Removed in LÖVE 0.10.0

Use SpriteBatch:getTexture instead.

Returns the image used by the SpriteBatch.

Function

Synopsis

```
image = SpriteBatch:getImage( )
```

Arguments

None

Returns

`Image` **`image`**

The image for the sprites.

See Also

- SpriteBatch
- SpriteBatch:setImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

SpriteBatch:getTexture

SpriteBatch:getTexture · revision 12959 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets the texture (Image or Canvas) used by the SpriteBatch.

Function

Synopsis

```
texture = SpriteBatch:getTexture( )
```

Arguments

None.

Returns

`Texture texture`

The Image or Canvas used by the SpriteBatch.

See Also

- SpriteBatch
- SpriteBatch:setTexture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:lock

SpriteBatch:lock · revision 9022 · last edited 2021-03-14

This has been changed to SpriteBatch:bind

SpriteBatch:set

SpriteBatch:set · revision 23648 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Changes a sprite in the batch. This requires the sprite index returned by SpriteBatch:add or SpriteBatch:addLayer.

Function

Synopsis

```
SpriteBatch:set( spriteindex, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

number spriteindex
: The index of the sprite that will be changed.

number x
: The position to draw the object (x-axis).

number y
: The position to draw the object (y-axis).

number r (0)
: Orientation (radians).

number sx (1)
: Scale factor (x-axis).

number sy (sx)
: Scale factor (y-axis).

number ox (0)
: Origin offset (x-axis).

number oy (0)
: Origin offset (y-axis).

number kx (0)
: Shear factor (x-axis).

number ky (0)
: Shear factor (y-axis).

Returns

Nothing.

Function

Available since LÖVE 0.9.0

This variant has replaced SpriteBatch:setq.

Changes a sprite with a Quad in the batch. This requires the index returned by SpriteBatch:add or SpriteBatch:addLayer.

Synopsis

```
SpriteBatch:set( spriteindex, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

number spriteindex
: The index of the sprite that will be changed.

Quad quad
: The Quad used on the image of the batch.

number x
: The position to draw the object (x-axis).

number y
: The position to draw the object (y-axis).

number r (0)
: Orientation (radians).

number sx (1)
: Scale factor (x-axis).

number sy (sx)
: Scale factor (y-axis).

number ox (0)
: Origin offset (x-axis).

number oy (0)
: Origin offset (y-axis).

number kx (0)
: Shear factor (x-axis).

number ky (0)
: Shear factor (y-axis).

Returns

Nothing.

Notes

SpriteBatches do not support removing individual sprites. One can do a pseudo removal (instead of clearing and re-adding everything) by:

```
SpriteBatch:set(id, 0, 0, 0, 0, 0)
```

This makes all the sprite's vertices equal (because the x and y scales are 0), which prevents the GPU from fully processing the sprite when drawing the SpriteBatch.

See Also

- SpriteBatch
- SpriteBatch:add
- SpriteBatch:addLayer
- SpriteBatch:setLayer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:setBufferSize

SpriteBatch:setBufferSize · revision 21480 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 11.0

SpriteBatches now automatically grow when necessary.

Sets the maximum number of sprites the SpriteBatch can hold. Existing sprites in the batch (up to the new maximum) will *not* be cleared when this function is called.



This function can be slow if it is called every frame, such as from love.update or love.draw.

Function

Synopsis

```
SpriteBatch:setBufferSize( size )
```

Arguments

`number size`
: The new maximum number of sprites the batch can hold.

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:getBufferSize
- SpriteBatch:getCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:setColor

SpriteBatch:setColor · revision 31552 · last edited 2026-05-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the color that will be used for the next add and set operations. Calling the function without arguments will disable all per-sprite colors for the SpriteBatch.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

In version 0.9.2 and older, the global color set with love.graphics.setColor will not work on the SpriteBatch if any of the sprites has its own color.

Function

Synopsis

```
SpriteBatch:setColor( r, g, b, a )
```

Arguments

number r
: The amount of red.

number g
: The amount of green.

number b
: The amount of blue.

number a (1)
: The amount of alpha.

Returns

Nothing.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
SpriteBatch:setColor( rgba )
```

Arguments

table rgba

A numerical indexed table with the red, green, blue and alpha values as numbers. The alpha is optional and defaults to 1 if it is left out.

Returns

Nothing.

Function

Disables all per-sprite colors for this SpriteBatch.

Synopsis

```
SpriteBatch:setColor( )
```

Arguments

None.

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:getColor
- SpriteBatch:add
- SpriteBatch:set

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:setDrawRange

SpriteBatch:setDrawRange · revision 22640 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Restricts the drawn sprites in the SpriteBatch to a subset of the total.

Function

Synopsis

```
SpriteBatch:setDrawRange( start, count )
```

Arguments

`number start`
: The index of the first sprite to draw. Index 1 corresponds to the first sprite added with SpriteBatch:add.

`number count`
: The number of sprites to draw.

Returns

Nothing.

Function

Allows all sprites in the SpriteBatch to be drawn.

Synopsis

```
SpriteBatch:setDrawRange( )
```

Arguments

None.

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:getDrawRange
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:setImage

SpriteBatch:setImage · revision 15738 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.2

This method is not supported in earlier versions.

Removed in LÖVE 0.10.0

Use SpriteBatch:setTexture instead.

Replaces the image used for the sprites.

Function

Synopsis

```
SpriteBatch:setImage( image )
```

Arguments

`Image image`
: The new Image to use for the sprites.

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:getImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:setLayer

SpriteBatch:setLayer · revision 23524 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Changes a sprite previously added with add or addLayer, in a batch created with an Array Texture.

Function

Changes the sprite in the SpriteBatch.

Synopsis

```
SpriteBatch:setLayer( spriteindex, layerindex, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

number spriteindex
: The index of the existing sprite to replace.

number layerindex
: The index of the layer in the Array Texture to use for this sprite.

number x (0)
: The position to draw the sprite (x-axis).

number y (0)
: The position to draw the sprite (y-axis).

number r (0)
: Orientation (radians).

number sx (1)
: Scale factor (x-axis).

number sy (sx)
: Scale factor (y-axis).

number ox (0)
: Origin offset (x-axis).

number oy (0)
: Origin offset (y-axis).

number kx (0)
: Shearing factor (x-axis).

number ky (0)

Shearing factor (y-axis).

Returns

Nothing.

Function

Adds a layer of the SpriteBatch's Array Texture using the specified Quad.

Synopsis

```
SpriteBatch:setLayer( spriteindex, layerindex, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

number spriteindex
The index of the existing sprite to replace.

number layerindex
The index of the layer to use for this sprite.

Quad quad
The subsection of the texture's layer to use when drawing the sprite.

number x (0)
The position to draw the sprite (x-axis).

number y (0)
The position to draw the sprite (y-axis).

number r (0)
Orientation (radians).

number sx (1)
Scale factor (x-axis).

number sy (sx)
Scale factor (y-axis).

number ox (0)
Origin offset (x-axis).

number oy (0)
Origin offset (y-axis).

number kx (0)
Shearing factor (x-axis).

number ky (0)
Shearing factor (y-axis).

Returns

Nothing.

Notes

The specified layer index overrides any layer index set on the Quad via Quad:setLayer.

Function

Adds a layer of the SpriteBatch's Array Texture using the specified Transform.

Synopsis

```
SpriteBatch:setLayer( spriteindex, layerindex, transform )
```

Arguments

number spriteindex
The index of the existing sprite to replace.

`number layerindex`
: The index of the layer to use for the sprite.

`Transform transform`
: A transform object.

Returns

Nothing.

Function

Adds a layer of the SpriteBatch's Array Texture using the specified Quad and Transform.

Synopsis

```
SpriteBatch:setLayer( spriteindex, layerindex, quad, transform )
```

Arguments

`number spriteindex`
: The index of the existing sprite to replace.

`number layerindex`
: The index of the layer to use for the sprite.

`Quad quad`
: The subsection of the texture's layer to use when drawing the sprite.

`Transform transform`
: A transform object.

Returns

Nothing.

Notes

The specified layer index overrides any layer index set on the Quad via Quad:setLayer.

See Also

- SpriteBatch
- SpriteBatch:addLayer
- love.graphics.newArrayImage
- love.graphics.newCanvas
- love.graphics.newShader
- TextureType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:setq

SpriteBatch:setq · revision 11009 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been merged into SpriteBatch:set.

Changes a sprite with a quad in the batch. This requires the identifier returned by add and addq.

Function

Synopsis

```
SpriteBatch:setq( id, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`number id`
: The identifier of the sprite that will be changed.

`Quad quad`
: The quad used on the image of the batch.

`number x`
: The position to draw the object (x-axis).

`number y`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shear factor (x-axis).

`number ky (0)`
: Shear factor (y-axis).

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:add
- SpriteBatch:addq
- SpriteBatch:set

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:setTexture

SpriteBatch:setTexture · revision 12958 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Sets the texture (Image or Canvas) used for the sprites in the batch, when drawing.

Function

Synopsis

```
SpriteBatch:setTexture( texture )
```

Arguments

`Texture texture`
: The new Image or Canvas to use for the sprites in the batch.

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:getTexture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:unbind

SpriteBatch:unbind · revision 22045 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from SpriteBatch:unlock.

Deprecated in LÖVE 0.9.2

It happens automatically since this version.

Removed in LÖVE 0.10.0

This function is not supported in that and later versions.

Unbinds the SpriteBatch.

All SpriteBatch:bind calls should have a matching *SpriteBatch:unbind* after the batch has been updated.

Function

Synopsis

```
SpriteBatch:unbind( )
```

Arguments

None.

Returns

Nothing.

See Also

- SpriteBatch
- SpriteBatch:bind

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SpriteBatch:unlock

SpriteBatch:unlock · revision 9020 · last edited 2021-03-14

This has been changed to SpriteBatch:unbind

SQLite3

SQLite3 · revision 16880 · last edited 2023-07-05

SQLite3

A simple, hassle-free, SQLite3 database solution for Lua

No Configuration Required

SQLite3 runs without the need to move or copy dll/so files or to set up any configuration whatsoever...it just works!

Easy to Use

SQLite3 is very easy to use with LOVE. All that's needed to get it working is to require it.

```
require("PathToMyPlugins.sqlite3.sqlite3");
```

That's it, you're ready to use local databases.

Download and Documentation

https://github.com/CentauriSoldier/SQLite3-for-Lua ↗

View Discussion Thread

https://love2d.org/forums/viewtopic.php?f=5&t=82760

SquareWorld

SquareWorld · revision 28119 · last edited 2022-11-17

SquareWorld is a 2D sandbox game with a focus on exploration and building.

You can join other players and explore together dozens of unique planets.

Get SquareWorld!

https://luke100000.itch.io/squareworld ↗

https://store.steampowered.com/app/950810/SquareWorld_Unpixeled/ ↗

Screenshots

- 
SquareWorldScreensho
- 
SquareWorldScreensho
- 
SquareWorldScreensho
- 
SquareWorldScreensho

sstrict.lua

sstrict.lua · revision 29363 · last edited 2024-10-15

On this page

- Undefined and unused variables
- Redefinition of variable names
- Empty and unnecessary code blocks
- Constant conditions
- Too many values in assignment
- Duplicate variables, arguments and table fields
- Documentation and source code

Super Strict is a pure Lua library that finds undeclared variables and other minor mistakes in your source code through static analysis. You do not need to execute any code to find your mistakes because Super Strict will check the code during loading. Just require the "sstrict.lua" file and any subsequent calls to "require","dofile","loadfile" or "loadstring" will be checked through Super Strict.

```
require('sstrict.lua')
```

To exclude a specific Lua file from the being checked place the "--!strict" line at the top of your source code.

Undefined and unused variables

```
function foo()
  a = 5 -- undefined variable 'a'
end
function bar(a, b)
  local c = a + b -- unused variable 'c'
  return a + b
end
```

Redefinition of variable names

```
for i = 1, 10 do
  for i = 1, 10 do
    -- variable name 'i' redefinition
  end
end
```

Empty and unnecessary code blocks

```
for i = 1, 3 do
  -- empty code block error
end
function baz()
  local n = 5
  n = n - 1 -- unnecessary code block error
end
```

Constant conditions

```
if 5+5 > 11 then
  -- constant condition error in if/else statement
end
```

Too many values in assignment

```
a = 1, 2 -- too many values on the right-hand side in assignment
```

Duplicate variables, arguments and table fields

```
t =
{
  ['a'] = 100,
```

```
  a = 100 -- duplicate field 'a' in table constructor
}
for b, b in pairs(t) do
  -- duplicate lvariable 'b'
end
c, c = 1, 2 -- duplicate variable 'c'
function foo(d, d)
  -- duplicate argument 'd'
end
```

Documentation and source code

https://github.com/2dengine/sstrict.lua ↗

https://2dengine.com/doc/sstrict.html ↗

StackingSceneMgr

StackingSceneMgr · revision 25506 · last edited 2023-07-05

StackingSceneMgr is a simple and quick scene manager, scenes are just considered as lua files that can be loaded and unloaded with ease.

Designed to be easy to modify, and easy to use.

Learn more and get it at Gitlab.com (https://gitlab.com/V3X3D/love-libs/-/tree/master/StackingSceneMgr).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

StackType

StackType · revision 15836 · last edited 2026-04-19

Available since LÖVE 0.9.2

This enum is not supported in earlier versions.

Graphics state stack types used with love.graphics.push.

Constants

transform

The transformation stack (love.graphics.translate, love.graphics.rotate, etc.)

all

All love.graphics state, including transform state.

Notes

Aside from the graphics transform state, the `all` Stack Type constant affects the following:

- background color
- blend mode
- active canvas
- current color
- color mask
- default filter mode
- active font
- scissor rectangle
- active shader
- wireframe mode
- stencil test mode
- line join mode
- line style
- line width
- point size
- point style

See Also

- love.graphics
- love.graphics.push
- love.graphics.pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

stateful.lua

stateful.lua · revision 24470 · last edited 2023-07-05

Original forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=3756).

Latest version:

https://github.com/kikito/stateful.lua ↗

stateful.lua allows the addition of states to objects in Lua. An object can have different states, and behave differently depending on which state it is in.

Please refer to the forum post above or the latest version link for examples of use.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Stateswitcher

Stateswitcher · revision 24472 · last edited 2023-07-05

Simple state switcher class for Love2D / Lua that uses one file per state.

Download: Stateswitcher class (http://www.ambience.sk/love2d-a-state-switcher-class-lua/)

Code example:

```
state=require("stateswitcher")
state.switch("credits") -- switching to credits.lua
```

Code example with passing a value:

```
state=require("stateswitcher")
state.switch("credits;2;hello") -- passing 2 as the first value, "hello" as the second value
```

Other advantages:

- edit your code & save to see changes immediately after state switch (without leaving your program)

- reuse the functions from the previous states (to keep same behaviour) or redeclare the functions (to change behaviour)

- automate debugging - create a sequence of state switching with variables passed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

StencilAction

StencilAction · revision 15686 · last edited 2026-04-19

Available since LÖVE 0.10.0
This enum is not supported in earlier versions.

How a stencil function modifies the stencil values of pixels it touches.

Constants

replace
: The stencil value of a pixel will be replaced by the value specified in love.graphics.stencil, if any object touches the pixel.

increment
: The stencil value of a pixel will be incremented by 1 for each object that touches the pixel. If the stencil value reaches 255 it will stay at 255.

decrement
: The stencil value of a pixel will be decremented by 1 for each object that touches the pixel. If the stencil value reaches 0 it will stay at 0.

incrementwrap
: The stencil value of a pixel will be incremented by 1 for each object that touches the pixel. If a stencil value of 255 is incremented it will be set to 0.

decrementwrap
: The stencil value of a pixel will be decremented by 1 for each object that touches the pixel. If the stencil value of 0 is decremented it will be set to 255.

invert
: The stencil value of a pixel will be bitwise-inverted for each object that touches the pixel. If a stencil value of 0 is inverted it will become 255.

See Also

- love.graphics
- love.graphics.stencil
- love.graphics.setStencilTest

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

StreamType

StreamType · revision 29276 · last edited 2024-06-15

Available since LÖVE 12.0

This enum is not supported in earlier versions.

Types of streaming for use with streaming audio Sources or Decoders.

Constants

file

Encoded data is read from the file on-demand and decoded, during playback.

memory

The file's encoded contents are fully loaded into memory when the Source is first created, and only decoding happens on-demand during playback.

The **file** stream type uses less memory especially for bigger files, whereas the **memory** stream type uses less disk IO and file reading time during audio playback.

See Also

- love.audio
- Source
- Decoder
- love.audio.newSource

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

SUIT

SUIT · revision 24530 · last edited 2023-07-05

SUIT - The Simple User Interface Toolkit for LÖVE. Make games, not GUIs!

SUIT supports layouts and themes.

Widgets

- Buttons
- Image Buttons
- Labels
- Text input boxes
- Checkboxes
- Sliders

Examples

Input and a button demo

```
suit = require 'suit'

local input = {text = ""}

function love.update(dt)
    suit.layout.reset(100,100)

    suit.Input(input, suit.layout.row(200,30))
    suit.Label("Hello, "..input.text, {align = "left"}, suit.layout.row())

    suit.layout.row() -- padding of one cell
    if suit.Button("Close", suit.layout.row()).hit then
        love.event.quit()
    end
end

function love.draw()
    suit.core.draw()
end

function love.textinput(t)
    suit.core.textinput(t)
end

function love.keypressed(key)
    suit.core.keypressed(key)
end
```

Links

- Documentation (http://suit.readthedocs.org/en/latest/)
- Source code (https://github.com/vrld/SUIT)
- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=81522)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Sukippu

Sukippu · revision 16913 · last edited 2021-03-14



Defend your barrio by skipping the rope - a funny little retro game made with LÖVE 2D engine.

It is still in early development status but constantly updated.

- Get the game and follow the development here: github.com/humansarepuppies/sukippu (https://github.com/humansarepuppies/sukippu) (404 not found)
- Forum thread(Download here): https://love2d.org/forums/viewtopic.php?t=12078

SURVIVE

SURVIVE · revision 18344 · last edited 2021-03-14

An entry for Ludum Dare 24 - Steal biomass from enemy organisms to increase your race's dominance.

Download (.love) (http://you.mongle.me/love/survive.love)

Ludum Dare 24 Entry Page (http://w w w.ludumdare.com/compo/ludum-dare-24/?action=preview &uid=13822)

Swingers

Swingers · revision 13586 · last edited 2023-07-05

Swingers (http://love2d.org/forums/viewtopic.php?f=5&t=1098&p=10725&hilit=swingers#p10725) is a gesture library for LÖVE.

Functions

- swingers.start(button)

  Starts the subsystem, button being the button to listen for

- swingers.checkGesture()

  Check if there's a new gesture, ALWAYS call this before using a get function

- swingers.getGesture()

  Get a gesture string, uses u for up, d for down, l for left and right for right, so rdul would be right, down, up left (in that order)

- swingers.getExtGesture()

  Get an extended gesture table, contains n, ne, e, se, s, sw, w and nw, one table entry per movement.

- swingers.update()

  This function does all the magic, call this every frame

Download

The library (http://love2d.org/forums/download/file.php?id=1821)

A demo (http://love2d.org/forums/download/file.php?id=1822)

See Also

Bartbes, the creator and maintainer of Swingers

The forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=1098)

SystemTheme

SystemTheme · revision 30611 · last edited 2026-02-13

Available since LÖVE 12.0

This enum is not supported in earlier versions.

System themes.

Constants

light
: Light colored system theme.

dark
: Dark colored system theme.

unknown
: Unknown system theme.

See Also

- love.window
- love.window.getSystemTheme
- love.themechanged

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tactile

Tactile · revision 24422 · last edited 2023-07-05

Tactile is a flexible and straightforward input library for LÖVE to help you manage multiple input sources. Get the code on GitHub (https://github.com/tesselode/tactile).

```
Control = {
  Horizontal = tactile.newControl()
    :addAxis(tactile.gamepadAxis(1, 'leftx'))
    :addButtonPair(tactile.keys 'left', tactile.keys 'right'),
  Fire = tactile.newControl()
    :addButton(tactile.gamepadButtons(1, 'a'))
    :addButton(tactile.keys 'x')
}

function love.update(dt)
  Control.Horizontal:update()
  Control.Fire:update()

  player.x = player.x + player.speed * Control.Horizontal() * dt
  if Control.Fire:isDown() then
    player:shoot()
  end
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Talkies.lua

Talkies.lua · revision 25338 · last edited 2023-07-05

A simple messagebox system for LÖVE.

Its a rewrite and update of the no longer mantained Moan.lua.

```
Talkies.say("Title", "Hello world!")
```

Features

- Multiple choices prompt
- Typing effect + sounds
- Pauses
- UTF-8 support
- Message box icons
- Autowrapped text
- General theming and per-message theming

Links

- Release Thread (https://love2d.org/forums/viewtopic.php?f=5&t=84110)
- Repository (https://github.com/tanema/talkies)
- Old youtube demo (https://w w w.youtube.com/w atch?v=ctu9hI-NhfI)

Screenshots



Other Language

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Terus

Terus · revision 7843 · last edited 2021-03-14

The game



This is a recreation of the old arcade game Gyruss (http://en.wikipedia.org/wiki/Gyruss). It probably won't be finished due to lack of community interest. Features tube-shooter gameplay style, some awesome sprite recreations, Galaga-style enemy waves, an advanced enemy action system, and slow but beautiful explosion effects.

Forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=1845)

Download

Direct from Dropbox (http://dl.dropbox.com/u/3713769/web/Love/Terus/TerusAlpha.love)

TEsound

TEsound · revision 24402 · last edited 2024-05-23

On this page

- About
  - Download
  - Contact
- Setup
- FAQ
- Functions
- Playback Functions
- Sound Modification Functions
- Utility Functions
- Internal Functions
- Other Languages

About

TEsound is a sound manager for the Love2D framework. TEsound is intended to make it easier to use sounds and music in your games.

It's under the ZLIB license (http://www.opensource.org/licenses/zlib-license.php). If you use the library and your game ends up making a lot of money, its creators politely request that you consider sending a little bit of it their way.

Download

The original Dropbox link is dead; leaving for posterity and in case it comes back. Until it does, here's the GitHub link (https://github.com/drhayes/TESound).

Direct from Dropbox (http://dl.dropbox.com/u/3713769/web/Love/TLTools/TEsound.lua)

Contact

- Forum Thread (http://love2d.org/forums/viewtopic.php?f=5&t=2334)
- Ensayia - Ensayia@gmail.com
- Taehl - SelfMadeSpirit@gmail.com

Setup

Everything in the module is contained in the TEsound namespace, so don't worry about it messing with your global variables. Memory and channel management is automatic.

1. Put TEsound.lua in your game's folder
2. At the top of main.lua, add the line `require"TEsound"`
3. In love.update(), add the line `TEsound.cleanup()`

FAQ

Q) How do I just play a sound?

A) All you need to do is `TEsound.play(sound)`, where `sound` is either a SoundData or a string filepath to a sound file (like `"sounds/boom.ogg"`).

Q) I have three different jump sounds, how can I play one at random?

A) `TEsound.play(list)`, where `list` is a table like `{"jump1.ogg", "jump2.ogg", "jump3.ogg"}`.

Q) Can I make it constantly play randomized music?

A) Sure! `TEsound.playLooping(list)`. When one song ends, a new one from the list will automatically start playing.

Q) Now how do I stop the music? / What are sound tags?

A) The best way is to use TEsound's "tagging" feature. Simply put, TEsound lets you add one or more tags to each sound that's playing, and you can call various functions on all sounds with a given tag. So you could do this:

```
TEsound.playLooping("song1.ogg", "music")
TEsound.stop("music")
```

Sounds can have multiple tags if you desire (use a list: `TEsound.play(sound, {"something", "whatever", "lol"})`, and multiple sounds can share tags (if you call something like `TEsound.pitch("sfx", .5)`, all sounds with the "sfx" tag will immediately be low-pitched). Tags can also be unique, in case you want to work with a specific sound.

Q) What if I want to change the volume/pitch of all sounds?

A) That's what the special "all" tag is for. You don't need to give the tag to sounds, it's automatically applied to them. So if you wanted to cut the volume of everything in half, you just need to `TEsound.volume("all",.5)`.

Q) I want to let the player choose different volume levels for sound effects, music, and voice-acting. Can TEsound handle it?

A) Yep! You can set a volume multiplier for any tag with `TEsound.volume(tag, volume)`. Tag-volume changes take immediate effect (even on sounds that are already playing!). So you could use `TEsound.volume("voice", .75)`, and any sound with the "voice" tag would play at .75 volume. This is multiplied by the sound's own volume and what the "all" tag is set to - if you `TEsound.play("splat.ogg", "sfx", .5)`, and you've set the "sfx" and "all" tags to .6 and 1 volume, then the sound would play at .3 volume.

Q) How do I pronounce the name of your module? "Tee ee sound"?

A) That works, but I, personally, say "teh sound" ;)

Functions

Playback Functions

TEsound.play

```
TEsound.play(sound, tags, volume, pitch, func)
```

Plays a sound. Returns either the number of the channel the sound is playing in, or nil and an error message.

`table sound`
: Either a SoundData or a string filepath to a sound file (example: "sounds/jump.ogg"), or a list of these. If a list is used, a random sound from that list will be played.

`table tags ({})`
: One or more tags that can be used to identify a sound (Example, single: "music". Example, multiple: {"sfx", "gun", "player"}). Multiple sounds may share tags, or tags may be unique. If omitted, no tags are assigned. Passing a string is shorthand for passing a table with only one item.

`number volume (1)`
: A number between 0 and 1 which specifies how loud a sound is. If the sound has a tag which a volume has been specified for, it will multiply this number (ie., if you use the tag "sfx" and volume 0.5, and "sfx" has been volume-ed to 0.6, then the sound will be played at 0.3 volume).

`number pitch (1)`
: A number which specifies the speed/pitch the sound will be played it. If the sound has a tag which a pitch has been specified for, it will multiply this number.

`function func (no callback)`
: A function which will be called when the sound is finished playing (it's passed one parameter - a list with the sound's volume and pitch). If omitted, no function will be used.

TEsound.playLooping

```
TEsound.playLooping(sound, tags, n, volume, pitch)
```

Plays a sound which will repeat be repeated n times. If n isn't given, it will loop until stopped manually with TEsound.stop. Returns either the number of the channel the sound is playing in, or nil and an error message.

`table sound`
: See TEsound.play. If a list is used, each loop will randomly choose from the list. Good for randomized background music.

`table tags`
: See TEsound.play

`number n`
: The number of times the sound will be looped. If omitted, the sound will loop forever.

`number volume`
: See TEsound.play

`number pitch`
: See TEsound.play

Sound Modification Functions

TEsound.volume

```
TEsound.volume(channel, volume)
```

Sets the volume of channel or tag and its loops (if any).

number channel
: Determines which channel will be affected. Since sound channels aren't static, it's generally advisable to use the tag method (see below).

number volume (no change)
: See TEsound.play. If omitted, the sound's volume won't be changed (pointless in most cases).

TEsound.volume

```
TEsound.volume(tag, volume)
```

Sets the volume of channel or tag and its loops (if any).

string tag
: Determines which channel will be affected. All channels playing a sound with that tag will be affected. Since sound channels aren't static, it's generally advisable to use the tag method.

number volume (no change)
: See TEsound.play. If omitted, the sound's volume won't be changed (pointless in most cases).

TEsound.pitch

```
TEsound.pitch(channel, pitch)
```

Sets the pitch of channel or tag and its loops (if any).

number channel
: See TEsound.volume

number pitch (no change)
: See TEsound.play. If omitted, the sound's pitch won't be changed.

TEsound.pitch

```
TEsound.pitch(tag, pitch)
```

Sets the pitch of channel or tag and its loops (if any).

string tag
: See TEsound.volume

number pitch (no change)
: See TEsound.play. If omitted, the sound's pitch won't be changed.

TEsound.pause

```
TEsound.pause(channel)
```

Pauses a channel or tag. Use TEsound.resume to unpause.

number channel
: See TEsound.volume

TEsound.pause

```
TEsound.pause(tag)
```

Pauses a channel or tag. Use TEsound.resume to unpause.

string tag

See TEsound.volume

TEsound.resume

```
TEsound.resume(channel)
```

Resumes a channel or tag from a pause.

number channel
: See TEsound.volume

TEsound.resume

```
TEsound.resume(tag)
```

Resumes a channel or tag from a pause.

string tag
: See TEsound.volume

TEsound.stop

```
TEsound.stop(channel, finish)
```

Stops a sound channel or tag either immediately or when finished, and prevents it from looping.

number channel
: See TEsound.volume

boolean finish (false)
: If true, the sound will be allowed to finish playing instead of stopping immediately. This is mainly used to end a sound loop.

TEsound.stop

```
TEsound.stop(tag, finish)
```

Stops a sound channel or tag either immediately or when finished, and prevents it from looping.

string tag
: See TEsound.volume

boolean finish (false)
: If true, the sound will be allowed to finish playing instead of stopping immediately. This is mainly used to end a sound loop.

Utility Functions

TEsound.cleanup

```
TEsound.cleanup()
```

Cleans up finished sounds, freeing memory. It's highly recommended you call this in love.update(). If not called, memory and channels won't be freed, and sounds won't loop.

TEsound.volume

```
TEsound.volume(tag, volume)
```

Change the volume level for a specified tag. Volume changes take effect immediately and last until changed again. This is recommended for changing entire categories of sounds, so you can independently adjust the volume of all sound effects, music, etc. (but don't forget to tag your sounds appropriately). If a sound has multiple tags with set volumes, the first one (in the order of its tag list) will be used.

string tag
: The tag to set the volume of. There is a special tag called "all" which will multiply ALL sounds (in addition to a regular tag, if present) - use it as a master volume control. Sounds do not need to be assigned the "all" tag for this to happen.

number volume (optional)

See TEsound.play. If omitted, the tag will no longer affect the volume of matching sounds.

TEsound.tagPitch

```
TEsound.tagPitch(tag, volume)
```

Change the pitch for a specified tag. Changes take effect immediately and last until changed again. This is recommended for changing entire categories of sounds. If a sound has multiple tags with set pitches, the first one (in the order of its tag list) will be used.

`string tag`
: See TEsound.volume

`number pitch (optional)`
: See TEsound.play. If omitted, the tag will no longer affect the pitch of matching sounds.

Internal Functions

TEsound.findTag

```
TEsound.findTag(tag)
```

Returns a list of all sound channels with a given tag.

`string tag`
: The channels of all sounds with this tag will be returned.

TEsound.findVolume

```
TEsound.findVolume(tag)
```

Returns a volume level for a given tag or tags, or 1 if the tag(s)'s volume hasn't been set. If a list of tags is given, it will return the level of the first tag with a setting.

`string tag`
: Chooses the tag to check the volume of. Passing a table will check only the first item.

TEsound.findPitch

```
TEsound.findPitch(tag)
```

Returns a pitch level for a given tag or tags, or 1 if the tag(s)'s pitch hasn't been set.

`string tag`
: See TEsound.findVolume

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text

Text · revision 30604 · last edited 2026-04-12

Available since LÖVE 0.10.0

This type is not supported in earlier versions.

One or more precalculated strings of text stored in a Drawable object.

A Text can be useful to avoid recalculating text layout when the contents of one or more strings to be printed don't change across frames. It is similar to a SpriteBatch.


Text may appear blurry if it's rendered at non-integer pixel locations.

Constructors

love.graphics.newText	Creates a new drawable Text object.	Added since 0.10.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
Text:add	Adds additional colored text to the Text object at the specified position.	Added since 0.10.0
Text:addf	Adds additional formatted / colored text to the Text object at the specified position.	Added since 0.10.0
Text:clear	Clears the contents of the Text object.	Added since 0.10.0
Text:getDimensions	Gets the width and height of the text.	Added since 0.10.1
Text:getFont	Gets the Font used with the Text object.	Added since 0.10.0
Text:getHeight	Gets the height of the text.	Added since 0.10.0
Text:getWidth	Gets the width of the text.	Added since 0.10.0
Text:set	Replaces the contents of the Text object with a new string.	Added since 0.10.0
Text:setFont	Replaces the Font used with the text.	Added since 0.10.0
Text:setf	Replaces the contents of the Text object with a new formatted string.	Added since 0.10.0

Supertypes

- Drawable
- Object

See Also

- love.graphics
- Font
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

text.lua

text.lua · revision 10492 · last edited 2023-07-05

This library is for printing pretty texts. The basic premise is to substitute tags in strings with a custom text object used for drawing.

Github Repo (https://github.com/markandgo/text.lua) Thread (https://love2d.org/forums/viewtopic.php?f=5&t=33179)

Text:add

Text:add · revision 20435 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Adds additional colored text to the Text object at the specified position.


Text may appear blurry if it's rendered at non-integer pixel locations.

Function

Synopsis

```
index = Text:add( textstring, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

`string textstring`
: The text to add to the object.

`number x (0)`
: The position of the new text on the x-axis.

`number y (0)`
: The position of the new text on the y-axis.

`number angle (0)`
: The orientation of the new text in radians.

`number sx (1)`
: Scale factor on the x-axis.

`number sy (sx)`
: Scale factor on the y-axis.

`number ox (0)`
: Origin offset on the x-axis.

`number oy (0)`
: Origin offset on the y-axis.

`number kx (0)`
: Shearing / skew factor on the x-axis.

`number ky (0)`
: Shearing / skew factor on the y-axis.

Returns

`number index`

An index number that can be used with Text:getWidth or Text:getHeight.

Function

Synopsis

```
index = Text:add( coloredtext, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext
A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1
A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1
A string of text which has a color specified by the previous color.

table color2
A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2
A string of text which has a color specified by the previous color.

tables and strings ...
Additional colors and strings.

number x (0)
The position of the new text on the x-axis.

number y (0)
The position of the new text on the y-axis.

number angle (0)
The orientation of the new text in radians.

number sx (1)
Scale factor on the x-axis.

number sy (sx)
Scale factor on the y-axis.

number ox (0)
Origin offset on the x-axis.

number oy (0)
Origin offset on the y-axis.

number kx (0)
Shearing / skew factor on the x-axis.

number ky (0)
Shearing / skew factor on the y-axis.

Returns

number index
An index number that can be used with Text:getWidth or Text:getHeight.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the Text object.

See Also

- Text
- Text:addf
- Text:set
- Text:setf

- Text:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:addf

Text:addf · revision 22713 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Adds additional formatted / colored text to the Text object at the specified position.

The word wrap limit is applied before any scaling, rotation, and other coordinate transformations. Therefore the amount of text per line stays constant given the same wrap limit, even if the scale arguments change.



Text may appear blurry if it's rendered at non-integer pixel locations.

Function

Synopsis

```
index = Text:addf( textstring, wraplimit, alignmode, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

`string textstring`
: The text to add to the object.

`number wraplimit`
: The maximum width in pixels of the text before it gets automatically wrapped to a new line.

`AlignMode align`
: The alignment of the text.

`number x`
: The position of the new text (x-axis).

`number y`
: The position of the new text (y-axis).

`number angle (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shearing / skew factor (x-axis).

`number ky (0)`
: Shearing / skew factor (y-axis).

Returns

`number index`

An index number that can be used with Text:getWidth or Text:getHeight.

Function

Synopsis

```
index = Text:addf( coloredtext, wraplimit, alignmode, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

number wraplimit

The maximum width in pixels of the text before it gets automatically wrapped to a new line.

AlignMode align

The alignment of the text.

number x

The position of the new text (x-axis).

number y

The position of the new text (y-axis).

number angle (0)

Orientation (radians).

number sx (1)

Scale factor (x-axis).

number sy (sx)

Scale factor (y-axis).

number ox (0)

Origin offset (x-axis).

number oy (0)

Origin offset (y-axis).

number kx (0)

Shearing / skew factor (x-axis).

number ky (0)

Shearing / skew factor (y-axis).

Returns

number index

An index number that can be used with Text:getWidth or Text:getHeight.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the Text object.

See Also

- Text
- Text:add
- Text:set
- Text:setf
- Text:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:clear

Text:clear · revision 15511 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Clears the contents of the Text object.

Function

Synopsis

```
Text:clear( )
```

Arguments

None.

Returns

Nothing.

See Also

- Text

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:getDimensions

Text:getDimensions · revision 16184 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.1

This function is not supported in earlier versions.

Gets the width and height of the text in pixels.

Function

Synopsis

```
width, height = Text:getDimensions( )
```

Arguments

None.

Returns

`number width`
: The width of the text. If multiple sub-strings have been added with Text:add, the width of the last sub-string is returned.

`number height`
: The height of the text. If multiple sub-strings have been added with Text:add, the height of the last sub-string is returned.

Function

Gets the width and height of a specific sub-string that was previously added to the Text object.

Synopsis

```
width, height = Text:getDimensions( index )
```

Arguments

`number index`
: An index number returned by Text:add or Text:addf.

Returns

`number width`
: The width of the sub-string (before scaling and other transformations).

`number height`
: The height of the sub-string (before scaling and other transformations).

See Also

- Text
- Text:set
- Text:setf
- Text:add

- Text:addf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:getFont

Text:getFont · revision 15564 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the Font used with the Text object.

Function

Synopsis

```
font = Text:getFont( )
```

Arguments

None.

Returns

`Font font`

The font used with this Text object.

See Also

- Text
- Text:setFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:getHeight

Text:getHeight · revision 25258 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the height of the text in pixels.

Function

Synopsis

```
height = Text:getHeight( )
```

Arguments

None.

Returns

number height
: The height of the text. If multiple sub-strings have been added with Text:add, the height of the last sub-string is returned.

Function

Gets the height of a specific sub-string that was previously added to the Text object.

Synopsis

```
height = Text:getHeight( index )
```

Arguments

number index
: An index number returned by Text:add or Text:addf.

Returns

number height
: The height of the sub-string (before scaling and other transformations).

See Also

- Text
- Text:getWidth
- Text:set
- Text:setf
- Text:add

- Text:addf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:getWidth

Text:getWidth · revision 15567 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the width of the text in pixels.

Function

Synopsis

```
width = Text:getWidth( )
```

Arguments

None.

Returns

`number width`
: The width of the text. If multiple sub-strings have been added with Text:add, the width of the last sub-string is returned.

Function

Gets the width of a specific sub-string that was previously added to the Text object.

Synopsis

```
width = Text:getWidth( index )
```

Arguments

`number index`
: An index number returned by Text:add or Text:addf.

Returns

`number width`
: The width of the sub-string (before scaling and other transformations).

See Also

- Text
- Text:getHeight
- Text:set
- Text:setf
- Text:add

- Text:addf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:set

Text:set · revision 21253 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Replaces the contents of the Text object with a new unformatted string.

Function

Synopsis

```
Text:set( textstring )
```

Arguments

`string textstring`
: The new string of text to use.

Returns

Nothing.

Function

Synopsis

```
Text:set( coloredtext )
```

Arguments

`table coloredtext`
: A table containing colors and strings to use as the new text, in the form of `{color1, string1, color2, string2, ...}`.

  `table color1`
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  `string string1`
  : A string of text which has a color specified by the previous color.

  `table color2`
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  `string string2`
  : A string of text which has a color specified by the previous color.

  `tables and strings ...`

Additional colors and strings.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the Text object.

Function

Removed in LÖVE 11.0
Use Text:clear instead.

Clears the contents of the Text object.

Synopsis

```
Text:set( )
```

Arguments

None.

Returns

Nothing.

See Also

- Text
- Text:setf
- Text:add
- Text:addf
- Text:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:setf

Text:setf · revision 20741 · last edited 2026-04-12

On this page
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0
This function is not supported in earlier versions.

Replaces the contents of the Text object with a new formatted string.

Function

Synopsis

```
Text:setf( textstring, wraplimit, alignmode )
```

Arguments

`string textstring`
The new string of text to use.

`number wraplimit`
The maximum width in pixels of the text before it gets automatically wrapped to a new line.

`AlignMode align`
The alignment of the text.

Returns

Nothing.

Function

Synopsis

```
Text:setf( coloredtext, wraplimit, alignmode )
```

Arguments

`table coloredtext`
A table containing colors and strings to use as the new text, in the form of `{color1, string1, color2, string2, ...}`.

- **`table color1`**
  A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.
- **`string string1`**
  A string of text which has a color specified by the previous color.
- **`table color2`**
  A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.
- **`string string2`**
  A string of text which has a color specified by the previous color.
- **`tables and strings ...`**
  Additional colors and strings.

`number wraplimit`
The maximum width in pixels of the text before it gets automatically wrapped to a new line.

`AlignMode align`
: The alignment of the text.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the Text object.

See Also

- Text
- Text:set
- Text:add
- Text:addf
- Text:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Text:setFont

Text:setFont · revision 15547 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Replaces the Font used with the text.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. Create multiple Text objects with different Fonts if you need to rapidly switch fonts.

Function

Synopsis

```
Text:setFont( font )
```

Arguments

`Font font`
: The new font to use with this Text object.

Returns

Nothing.

See Also

- Text
- Text:getFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch

TextBatch · revision 29236 · last edited 2024-06-11

Available since LÖVE 12.0

This type has been renamed from Text.

One or more precalculated strings of text stored in a Drawable object.

A TextBatch can be useful to avoid recalculating text layout when the contents of one or more strings to be printed don't change across frames. It is similar to a SpriteBatch.


Text may appear blurry if it's rendered at non-integer pixel locations.

Constructors

love.graphics.newTextBatch	Creates a new drawable TextBatch object.	Added since 12.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
TextBatch:add	Adds additional colored text to the TextBatch object at the specified position.	Added since 12.0
TextBatch:addf	Adds additional formatted / colored text to the TextBatch object at the specified position.	Added since 12.0
TextBatch:clear	Clears the contents of the TextBatch object.	Added since 12.0
TextBatch:getDimensions	Gets the width and height of a given string within the TextBatch.	Added since 12.0
TextBatch:getFont	Gets the Font used with the TextBatch object.	Added since 12.0
TextBatch:getHeight	Gets the height of a given string within the TextBatch.	Added since 12.0
TextBatch:getWidth	Gets the width of a given string within the TextBatch.	Added since 12.0
TextBatch:set	Replaces the contents of the TextBatch object with a new string.	Added since 12.0
TextBatch:setFont	Replaces the Font used with the TextBatch.	Added since 12.0
TextBatch:setf	Replaces the contents of the TextBatch object with a new formatted string.	Added since 12.0

Supertypes

- Drawable
- Object

See Also

- love.graphics
- Font
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:add

TextBatch:add · revision 30572 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:add.

Adds additional colored text to the TextBatch object at the specified position.



Text may appear blurry if it's rendered at non-integer pixel locations.

Function

Synopsis

```
index = TextBatch:add( textstring, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

`string textstring`
: The text to add to the object.

`number x (0)`
: The position of the new text on the x-axis.

`number y (0)`
: The position of the new text on the y-axis.

`number angle (0)`
: The orientation of the new text in radians.

`number sx (1)`
: Scale factor on the x-axis.

`number sy (sx)`
: Scale factor on the y-axis.

`number ox (0)`
: Origin offset on the x-axis.

`number oy (0)`
: Origin offset on the y-axis.

`number kx (0)`
: Shearing / skew factor on the x-axis.

`number ky (0)`
: Shearing / skew factor on the y-axis.

Returns

`number index`

An index number that can be used with TextBatch:getWidth or TextBatch:getHeight.

Function

Synopsis

```
index = TextBatch:add( coloredtext, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext
A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1
A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1
A string of text which has a color specified by the previous color.

table color2
A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2
A string of text which has a color specified by the previous color.

tables and strings ...
Additional colors and strings.

number x (0)
The position of the new text on the x-axis.

number y (0)
The position of the new text on the y-axis.

number angle (0)
The orientation of the new text in radians.

number sx (1)
Scale factor on the x-axis.

number sy (sx)
Scale factor on the y-axis.

number ox (0)
Origin offset on the x-axis.

number oy (0)
Origin offset on the y-axis.

number kx (0)
Shearing / skew factor on the x-axis.

number ky (0)
Shearing / skew factor on the y-axis.

Returns

number index
An index number that can be used with TextBatch:getWidth or TextBatch:getHeight.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the TextBatch object.

See Also

- TextBatch
- TextBatch:addf
- TextBatch:set
- TextBatch:setf

- TextBatch:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:addf

TextBatch:addf · revision 30573 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:addf.

Adds additional formatted / colored text to the TextBatch object at the specified position.

The word wrap limit is applied before any scaling, rotation, and other coordinate transformations. Therefore the amount of text per line stays constant given the same wrap limit, even if the scale arguments change.



Text may appear blurry if it's rendered at non-integer pixel locations.

Function

Synopsis

```
index = TextBatch:addf( textstring, wraplimit, alignmode, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

`string textstring`
: The text to add to the object.

`number wraplimit`
: The maximum width in pixels of the text before it gets automatically wrapped to a new line.

`AlignMode align`
: The alignment of the text.

`number x`
: The position of the new text (x-axis).

`number y`
: The position of the new text (y-axis).

`number angle (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shearing / skew factor (x-axis).

`number ky (0)`
: Shearing / skew factor (y-axis).

Returns

`number index`

An index number that can be used with TextBatch:getWidth or TextBatch:getHeight.

Function

Synopsis

```
index = TextBatch:addf( coloredtext, wraplimit, alignmode, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

number wraplimit

The maximum width in pixels of the text before it gets automatically wrapped to a new line.

AlignMode align

The alignment of the text.

number x

The position of the new text (x-axis).

number y

The position of the new text (y-axis).

number angle (0)

Orientation (radians).

number sx (1)

Scale factor (x-axis).

number sy (sx)

Scale factor (y-axis).

number ox (0)

Origin offset (x-axis).

number oy (0)

Origin offset (y-axis).

number kx (0)

Shearing / skew factor (x-axis).

number ky (0)

Shearing / skew factor (y-axis).

Returns

number index

An index number that can be used with TextBatch:getWidth or TextBatch:getHeight.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the Text object.

See Also

- TextBatch
- TextBatch:add
- TextBatch:set
- TextBatch:setf
- TextBatch:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:clear

TextBatch:clear · revision 30574 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:clear.

Clears the contents of the TextBatch object.

Function

Synopsis

```
TextBatch:clear( )
```

Arguments

None.

Returns

Nothing.

See Also

- TextBatch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:getDimensions

TextBatch:getDimensions · revision 30575 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:getDimensions.

Gets the width and height of a given string within the TextBatch.

Function

Synopsis

```
width, height = TextBatch:getDimensions( )
```

Arguments

None.

Returns

`number width`
: The width of the text. If multiple sub-strings have been added with TextBatch:add, the width of the last sub-string is returned.

`number height`
: The height of the text. If multiple sub-strings have been added with TextBatch:add, the height of the last sub-string is returned.

Function

Gets the width and height of a specific sub-string that was previously added to the TextBatch object.

Synopsis

```
width, height = TextBatch:getDimensions( index )
```

Arguments

`number index`
: An index number returned by TextBatch:add or TextBatch:addf.

Returns

`number width`
: The width of the sub-string (before scaling and other transformations).

`number height`
: The height of the sub-string (before scaling and other transformations).

See Also

- TextBatch
- TextBatch:set
- TextBatch:setf
- TextBatch:add

- TextBatch:addf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:getFont

TextBatch:getFont · revision 30576 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:getFont.

Gets the Font used with the TextBatch object.

Function

Synopsis

```
font = TextBatch:getFont( )
```

Arguments

None.

Returns

`Font font`

The font used with this TextBatch object.

See Also

- TextBatch
- TextBatch:setFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:getHeight

TextBatch:getHeight · revision 30580 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:getHeight.

Gets the height of a given string within the TextBatch.

Function

Synopsis

```
height = TextBatch:getHeight( )
```

Arguments

None.

Returns

number height

The height of the text. If multiple sub-strings have been added with TextBatch:add, the height of the last sub-string is returned.

Function

Gets the height of a specific sub-string that was previously added to the TextBatch object.

Synopsis

```
height = TextBatch:getHeight( index )
```

Arguments

number index

An index number returned by TextBatch:add or TextBatch:addf.

Returns

number height

The height of the sub-string (before scaling and other transformations).

See Also

- TextBatch
- TextBatch:getWidth
- TextBatch:getDimensions
- TextBatch:set
- TextBatch:setf
- TextBatch:add

- TextBatch:addf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:getWidth

TextBatch:getWidth · revision 30581 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:getWidth.

Gets the width of a given string within the TextBatch.

Function

Synopsis

```
width = TextBatch:getWidth( )
```

Arguments

None.

Returns

`number width`
: The width of the text. If multiple sub-strings have been added with TextBatch:add, the width of the last sub-string is returned.

Function

Gets the width of a specific sub-string that was previously added to the TextBatch object.

Synopsis

```
width = TextBatch:getWidth( index )
```

Arguments

`number index`
: An index number returned by TextBatch:add or TextBatch:addf.

Returns

`number width`
: The width of the sub-string (before scaling and other transformations).

See Also

- TextBatch
- TextBatch:getHeight
- TextBatch:getDimensions
- TextBatch:set
- TextBatch:setf
- TextBatch:add

- TextBatch:addf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:set

TextBatch:set · revision 30578 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:set.

Replaces the contents of the TextBatch object with a new unformatted string.

Function

Synopsis

```
TextBatch:set( textstring )
```

Arguments

`string textstring`
: The new string of text to use.

Returns

Nothing.

Function

Synopsis

```
TextBatch:set( coloredtext )
```

Arguments

`table coloredtext`
: A table containing colors and strings to use as the new text, in the form of `{color1, string1, color2, string2, ...}`.

  `table color1`
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  `string string1`
  : A string of text which has a color specified by the previous color.

  `table color2`
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  `string string2`
  : A string of text which has a color specified by the previous color.

  `tables and strings ...`

Additional colors and strings.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the TextBatch object.

See Also

- TextBatch
- TextBatch:setf
- TextBatch:add
- TextBatch:addf
- TextBatch:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:setf

TextBatch:setf · revision 30579 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:setf.

Replaces the contents of the TextBatch object with a new formatted string.

Function

Synopsis

```
TextBatch:setf( textstring, wraplimit, alignmode )
```

Arguments

`string textstring`
: The new string of text to use.

`number wraplimit`
: The maximum width in pixels of the text before it gets automatically wrapped to a new line.

`AlignMode align`
: The alignment of the text.

Returns

Nothing.

Function

Synopsis

```
TextBatch:setf( coloredtext, wraplimit, alignmode )
```

Arguments

`table coloredtext`
: A table containing colors and strings to use as the new text, in the form of `{color1, string1, color2, string2, ...}`.

  `table color1`
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  `string string1`
  : A string of text which has a color specified by the previous color.

  `table color2`
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  `string string2`
  : A string of text which has a color specified by the previous color.

  `tables and strings ...`
  : Additional colors and strings.

`number wraplimit`
: The maximum width in pixels of the text before it gets automatically wrapped to a new line.

`AlignMode align`
: The alignment of the text.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the TextBatch object.

See Also

- TextBatch
- TextBatch:set
- TextBatch:add
- TextBatch:addf
- TextBatch:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextBatch:setFont

TextBatch:setFont · revision 30577 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text:setFont.

Replaces the Font used with the TextBatch.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. Create multiple TextBatch objects with different Fonts if you need to rapidly switch fonts.

Function

Synopsis

```
TextBatch:setFont( font )
```

Arguments

`Font font`
: The new font to use with this TextBatch object.

Returns

Nothing.

See Also

- TextBatch
- TextBatch:getFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextInputField

TextInputField · revision 30154 · last edited 2025-11-02

based on TextInput (https://github.com/darkfrei/love2d-lua-tests/tree/main/textinput)

Text Input Field

Minimal example for full featured text input field, key features:

1. Supports Unicode.

2. Supports multilines as "\n".

3. Supports new lines caused due too long strings.

4. Supports arrows to move cursor, also Home and End keys.

5. Supports text editing and erasing with Backspace and Delete.

```lua
-- minimal example of full featured text input field

local utf8 = require("utf8")

-- utf8-aware substring
function utf8.sub(s, i, j)
    if not s then return "" end
    local len = utf8.len(s) or 0
    i = i or 1
    j = j or len
    if i < 0 then i = len + i + 1 end
    if j < 0 then j = len + j + 1 end
    if i < 1 then i = 1 end
    if j > len then j = len end
    if i > j then return "" end
    local startByte = utf8.offset(s, i)
    local endByte = utf8.offset(s, j + 1)
    return string.sub(s, startByte, endByte and endByte - 1 or -1)
end

local font
local major, minor, revision, codename = love.getVersion()
--print (major, minor, revision, codename)
if major == 12 then
    love.graphics.newFont(30)
else
    -- see https://fonts.google.com/noto/specimen/Noto+Sans
    font = love.graphics.newFont("NotoSans-Regular.ttf", 30)
end
local lineHeight = font:getHeight()

-- all field state in one table
local field = {
    x = 100,
    y = 100,
    w = 400,
    h = 0, -- computed dynamically
    paddingX = 5,
    text = [[1. Really long strings are supported
2. And multiline
3. И юникод (and unicode)
4. Also all arrow keys, backspace, delete, Home, End with blinking cursor]],
    cursorPos = 0,
    desiredCursorX = nil
}

-- key repeat state
local keyRepeat = {
    key = nil,
    timer = 0,
    delay = 0.5,
    interval = 0.03
```

```
}



-- helper: build visual wrapped lines and absolute start indices
local function getWrappedLinesWithStarts(textStr, maxWidth)
    local pos = 0
    local lines, starts = {}, {}
    for rawLine in (textStr .. "\n"):gmatch("(.-)\n") do
        if rawLine == "" then
            table.insert(lines, "")
            table.insert(starts, pos)
        else
            local _, wrapped = font:getWrap(rawLine, maxWidth)
            for _, w in ipairs(wrapped) do
                table.insert(lines, w)
                table.insert(starts, pos)
                pos = pos + utf8.len(w)
            end
        end
        pos = pos + 1
    end
    if #textStr > 0 then pos = pos - 1 end
    return lines, starts
end


local function updateFieldHeight()
    local maxWidth = field.w - 2 * field.paddingX
    local totalLines = 0

    -- split text into raw lines by newline
    for rawLine in (field.text .. "\n"):gmatch("(.-)\n") do
        -- wrap each raw line into visual lines
        local _, wrapped = font:getWrap(rawLine, maxWidth)
        totalLines = totalLines + #wrapped
    end

    -- ensure at least 3 lines
    local lineCount = math.max(totalLines, 3)
    field.h = lineCount * lineHeight + 2 * field.paddingX
end




-- update desired x using wrapped lines and absolute starts
local function updateDesiredX()
    local maxWidth = field.w - 2 * field.paddingX
    local allLines, allStarts = getWrappedLinesWithStarts(field.text, maxWidth)

    -- find visual line index for current cursorPos
    local idx = 1
    for i = 1, #allLines do
        local startPos = allStarts[i]
        local len = utf8.len(allLines[i])
        if field.cursorPos >= startPos and field.cursorPos <= startPos + len then
            idx = i
            break
        end
        if i == #allLines and field.cursorPos > startPos + len then
            idx = i
        end
    end

    -- measure width from line start to cursor within that visual line
    local lineStartPos = allStarts[idx]
    local charsInLineToCursor = math.max(field.cursorPos - lineStartPos, 0)
    local lineText = allLines[idx] or ""
    local subToCursor = utf8.sub(lineText, 1, charsInLineToCursor)

    field.desiredCursorX = font:getWidth(subToCursor)
```

```lua
end


-- move cursor vertically, keeping closest x position across wrapped lines
local function moveCursorVertical(direction)
    local maxWidth = field.w - 2 * field.paddingX
    local allLines, allStarts = getWrappedLinesWithStarts(field.text, maxWidth)

    -- find current visual line index
    local currentIdx = 1
    for i = 1, #allLines do
        local startPos = allStarts[i]
        local len = utf8.len(allLines[i])
        if field.cursorPos >= startPos and field.cursorPos <= startPos + len then
            currentIdx = i
            break
        end
        if i == #allLines and field.cursorPos > startPos + len then
            currentIdx = i
        end
    end

    -- target line index
    local targetIdx = currentIdx + direction
    if targetIdx < 1 or targetIdx > #allLines then
        return
    end

    -- ensure desiredCursorX is set to the current visual x if nil
    if field.desiredCursorX == nil then
        local lineStartPos = allStarts[currentIdx]
        local charsInLineToCursor = math.max(field.cursorPos - lineStartPos, 0)
        local lineText = allLines[currentIdx] or ""
        local subToCursor = utf8.sub(lineText, 1, charsInLineToCursor)
        field.desiredCursorX = font:getWidth(subToCursor)
    end

    -- project desired x onto the target line by measuring character widths
    local targetText = allLines[targetIdx] or ""
    local targetStart = allStarts[targetIdx]
    local targetLen = utf8.len(targetText) or 0

    local newCursorPosInLine = 0
    local prevWidth = 0
    local found = false
    for i = 1, targetLen do
        local subtext = utf8.sub(targetText, 1, i)
        local w = font:getWidth(subtext)
        if w >= field.desiredCursorX then
            -- choose the closer of the two positions
            if math.abs(field.desiredCursorX - prevWidth) <= math.abs(w - field.desiredCursorX) then
                newCursorPosInLine = i - 1
            else
                newCursorPosInLine = i
            end
            found = true
            break
        end
        prevWidth = w
    end
    if not found then
        newCursorPosInLine = targetLen
    end

    field.cursorPos = targetStart + newCursorPosInLine
end


local function replaceText(startIdx, endIdx, replacement)
    local before = utf8.sub(field.text, 1, startIdx - 1)
    local after = utf8.sub(field.text, endIdx + 1)
    field.text = before .. (replacement or "") .. after
end
```

```lua
-- handle key input
local function handleKey(key)
    if key == "backspace" and field.cursorPos > 0 then
        replaceText(field.cursorPos, field.cursorPos, "")
        field.cursorPos = field.cursorPos - 1
        updateDesiredX()
    elseif key == "delete" and field.cursorPos < utf8.len(field.text) then
        replaceText(field.cursorPos + 1, field.cursorPos + 1, "")

    elseif key == "left" and field.cursorPos > 0 then
        field.cursorPos = field.cursorPos - 1
        updateDesiredX()

    elseif key == "right" and field.cursorPos < utf8.len(field.text) then
        field.cursorPos = field.cursorPos + 1
        updateDesiredX()

    elseif key == "up" then
        moveCursorVertical(-1)

    elseif key == "down" then
        moveCursorVertical(1)

    elseif key == "return" then
        replaceText(field.cursorPos + 1, field.cursorPos, "\n")
        field.cursorPos = field.cursorPos + 1
        updateDesiredX()
        updateFieldHeight()

    elseif key == "home" then
        field.cursorPos = 0

    elseif key == "end" then
        field.cursorPos = utf8.len(field.text)
    end
end



-- separate function for rendering the cursor
local function drawCursor(field, font, lineHeight, maxWidth)

    -- blink speed in blinks per second
    local blinkSpeed = 3
    if math.floor(love.timer.getTime() * blinkSpeed) % 2 == 0 then
        -- get all visual lines and their start positions
        local allLines, allStarts = getWrappedLinesWithStarts(field.text, maxWidth)

        -- find the line where the cursor is located
        local cursorLineIndex = 1
        for i = 1, #allLines do
            local startPos = allStarts[i]
            local lineLen = utf8.len(allLines[i])
            if field.cursorPos >= startPos and field.cursorPos <= startPos + lineLen then
                cursorLineIndex = i
                break
            end
            if i == #allLines and field.cursorPos > startPos + lineLen then
                cursorLineIndex = i
            end
        end

        -- calculate offset inside the line
        local lineStartPos = allStarts[cursorLineIndex]
        local charsInLineToCursor = math.max(field.cursorPos - lineStartPos, 0)
        local lineText = allLines[cursorLineIndex] or ""
        local subToCursor = utf8.sub(lineText, 1, charsInLineToCursor)

        local cursorX = field.x + field.paddingX + font:getWidth(subToCursor)
        local cursorY = field.y + field.paddingX + (cursorLineIndex - 1) * lineHeight
```

```
        -- blinking cursor

        love.graphics.line(cursorX, cursorY, cursorX, cursorY + lineHeight)
    end
end




---------------------------
-- love2d -----------------
---------------------------


function love.load()
    -- enable text input
    love.keyboard.setTextInput(true)

    -- recalc field height
    updateFieldHeight()

    -- put cursor at the end of the text
    field.cursorPos = utf8.len(field.text)
    updateDesiredX()
end


function love.textinput(t)
    local before = utf8.sub(field.text, 1, field.cursorPos)
    local after = utf8.sub(field.text, field.cursorPos + 1)
    field.text = before .. t .. after
    field.cursorPos = field.cursorPos + utf8.len(t)
    updateDesiredX()
    updateFieldHeight()

end



function love.keypressed(key)
    if key == "escape" then
        love.event.quit()
        return
    end
    handleKey(key)
    keyRepeat.key = key
    keyRepeat.timer = 0
end

function love.keyreleased(key)
    if keyRepeat.key == key then
        keyRepeat.key = nil
        keyRepeat.timer = 0
    end
end

function love.update(dt)
    if keyRepeat.key then
        keyRepeat.timer = keyRepeat.timer + dt
        if keyRepeat.timer >= keyRepeat.delay then
            local elapsed = keyRepeat.timer - keyRepeat.delay
            local repeats = math.floor(elapsed / keyRepeat.interval)
            if repeats > 0 then
                handleKey(keyRepeat.key)
                keyRepeat.timer = keyRepeat.delay + (repeats * keyRepeat.interval)
            end
        end
    end
end


function love.draw()
    -- compute max text width inside the field
```

```
    local maxWidth = field.w - 2 * field.paddingX

    -- field background and border
    love.graphics.setColor(0.2, 0.2, 0.25)
    love.graphics.rectangle("fill", field.x, field.y, field.w, field.h)
    love.graphics.setColor(0.3, 0.5, 0.7)
    love.graphics.rectangle("line", field.x, field.y, field.w, field.h)

    -- draw text
    love.graphics.setColor(1, 1, 1)
    love.graphics.setFont(font)
    love.graphics.printf(field.text, field.x + field.paddingX, field.y + field.paddingX, maxWidth, "left")

    -- draw cursor
    drawCursor(field, font, lineHeight, maxWidth)

    -- debug info
    love.graphics.print(
        "CursorPos: " .. field.cursorPos ..
        "; TextLen: " .. utf8.len(field.text) ..
        "; desiredCursorX: " .. tostring(field.desiredCursorX),
        10, 10
    )
end
```

Texture

Texture · revision 23220 · last edited 2026-03-22

On this page

- Functions
- Supertypes
- Subtypes
- Enums
- See Also
- Other Languages

Available since LÖVE 0.9.1

This type is not supported in earlier versions, but the methods were still available on Canvas and Image objects.

Superclass for drawable objects which represent a texture. All Textures can be drawn with Quads. This is an abstract type that can't be created directly.

Functions

Texture:getDPIScale	Gets the DPI scale factor of the Texture.	Added since 11.0
Texture:getDepth	Gets the depth of a Volume Texture.	Added since 11.0
Texture:getDepthSampleMode	Gets the comparison mode used when sampling from a depth texture in a shader.	Added since 11.0
Texture:getDimensions	Gets the width and height of the Texture.	Added since 0.9.0
Texture:getFilter	Gets the filter mode of the Texture.	
Texture:getFormat	Gets the pixel format of the Texture.	Added since 11.0
Texture:getHeight	Gets the height of the Texture.	
Texture:getLayerCount	Gets the number of layers / slices in an Array Texture.	Added since 11.0
Texture:getMipmapCount	Gets the number of mipmaps contained in the Texture.	Added since 11.0
Texture:getMipmapFilter	Gets the mipmap filter mode for a Texture.	Added since 0.9.0
Texture:getPixelDimensions	Gets the width and height in pixels of the Texture.	Added since 11.0
Texture:getPixelHeight	Gets the height in pixels of the Texture.	Added since 11.0
Texture:getPixelWidth	Gets the width in pixels of the Texture.	Added since 11.0
Texture:getTextureType	Gets the type of the Texture.	Added since 11.0
Texture:getWidth	Gets the width of the Texture.	
Texture:getWrap	Gets the wrapping properties of a Texture.	
Texture:isReadable	Gets whether the Texture can be drawn and sent to a Shader.	Added since 11.0
Texture:setDepthSampleMode	Sets the comparison mode used when sampling from a depth texture in a shader.	Added since 11.0
Texture:setFilter	Sets the filter mode of the Texture.	
Texture:setMipmapFilter	Sets the mipmap filter mode for a Texture.	Added since 0.9.0
Texture:setWrap	Sets the wrapping properties of a Texture.	

Supertypes

- Drawable
- Object

Subtypes

Canvas	Off-screen render target.	Added since 0.8.0
Image	Drawable image type.	

Enums

TextureType	Types of textures (2D, cubemap, etc.)	Added since 11.0

See Also

- love.graphics
- love.graphics.draw
- Mesh:setTexture
- ParticleSystem:setTexture
- SpriteBatch:setTexture

- Shader:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:generateMipmaps

Texture:generateMipmaps · revision 29251 · last edited 2024-12-02

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been moved from Canvas:generateMipmaps.

Generates mipmaps for the Texture, based on the contents of the highest-resolution mipmap level.

The Texture must be created with mipmaps set to a MipmapMode other than "none" for this function to work. It should only be called while the Texture is not the active render target.

If the mipmap mode is set to "auto", this function is automatically called inside love.graphics.setCanvas when switching the active canvas from this Texture to another Texture or to the main screen.

Function

Synopsis

```
Texture:generateMipmaps( )
```

Arguments

None.

Returns

Nothing.

See Also

- Texture
- love.graphics.newCanvas
- love.graphics.newTexture
- Texture:getMipmapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getDebugName

Texture:getDebugName · revision 30571 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Gets the debug name that was set in love.graphics.newTexture, love.graphics.newImage, and other Texture creation functions.

Graphics debugging tools such as RenderDoc will recognize and display the debug name inside the tool's interface, if it's set.

Function

Synopsis

```
name = Texture:getDebugName( )
```

Arguments

None.

Returns

`string name`
: The previously set debug name for this Shader.

See Also

- Texture
- love.graphics.newTexture
- love.graphics.newImage
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getDepth

Texture:getDepth · revision 22648 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the depth of a Volume Texture. Returns 1 for 2D, Cubemap, and Array textures.

Function

Synopsis

```
depth = Texture:getDepth( )
```

Arguments

None.

Returns

`number depth`

The depth of the volume Texture.

See Also

- Texture
- love.graphics.newVolumeImage
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getDepthSampleMode

Texture:getDepthSampleMode · revision 22681 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the comparison mode used when sampling from a depth texture in a shader.

Depth texture comparison modes are advanced low-level functionality typically used with shadow mapping in 3D.

Function

Synopsis

```
compare = Texture:getDepthSampleMode( )
```

Arguments

None.

Returns

`CompareMode compare (nil)`

The comparison mode used when sampling from this texture in a shader, or nil if setDepthSampleMode has not been called on this Texture.

See Also

- Texture
- Texture:setDepthSampleMode
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getDimensions

Texture:getDimensions · revision 20173 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width and height of the Texture.

Function

Synopsis

```
width, height = Texture:getDimensions( )
```

Arguments

None.

Returns

`number width`
: The width of the Texture, in pixels.

`number height`
: The height of the Texture, in pixels.

See Also

- Texture
- Texture:getWidth
- Texture:getHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getDPIScale

Texture:getDPIScale · revision 22678 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This method is not supported in earlier versions.

Gets the DPI scale factor of the Texture.

The DPI scale factor represents relative pixel density. A DPI scale factor of 2 means the texture has twice the pixel density in each dimension (4 times as many pixels in the same area) compared to a texture with a DPI scale factor of 1.

For example, a texture with pixel dimensions of 100x100 with a DPI scale factor of 2 will be drawn as if it was 50x50. This is useful with high-dpi / retina displays to easily allow swapping out higher or lower pixel density Images and Canvases without needing any extra manual scaling logic.

Function

Synopsis

```
dpiscale = Texture:getDPIScale( )
```

Arguments

None.

Returns

`number dpiscale`
The DPI scale factor of the Texture.

See Also

- Texture
- Texture:getDimensions
- Texture:getPixelDimensions
- love.graphics.newImage
- love.graphics.newCanvas
- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getFilter

Texture:getFilter · revision 28336 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Gets the filter mode of the Texture.

Function

Synopsis

```
min, mag = Texture:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode to use when minifying the texture (rendering it at a smaller size on-screen than its size in pixels).

`FilterMode mag`
: Filter mode to use when magnifying the texture (rendering it at a larger size on-screen than its size in pixels).

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
min, mag, anisotropy = Texture:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode to use when minifying the texture (rendering it at a smaller size on-screen than its size in pixels).

`FilterMode mag`
: Filter mode to use when magnifying the texture (rendering it at a larger size on-screen than its size in pixels).

`number anisotropy`
: Maximum amount of anisotropic filtering used.

Notes

When mipmapping is used, higher anisotropic filtering values increase the quality of the texture when rendering it with a non-uniform scale, at the expense of a bit of performance.

Most systems support up to 8x or 16x anisotropic filtering.

See Also

- Texture
- Texture:setFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getFormat

Texture:getFormat · revision 22650 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the pixel format of the Texture.

Function

Synopsis

```
format = Texture:getFormat( )
```

Arguments

None.

Returns

`PixelFormat format`

The pixel format the Texture was created with.

See Also

- Texture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getHeight

Texture:getHeight · revision 20172 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the height of the Texture.

Function

Synopsis

```
height = Texture:getHeight( )
```

Arguments

None.

Returns

`number height`
: The height of the Texture, in pixels.

See Also

- Texture
- Texture:getWidth
- Texture:getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getLayerCount

Texture:getLayerCount · revision 22649 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the number of layers / slices in an Array Texture. Returns 1 for 2D, Cubemap, and Volume textures.

Function

Synopsis

```
layers = Texture:getLayerCount( )
```

Arguments

None.

Returns

`number layers`

The number of layers in the Array Texture.

See Also

- Texture
- love.graphics.newArrayImage
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getMipmapCount

Texture:getMipmapCount · revision 22639 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the number of mipmaps contained in the Texture. If the texture was not created with mipmaps, it will return 1.

Function

Synopsis

```
mipmaps = Texture:getMipmapCount( )
```

Arguments

None.

Returns

`number mipmaps`

The number of mipmaps in the Texture.

See Also

- Texture
- Texture:setMipmapFilter
- love.graphics.newImage
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getMipmapFilter

Texture:getMipmapFilter · revision 21375 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the mipmap filter mode for a Texture. Prior to 11.0 this method only worked on Images.

Function

Synopsis

```
mode, sharpness = Texture:getMipmapFilter( )
```

Arguments

None.

Returns

`FilterMode mode`

The filter mode used in between mipmap levels. nil if mipmap filtering is not enabled.

`number sharpness`

Value used to determine whether the image should use more or less detailed mipmap levels than normal when drawing.

See Also

- Texture
- Texture:setMipmapFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getMSAA

Texture:getMSAA · revision 29246 · last edited 2024-12-02

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been moved from Canvas:getMSAA.

Gets the number of multisample antialiasing (MSAA) samples used for this Texture.

This may be different than the number used as an argument to love.graphics.newCanvas or love.graphics.newTexture if the system running LÖVE doesn't support that number.

Function

Synopsis

```
samples = Texture:getMSAA( )
```

Arguments

None.

Returns

`number samples`
: The number of multisample antialiasing samples used for this Texture.

See Also

- Texture
- love.graphics.newCanvas
- love.graphics.newTexture
- love.graphics.getSystemLimits

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getPixelDimensions

Texture:getPixelDimensions · revision 22661 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This method is not supported in earlier versions.

Gets the width and height in pixels of the Texture.

Texture:getDimensions gets the dimensions of the texture in units scaled by the texture's DPI scale factor, rather than pixels. Use getDimensions for calculations related to drawing the texture (calculating an origin offset, for example), and getPixelDimensions only when dealing specifically with pixels, for example when using Canvas:newImageData.

Function

Synopsis

```
pixelwidth, pixelheight = Texture:getPixelDimensions( )
```

Arguments

None.

Returns

number pixelwidth
: The width of the Texture, in pixels.

number pixelheight
: The height of the Texture, in pixels.

See Also

- Texture
- Texture:getDPIScale
- Texture:getPixelWidth
- Texture:getPixelHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getPixelHeight

Texture:getPixelHeight · revision 22660 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This method is not supported in earlier versions.

Gets the height in pixels of the Texture.

Texture:getHeight gets the height of the texture in units scaled by the texture's DPI scale factor, rather than pixels. Use getHeight for calculations related to drawing the texture (calculating an origin offset, for example), and getPixelHeight only when dealing specifically with pixels, for example when using Canvas:newImageData.

Function

Synopsis

```
pixelheight = Texture:getPixelHeight( )
```

Arguments

None.

Returns

`number pixelheight`
The height of the Texture, in pixels.

See Also

- Texture
- Texture:getDPIScale
- Texture:getPixelWidth
- Texture:getPixelDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getPixelWidth

Texture:getPixelWidth · revision 22659 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This method is not supported in earlier versions.

Gets the width in pixels of the Texture.

Texture:getWidth gets the width of the texture in units scaled by the texture's DPI scale factor, rather than pixels. Use getWidth for calculations related to drawing the texture (calculating an origin offset, for example), and getPixelWidth only when dealing specifically with pixels, for example when using Canvas:newImageData.

Function

Synopsis

```
pixelwidth = Texture:getPixelWidth( )
```

Arguments

None.

Returns

`number pixelwidth`

The width of the Texture, in pixels.

See Also

- Texture
- Texture:getDPIScale
- Texture:getPixelHeight
- Texture:getPixelDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getTextureType

Texture:getTextureType · revision 22647 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the type of the Texture.

Function

Synopsis

```
texturetype = Texture:getTextureType( )
```

Arguments

None.

Returns

`TextureType texturetype`

The type of the Texture.

See Also

- Texture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getWidth

Texture:getWidth · revision 30064 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Gets the width of the Texture.

Function

Synopsis

```
width = Texture:getWidth( )
```

Arguments

None.

Returns

`number width`
The width of the Texture, in pixels.

See Also

- Texture
- Texture:getHeight
- Texture:getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:getWrap

Texture:getWrap · revision 22726 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the wrapping properties of a Texture.

This function returns the currently set horizontal and vertical wrapping modes for the texture.

Function

Synopsis

```
horiz, vert, depth = Texture:getWrap( )
```

Arguments

None.

Returns

`WrapMode horiz`
: Horizontal wrapping mode of the texture.

`WrapMode vert`
: Vertical wrapping mode of the texture.

Available since LÖVE 11.0

`WrapMode depth`
: Wrapping mode for the z-axis of a Volume texture.

See Also

- Texture
- Texture:setWrap
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:isCanvas

Texture:isCanvas · revision 29255 · last edited 2026-03-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets whether the Texture has been created with canvas capabilities via love.graphics.newCanvas or love.graphics.newTexture.

Function

Synopsis

```
canvas = Texture:isCanvas( )
```

Arguments

None.

Returns

`boolean canvas`

True if the Texture can be rendered to with love.graphics.setCanvas, false otherwise.

See Also

- Texture
- love.graphics.newTexture
- love.graphics.newCanvas
- love.graphics.setCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:isComputeWritable

Texture:isComputeWritable · revision 30582 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets whether the Texture has been created with compute shader write capabilities via love.graphics.newTexture, love.graphics.newImage, or love.graphics.newCanvas.

If the Texture has compute shader write capabilities enabled, then it can be written to in a Compute Shader using an `image2D` uniform variable type and the `imageStore` GLSL function.

Not all systems support compute shader write capabilities for every PixelFormat. Use love.graphics.getTextureFormats to check.

Function

Synopsis

```
computewritable = Texture:isComputeWritable( )
```

Arguments

None.

Returns

`boolean computewritable`

True if the Texture can be rendered to with love.graphics.setCanvas, false otherwise.

See Also

- Texture
- love.graphics.newTexture
- love.graphics.newComputeShader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:isFormatLinear

Texture:isFormatLinear · revision 29254 · last edited 2024-12-02

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has been moved from Image:isFormatLinear.

Gets whether the Texture was created with the linear (non-gamma corrected) flag set to true or with a non-sRGB PixelFormat.

This method always returns false when gamma-correct rendering is not enabled.

Function

Synopsis

```
linear = Texture:isFormatLinear( )
```

Arguments

None.

Returns

`boolean linear`

Whether the Texture's internal pixel format is linear (not gamma corrected), when gamma-correct rendering is enabled.

See Also

- Texture
- Texture:getFormat
- love.graphics.newTexture
- love.graphics.newImage
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:isReadable

Texture:isReadable · revision 26787 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether the Texture can be drawn and sent to a Shader.

Canvases created with stencil and/or depth PixelFormats are not readable by default, unless `readable=true` is specified in the settings table passed into love.graphics.newCanvas.

Non-readable Canvases can still be rendered to.

Function

Synopsis

```
readable = Texture:isReadable( )
```

Arguments

None.

Returns

`boolean readable`

Whether the Texture is readable.

See Also

- Texture
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:renderTo

Texture:renderTo · revision 29249 · last edited 2024-06-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Using an anonymous function for drawing to a canvas Texture
- See Also
- Other Languages

Available since LÖVE 12.0

It has been moved from Canvas:renderTo.

Render to the Texture using a function. The Texture must have been created with the canvas capability.

This is a shortcut to love.graphics.setCanvas:

```
texture:renderTo( func )
```

is similar to

```
local prevcanvas = love.graphics.getCanvas()
love.graphics.setCanvas( canvas )
func()
love.graphics.setCanvas( prevcanvas )
```

Function

Synopsis

```
Texture:renderTo( func, ... )
```

Arguments

`function func`
: A function performing drawing operations.

`any ...`
: Additional arguments to call the function with.

Returns

Nothing.

Function

Synopsis

```
Canvas:renderTo( index, func, ... )
```

Arguments

`number index`
: An index to a layer (for array textures and volume textures) or an index to a cubemap face (for cubemap textures).

`function func`
: A function performing drawing operations.

`any ...`

Additional arguments to call the function with.

Returns

Nothing.

Examples

Using an anonymous function for drawing to a canvas Texture

This example randomly draws a bunch of red lines from the top left corner of the screen to the bottom.

```
local canvas = love.graphics.newCanvas()
function love.update()
    canvas:renderTo(function()
        love.graphics.setColor(love.math.random(), 0, 0);
        love.graphics.line(0, 0, love.math.random(0, love.graphics.getWidth()), love.math.random(0, love.graphics.getHeight()));
    end);
end

function love.draw()
    love.graphics.setColor(1, 1, 1);
    love.graphics.draw(canvas);
end
```

See Also

- Texture
- love.graphics.setCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:replacePixels

Texture:replacePixels · revision 29252 · last edited 2024-06-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 12.0

This function replaces Image:replacePixels.

Replace the contents of a Texture with the contents of the given ImageData.

Function

Synopsis

```
Texture:replacePixels( data, slice, mipmap, x, y, reloadmipmaps )
```

Arguments

`ImageData data`
: The new ImageData to replace the contents with.

`number slice (nil)`
: Which cubemap face, array index, or volume layer to replace, if applicable; the value is ignored otherwise.

`number mipmap (1)`
: The mimap level to replace, if the Texture has mipmaps.

`number x (0)`
: The x-offset in pixels from the top-left of the texture to replace. The given ImageData's width plus this value must not be greater than the pixel width of the Texture's specified mipmap level.

`number y (0)`
: The y-offset in pixels from the top-left of the texture to replace. The given ImageData's height plus this value must not be greater than the pixel height of the Texture's specified mipmap level.

`boolean reloadmipmaps (true/false)`
: Whether to generate new mipmaps after replacing the Texture's pixels. True by default if the Texture was created with automatically generated mipmaps, false by default otherwise.

Returns

Nothing.

Examples

```
function love.load()
    imagedata = love.image.newImageData("pig.png")
    texture = love.graphics.newImage(imagedata)
end

function love.draw()
    love.graphics.draw(texture)
end

function love.keypressed(key)
    if key == "e" then
        -- Modify the original ImageData and apply the changes to the Texture.
        imagedata:mapPixel(function(x, y, r, g, b, a) return r/2, g/2, b/2, a/2 end)
        texture:replacePixels(imagedata)
```

```
    end
end
```

See Also

- Texture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:setDepthSampleMode

Texture:setDepthSampleMode · revision 22680 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets the comparison mode used when sampling from a depth texture in a shader.

Depth texture comparison modes are advanced low-level functionality typically used with shadow mapping in 3D.

When using a depth texture with a comparison mode set in a shader, it must be declared as a `sampler2DShadow` and used in a GLSL 3 Shader. The result of accessing the texture in the shader will return a float between 0 and 1, proportional to the number of samples (up to 4 samples will be used if bilinear filtering is enabled) that passed the test set by the comparison operation.

Depth texture comparison can only be used with readable depth-formatted Canvases.

Function

Synopsis

```
Texture:setDepthSampleMode( compare )
```

Arguments

`CompareMode compare`
: The comparison mode used when sampling from this texture in a shader.

Returns

Nothing.

Function

Disables depth texture comparison functionality for this texture.

Synopsis

```
Texture:setDepthSampleMode( )
```

Arguments

None.

Returns

Nothing.

See Also

- Texture
- Texture:getDepthSampleMode

- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:setFilter

Texture:setFilter · revision 26714 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Example
- See Also
- Other Languages

Sets the filter mode of the Texture.

Function

Synopsis

```
Texture:setFilter( min, mag, anisotropy )
```

Arguments

`FilterMode min`
: Filter mode to use when minifying the texture (rendering it at a smaller size on-screen than its size in pixels).

`FilterMode mag (min)`
: Filter mode to use when magnifying the texture (rendering it at a larger size on-screen than its size in pixels).

Available since LÖVE 0.9.0

`number anisotropy (1)`
: Maximum amount of anisotropic filtering to use.

Returns

Nothing.

Notes

When mipmapping is used, higher anisotropic filtering values increase the quality of the texture when rendering it with a non-uniform scale, at the expense of a bit of performance.

Most systems support up to 8x or 16x anisotropic filtering.

Example

```
Texture = love.graphics.newImage("char.png")
Texture:setFilter("linear", "nearest")
```

See Also

- Texture
- Texture:getFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:setMipmapFilter

Texture:setMipmapFilter · revision 25486 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the mipmap filter mode for a Texture. Prior to 11.0 this method only worked on Images.

Mipmapping (http://en.wikipedia.org/wiki/Mipmap) is useful when drawing a texture at a reduced scale. It can improve performance and reduce aliasing issues.

In 0.10.0 and newer, the texture must be created with the `mipmaps` flag enabled for the mipmap filter to have any effect. In versions prior to 0.10.0 it's best to call this method directly after creating the image with love.graphics.newImage, to avoid bugs in certain graphics drivers.

Due to hardware restrictions and driver bugs, in versions prior to 0.10.0 images that weren't loaded from a CompressedData must have power-of-two dimensions (64x64, 512x256, etc.) to use mipmaps.

Function

Synopsis

```
Texture:setMipmapFilter( filtermode, sharpness )
```

Arguments

`FilterMode filtermode`
: The filter mode to use in between mipmap levels. "nearest" will often give better performance.

`number sharpness (0)`
: A positive sharpness value makes the texture use a more detailed mipmap level when drawing, at the expense of performance. A negative value does the reverse.

Returns

Nothing.

Notes

On mobile devices (Android and iOS), the sharpness parameter is not supported and will do nothing. You can use a custom Shader instead, and specify the mipmap sharpness in the optional third parameter to the `Texel` function in the shader (a *negative* value makes the texture use a more detailed mipmap level.)

In versions prior to 0.10.0, calling this function automatically creates mipmaps for the Image if none exist yet. If the image is compressed and its CompressedData has mipmap data included, it will use that.

Function

Disables mipmap filtering.

Synopsis

```
Texture:setMipmapFilter( )
```

Arguments

None.

Returns

Nothing.

See Also

- Texture
- Texture:getMipmapFilter
- love.graphics.newImage
- love.graphics.newCanvas
- MipmapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Texture:setWrap

Texture:setWrap · revision 22725 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the wrapping properties of a Texture.

This function sets the way a Texture is repeated when it is drawn with a Quad that is larger than the texture's extent, or when a custom Shader is used which uses texture coordinates outside of [0, 1]. A texture may be clamped or set to repeat in both horizontal and vertical directions.

Clamped textures appear only once (with the edges of the texture stretching to fill the extent of the Quad), whereas repeated ones repeat as many times as there is room in the Quad.

Function

Synopsis

```
Texture:setWrap( horiz, vert, depth )
```

Arguments

`WrapMode horiz`
: Horizontal wrapping mode of the texture.

`WrapMode vert (horiz)`
: Vertical wrapping mode of the texture.

Available since LÖVE 11.0

`WrapMode depth (horiz)`
: Wrapping mode for the z-axis of a Volume texture.

Returns

Nothing.

See Also

- Texture
- Texture:getWrap
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TexturedPolygon

TexturedPolygon · revision 19081 · last edited 2021-03-14

A function to create an Image from a polygon texturised by a texture.

function TexturedPolygon(polygon,texture, xx,yy,rr,sx,sy,ox,oy, smooth)

Parameters

`polygon` a polygon to texturize in a form of table of coordinates of points: `{x1,y1, x2,y2, ...}`

`texture` may be an ImageData object or a filename to picture. Used to fill the entire polygon, wrapping vertically and horizontally.

All other parameters are optional:

`xx,yy,r,...` are used to shift, rotate and scale texture before applying to polygon. Think of it as about a sheet of paper, filled with repeated images of transformed texture, and you use polygon points coordinates to cut out some figure using scissors. For instance, increasing an `xx` value will move texture right while the polygon itself will stay at place.

`xx,yy` coordinates of a (origin of) a texture.

`rr` rotation angle of a texture (related to origin), in degrees.

`sx,xy` scale factors of a texture along x and y axes (related to origin).

`ox,oy` position of an origin on a texture. It affects how rotation and scale are applied.

On default, no some transformation is applied. It means: `xx,yy,rr,sx,sy,ox,oy=0,0,0,1,1,0,0`

`smooth` whether to smooth image (`true` or `false`). Smoothed image looks better, but calculates slower. `smooth` is set to `true` by default.

Return value

Function return an object which is very simple to draw:

`{img, x,y, w,h, imgData}`

Here:

`img` an Image object, may be drawn by love.graphics.draw function.

`x,y` where to draw image. Use it like this: `love.graphics.draw(obj.img, obj.x,obj.y)`

`w,h` width and height of an image.

`imgData` an ImageData object. If you want to use it, you must to know how to do it.

Implementation

Function uses ImageData:mapPixel function to draw resulting image pixel-by-pixel. Area which is not within polygon is left fully transparent.

As soon as we need to calculate every pixel, function gets much time. Hence you'd better use it as rare as possible, for example on loading the game or new level. Don't try to use it inside of love.update function. If you still need it, try to set `smooth` to `false` and see what happens.

See text itself for more details. It also contains several auxiliary functions which you can use as well. (Scroll bravely, example is right below).

```
function TexturedPolygon(polygon,texture, xx,yy,rr,sx,sy,ox,oy, smooth) --return array of images. If drawn all, they represent textured polygon
    if type(polygon)~="table" or #polygon%2~=0 then return nil end  --polygon must be a table of x1,y1,x2,y2...
    if #polygon<6 then return nil end --for line return nil: nothing to draw here

    --load texture if needed
    if type(texture)=="string" then
        texture=love.image.newImageData(texture)
    end

    --default values for texture position parameters
```

```
    if xx==nil then xx=0 end
    if yy==nil then yy=0 end
    if rr==nil then rr=0 end
    if sx==nil then sx=1 end
    if sy==nil then sy=1 end
    if ox==nil then ox=0 end
    if oy==nil then oy=0 end

    if sx==0 then sx=1 end --avoid bad scenario
    if sy==0 then sy=1 end

    rr=rr/180*3.14159 --convert degrees to radians

    if smooth==nil then smooth=true end --we draw beautiful things on default

    local bb=PolygonBoundingBox(polygon) --bounding box of polygon

    --simple==0 means no simplicity, 1 means we don't transform texture, 2 means we don't even need to round x,y values
    local simple=0
    if xx==0 and yy==0 and rr==0 and sx==1 and sy==1 and ox==0 and oy==0 then
        simple=1
        if bb.x==math.floor(bb.x) and bb.y==math.floor(bb.y) then
            simple=2
        end
    end
    --additionally, simple==3 means we need to transform our texture but user don't want to smooth the image
    if simple==0 and not smooth then
        simple=3
    end

    --calculate some oftenly used numbers once to optimize our lovely code
    local cr,sr=math.cos(rr), math.sin(rr)
    local tw,th=texture:getWidth(), texture:getHeight()

    --triangles is array of triangles (what a concidence!)
    local triangles=nil
    if #polygon==6 then
        triangles={polygon}
    else
        triangles=love.math.triangulate(polygon)
    end

    local imageData=love.image.newImageData(math.ceil(bb.w),math.ceil(bb.h)) --empty ImageData for drawing. "empty" means every pixel has value 0,0,0,0

    for i,triangle in ipairs(triangles) do --for every tirangle...

        --we need three criteria to check whether pixel is within our triangle
        --we define three linear functions y=a#+b#*(x-c#) or x=a#+b#*(y-c#) (depend on side's inclination)
        --m# is mode: above (1), below (2), right (3) or left (4)
        --e#: whether this line is the edge of original polygon: nil(false) or range of x or y(true)
        local a0={}
        local b0={}
        local c0={}
        local m0={}
        local e0={}

        --I'd love to make a cycle over pair of points of triangle, but 3rd pair is (1,3) points, not (3,4)

        --for 1st pair of points    --we keep in mind that triangle[1]->x1,[2]->y1, [3]->x2,[4]->y2, [5]->x3,[6]->y3
        if triangle[1]==triangle[3] or math.abs( (triangle[4]-triangle[2])/(triangle[3]-triangle[1]) ) > 1 then --inclination is close to vertical
            m0[1]=2
        else
            m0[1]=1
        end
        if m0[1]==1 then --horizontal criterion
            a0[1],b0[1],c0[1] = triangle[2], (triangle[4]-triangle[2])/(triangle[3]-triangle[1]), triangle[1]
            if triangle[6] > a0[1] + b0[1]*(triangle[5]-c0[1]) then --check how another point of triangle pass this criteron
                m0[1]=1 --above criterion
            else
                m0[1]=2 --under criterion
            end
```

```
        else --vertial criterion
            a0[1],b0[1],c0[1] = triangle[1], (triangle[3]-triangle[1])/(triangle[4]-triangle[2]), triangle[2]
            if triangle[5] > a0[1] + b0[1]*(triangle[6]-c0[1]) then --check how another point of triangle pass this criteron
                m0[1]=3 --right criterion
            else
                m0[1]=4 --left  criterion
            end
        end
        if PolygonEdgeLine(polygon, triangle[1],triangle[2],triangle[3],triangle[4]) then --check whether this line is the edge of polygon
            if m0[1]==1 or m0[1]==2 or m0[1]==5 or m0[1]==6 then
                e0[1]={math.min(triangle[1],triangle[3]), math.max(triangle[1],triangle[3])} --range of x where it is true
            else
                e0[1]={math.min(triangle[2],triangle[4]), math.max(triangle[2],triangle[4])} --range of y where it is true
            end
        else
            e0[1]=nil --no, it's internal splitting line
        end
        --and so on...

        --for 2nd pair of points
        if triangle[3]==triangle[5] or math.abs( (triangle[6]-triangle[4])/(triangle[5]-triangle[3]) ) > 1 then --inclination is close to
vertical
            m0[2]=2
        else
            m0[2]=1
        end
        if m0[2]==1 then --horizontal criterion
            a0[2],b0[2],c0[2] = triangle[4], (triangle[6]-triangle[4])/(triangle[5]-triangle[3]), triangle[3]
            if triangle[2] > a0[2] + b0[2]*(triangle[1]-c0[2]) then
                m0[2]=1 --above criterion
            else
                m0[2]=2
            end
        else --vertial criterion
            a0[2],b0[2],c0[2] = triangle[3], (triangle[5]-triangle[3])/(triangle[6]-triangle[4]), triangle[4]
            if triangle[1] > a0[2] + b0[2]*(triangle[2]-c0[2]) then
                m0[2]=3
            else
                m0[2]=4
            end
        end
        if PolygonEdgeLine(polygon, triangle[3],triangle[4],triangle[5],triangle[6]) then
            if m0[2]==1 or m0[2]==2 or m0[2]==5 or m0[2]==6 then
                e0[2]={math.min(triangle[3],triangle[5]), math.max(triangle[3],triangle[5])}
            else
                e0[2]={math.min(triangle[4],triangle[6]), math.max(triangle[4],triangle[6])}
            end
        else
            e0[2]=nil
        end

        --for 3rd pair of points
        if triangle[1]==triangle[5] or math.abs( (triangle[6]-triangle[2])/(triangle[5]-triangle[1]) ) > 1 then --inclination is close to
vertical
            m0[3]=2
        else
            m0[3]=1
        end
        if m0[3]==1 then --horizontal criterion
            a0[3],b0[3],c0[3] = triangle[2], (triangle[6]-triangle[2])/(triangle[5]-triangle[1]), triangle[1]
            if triangle[4] > a0[3] + b0[3]*(triangle[3]-c0[3]) then
                m0[3]=1
            else
                m0[3]=2
            end
        else --vertial criterion
            a0[3],b0[3],c0[3] = triangle[1], (triangle[5]-triangle[1])/(triangle[6]-triangle[2]), triangle[2]
            if triangle[3] > a0[3] + b0[3]*(triangle[4]-c0[3]) then
                m0[3]=3
            else
                m0[3]=4
            end
        end
```

```
if PolygonEdgeLine(polygon, triangle[1],triangle[2],triangle[5],triangle[6]) then
    if m0[3]==1 or m0[3]==2 or m0[3]==5 or m0[3]==6 then
        e0[3]={math.min(triangle[1],triangle[5]), math.max(triangle[1],triangle[5])}
    else
        e0[3]={math.min(triangle[2],triangle[6]), math.max(triangle[2],triangle[6])}
    end
else
    e0[3]=nil
end

local function func(x,y,r,g,b,a) --function to apply to mapPixel method of ImageData polygon
    if r~=0 or g~=0 or b~=0 or a~=0 then return r,g,b,a end --pixel was already drawn, skip

    x=x+bb.x --go from resulting image coordinates to internal texture coordinates
    y=y+bb.y

    local alpha=1 --transparency to smooth edge (only used if smooth==true)

    --check all three criteria
    for j=1,3 do

        if m0[j]==1 then
            local y0=a0[j]+b0[j]*(x-c0[j]) --what to compare with
            if smooth and e0[j]~=nil and x>=e0[j][1] and x<e0[j][2] then --whether we need to smooth edge
                local z=y0-y --how far we are from ideal position of edge
                if z>1 then --we are too far
                    return 0,0,0,0
                elseif z>0 then --we are within one-pixel edge area of smoothing
                    alpha=alpha*(1-z)
                end
            elseif y<y0 then --we are outside of the triangle
                return 0,0,0,0
            end
            --and so on...

        elseif m0[j]==2 then
            local y0=a0[j]+b0[j]*(x-c0[j])
            if smooth and e0[j]~=nil and x>=e0[j][1] and x<e0[j][2] then
                local z=y0-y
                if z<0 then
                    return 0,0,0,0
                elseif z<1 then
                    alpha=alpha*z
                end
            elseif y>=y0 then
                return 0,0,0,0
            end

        elseif m0[j]==3 then
            local x0=a0[j]+b0[j]*(y-c0[j])
            if smooth and e0[j]~=nil and y>=e0[j][1] and y<e0[j][2] then
                local z=x0-x
                if z>1 then
                    return 0,0,0,0
                elseif z>0 then
                    alpha=alpha*(1-z)
                end
            elseif x<x0 then
                return 0,0,0,0
            end

        elseif m0[j]==4 then
            local x0=a0[j]+b0[j]*(y-c0[j])
            if smooth and e0[j]~=nil and y>=e0[j][1] and y<e0[j][2] then
                local z=x0-x
                if z<0 then
                    return 0,0,0,0
                elseif z<1 then
                    alpha=alpha*z
                end
            elseif x>=x0 then
                return 0,0,0,0
            end
```

```
                end
            end

            if simple==0 then --complicated case
                r,g,b,a=getAveragePixel(texture, ((x+0.5-xx)*cr+(y+0.5-yy)*sr)/sx+ox, ((y+0.5-yy)*cr-(x+0.5-xx)*sr)/sy+oy, tw,th)
            elseif simple==1 then --simpler case
                r,g,b,a=texture:getPixel( math.floor(x)%tw, math.floor(y)%th)
            elseif simple==2 then --simplest case
                r,g,b,a=texture:getPixel( x%tw, y%th)
            else--if simple==3 then --ugly but fast case
                r,g,b,a=texture:getPixel( math.floor(((x-xx)*cr+(y-yy)*sr)/sx+ox)%tw, math.floor(((y-yy)*cr-(x-xx)*sr)/sy+oy)%th)
            end

            if smooth then a=a*alpha end --make edge as beautiful as we can
            --return texture:getPixel( (x)%texture:getWidth(), (y)%texture:getHeight())
            return r,g,b,a
        end

        imageData:mapPixel(func) --applying will draw one more triangle on the imageData
    end

    local image=love.graphics.newImage(imageData) --creating drawable object

    return {img=image, x=bb.x,y=bb.y,w=bb.w,h=bb.h, imgData=imageData}
end

function PolygonBoundingBox(polygon) --simple function that find max and min x,y coordinates of a polygon and return them in a form of
BoundingBox object: {x,y,w,h}
    if type(polygon)~="table" or #polygon<2 or #polygon%2~=0 then return nil end

    local minX=polygon[1]
    local maxX=polygon[1]
    local minY=polygon[2]
    local maxY=polygon[2]

    for i=1,#polygon,2 do
        if polygon[i]<minX then minX=polygon[i] end
        if polygon[i]>maxX then maxX=polygon[i] end
        if polygon[i+1]<minY then minY=polygon[i+1] end
        if polygon[i+1]>maxY then maxY=polygon[i+1] end
    end

    return {x=minX,y=minY,w=maxX-minX,h=maxY-minY}
end

function PolygonEdgeLine(polygon, x1,y1,x2,y2) --specific function that checks whether given pair of points belongs to one edge of a polygon
    local length=#polygon
    if type(polygon)~="table" or length<4 or length%2~=0 then return nil end

    local buf={} --comparison of non-integer values may be safe only if these values are rounded (to two digits, for example)
    for i,v in ipairs(polygon) do
        buf[i]=math.floor(v*100)/100
    end

    local x1=math.floor(x1*100)/100
    local y1=math.floor(y1*100)/100
    local x2=math.floor(x2*100)/100
    local y2=math.floor(y2*100)/100

    for i=1,length-2,2 do --compare all but the last line
        if buf[i]==x1 and buf[i+1]==y1 and buf[i+2]==x2 and buf[i+3]==y2 then
            return true
        elseif buf[i]==x2 and buf[i+1]==y2 and buf[i+2]==x1 and buf[i+3]==y1 then
            return true
        end
    end

    --compare last line separately
    if buf[1]==x1 and buf[2]==y1 and buf[length-1]==x2 and buf[length]==y2 then
        return true
    elseif buf[1]==x2 and buf[2]==y2 and buf[length-1]==x1 and buf[length]==y1 then
        return true
```

```
    end

    --if we have passed all previous checks, we fail:
    return false
end

function getAveragePixel(texture, x,y,w,h) --return a pixel which color is averaged over neighbour pixels of a texture around x,y coordinates. w,h calculated previously to speed up the code
    local x1,x2,y1,y2 = math.floor(x), math.ceil(x), math.floor(y), math.ceil(y) --find neighbour integer-indexed pixels
    if x2==x1 then x2=x2+1 end
    if y2==y1 then y2=y2+1 end

    local d1,d2,d3,d4 = (x2-x)*(y2-y), (x-x1)*(y2-y), (x2-x)*(y-y1), (x-x1)*(y-y1) --find fraction of each pixel in a result

    x1=x1%w --ensure we are within texture (wrapped around)
    x2=x2%w
    y1=y1%h
    y2=y2%h

    local r1,g1,b1,a1 = texture:getPixel(x1,y1) --obtain four raw colors
    local r2,g2,b2,a2 = texture:getPixel(x2,y1)
    local r3,g3,b3,a3 = texture:getPixel(x1,y2)
    local r4,g4,b4,a4 = texture:getPixel(x2,y2)

    --...and average it all
    return d1*r1+d2*r2+d3*r3+d4*r4, d1*g1+d2*g2+d3*g3+d4*g4, d1*b1+d2*b2+d3*b3+d4*b4, d1*a1+d2*a2+d3*a3+d4*a4
end
```

Run an example to see how it works:

```
require "textured_polygon" --link to a file where TexturedPolygon function is, or paste it below here

function love.load()
    texture=love.image.newImageData('texture.png')
    bkg=love.graphics.newImage(texture)
    polygon={100,100, 200,100, 300,200, 400,100, 520,100, 530,200, 400,200, 300,300, 200,200, 100,200}

    t_start=love.timer.getTime()

    image=TexturedPolygon(polygon,'texture.png')

    t_end=love.timer.getTime()

    love.graphics.setBackgroundColor(128,50,100)
end

function love.draw()
    love.graphics.draw( bkg, 0,20, 0, 10,10)
    love.graphics.draw( image.img, image.x, image.y)

    love.graphics.print("Spent "..(t_end-t_start).." seconds to create textured polygon")
    love.graphics.print("<- see theese pixels smoothed!", 530,140)
    love.graphics.print("| works with concave polygons as well!", 302,80)
    love.graphics.print("V                                    ", 300,90)
    love.graphics.print("^                            ", 420,200)
    love.graphics.print("| semi-transparent textures!", 422,205)
end
```

Use this texture:


texture.png

How it works:


screenshot1.png

TextureFormat

TextureFormat · revision 16411 · last edited 2026-05-12

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0
This enum is not supported in earlier or later versions.

Image texture formats.

As of LÖVE 0.9.2, Canvases use CanvasFormats rather than TextureFormats.

Constants

normal
: The default texture format: 8 bits per channel (32 bpp) RGBA. Color channel values range from 0-255 (0-1 in shaders.)

hdr
: Only usable in Canvases. The high dynamic range texture format: floating point 16 bits per channel (64 bpp) RGBA. Color channel values inside the Canvas range from -infinity to +infinity.

Available since LÖVE 0.9.1

srgb
: The same as `normal`, but the texture is *interpreted* as being in the sRGB color space. It will be decoded from sRGB to linear RGB when drawn or sampled from in a shader. For Canvases, this will also convert everything drawn *to* the Canvas from linear RGB to sRGB.

Notes

The HDR format is most useful when combined with pixel shaders. Effects such as tonemapped HDR with bloom can be accomplished, or the canvas can be used to store arbitrary non-color data such as positions which are then interpreted and used in a shader, as long as you draw the right things to the canvas.

The sRGB format should only be used when doing gamma-correct rendering, which is an advanced topic and it's easy to get color-spaces mixed up. If you're not sure whether you need this, you might want to avoid it. Read more about gamma-correct rendering here (http://http.developer.nvidia.com/GPUGems3/gpugems3_ch24.html), here (http://filmicgames.com/archives/299), and here (http://renderwonk.com/blog/index.php/archive/adventures-with-gamma-correct-rendering/).

Not all systems support the HDR and sRGB formats. Use love.graphics.isSupported to check before creating the Canvas or Image.

See Also

- love.graphics
- love.graphics.newImage
- love.graphics.newCanvas
- love.graphics.isSupported

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TextureType

TextureType · revision 21436 · last edited 2026-04-09

Available since LÖVE 11.0

This enum is not supported in earlier versions.

Types of textures (2D, cubemap, etc.)

Constants

2d

Regular 2D texture with width and height.

array

Several same-size 2D textures organized into a single object. Similar to a texture atlas / sprite sheet, but avoids sprite bleeding and other issues.

cube

Cubemap (https://en.wikipedia.org/wiki/Cube_mapping) texture with 6 faces. Requires a custom shader (and Shader:send) to use. Sampling from a cube texture in a shader takes a 3D direction vector instead of a texture coordinate.

volume

3D texture with width, height, and depth. Requires a custom shader to use. Volume textures can have texture filtering applied along the 3rd axis.

Notes

Array textures (http://codeflow.org/entries/2010/dec/09/minecraft-like-rendering-experiments-in-opengl-4/illustrations/textures.jpg) can be rendered with love.graphics.drawLayer, with love.graphics.draw and a Quad via Quad:setLayer, or by sending it to a custom Shader. When getting the pixels of a layer of an Array Texture in a shader, a third texture coordinate component is used to choose which layer to get.

When using a custom Shader, each texture type has a different GLSL type.

- `Image` (or `sampler2D`) is a 2D texture.
- `ArrayImage` (or `sampler2DArray`) is an array texture.
- `CubeImage` (or `samplerCube`) is a cubemap texture.
- `VolumeImage` (or `sampler3D`) is a volume texture.

The `Texel` shader function can be used to sample from all types of textures in a shader. `Texel` is recommended instead of `texture2D`, `textureCube` etc., because the latter do not work across all versions of GLSL supported by LÖVE, whereas Texel does (GLSL 3 removed `texture2D` and added a generic `texture` function).

Each texture type has a different maximum size on a user's system. Use love.graphics.getSystemLimits to check.

Not all texture types are supported by all systems. love.graphics.getTextureTypes can check for support.

2D and cube texture types are supported everywhere. Array textures require a system that supports OpenGL 3 or OpenGL ES 3. Volume / 3D textures require a system that supports OpenGL 2 or OpenGL ES 3.

See Also

- love.graphics
- Texture
- Texture:getTextureType
- love.graphics.newImage
- love.graphics.newArrayImage
- love.graphics.newCubeImage
- love.graphics.newVolumeImage
- love.graphics.newCanvas

- love.graphics.drawLayer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thranduil

Thranduil · revision 24520 · last edited 2023-07-05

Thranduil is an UI library created by "adonaac" for LÖVE. **sources & website : 404**

Links

- Website (https://github.com/adonaac/)
- Documentation (https://github.com/adonaac/thranduil/blob/master/README.md)
- Release Thread (https://love2d.org/forums/viewtopic.php?f=5&t=79553)
- Repository (https://github.com/adonaac/thranduil)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread

Thread · revision 15972 · last edited 2025-09-30

On this page

- Constructors
- Functions
- Supertypes
- Notes
- See Also
- Other Languages

Available since LÖVE 0.7.0

This type is not supported in earlier versions.

A Thread is a chunk of code that can run in parallel with other threads. Data can be sent between different threads with Channel objects.

Constructors

love.thread.newThread	Creates a new Thread from a filename, string or FileData object containing Lua code.	Added since 0.7.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Thread:demand	Receive a message from a thread. Wait for the message to exist before returning.	Added since 0.7.0	Removed in 0.9.0
Thread:get	Get a value.	Added since 0.8.0	Removed in 0.9.0
Thread:getError	Retrieves the error string from the thread.	Added since 0.9.0	
Thread:getKeys	Returns the names of all messages as a table.	Added since 0.8.0	Removed in 0.9.0
Thread:getName	Get the name of a thread.	Added since 0.7.0	Removed in 0.9.0
Thread:isRunning	Returns whether the thread is currently running.	Added since 0.9.0	
Thread:kill	Forcefully terminate the thread.	Added since 0.7.0	Removed in 0.8.0
Thread:peek	Receive a message from a thread, but leave it in the message box.	Added since 0.7.0	Removed in 0.9.0
Thread:receive	Receive a message from a thread.	Added since 0.7.0	Removed in 0.8.0
Thread:send	Send a message.	Added since 0.7.0	Removed in 0.8.0
Thread:set	Set a value.	Added since 0.8.0	Removed in 0.9.0
Thread:start	Starts the thread.	Added since 0.7.0	
Thread:wait	Wait for a thread to finish.	Added since 0.7.0	

Supertypes

- Object

Notes

Removed in LÖVE 0.9.0

This method for retrieving errors has been replaced by Thread:getError and love.threaderror.

If a Lua error occurs in the thread, a message with the name "error" gets pushed to its message pool. Retrieve the message with `Thread:get('error')`.

See Also

- love.thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:demand

Thread:demand · revision 19338 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.9.0

It has been moved to the Channel API.

Receive a message from a thread. Wait for the message to exist before returning. (Can return nil in case of an error in the thread.)

Function

Synopsis

```
value = Thread:demand( name )
```

Arguments

`string name`
: The name of the message.

Returns

`Variant value`
: The contents of the message.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:get

Thread:get · revision 19340 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been moved to the Channel api.

Retrieves the value of a message and removes it from the thread's message box.

Function

Synopsis

```
value = Thread:get( name )
```

Arguments

`string name`

The name of the message.

Returns

`Variant value`

The contents of the message or nil when no message in message box.

See Also

- Thread
- Thread:set

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Thread:getError

Thread:getError · revision 20775 · last edited 2025-09-30

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Retrieves the error string from the thread if it produced an error.

Function

Synopsis

```
err = Thread:getError( )
```

Arguments

None.

Returns

`string err (nil)`

The error message, or nil if the Thread has not caused an error.

See Also

- Thread
- love.threaderror

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:getKeys

Thread:getKeys · revision 15970 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

This method is not supported in earlier or later versions.

Returns a table with the names of all messages in the message box.

Function

Synopsis

```
msgNames = Thread:getKeys()
```

Arguments

None.

Returns

`table msgNames`

A sequence with all the message names.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:getName

Thread:getName · revision 15969 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Get the name of a thread.

Function

Synopsis

```
name = Thread:getName( )
```

Arguments

None.

Returns

`string name`

The name of the thread.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Thread:isRunning

Thread:isRunning · revision 10875 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Returns whether the thread is currently running.

Threads which are not running can be (re)started with Thread:start.

Function

Synopsis

```
running = Thread:isRunning( )
```

Arguments

None.

Returns

`boolean value`

True if the thread is running, false otherwise.

See Also

- Thread
- Thread:start

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:kill

Thread:kill · revision 23081 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

This method is not supported in earlier or later versions.

Forcefully terminate the thread.


This function terminates the thread wherever it is, which is capable of locking the game, or doing anything evil for that matter, its use is strongly discouraged.

Function

Synopsis

```
Thread:kill( )
```

Arguments

None.

Returns

Nothing.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:peek

Thread:peek · revision 15968 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.9.0

It has been moved to the Channel api.

Retrieves the value of a message, but leaves it in the thread's message box. The name of the message can be any string. The value of the message can be a boolean, string, number or a LÖVE userdata. It returns nil, if there's no message with the given name.

Function

Synopsis

```
value = Thread:peek( name )
```

Arguments

`string name`
: The name of the message.

Returns

`boolean, number, string or LÖVE userdata value`
: The contents of the message.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:receive

Thread:receive · revision 19337 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

It has been renamed to Thread:get.

Receive a message from a thread. Returns nil when a message is not in the message box.

Function

Synopsis

```
value = Thread:receive(name)
```

Arguments

`string name`
: The name of the message.

Returns

`Variant value`
: The contents of the message or nil when no message in message box.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:send

Thread:send · revision 19339 · last edited 2023-07-05

On this page
- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0
It has been renamed to Thread:set.

Send a message (put it in the message box).

Function

Synopsis

```
Thread:send(name, value)
```

Arguments

`string name`
: The name of the message.

`Variant value`
: The contents of the message.

Returns

None.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:set

Thread:set · revision 15965 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been moved to the Channel API.

Sets a value in the message box of the thread. The name of the message can be any string. The value of the message can be a boolean, string, number or a LÖVE userdata. Foreign userdata (Lua's files, LuaSocket, ...), functions or tables are not supported.

Function

Synopsis

```
Thread:set(name, value)
```

Arguments

`string name`
: The name of the message.

`boolean, string, number or LÖVE userdata value`
: The contents of the message.

Returns

None.

See Also

- Thread
- Thread:get

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:start

Thread:start · revision 19336 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Starts the thread.

Beginning with version 0.9.0, threads can be restarted after they have completed their execution.

Function

Synopsis

```
Thread:start( )
```

Arguments

None.

Returns

Nothing.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
Thread:start( arg1, arg2, ... )
```

Arguments

`Variant arg1`
: A string, number, boolean, LÖVE object, or simple table.

`Variant arg2`
: A string, number, boolean, LÖVE object, or simple table.

`Variant ...`
: You can continue passing values to the thread.

Returns

Nothing.

Notes

Arguments passed to Thread:start are accessible in the thread's main file via **...** (the vararg expression.)

See Also

- Thread
- Thread:wait

- Thread:isRunning

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Thread:wait

Thread:wait · revision 6203 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Wait for a thread to finish. This call will block until the thread finishes.

Function

Synopsis

```
Thread:wait( )
```

Arguments

None.

Returns

None.

See Also

- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tiled

Tiled · revision 7773 · last edited 2021-03-14

Tiled is a general purpose tile map editor. It's built to be easy to use, yet flexible enough to work with varying game engines, whether your game is an RPG, platformer or Breakout clone. Tiled is free software and written in C++, using the Qt application framework. The main features in a nutshell:

- General purpose tile map editor with XML-based map format
- Supports orthogonal and isometric maps
- Custom objects can be placed with pixel precision
- Full undo/redo and copy/paste support
- Add custom properties to tiles, layers, objects or the map
- Automatically reloads tilesets when changed externally
- Resize or offset your tile map later as needed
- Efficient tile editing tools like stamp and fill brushes

- see TiledMapLoader for example snippet of how to load map files

- Tiled (http://www.mapeditor.org/)

TiledMapLoader

TiledMapLoader · revision 19083 · last edited 2021-03-14

loader for Tiled (mapeditor) map files.

see this forum thread for demo.zip : http://love2d.org/forums/viewtopic.php?f=5&t=2411&p=25929#p25929

note : for an alternative implementation see also http://love2d.org/forums/viewtopic.php?f=5&t=2097&hilit=tiled



Probably outdated, use this instead: https://github.com/karai17/Simple-Tiled-Implementation ↗

```
-- see https://love2d.org/wiki/TiledMapLoader for latest version
-- loader for "tiled" map editor maps (.tmx,xml-based) http://www.mapeditor.org/
-- supports multiple layers
-- NOTE : function ReplaceMapTileClass (tx,ty,oldTileType,newTileType,fun_callback) end
-- NOTE : function TransmuteMap (from_to_table) end -- from_to_table[old]=new
-- NOTE : function GetMousePosOnMap () return gMouseX+gCamX-gScreenW/2,gMouseY+gCamY-gScreenH/2 end

kTileSize = 32
kMapTileTypeEmpty = 0
local floor = math.floor
local ceil = math.ceil
local max = math.max
local min = math.min
local abs = math.abs
gTileMap_LayerInvisByName = {}

function TiledMap_Load (filepath,tilesize,spritepath_removeold,spritepath_prefix)
    spritepath_removeold = spritepath_removeold or "../"
    spritepath_prefix = spritepath_prefix or ""
    kTileSize = tilesize or kTileSize or 32
    gTileGfx = {}

    local tiletype,layers = TiledMap_Parse(filepath)
    gMapLayers = layers
    for first_gid,path in pairs(tiletype) do
        path = spritepath_prefix .. string.gsub(path,"^"..string.gsub(spritepath_removeold,"%.","%%."),"")
        local raw = love.image.newImageData(path)
        local w,h = raw:getWidth(),raw:getHeight()
        local gid = first_gid
        local e = kTileSize
        for y=0,floor(h/kTileSize)-1 do
        for x=0,floor(w/kTileSize)-1 do
            local sprite = love.image.newImageData(kTileSize,kTileSize)
            sprite:paste(raw,0,0,x*e,y*e,e,e)
            gTileGfx[gid] = love.graphics.newImage(sprite)
            gid = gid + 1
        end
        end
    end
end

function TiledMap_GetMapW () return gMapLayers.width end
function TiledMap_GetMapH () return gMapLayers.height end

-- returns the mapwidth actually used by tiles
function TiledMap_GetMapWUsed ()
    local maxx = 0
    local miny = 0
    local maxy = 0
    for layerid,layer in pairs(gMapLayers) do
        if (type(layer) == "table") then for ty,row in pairs(layer) do
            if (type(row) == "table") then for tx,t in pairs(row) do
                if (t and t ~= kMapTileTypeEmpty) then
                    miny = min(miny,ty)
                    maxy = max(maxy,ty)
                    maxx = max(maxx,tx)
```

```
                end
            end end
        end end
    end
    return maxx + 1,miny,maxy+1
end

-- x,y= position for nearest-distance(square,not round), z= layer, maxrad= optional limit for searching
-- returns x,y
-- if x,y can be far outside map, set a sensible maxrad, otherwise it'll get very slow since searching outside map isn't optimized
function TiledMap_GetNearestTileByTypeOnLayer (x,y,z,iTileType,maxrad)
    local w = TiledMap_GetMapW()
    local h = TiledMap_GetMapW()
    local maxrad2 = max(x,w-x,y,h-y) if (maxrad) then maxrad2 = min(maxrad2,maxrad) end
    if (TiledMap_GetMapTile(x,y,z) == iTileType) then return x,y end
    for r = 1,maxrad2 do
        for i=-r,r do
            local xa,ya = x+i,y-r if (TiledMap_GetMapTile(xa,ya,z) == iTileType) then return xa,ya end -- top
            local xa,ya = x+i,y+r if (TiledMap_GetMapTile(xa,ya,z) == iTileType) then return xa,ya end -- bot
            local xa,ya = x-r,y+i if (TiledMap_GetMapTile(xa,ya,z) == iTileType) then return xa,ya end -- left
            local xa,ya = x+r,y+i if (TiledMap_GetMapTile(xa,ya,z) == iTileType) then return xa,ya end -- right
        end
    end
end

function TiledMap_GetMapTile (tx,ty,layerid) -- coords in tiles
    local row = gMapLayers[layerid][ty]
    return row and row[tx] or kMapTileTypeEmpty
end

function TiledMap_SetMapTile (tx,ty,layerid,v) -- coords in tiles
    local row = gMapLayers[layerid][ty]
    if (not row) then row = {} gMapLayers[layerid][ty] = row end
    row[tx] = v
end

-- todo : maybe optimize during parse xml for types registered as to-be-listed before parsing ?
function TiledMap_ListAllOfTypeOnLayer (layerid,iTileType)
    local res = {}
    local w = TiledMap_GetMapW()
    local h = TiledMap_GetMapH()
    for x=0,w-1 do
    for y=0,h-1 do
        if (TiledMap_GetMapTile(x,y,layerid) == iTileType) then table.insert(res,{x=x,y=y}) end
    end
    end
    return res
end

function TiledMap_GetLayerZByName (layername) for z,layer in ipairs(gMapLayers) do if (layer.name == layername) then return z end end end
function TiledMap_SetLayerInvisByName (layername) gTileMap_LayerInvisByName[layername] = true end

function TiledMap_IsLayerVisible (z)
    local layer = gMapLayers[z]
    return layer and (not gTileMap_LayerInvisByName[layer.name or "?"])
end

function TiledMap_GetTilePosUnderMouse (mx,my,camx,camy)
    return  floor((mx+camx-love.graphics.getWidth()/2)/kTileSize),
            floor((my+camy-love.graphics.getHeight()/2)/kTileSize)
end

function TiledMap_DrawNearCam (camx,camy,fun_layercallback)
    camx,camy = floor(camx),floor(camy)
    local screen_w = love.graphics.getWidth()
    local screen_h = love.graphics.getHeight()
    local minx,maxx = floor((camx-screen_w/2)/kTileSize),ceil((camx+screen_w/2)/kTileSize)
    local miny,maxy = floor((camy-screen_h/2)/kTileSize),ceil((camy+screen_h/2)/kTileSize)
    for z = 1,#gMapLayers do
    if (fun_layercallback) then fun_layercallback(z,gMapLayers[z]) end
    if (TiledMap_IsLayerVisible(z)) then
    for x = minx,maxx do
    for y = miny,maxy do
```

```
        local gfx = gTileGfx[TiledMap_GetMapTile(x,y,z)]
        if (gfx) then
            local sx = x*kTileSize - camx + screen_w/2
            local sy = y*kTileSize - camy + screen_h/2
            love.graphics.draw(gfx,sx,sy) -- x, y, r, sx, sy, ox, oy
        end
    end
    end
    end
    end
end


-- ***** ***** ***** ***** ***** xml parser


-- LoadXML from http://lua-users.org/wiki/LuaXml
function LoadXML(s)
  local function LoadXML_parseargs(s)
    local arg = {}
    string.gsub(s, "(%w+)=([\"'])(.-)%2", function (w, _, a)
    arg[w] = a
    end)
    return arg
  end
  local stack = {}
  local top = {}
  table.insert(stack, top)
  local ni,c,label,xarg, empty
  local i, j = 1, 1
  while true do
    ni,j,c,label,xarg, empty = string.find(s, "<(%/?)([%w:]+)(.-)(%/?)>", i)
    if not ni then break end
    local text = string.sub(s, i, ni-1)
    if not string.find(text, "^%s*$") then
      table.insert(top, text)
    end
    if empty == "/" then  -- empty element tag
      table.insert(top, {label=label, xarg=LoadXML_parseargs(xarg), empty=1})
    elseif c == "" then   -- start tag
      top = {label=label, xarg=LoadXML_parseargs(xarg)}
      table.insert(stack, top)   -- new level
    else  -- end tag
      local toclose = table.remove(stack)  -- remove top
      top = stack[#stack]
      if #stack < 1 then
        error("nothing to close with "..label)
      end
      if toclose.label ~= label then
        error("trying to close "..toclose.label.." with "..label)
      end
      table.insert(top, toclose)
    end
    i = j+1
  end
  local text = string.sub(s, i)
  if not string.find(text, "^%s*$") then
    table.insert(stack[#stack], text)
  end
  if #stack > 1 then
    error("unclosed "..stack[stack.n].label)
  end
  return stack[1]
end


-- ***** ***** ***** ***** ***** parsing the tilemap xml file

local function getTilesets(node)
    local tiles = {}
    for k, sub in ipairs(node) do
        if (sub.label == "tileset") then
            tiles[tonumber(sub.xarg.firstgid)] = sub[1].xarg.source
```

```
        end
    end
    return tiles
end

local function getLayers(node)
    local layers = {}
    layers.width = 0
    layers.height = 0
    for k, sub in ipairs(node) do
        if (sub.label == "layer") then --  and sub.xarg.name == layer_name
            layers.width  = max(layers.width ,tonumber(sub.xarg.width ) or 0)
            layers.height = max(layers.height,tonumber(sub.xarg.height) or 0)
            local layer = {}
            table.insert(layers,layer)
            layer.name = sub.xarg.name
            --~ print("layername",layer.name)
            width = tonumber(sub.xarg.width)
            i = 0
            j = 0
            for l, child in ipairs(sub[1]) do
                if (j == 0) then
                    layer[i] = {}
                end
                layer[i][j] = tonumber(child.xarg.gid)
                j = j + 1
                if j >= width then
                    j = 0
                    i = i + 1
                end
            end
        end
    end
    return layers
end

function TiledMap_Parse(filename)
    local xml = LoadXML(love.filesystem.read(filename))
    local tiles = getTilesets(xml[2])
    local layers = getLayers(xml[2])
    return tiles, layers
end
```

usage demo

(see zip file in forum for map files and graphics)

```
-- small demo for TiledMap loader
love.filesystem.load("tiledmap.lua")()

gKeyPressed = {}
gCamX,gCamY = 100,100

function love.load()
    TiledMap_Load("map/map01.tmx")
end

function love.keyreleased( key )
    gKeyPressed[key] = nil
end

function love.keypressed( key, unicode )
    gKeyPressed[key] = true
    if (key == "escape") then os.exit(0) end
    if (key == " ") then -- space = next map
        gMapNum = (gMapNum or 1) + 1
        if (gMapNum > 10) then gMapNum = 1 end
        TiledMap_Load(string.format("map/map%02d.tmx",gMapNum))
        gCamX,gCamY = 100,100
    end
end
```

```
function love.update( dt )
    local s = 500*dt
    if (gKeyPressed.up) then gCamY = gCamY - s end
    if (gKeyPressed.down) then gCamY = gCamY + s end
    if (gKeyPressed.left) then gCamX = gCamX - s end
    if (gKeyPressed.right) then gCamX = gCamX + s end
end

function love.draw()
    love.graphics.print('arrow-keys=scroll, space=next map', 50, 50)
    love.graphics.setBackgroundColor(0x80,0x80,0x80)
    TiledMap_DrawNearCam(gCamX,gCamY)
end
```

TileMerging

TileMerging · revision 29097 · last edited 2024-01-24

This algorithm is for 2D tile maps. The algorithm generates a (hopefully) minimum set of rectangles that cover all tiles of a certain type.

This is useful for physics where you can generate rectangles to cover all wall tiles. You use the rectangles to create physics bodies that can cover multiple wall tiles, instead of create a physics body for every single tile.

First version

```lua
-- map_width and map_height are the dimensions of the map
-- is_wall_f checks if a tile is a wall

local function tilesMerging (grid, tileSize, is_wall_f) -- tilesMerging_v1
    local function newRect (start_x, start_y, end_x, end_y)
        local new_rect = {
            start_x = start_x,
            start_y = start_y,
            end_x = end_x,
            end_y = end_y,
        }
        return new_rect
    end

    local sortStart_y = function (a, b)
        return a.start_y < b.start_y
    end

    local function rectOverlaps (r, x, start_y, end_y)
        return r.end_x == x - 1
        and start_y <= r.start_y
        and end_y >= r.end_y
    end

    local   map_width = #grid[1]
    local map_height = #grid
    local rectangles = {}

    for x = 0, map_width - 1 do
        local start_y
        local end_y
        for y = 0, map_height - 1 do
            if is_wall_f(grid, x, y) then
                if not start_y then
                    start_y = y
                end
                end_y = y
            elseif start_y then
                local overlaps = {}
                for _, r in ipairs(rectangles) do
                    if rectOverlaps (r, x, start_y, end_y) then
                        table.insert(overlaps, r)
                    end
                end
                table.sort(overlaps, sortStart_y)
                for _, r in ipairs(overlaps) do
                    if start_y < r.start_y then
                        local new_rect = newRect (x, start_y, x, r.start_y - 1)
                        table.insert(rectangles, new_rect)
                        start_y = r.start_y
                    end
                    if start_y == r.start_y then
                        r.end_x = r.end_x + 1
                        if end_y == r.end_y then
                            start_y = nil
                            end_y = nil
                        elseif end_y > r.end_y then
                            start_y = r.end_y + 1
```

```
                    end
                end
            end

            if start_y or (y == map_height - 1) then
                local new_rect = newRect (x, start_y, x, end_y)
                table.insert(rectangles, new_rect)
                start_y = nil
            end
        end
    end
    if start_y then
        local new_rect = newRect (x, start_y, x, end_y)
        table.insert(rectangles, new_rect)
    end
end

-- resize rectangles
for _, r in ipairs(rectangles) do
    r.x = r.start_x * tileSize
    r.y = r.start_y * tileSize
    r.w = (r.end_x - r.start_x + 1) * tileSize
    r.h = (r.end_y - r.start_y + 1) * tileSize
end

return rectangles
end
```

Example:

```
local grid = {
    {1,1,1,1,1,1,1,1},
    {1,0,0,1,1,1,0,1},
    {1,0,1,1,1,1,0,1},
    {1,1,1,0,1,0,1,1},
    {1,1,1,1,0,1,1,1},
    {1,1,1,0,1,0,1,1},
    {1,0,1,1,1,1,0,1},
    {1,0,0,1,1,1,0,1},
    {1,1,1,1,1,1,1,1},
}

local tileSize = 30

local function is_wall_f(grid, x, y)
    if grid[y+1] and grid[y+1][x+1] then
        if grid[y+1][x+1] == 1 then
            return true -- wall
        else
            return false -- not wall
        end
    else
        return false -- out of map, also wall
    end
end

local rectangles = tilesMerging (grid, tileSize, is_wall_f)
print ('amount rectangles:', #rectangles)

function love.draw ()
    love.graphics.translate (5,5)
    for i, r in ipairs (rectangles) do
        love.graphics.setColor (0.8,0.8,0.8,0.8)
        love.graphics.rectangle ('fill', r.x, r.y, r.w, r.h)
        love.graphics.setColor (1,1,1)
        love.graphics.rectangle ('line', r.x, r.y, r.w, r.h)
    end
end
```

It gives 20 following rectangles:


tilesMerging v1.png

Second version

```lua
local function tilesMerging (grid, tileSize, is_wall_f) -- v2
    local function isRectangle(hashMap, x0, y0, w, h)
        for y = y0, y0 + h - 1 do
            for x = x0, x0 + w - 1 do
                if not (hashMap[y] and hashMap[y][x]) then
                    return false
                end
            end
        end
        return true
    end

    local function newRectangle(hashMap, start_x, start_y)
        local w, h = 1, 1
        local end_x, end_y = start_x, start_y
        while true do
--          print ('x, y, w, h', start_x, start_y, w, h)
            local right = isRectangle(hashMap, start_x + w, start_y, 1, h)
            local rightT = isRectangle(hashMap, start_x + w, start_y - 1, 1, h + 2)
            local down = isRectangle(hashMap, start_x, start_y + h, w, 1)
            local downT = isRectangle(hashMap, start_x - 1, start_y + h, w + 2, 1)
            local canDiag = right and down and hashMap[start_y + w] and hashMap[start_x + w][start_y + h]
            if canDiag then
                w, h = w + 1, h + 1
            elseif right and not rightT then
                w = w + 1
            elseif down and not downT then
                h = h + 1
            else
                for y0 = start_y, start_y + h - 1 do
                    for x0 = start_x, start_x + w - 1 do
                        hashMap[y0][x0] = false
                    end
                end
                start_x = start_x-1
                start_y = start_y-1
                local end_x = start_x+w-1
                local end_y = start_y+h-1
--              print (start_x, start_y, end_x, end_y)
                return { start_x = start_x, start_y = start_y, end_x = end_x, end_y = end_y}
            end
        end
    end

    -- create hash of booleans
    local hashMap = {}
    for y = 1, #grid do
        hashMap[y] = {}
        for x = 1, #grid[y] do
            hashMap[y][x] = is_wall_f(grid, x-1, y-1) -- true by 1
        end
    end

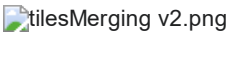
    -- main loop
    local   map_width = #grid[1]
    local map_height = #grid
    local rectangles = {}
    for y = 1, map_height do
        for x = 1, map_width do
            if hashMap[y][x] then
                local rectangle = newRectangle(hashMap, x, y)
                table.insert(rectangles, rectangle)
            end
        end
    end

    -- resize rectangles
```

```
    for _, r in ipairs(rectangles) do
        r.x = r.start_x * tileSize
        r.y = r.start_y * tileSize
        r.w = (r.end_x - r.start_x + 1) * tileSize
        r.h = (r.end_y - r.start_y + 1) * tileSize
    end

    return rectangles
end
```

And it gives 16 rectangles:



Adding physics

Here's how the rectangles would be used for physics.

```
-- Use contents of rectangles to create physics bodies
-- phys_world is the world, wall_rects is the list of...
-- wall rectangles



for _, r in ipairs(rectangles) do
    local start_x = r.start_x * TILE_SIZE
    local start_y = r.start_y * TILE_SIZE
    local width = (r.end_x - r.start_x + 1) * TILE_SIZE
    local height = (r.end_y - r.start_y + 1) * TILE_SIZE

    local x = start_x + (width / 2)
    local y = start_y + (height / 2)

    local body = love.physics.newBody(phys_world, x, y, 0, 0)
    local shape = love.physics.newRectangleShape(body, 0, 0,
      width, height)

    shape:setFriction(0)

    table.insert(wall_rects, {body = body, shape = shape})
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TimeUnit

TimeUnit · revision 28908 · last edited 2025-11-25

Available since LÖVE 0.8.0

This enum is not supported in earlier versions.

Units that represent time.

Constants

seconds
: Regular seconds.

samples
: Audio samples.

See Also

- love.audio
- Source:getDuration
- Source:seek
- Source:tell

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TLbind

TLbind · revision 24435 · last edited 2023-07-05

On this page

- About
  - Download
  - Contact
- Setup
- FAQ
- Functions
- Input types and configuration
- A note about Xbox controllers
- Other Languages

About

TLbind is a system which turns a mass of keyboard/joystick input into easy-to-use game-oriented controls. Making this divide also makes it easy to rebind controls, enable or disable keyboard, mouse, or joystick input at will, and make digital and analogue input work easily together. It's goal is to make it simple to add professional-style configurable "2 sets of keys + optional joystick" controls to Love2D games.

It's under the ZLIB license (http://www.opensource.org/licenses/zlib-license.php).

Download

Direct from Dropbox (http://dl.dropbox.com/u/3713769/web/Love/TLTools/TLbind.lua)

Contact

- Forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=3781)
- Taehl - SelfMadeSpirit@gmail.com

Setup

Procedural style

1. Add TLbind.lua to your game's folder
2. In love.load, add the code `TLbind,control = love.filesystem.load("TLbind.lua")()`
3. In love.update(), add the code `TLbind:update()`

Object-oriented style

1. Add TLbind.lua to your game's folder
2. Assuming your player object is called "player", and you want its controls to be listed in player.control, use the code `player.binds, player.control = love.filesystem.load("TLbind.lua")()`
3. In your player object's update function (or otherwise once per frame), call `player.binds:update()`
4. TLbind itself acts like an object, so you can give each player their own separate TLbind (using `TLbind.giveInstance()`), if you so desire (could be handy for splitscreen)

FAQ

Q) How do I know when a control is pressed?

A) Whenever a control is pressed or held, control[control name] will be true. You could use it with a statement like `if control.up then...`

Q) How do I change a control?

A) To bind a "fire" control to the third button of joystick 1, for example, you could use `TLbind.joyBtns[1][3]="fire"`. From then on, control.fire would be true whenever Joy1 button 3 is pressed.

Q) How do I know if the player has simply pressed the button, instead of holding it? What if I want to know when it's released?

A) When any control is first pressed, control.tap[control name] will be true for one frame. Likewise, when it's released, control.released[control name] will be true for one frame. These would be used the same way you'd use a normal control - `if control.tap.jump then`. You can also use the control.controlPressed and control.controlReleased callbacks (see below).

Q) How does TLbind handle mouse input?

A) Clicking is bound to digital controls, like a joystick button. Cursor position is an analogue pair where the center of the screen is (0,0), the top-left corner is (-1,-1), and the bottom-right corner is (1,1). If the mouse position is bound to the same control as joystick axes, then the control will only use the mouse position when the joystick axes are centered. If you can't seem to move the mouse axes, ensure your joystick is centered (both axes are 0).

Q) What functionality does TLbind have?

A) It separates controls (things your game cares about "jump", "left", etc.) and input (spacebar, joystick coordinates, etc.). This allows a game's controls to be easily configured at any time; lets a game have multiple inputs per control (or controls per input); and makes coding simpler. It makes it easy to bridge analogue and digital control; disable keyboard or joystick input easily; detection of taps instead of holding (no more "hold jump to bounce over and over the second you touch the ground"); and can apply analogue "deadzones" (fixes the "drifting when not touching the stick" problem) and normalizing (preventing the all-too-common "you run faster diagonally" bug).

Q) Is it hard to use?

A) I tried to make it as simple as possible, and list example binds below. With six lines of code, you can give your game industry-standard WASD + Arrows + Gamepad controls.

Q) What's the difference between digital and analogue controls, and how do you make them work together?

A) In TLbind, digital controls are keys or buttons, and are represented as either true or false. Analogue controls take input from joystick axes or balls, and are represented as a decimal between -1 and 1, with 0 being centered. TLbind lets you map an analogue control to two digital controls, which makes the analogue input trigger the digital controls and the digital input push the analogue control to -1 or 1. As an example: If using the example binds below, pressing the joystick a little to the left might make control.horiz = -0.22191, control.left = true, and control.right = false.

Functions

bind:update

```
TLbind:update()
```

This updates the controls, and thus should be called once each frame. Please note that it has a colon, not a period.

TLbind.giveInstance

```
TLbind.giveInstance( binds )
```

This will create a new set of controls from the given binds table. Returns the new instance's binds, control, control.tap, and control.release. Good for OO-style programming, multiplayer controls, etc..

`table binds`

A table of binds, as explained below.

control.controlPressed

```
control.controlPressed(c)
```

This callback works just like love.keypressed, except for controls (meaning it'll work properly with binding changes and joystick input). It gets called exactly once when a control is activated.

`string c`

The name of the control that was pressed.

control.controlReleased

```
control.controlReleased(c)
```

This callback works just like love.keyreleased, except for controls.

`string c`

The name of the control that was released.

Input types and configuration

Bindings are managed under TLbind (procedural example / default) or player.binds (OOP example). A list of bindings is a table, grouped together by input, joystick number, etc.. For clarity, please refer to the following terminology:

- *Control* - an abstract, game-centric variable which will either equal true/false (for digital controls) or a number between -1 and 1 (analogue controls). Controls can have any name you want to give them, but each name must be unique.
- *Input* - the signal a keyboard or joystick gives off when manipulated. Love2D defines these as KeyConstants, MouseConstants, JoystickConstants, joystick numbers, and joystick button, axis, hat, and ball numbers.
- *Bind* - the link between input and controls. "When this input is triggered, set that control". Each input can be bound to only one control, but a control may be bound to many inputs.
- *Map* - links an analogue control to two digital controls. Mapping makes it simpler to work with analogue controls since you don't have to worry about whether or not the player is using analogue or digital input.

bind.keys

This is for keyboard keys. It's a table where the keys are KeyConstants and the values are the names of controls. In other words, you make bindings in the format `TLbind.keys.KeyConstant = "control"`. See

```
TLbind.keys = {
    w="up", a="left", s="down", d="right", [" "]="jump", lctrl="attack", escape="menu",
    up="up", left="left", down="down", right="right", z="jump", rctrl="attack",
}
```

bind.mouseAxes

This is a table that binds the mouse x and y coordinates. It's in the form of `TLbind.mouseAxes[1 for x or 2 for y] = "control"`.

```
TLbind.mouseAxes = { "horiz", "vert" }
```

bind.mouseBtns

This lists mouse buttons, in the form of `TLbind.mouseBtns.MouseConstant = "control"`.

```
TLbind.mouseBtns = { l="target", r="shoot" }
```

bind.joyAxes

This is the list of joystick axes (the stick itself or gamepad thumbstick(s), but may also include rudders, etc.). Using bind.deadzoneAxes on axis pairs is highly recommended. It's in the form of `TLbind.joyAxes[joystick#][axis#] = "control"`.

```
TLbind.joyAxes = { {"horiz", "vert"} }
```

bind.joyBtns

This lists joystick buttons, in the form of `TLbind.joyBtns[joystick#][button#] = "control"`

```
TLbind.joyBtns = { {"jump", "attack", [8]="menu"} }
```

bind.joyBalls

This is for any ball device a joystick may be sporting. Since these are rare, it's advisable to not make your game depend on using one. Their binds are in form of `TLbind.joyBalls[joystick#][ball#] = {"x control", "y control"}`

```
TLbind.joyBalls = { { {"horiz","vert"}, {"yaw","pitch"} } }
```

bind.joyHats

This table is for joystick hats (those mini-thumbstick things seen on some joysticks). Please note that they're digital, not analogue. They're bound in the form of `TLbind.joyHats[joystick#][hat#] = {"l control", "r control", "u control", "d control"}`

```
TLbind.joyHats = { {{"left","right","up","down"}} }
```

bind.maps

This table lists the analogue controls that should be mapped to and from digital controls. Using this, you can easily code for one type of input without having to worry about coding for the other type. They're created in the form of `TLbind.maps[analogue] = {"negative digital", "positive digital"}`

```
TLbind.maps = { horiz={"left","right"}, vert={"up","down"} }
```

bind.deadzoneAxes

This feature applies scaled radial deadzones to a pair of analogue controls. This prevents "stick drift" on old/cheap/uncalibrated devices. It also restricts the analogue pair to the inside of a circle, rather than the default square. Using this, you can avoid the classic "you move faster diagonally" bug. Pairs will reach about 0.7 when at a perfect 45-degree angle (but still reach a full 1 when straight). It's made in the form of `TLbind.deadzoneAxes[entry#] = {"analogue 1", "analogue 2"}`

```
TLbind.deadzoneAxes = { {"horiz", "vert"} }
```

A note about Xbox controllers

Microsoft has decided that Xbox controllers are the official gamepad for Windows, and as such, are the most common controller a player may have. Therefore, it's advisable to tailor your game's controls to work with them.

One important caveat about Xbox controllers: Both of their triggers (L2 and R2 for those of you more familiar with Playstations) are treated as **one axis**, rather than two buttons! Consider using the following binds to make up for it:

```
joyAxes = { {[3]="triggers"} }          -- r trigger is -1!
maps = { triggers={"rtrig","ltrig"} }   -- ltrig and rtrig are the digital controls
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TLfres

TLfres · revision 26720 · last edited 2023-07-05

On this page

- About
- Setup
- FAQ
- Functions
- tlfres.lua
- Example
- Other Languages

About

TLfres aims to make it easy to make any game run at any screen resolution. Using it doesn't even require you to rewrite your graphics code!

It's under the ZLIB license (http://www.opensource.org/licenses/zlib-license.php).

Setup

Everything in the module is contained in the TLfres namespace, so don't worry about it messing with your global variables. Assuming your game was built to use the Love-default resolution of 800×600:

1. Put tlfres.lua in your game's folder
2. At the top of main.lua, add the line `local TLfres = require "tlfres"`
3. At the beginning of love.draw(), add the line `TLfres.beginRendering(800, 600)`
4. At the bottom of love.draw() add the line `TLfres.endRendering()`
5. Whenever you need to know the current scaling factor, call `TLfres.getScale(800, 600)`
6. Whenever you would call love.mouse.getPosition, instead call `TLfres.getMousePosition(800, 600)`
7. Whenever you would call love.graphics.setPointSize(x), instead call `love.graphics.setPointSize(x * TLfres.getScale())` (Love doesn't scale point sizes automatically)

FAQ

Q) Why should I use this instead of love.graphics.scale?

A) Because even stretching the screen to a given res requires tricky math, which TLfres can do for you. Furthermore, most people hate stretched graphics - that's why you should use TLfres's letterboxing option. Instead of stretching the screen, it resizes it in the correct ratio and draws black boxes at the top and bottom if needed.

Q) What resolutions / aspect ratios does it support?

A) Any.

Q) I made my game to use a non-default resolution. Will I have to change everything to make TLfres work with it?

A) Nope! Just call TLfres.beginRendering() with the resolution you built your game for (see below).

Functions

TLfres.beginRendering

```
TLfres.beginRendering(width, height, centered)
```

Call this at the very beginning of love.draw(), before drawing anything. This scales and translates the screen coordinates as needed by the current screen resolution. If you want to draw something at absolute screen coordinates, do it before this function (or after endRendering()).

`number width`

Width of the target canvas.

`number height`

Width of the target canvas.

`boolean centered (false)`

If true, (0, 0) is the middle of the canvas, otherwise it is the top-left corner.

TLfres.endRendering

```
TLfres.endRendering(letterboxColor)
```

Call this at the end of love.draw(), after drawing everything. This draws letterboxes to trim any graphics which should be out of the screen. It defaults to black letterboxes.

table letterboxColor ({0,0,0, 255})
The color of the letterboxes, given in RGBA (just like love.graphics.setColor). Defaults to black.

TLfres.getScale

```
TLfres.getScale(width, height)
```

number width
The canvas width expected by your code.

number height
The canvas height expected by your code.

TLfres.getMousePosition

```
TLfres.getMousePosition(width, height)
```

Use this any time you would normally call love.mouse.getPosition. The returned position is scaled to the given dimensions.

number width
The canvas width expected by your input code.

number height
The canvas height expected by your input code.

tlfres.lua

```lua
local lwGetMode     = _G.love.window.getMode
local lgPush        = _G.love.graphics.push
local lgPop         = _G.love.graphics.pop
local lgTranslate   = _G.love.graphics.translate
local lgScale       = _G.love.graphics.scale
local lgRectangle   = _G.love.graphics.rectangle
local lgSetColor    = _G.love.graphics.setColor
local lmGetPosition = _G.love.mouse.getPosition
local min = math.min

local TLfres = {}

local lastMouseX, lastMouseY = 0, 0
local currentlyRendering

-- Internal helper function
local function _getRawMousePosition(width, height)
   local x, y = lmGetPosition()
   local w, h = lwGetMode()
   local scale = min(w/width, h/height)
   return (x - (w - width * scale) * 0.5)/scale, (y - (h - height * scale) * 0.5)/scale
end

-- Use this any time you would normally call love.mouse.getPosition.
-- The returned position is scaled to the given dimensions.
function TLfres.getMousePosition(width, height)
   local x, y = _getRawMousePosition(width, height)
   if x >= 0 and x <= width and y >= 0 and y <= height then
      lastMouseX, lastMouseY = _getRawMousePosition(width, height)
   end
   return lastMouseX, lastMouseY
end

-- Calculate the current scale based on the desired dimensions and current ones
```

```lua
-- If called within a rendering block, width and height are optional.
function TLfres.getScale(width, height)
   if currentlyRendering then
      width  = width  or currentlyRendering[1]
      height = height or currentlyRendering[2]
   end
   local w, h = lwGetMode()
   return min(w/width, h/height)
end

-- Zooms and centers to fit width×height into the current window.
-- 0,0 is at the top-left of the canvas, or the middle if centered is true.
-- Use love.graphics.push before this and love.graphics.pop after done rendering
function TLfres.beginRendering(width, height, centered)
   if currentlyRendering then
      error("Must call tlfres.endRendering before calling beginRendering.")
      return
   end
   currentlyRendering = {width, height}
   lgPush()

   local w, h = lwGetMode()
   local scale = min(w/width, h/height)
   lgTranslate((w - width * scale) * 0.5, (h - height * scale) * 0.5)
   lgScale(scale)
   if centered then
      lgTranslate(0.5 * width, 0.5 * height)
   end
   return scale
end

local _black = {0, 0, 0, 255}

-- Pops out of the transform; if letterboxColor is true, draws black letterbox
-- bars. letterboxColor can also be any {r, g, b, a} table.
function TLfres.endRendering(letterboxColor)
   if not currentlyRendering then
      error("Must call tlfres.beginRendering before calling endRendering.")
      return
   end
   local width, height = currentlyRendering[1], currentlyRendering[2]
   currentlyRendering = nil
   lgPop()

   local w, h = lwGetMode()
   local scale = min(w/width, h/height)
   width, height = width * scale, height * scale

   lgSetColor(letterboxColor or _black)
   lgRectangle("fill", 0, 0,  w,  0.5 * (h - height)) -- top
   lgRectangle("fill", 0, h,  w, -0.5 * (h - height)) -- bottom
   lgRectangle("fill", 0, 0,  0.5 * (w - width), h)   -- left
   lgRectangle("fill", w, 0, -0.5 * (w - width), h)   -- right
end

return TLfres
```

Example

```lua
local TLfres = require "tlfres"

local CANVAS_WIDTH = 800
local CANVAS_HEIGHT = 600
local POINT_SIZE = 1

function love.mouse.getPosition() -- Override the standard function with our helper function
   return TLfres.getMousePosition(CANVAS_WIDTH, CANVAS_HEIGHT)
end

function love.draw()
   tlfres.beginRendering(CANVAS_WIDTH, CANVAS_HEIGHT)
```

```
      love.graphics.setPointSize(tlfres.getScale()*POINT_SIZE) -- Point size doesn't scale automatically, so multiply it manually.
      love.graphics.points(400, 300) -- Will draw at the center of the canvas no matter how the screen is resized.

   tlfres.endRendering() -- Draw black letterbox
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TLpath

TLpath · revision 28004 · last edited 2023-07-05

On this page

About
- Download
- Contact

Setup

FAQ

Functions

About

TLpath adds pathfinding to LÖVE! It uses an optimized implementation of the excellent A* search algorithm (https://secure.wikimedia.org/wikipedia/en/wiki/A*) used in many commercial games. It comes with a demo which demonstrates its ability to find optimal paths in as little as 0.015 seconds in mazes comprised of over fifty nodes. TLpath uses threading, meaning it will only work in Love2D version 0.7.1 or higher, and requires TSerial (included).

It's under the ZLIB license (http://www.opensource.org/licenses/zlib-license.php).

Download

Direct from Dropbox (http://dl.dropbox.com/u/3713769/web/Love/TLpath.zip)

Contact

- Forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=2680)
- Taehl - SelfMadeSpirit@gmail.com

Setup

Everything in the module is contained in the TLpath namespace, so don't worry about it messing with your global variables. It does its work in a separate thread, allowing the host game to continue running normally.

1. Put TLpath.lua and TSerial.lua in your game's folder
2. At the top of main.lua, add the line `require"TLpath"`
3. In love.update(), add the line `TLpath.update()`

FAQ

Q) What are nodes?

A) A*, like nearly every pathfinding algorithm, uses nodes for increased speed. They are simply points which have connections to other nodes. In TLpath, they are expected to be a list of tables, for example, `{{x=1,y=1}, {x=5,y=3}, {x=2,y=8}, {x=9,y=6}}`. Technically speaking, nodes don't need x and y values, but may contain any relevant information. Nodes may also have a dests table, which contains the indexen of reachable nodes and the g-factor of that connection.

Q) What's a g-factor? An h-factor? An f-factor?

A) I would highly recommend you read about the A* algorithm on the Wikipedia (https://secure.wikimedia.org/wikipedia/en/wiki/A*) for an explanation.

Q) Can TLpath use different functions to calculate g- and h-factors?

A) Yes, with TLpath.setGfunction() and TLpath.setHfunction()

Q) My game has node relationships which are too complex to calculate in one function. Can I set them myself?

A) Yes. If you TLpath.calcNodes() a list of nodes which already have dests tables, the tables will be left alone. Dests are required to be in form of `node.dests = {neighbor_index = g_factor_number}`.

Q) What's the difference between dynamic and static nodes?

A) Static nodes pre-calculate the node graph and g-factors, allowing all pathfinding calculations to be faster, but this calculation may take a moment to complete (your game won't freeze during this time). Dynamic nodes don't have this delay, but it will take longer to find paths. Static mode is generally preferable, unless your game frequently changes nodes and/or their connections.

Q) How do I know what TLpath is doing? What happens if it has an error?

A) TLpath updates the string TLpath.status with messages detailing what it's doing (including timing information if love.timer is available). If TLpath or its thread has an error, it will summon the standard Love2D error screen.

Functions

TLpath.calcNodes

```
TLpath.calcNodes(nodes, dynamic)
```

This function updates the node graph that TLpath uses. Use it whenever the nodes or their connections change. Non-dynamic nodes take a moment to calculate, but are generally preferable to dynamic ones.

table nodes
: A list of nodes. For example: `{{x=1,y=1}, {x=5,y=3}, {x=2,y=8}, {x=9,y=6}}`

boolean dynamic
: If true, node relationships won't be pre-calculated, allowing paths to start being calculated instantly and under changing conditions, but take longer than static nodes. Non-dynamic nodes are recommended.

TLpath.setGfunction

```
TLpath.setGfunction(f)
```

This function is used to specify the g-function. The g-function is called to determine whether a node can reach another, and how "expensive" the resulting path will be. This will likely need to be changed to suit your game.

string f
: A string which is Lua code for a function that will be called when calculating the node graph. The function is passed two nodes as parameters and is expected to return either nil (if a path can't extend from the first to the second) or a number (the g-factor of the path). By default, it is `[[local d = ((b.x-a.x)^2+(b.y-a.y)^2)^0.5 return d<=100 and d or nil]]`, which means that nodes can only be traveled between if they're less than 100 pixels apart.

TLpath.setHfunction

```
TLpath.setHfunction(f)
```

Much like TLpath.setGfunction(), except for calculating the heuristic factor of a node to the goal. This function must return a number which is less than or equal to (NOT greater than) the actual cost of a path from the node to the goal. The closer the h-factor is to the real cost of the path, the faster pathfinding calculations will be. The default h-function simply calculates the distance between the node and the goal, which is fast and should be fine for most games.

string f
: Defaults to `[[return ((goal.x-a.x)^2+(goal.y-a.y)^2)^0.5]]`.

TLpath.findPath

```
TLpath.findPath(ent, goal)
```

This function tells TLpath to calculate a path from ent to goal, where ent is a table that includes a path list, and goal is a node index. Paths are calculated one at a time in the order they're requested, and may take a little bit of time to find, so there may be a small delay between requesting a path and receiving it. When a path is found, the path table of ent will be updated. If the goal isn't able to be reached from ent's current position, ent.path.unreachable will equal true and the ent's current path will be left unchanged.

table ent
: A table which includes a paths table which lists at least one node index (i.e., where the ent is currently located).

number goal
: The index of the goal node (that is, nodes[goal] = the desired node).

TLpath.cancelPathfinding

```
TLpath.cancelPathfinding()
```

Calling this will empty the pathfinding queue and cancel any currently-processing searches. TLpath.calcNodes automatically calls it to prevent bizarre behavior that emerges from changing the node graph while in the middle of finding a path.

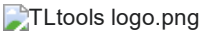
TLpath.update

```
TLpath.update()
```

Call this in love.update() to handle TLpath's events (path request spooling, receiving data from thread, etc.).

TLTools

TLTools · revision 10933 · last edited 2021-03-14



TLtools is a collection of high-quality libraries made by Taehl:

TEsound	A sound manager that makes it easy to use sound and music
TEsound (Español)	Un administrador de sonido que hace más fácil usar sonido y música
TLbind	A simple system for creating professional control schemes
TLfres	Lets games run at any resolution, easily and without stretching
TLpath	Add A* pathfinding to games
Tserial	Converts tables into strings and back. Good for saving games, multiplayer, etc.

Outdated

TLibCompress

Can zip and unzip strings, reducing their size. A good way to save bandwidth when sending strings across a network.

- *TLibCompress.CompressLZW(string)* returns a zipped string
- *TLibCompress.DecompressLZW(string)* returns an unzipped version of a zipped string

Download (http://dl.dropbox.com/u/3713769/w eb/Love/TLTools/TLibCompress.lua)

TouchDeviceType

TouchDeviceType · revision 30602 · last edited 2026-02-04

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Represents the type of device that generated a touch event.

Constants

`string "touchscreen"`
: A physical touchscreen display. Touch coordinates are in window-relative DPI-scaled pixels.

`string "touchpad"`
: A touchpad that provides absolute position. Touch coordinates are normalized values in the range [0, 1].

`string "touchpadrelative"`
: A touchpad that provides relative motion. Touch coordinates are normalized values in the range [0, 1].

Notes

The coordinate system differs based on the device type:

- **touchscreen**: Coordinates are in pixels, relative to the window, and scaled for DPI. These match the coordinate system used by love.graphics.
- **touchpad** and **touchpadrelative**: Coordinates are normalized to the range [0, 1], where (0, 0) is the top-left and (1, 1) is the bottom-right of the touchpad surface.

Trackpad touch events are not generated by default. To receive them, set `t.trackpadtouch = true` in love.conf.

See Also

- love.touch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform

Transform · revision 31478 · last edited 2026-04-20

On this page

- Constructors
- Functions
- Enums
- Supertypes
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This type is not supported in earlier versions.

Object containing a coordinate system transformation.

The love.graphics module has several functions and function variants which accept Transform objects.

Constructors

love.math.newTransform	Creates a new Transform object.	Added since 11.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
Transform:apply	Applies the given other Transform object to this one.	Added since 11.0
Transform:clone	Creates a new copy of this Transform.	Added since 11.0
Transform:getMatrix	Gets the internal transformation matrix stored by this Transform.	Added since 11.0
Transform:inverse	Creates a new Transform containing the inverse of this Transform.	Added since 11.0
Transform:inverseTransformPoint	Applies the reverse of the Transform object's transformation to the given 2D position.	Added since 11.0
Transform:isAffine2DTransform	Checks whether the Transform is an affine transformation.	Added since 11.0
Transform:reset	Resets the Transform to an identity state.	Added since 11.0
Transform:rotate	Applies a rotation to the Transform's coordinate system.	Added since 11.0
Transform:scale	Scales the Transform's coordinate system.	Added since 11.0
Transform:setMatrix	Directly sets the Transform's internal transformation matrix.	Added since 11.0
Transform:setTransformation	Resets the Transform to the specified transformation parameters.	Added since 11.0
Transform:shear	Applies a shear factor (https://en.wikipedia.org/wiki/Shear_mapping) (skew) to the Transform's coordinate system.	Added since 11.0
Transform:transformPoint	Applies the Transform object's transformation to the given 2D position.	Added since 11.0

Transform:translate	Applies a translation to the Transform's coordinate system.	Added since 11.0

Enums

MatrixLayout	The layout of matrix elements (row-major or column-major).	Added since 11.0

Supertypes

- Object

Notes

Transform objects have a custom `*` (multiplication) operator. `result = tA * tB` is equivalent to `result = tA:clone():apply(tB)`. It maps to the matrix multiplication operation that Transform:apply performs.

The `*` operator creates a new Transform object, so it is not recommended to use it heavily in per-frame code.

See Also

- love.math
- love.graphics.applyTransform
- love.graphics.replaceTransform
- love.graphics.draw
- love.graphics.print
- SpriteBatch:add
- Text:add
- Shader:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:apply

Transform:apply · revision 22673 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies the given other Transform object to this one.

This effectively multiplies this Transform's internal transformation matrix with the other Transform's (i.e. `self * other`), and stores the result in this object.

Function

Synopsis

```
transform = Transform:apply( other )
```

Arguments

`Transform other`
: The other Transform object to apply to this Transform.

Returns

`Transform transform`
: The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:clone

Transform:clone · revision 24272 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new copy of this Transform.

Function

Synopsis

```
clone = Transform:clone( )
```

Arguments

None.

Returns

`Transform clone`

The copy of this Transform.

See Also

- Transform
- love.math.newTransform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:getMatrix

Transform:getMatrix · revision 29486 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the internal 4x4 transformation matrix stored by this Transform. The matrix is returned in row-major order.

Function

Synopsis

```
e1_1, e1_2, ..., e4_4 = Transform:getMatrix( )
```

Arguments

None.

Returns

`number e1_1`
: The first column of the first row of the matrix.

`number e1_2`
: The second column of the first row of the matrix.

`number ...`
: Additional matrix elements.

`number e4_4`
: The fourth column of the fourth row of the matrix.

See Also

- Transform
- Transform:setMatrix

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:inverse

Transform:inverse · revision 24274 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new Transform containing the inverse of this Transform.

Function

Synopsis

```
inverse = Transform:inverse( )
```

Arguments

None.

Returns

`Transform inverse`

A new Transform object representing the inverse of this Transform's matrix.

See Also

- Transform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:inverseTransformPoint

Transform:inverseTransformPoint · revision 22665 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies the reverse of the Transform object's transformation to the given 2D position.

This effectively converts the given position from the local coordinate space of the Transform into global coordinates.

One use of this method can be to convert a screen-space mouse position into global world coordinates, if the given Transform has transformations applied that are used for a camera system in-game.

Function

Synopsis

```
globalX, globalY = Transform:inverseTransformPoint( localX, localY )
```

Arguments

`number localX`
: The x component of the position with the transform applied.

`number localY`
: The y component of the position with the transform applied.

Returns

`number globalX`
: The x component of the position in global coordinates.

`number globalY`
: The y component of the position in global coordinates.

See Also

- Transform
- Transform:transformPoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:isAffine2DTransform

Transform:isAffine2DTransform · revision 22374 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Checks whether the Transform is an affine transformation.

Function

Synopsis

```
affine = Transform:isAffine2DTransform()
```

Arguments

None.

Returns

`boolean affine`

`true` if the transform object is an affine transformation, `false` otherwise.

See Also

- Transform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:reset

Transform:reset · revision 24275 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Resets the Transform to an identity state. All previously applied transformations are erased.

Function

Synopsis

```
transform = Transform:reset( )
```

Arguments

None.

Returns

`Transform transform`

The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:rotate

Transform:rotate · revision 22667 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies a rotation to the Transform's coordinate system. This method does not reset any previously applied transformations.

Function

Synopsis

```
transform = Transform:rotate( angle )
```

Arguments

`number angle`
: The relative angle in radians to rotate this Transform by.

Returns

`Transform transform`
: The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform
- Transform:reset
- Transform:translate
- Transform:scale
- Transform:shear
- Transform:setTransformation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:scale

Transform:scale · revision 22668 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Scales the Transform's coordinate system. This method does not reset any previously applied transformations.

Function

Synopsis

```
transform = Transform:scale( sx, sy )
```

Arguments

`number sx`
: The relative scale factor along the x-axis.

`number sy (sx)`
: The relative scale factor along the y-axis.

Returns

`Transform transform`
: The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform
- Transform:reset
- Transform:translate
- Transform:rotate
- Transform:shear
- Transform:setTransformation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:setMatrix

Transform:setMatrix · revision 22675 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Directly sets the Transform's internal 4x4 transformation matrix.

Function

Synopsis

```
transform = Transform:setMatrix( e1_1, e1_2, ..., e4_4 )
```

Arguments

`number e1_1`
: The first column of the first row of the matrix.

`number e1_2`
: The second column of the first row of the matrix.

`number ...`
: Additional matrix elements.

`number e4_4`
: The fourth column of the fourth row of the matrix.

Returns

`Transform transform`
: The Transform object the method was called on. Allows easily chaining Transform methods.

Function

Synopsis

```
transform = Transform:setMatrix( layout, e1_1, e1_2, ..., e4_4 )
```

Arguments

`MatrixLayout layout`
: How to interpret the matrix element arguments (row-major or column-major).

`number e1_1`
: The first column of the first row of the matrix.

`number e1_2`
: The second column of the first row or the first column of the second row of the matrix, depending on the specified layout.

`number ...`
: Additional matrix elements.

`number e4_4`

The fourth column of the fourth row of the matrix.

Returns

`Transform transform`
The Transform object the method was called on. Allows easily chaining Transform methods.

Function

Synopsis

```
transform = Transform:setMatrix( layout, matrix )
```

Arguments

`MatrixLayout layout`
How to interpret the matrix element arguments (row-major or column-major).

`table matrix`
A flat table containing the 16 matrix elements.

Returns

`Transform transform`
The Transform object the method was called on. Allows easily chaining Transform methods.

Function

Synopsis

```
transform = Transform:setMatrix( layout, matrix )
```

Arguments

`MatrixLayout layout`
How to interpret the matrix element arguments (row-major or column-major).

`table matrix`
A table of 4 tables, with each sub-table containing 4 matrix elements.

Returns

`Transform transform`
The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform
- Transform:getMatrix

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:setTransformation

Transform:setTransformation · revision 22663 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Resets the Transform to the specified transformation parameters.

Function

Synopsis

```
transform = Transform:setTransformation( x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

`number x`
: The position of the Transform on the x-axis.

`number y`
: The position of the Transform on the y-axis.

`number angle (0)`
: The orientation of the Transform in radians.

`number sx (1)`
: Scale factor on the x-axis.

`number sy (sx)`
: Scale factor on the y-axis.

`number ox (0)`
: Origin offset on the x-axis.

`number oy (0)`
: Origin offset on the y-axis.

`number kx (0)`
: Shearing / skew factor on the x-axis.

`number ky (0)`
: Shearing / skew factor on the y-axis.

Returns

`Transform transform`
: The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform
- love.math.newTransform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:shear

Transform:shear · revision 22669 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies a shear factor (https://en.w ikipedia.org/w iki/Shear_mapping) (skew) to the Transform's coordinate system. This method does not reset any previously applied transformations.

Function

Synopsis

```
transform = Transform:shear( kx, ky )
```

Arguments

`number kx`
: The shear factor along the x-axis.

`number ky`
: The shear factor along the y-axis.

Returns

`Transform transform`
: The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform
- Transform:reset
- Transform:translate
- Transform:rotate
- Transform:scale
- Transform:setTransformation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:transformPoint

Transform:transformPoint · revision 22664 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies the Transform object's transformation to the given 2D position.

This effectively converts the given position from global coordinates into the local coordinate space of the Transform.

Function

Synopsis

```
localX, localY = Transform:transformPoint( globalX, globalY )
```

Arguments

`number globalX`
: The x component of the position in global coordinates.

`number globalY`
: The y component of the position in global coordinates.

Returns

`number localX`
: The x component of the position with the transform applied.

`number localY`
: The y component of the position with the transform applied.

See Also

- Transform
- Transform:inverseTransformPoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Transform:translate

Transform:translate · revision 22666 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies a translation to the Transform's coordinate system. This method does not reset any previously applied transformations.

Function

Synopsis

```
transform = Transform:translate( dx, dy )
```

Arguments

`number dx`
: The relative translation along the x-axis.

`number dy`
: The relative translation along the y-axis.

Returns

`Transform transform`
: The Transform object the method was called on. Allows easily chaining Transform methods.

See Also

- Transform
- Transform:reset
- Transform:rotate
- Transform:scale
- Transform:shear
- Transform:setTransformation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TransitionMgr

TransitionMgr · revision 25507 · last edited 2023-07-05

TransitionMgr is a simple and quick transition manager, transitions are just considered as lua files. When running a transition you can pass any arbitrary function to it, use this function for things like scene changing.

Designed to be easy to modify, and easy to use.

Learn more and get it at Gitlab.com (https://gitlab.com/V3X3D/love-libs/-/tree/master/TransitionMgr).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Tserial

Tserial · revision 26643 · last edited 2023-07-05

On this page

- About
  - Dow nload
  - Contact
- Setup
- FAQ
- Functions
- Other Languages

About

Tserial is a library for turning tables into strings ("serialization") and vice-versa. This has several uses, including making a game save/load system (you write all your game state to a file) and multiplayer (you can pass tables across the network). Tserial turns tables into Lua script, for maximum ease-of-use.

Download

https://github.com/zhsso/Tserial.lua ↗

Contact

- Taehl - SelfMadeSpirit@gmail.com

Setup

1. Put Tserial.lua in your game's folder
2. At the top of main.lua, add the line `require"Tserial"`

FAQ

Q) Can Tserial pack a table that contains other tables?

A) Yes. Any number of nested tables is fine (it's suitable even for things like tile maps). However, Tserial can NOT handle self-referential tables (such as `t={t}` or `t={{1,2,3},{t}}`).

Q) What data types can/can't be handled by Tserial?

A) It will properly serialize strings, numbers, booleans, and tables. A table may have any of these types as either keys or values. Userdata and functions can't be serialized by default, but can be supported with a little extra work.

Q) How can I serialize userdata?

A) The second parameter of Tserial.pack is used for these. If it's true, Tserial will simply skip userdata and functions. If it's a function, this function will be used to serialize the data. If it's a table, the table will be used for serial lookups.

Functions

Tserial.pack

```
Tserial.pack(t, drop, indent)
```

Serializes a table into a string. Returns a string of Lua script which recreates the table.

`table t`

The table to be serialized. May not be self-referential.

`table drop`

What to do upon encountering data that can't be serialized, like userdata and functions. If absent, Tserial will throw an error upon encountering them. If drop is a function, then it will be called with one parameter (the data) and expect a string in return (so you can supply your own serializing function). If drop is a table, it will be used to look up serials (with data as keys and serials as values).

`boolean indent`

If true, output will be in "human-readable mode" with newlines and proper indentation. Otherwise, the string will all be on one line (resulting in smaller sizes).

Tserial.unpack

```
Tserial.unpack(s, safe)
```

Turns a serial back into a table. Returns a table recreated from the string (or nil and an error message if safe is true and a malformed string was given).

`string s`
: The serial to be turned into a table.

`string safe`
: If true, all extraneous parts of the string will be removed, leaving only a table (this prevents running anomalous code when unpacking untrusted strings). Will also cause malformed tables to quietly return nil and an error message, instead of throwing an error (so your program can't be crashed with a bad string).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

TutorialTestPage

TutorialTestPage · revision 27170 · last edited 2021-09-25

Example browser containing many helpful examples.

For LÖVE 0.9.0: Download (https://github.com/Codingale/LOVE/raw/master/0.9.0%20examples/0.9.0%20examples.love)

For LÖVE 0.8.0: Download (https://github.com/downloads/bartoleo/Love-Examples/examples.love)

For LÖVE 0.7.0: Download (http://commondatastorage.googleapis.com/loveclub/examples.love)



The main menu

tween.lua

tween.lua · revision 19229 · last edited 2023-07-05

Original forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=3015).

Latest version:

https://github.com/kikito/tween.lua ↗

tween.lua simplifies "interpolationish" transitions, like fadeins/fadeouts, gradual movement vs time.

It is based on jQuery's animate (http://api.jquery.com/animate/) method.

Types

Types · revision 24812 · last edited 2023-07-05

On this page

- Types
  - Lua Types
  - Löve Types
- Semantic (Wiki) Types
- Other
- Other Languages

These are the data types that are used by this wiki; some of them are wiki-specific aggregates for clearer documentation reasons.

Types

Lua Types

You can use the **type** function to get the type of a variable.

`nil`
: Denotes the lack of a value. Only 'nil' has the type 'nil'.

`boolean`
: Denotes a logical value. Can be 'true' or 'false'.

`number`
: Denotes a double-precision floating-point value. Includes infinities and not-a-numbers.

`string`
: Denotes an array of characters; immutable.

`table`
: Denotes a combined numerical-array and hash-table.

`function`
: Denotes a callable function.

`userdata`
: Denotes arbitrary C data.

`thread`
: Denotes coroutines.

Löve Types

`Object`
: Superclass of all Löve-specific types below.

Data

`Data`
: Superclass of the below types.

`CompressedData`
: Holds arbitrary compressed data.

`CompressedImageData`
: Holds compressed image data.

`FileData`
: Holds file data.

`FontData`
: Holds font data.

`GlyphData`
: Holds glyph data.

`ImageData`
: Holds image data.

`SoundData`
: Holds sound data.

Math

`BezierCurve`

`RandomGenerator`

Physics

Body

Contact

Fixture

World

Shape

ChainShape

CircleShape

EdgeShape

PolygonShape

Joint

FrictionJoint

GearJoint

MotorJoint

MouseJoint

PrismaticJoint

PulleyJoint

RevoluteJoint

RopeJoint

WeldJoint

WheelJoint

Thread

Channel

Thread

Graphics

Cursor

PixelEffect

Deprecated version of Shader type.

Shader

Quad

Drawable

Texture

Canvas

Framebuffer

Deprecated version of Canvas type.

Image

Mesh

ParticleSystem

SpriteBatch

Text

Video

Sound

Decoder

Source

QueuableSource

Filesystem

File

Fonts

Font

Rasterizer

Input

Joystick

Video

VideoStream

Semantic (Wiki) Types

table flat table
: Denotes a combined numerical-array and hash-table that does not have any table values inside it.

Variant
: Can be a flat table, a boolean, a number, a string, or a LÖVE userdata.

Other

LUBE

ClientObject

ServerObject

Enet

enet.host

enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

UaLove

UaLove · revision 14759 · last edited 2021-03-14

UaLove is a library that changes the way you LÖVE. It is developed and maintained by thelinx.

It changes the global callback way and aims to split everything up in separate hooks.

Current features:

- Dynamic hooks.
- Basic game states.
- pixelMap

Future improvements:

- Class-based system for drawing stuff.
- UI library.

Download, fork and extend today! (http://github.com/TheLinx/UaLove)

UbuntuSecurity

UbuntuSecurity · revision 6309 · last edited 2021-03-14

On this page

- Overview
- Requirements
- The Instructions
- Removing the profile
- Advanced Geek AppArmor Stuff
- Bugs
- Known Working Games
- See also

Akima's simple guide to LÖVE security sandboxing for Ubuntu users.

Overview

AppArmor is a piece of security software that is already present on your Ubuntu computer and already configured by default to protect some software services running on your machine.

This page provides straight forward instructions that explain how to secure the LÖVE engine using an AppArmor profile. The effect of securing the LÖVE engine will be that .love games will have very restricted access to your computer so they can only execute, read or write the files that are likely to be necessary. The games will not be able to access your private files and will not be able to start any other executables on your system.

One drawback of security sandboxing the LÖVE engine is that some .love games may try to do things which the AppArmor profile does not permit. This may partially or completely stop the .love game from working. Nothing bad can happen to your computer or your files as a result of a .love game being denied permission to do something though.

Disclaimer: Until otherwise stated, this LÖVE security enhancement is not endorsed or supported by the LÖVE development team.

Requirements

You must be using an officially packaged .deb install of LÖVE. They can be downloaded on the LÖVE home page (http://love2d.org/).

The following setups have been tested and are confirmed working:

LÖVE versions:

- 0.6.2
- 0.7.1

Ubuntu versions:

- Ubuntu Desktop 10.04 (64bit).

The apparmor profile *should* work on other Canonical (http://www.canonical.com/) supported versions of Ubuntu Desktop.

The Instructions

1) Save the apparmor profile. To do that: select and copy the apparmor profile text below, paste it into a text editor and save it to a file called *usr.bin.love* in the root of your home directory: eg: /home/your_user_name/usr.bin.love

```
# Last Modified: Mon Mar  28 21:50:00 2011
#include <tunables/global>

/usr/bin/love {
  #include <abstractions/X>
  #include <abstractions/audio>
  #include <abstractions/base>

  # Read access to the LOVE executable
  /usr/bin/love r,

  # Read access to misc files required by LOVE
  /etc/openal/alsoft.conf r,
  owner /proc/*/cmdline r,
  /var/lib/dbus/machine-id r,
  owner /var/run/gdm/auth*/database r,

  # Read access to all files with a .love extension
  /**.love r,
```

```
  # Read & write access to user love config & save directories
  owner @{HOME}/.love/ rw,
  owner @{HOME}/.love/** rw,
  owner @{HOME}/.local/share/love/ rw,
  owner @{HOME}/.local/share/love/** rw,

}
```

2) Open a terminal. You should be able to find Gnome Terminal under: Applications -> Accessories -> Terminal

3) Activate the profile. One line at a time enter each of these commands into the terminal:

```
sudo mv ~/usr.bin.love /etc/apparmor.d
sudo chown root:root /etc/apparmor.d/usr.bin.love
sudo apparmor_parser --add /etc/apparmor.d/usr.bin.love
```

(note: each command uses sudo to elevate your user to an administrator. When you enter the first command, using sudo for the first time, you will be prompted for your password. This is normal.)

4) (optional step). If you want to receive desktop notifications when AppArmor restricts a .love game from doing something that is not permitted then install the apparmor-notify package. To do this, simply enter this command into the terminal:

```
sudo apt-get install apparmor-notify
```

Removing the profile

If you ever want to remove the AppArmor security profile you added and go back to how it was before, it is quite simple. Open a terminal again and one line at a time enter each of these commands into the terminal:

```
sudo apparmor_parser --remove /etc/apparmor.d/usr.bin.love
sudo rm /etc/apparmor.d/usr.bin.love
```

That's it!

Advanced Geek AppArmor Stuff

This section is for people who want to know more about AppArmor and perhaps customize, extend or otherwise improve their AppArmor profile.

An AppArmor profile simply contains a series of instructions telling AppArmor what a piece of software can and cannot do. If you want to understand the rules inside a profile better read the **apparmor.d** man page.

If you want to allow .love files to have TCP and UDP network access as well as the ability to resolve hostnames to ip addresses the make then following changes to the AppArmor profile given in *The Instructions*: Add the following line underneith the other #includes:

```
  #include <abstractions/nameservice>
```

After the line:

```
owner @{HOME}/.local/share/love/** rw,
```

...add the following lines:

```
  # Network access
  network udp,
  network tcp,
```

A good place to learn more is the Ubuntu AppArmor wiki page (https://w iki.ubuntu.com/AppArmor).

Bugs

Found a bug? Have a problem? Got a suggested update for this page? Then post in the forums (http://love2d.org/forums/) or PM Akima! (http://love2d.org/forums/memberlist.php?mode=viewprofile&u=2076)

If you confirm the apparmor profile to work on an unlisted version of the LÖVE engine or Ubuntu then update the "Requirements" section of this page accordingly.

Known Working Games

The following .love files have been tested and work fine in the AppArmor sandbox:

- Dave Gone Apeshit 1.2
- AutoBenchmark 0.90
- Desert Loot

See also

- SELÖVE

Underlife

Underlife · revision 7840 · last edited 2021-03-14





(These screenshots are from an early beta build - the graphics are unfinished)

The game

A simulation in entropic oppression: A microcosm of noise, where the inhabitant may strive to create order, but space itself is against you. Features sidescrolling gameplay with building mechanics and exploration as the goal.

Forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=1280)

Download

This game is very incomplete, and isn't ready for distribution yet.

Media

- An .avi video of gameplay, taken 2011/9/21 (2.8 MB) (http://dl.dropbox.com/u/3713769/w eb/Love/Underlife/underlife%202011-09-21%2022-12-08-75.avi)
- A video demonstrating the dynamic shadow system (Youtube) (http://w w w.youtube.com/embed/dgFAyGl9nKs)
- A screenshot showing that the game world can become very, very big (http://dl.dropbox.com/u/3713769/w eb/Love/Underlife/UnderlifeBeta.png)

Unrequited

Unrequited · revision 5317 · last edited 2021-03-14

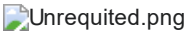
Disregard lady birds, acquire coins. A frantic platformer where the goal is to evade a crafty AI controlled opponent. You can jump, she can use ladders. Use that to your advantage!

This game features a working A* implementation.

- Forum Thread (http://love2d.org/forums/viewtopic.php?f=5&t=2347)

Download

- .love (http://gloryfish.s3.amazonaws.com/games/RedditGameJam-05.love)
- Mac (http://gloryfish.s3.amazonaws.com/games/Unrequited.zip)
- Windows (http://gloryfish.s3.amazonaws.com/games/unrequited_win.zip)
- Source (http://github.com/GloryFish/RedditGameJam-05)



Variant

Variant · revision 29283 · last edited 2026-04-05

The Variant type is not a real lua type, but instead indicates what lua values LÖVE can store internally. It is used in love.thread and love.event. Indeed, as it is a "virtual" type, it has no specific representation in lua, and no methods.

Types

A Variant can be a table, a boolean, a string, a number or LÖVE Objects.

Notes

- Foreign userdata (Lua's files, LuaSocket, ENet, ...), functions and `nil` are not supported.
- There are various restrictions on using tables for Variant:
  - Values within a table are also subject to the Variant rules (i.e. must only contain Variant types).
  - Tables will lose their metatables when used as a Variant.
  - Prior to 11.0 nested tables are not officially supported.
  - Nested tables cannot have cyclical references

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Vectors

Vectors · revision 28223 · last edited 2023-02-23

Lua has operator overloading:

```lua
 -- vector metatable:
local Vector = {}
Vector.__index = Vector

-- vector constructor:
function Vector.new(x, y)
  local v = {x = x or 0, y = y or 0}
  setmetatable(v, Vector)
  return v
end

-- vector addition:
function Vector.__add(a, b)
  return Vector.new(a.x + b.x, a.y + b.y)
end

-- vector subtraction:
function Vector.__sub(a, b)
  return Vector.new(a.x - b.x, a.y - b.y)
end

-- multiplication of a vector by a scalar:
function Vector.__mul(a, b)
  if type(a) == "number" then
    return Vector.new(b.x * a, b.y * a)
  elseif type(b) == "number" then
    return Vector.new(a.x * b, a.y * b)
  else
    error("Can only multiply vector by scalar.")
  end
end

-- dividing a vector by a scalar:
function Vector.__div(a, b)
   if type(b) == "number" then
      return Vector.new(a.x / b, a.y / b)
   else
      error("Invalid argument types for vector division.")
   end
end

-- vector equivalence comparison:
function Vector.__eq(a, b)
    return a.x == b.x and a.y == b.y
end

-- vector not equivalence comparison:
function Vector.__ne(a, b)
    return not Vector.__eq(a, b)
end

-- unary negation operator:
function Vector.__unm(a)
    return Vector.new(-a.x, -a.y)
end

-- vector < comparison:
function Vector.__lt(a, b)
     return a.x < b.x and a.y < b.y
end

-- vector <= comparison:
function Vector.__le(a, b)
     return a.x <= b.x and a.y <= b.y
end
```

```
-- vector value string output:
function Vector.__tostring(v)
     return "(" .. v.x .. ", " .. v.y .. ")"
end
```

Example:

```
local v1 = Vector.new(2, 3)
local v2 = Vector.new(4, 6)


print ( tostring (v1))  -- (2, 3)
print ( tostring (v2))  -- (4, 6)
print ( tostring (v1+v2))   -- (6, 9)
print ( tostring (v1-v2))   -- (-2, -3)
print ( tostring (v1*2))    -- (4, 6)
print ( tostring (v2/2))    -- (2.0, -3.0)
print ( tostring (-v2)) -- (-4, -6)
print ( tostring (v1==v2)) -- false
print ( tostring (v2==v1*2)) -- true
print ( tostring (v1~=v2)) -- true
print ( tostring (v2~=v1*2)) -- false
print ( tostring (v1<=v2)) -- true
print ( tostring (v2<v1*2)) -- false
```

VertexAttributeStep

VertexAttributeStep · revision 25522 · last edited 2026-04-10

Available since LÖVE 11.0

This enum is not supported in earlier versions.

The frequency at which a vertex shader fetches the vertex attribute's data from the Mesh when it's drawn.

Per-instance attributes can be used to render a Mesh many times with different positions, colors, or other attributes via a single love.graphics.drawInstanced call, without using the `love_InstanceID` vertex shader variable.

Constants

pervertex

The vertex attribute will have a unique value for each vertex in the Mesh within a single instance.

perinstance

The vertex attribute will have a unique value for each instance of the Mesh.

See Also

- love.graphics
- Mesh:attachAttribute
- love.graphics.drawInstanced
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VertexWinding

VertexWinding · revision 25482 · last edited 2026-04-19

Available since LÖVE 11.0

This enum is not supported in earlier versions.

How Mesh geometry vertices are ordered.

Constants

cw

Clockwise

ccw

Counter-clockwise

See Also

- love.graphics
- love.graphics.getFrontFaceWinding

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Video

Video · revision 15632 · last edited 2026-04-06

Available since LÖVE 0.10.0
This type is not supported in earlier versions.

A drawable video.

Constructors

love.graphics.newVideo	Creates a new Video.	Added since 0.10.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
Video:getDimensions	Gets the width and height of the Video.	Added since 0.10.0
Video:getFilter	Gets the scaling filters used when drawing the Video.	Added since 0.10.0
Video:getHeight	Gets the height of the Video.	Added since 0.10.0
Video:getSource	Gets the audio Source used for playing back the video's audio.	Added since 0.10.0
Video:getStream	Gets the VideoStream object used for decoding and controlling the video.	Added since 0.10.0
Video:getWidth	Gets the width of the Video.	Added since 0.10.0
Video:isPlaying	Gets whether the Video is currently playing.	Added since 0.10.0
Video:pause	Pauses the Video.	Added since 0.10.0
Video:play	Starts playing the Video.	Added since 0.10.0
Video:rewind	Rewinds the Video to the beginning.	Added since 0.10.0
Video:seek	Sets the current playback position of the Video.	Added since 0.10.0
Video:setFilter	Sets the scaling filters used when drawing the Video.	Added since 0.10.0
Video:setSource	Sets the audio Source used for playing back the video's audio.	Added since 0.10.0
Video:tell	Gets the current playback position of the Video.	Added since 0.10.0

Supertypes

- Drawable
- Object

See Also

- love.graphics
- love.graphics.draw
- VideoStream

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:getDimensions

Video:getDimensions · revision 15672 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the width and height of the Video in pixels.

Function

Synopsis

```
width, height = Video:getDimensions( )
```

Arguments

None.

Returns

number width
: The width of the Video.

number height
: The height of the Video.

See Also

- Video
- Video:getWidth
- Video:getHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:getFilter

Video:getFilter · revision 15663 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the scaling filters used when drawing the Video.

Function

Synopsis

```
min, mag, anisotropy = Video:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: The filter mode used when scaling the Video down.

`FilterMode mag`
: The filter mode used when scaling the Video up.

`number anisotropy (1)`
: Maximum amount of anisotropic filtering used.

See Also

- Video
- Video:setFilter
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:getHeight

Video:getHeight · revision 15670 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the height of the Video in pixels.

Function

Synopsis

```
height = Video:getHeight( )
```

Arguments

None.

Returns

`number height`
: The height of the Video.

See Also

- Video
- Video:getWidth
- Video:getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:getSource

Video:getSource · revision 15658 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the audio Source used for playing back the video's audio. May return nil if the video has no audio, or if Video:setSource is called with a nil argument.

Function

Synopsis

```
source = Video:getSource( )
```

Arguments

None.

Returns

`Source` **`source`**

The audio Source used for audio playback, or nil if the video has no audio.

See Also

- Video
- Video:setSource
- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:getStream

Video:getStream · revision 15665 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the VideoStream object used for decoding and controlling the video.

Function

Synopsis

```
stream = Video:getStream( )
```

Arguments

None.

Returns

`VideoStream stream`

The VideoStream used for decoding and controlling the video.

See Also

- Video
- VideoStream

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:getWidth

Video:getWidth · revision 15669 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the width of the Video in pixels.

Function

Synopsis

```
width = Video:getWidth( )
```

Arguments

None.

Returns

`number width`
: The width of the Video.

See Also

- Video
- Video:getHeight
- Video:getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:isPlaying

Video:isPlaying · revision 15650 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whether the Video is currently playing.

Function

Synopsis

```
playing = Video:isPlaying( )
```

Arguments

None.

Returns

`boolean playing`

Whether the video is playing.

See Also

- Video
- Video:play
- Video:pause

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:pause

Video:pause · revision 15648 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Pauses the Video.

Function

Synopsis

```
Video:pause( )
```

Arguments

None.

Returns

Nothing.

See Also

- Video
- Video:play
- Video:isPlaying

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:play

Video:play · revision 15647 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Starts playing the Video. In order for the video to appear onscreen it must be drawn with love.graphics.draw.

Function

Synopsis

```
Video:play( )
```

Arguments

None.

Returns

Nothing.

See Also

- Video
- Video:pause
- Video:isPlaying

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:rewind

Video:rewind · revision 15649 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Rewinds the Video to the beginning.

Function

Synopsis

```
Video:rewind( )
```

Arguments

None.

Returns

Nothing.

See Also

- Video
- Video:play
- Video:seek
- Video:tell

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:seek

Video:seek · revision 15661 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets the current playback position of the Video.

Function

Synopsis

```
Video:seek( offset )
```

Arguments

`number offset`
: The time in seconds since the beginning of the Video.

Returns

Nothing.

See Also

- Video
- Video:rewind
- Video:tell

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:setFilter

Video:setFilter · revision 15662 · last edited 2026-04-06

On this page
- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0
This function is not supported in earlier versions.

Sets the scaling filters used when drawing the Video.

Function

Synopsis

```
Video:setFilter( min, mag, anisotropy )
```

Arguments

`FilterMode min`
: The filter mode used when scaling the Video down.

`FilterMode mag`
: The filter mode used when scaling the Video up.

`number anisotropy (1)`
: Maximum amount of anisotropic filtering used.

Returns

Nothing.

See Also

- Video
- Video:getFilter
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:setSource

Video:setSource · revision 15666 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets the audio Source used for playing back the video's audio. The audio Source also controls playback speed and synchronization.

Function

Synopsis

```
Video:setSource( source )
```

Arguments

`Source source (nil)`

The audio Source used for audio playback, or nil to disable audio synchronization.

Returns

Nothing.

See Also

- Video
- Video:getSource
- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Video:tell

Video:tell · revision 15660 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the current playback position of the Video.

Function

Synopsis

```
seconds = Video:tell( )
```

Arguments

None.

Returns

`number seconds`

The time in seconds since the beginning of the Video.

See Also

- Video
- Video:rewind
- Video:seek

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream

VideoStream · revision 15797 · last edited 2026-04-06

Available since LÖVE 0.10.0
This type is not supported in earlier versions.

An object which decodes, streams, and controls Videos.

Constructors

love.video.newVideoStream	Creates a new VideoStream.	Added since 0.10.0

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	
VideoStream:getFilename	Gets filename of video stream.	Added since 0.10.0
VideoStream:isPlaying	Gets whatever the video stream is playing.	Added since 0.10.0
VideoStream:pause	Pauses video stream.	Added since 0.10.0
VideoStream:play	Plays video stream.	Added since 0.10.0
VideoStream:rewind	Rewinds video stream.	Added since 0.10.0
VideoStream:seek	Sets the current playback position of the Video.	Added since 0.10.0
VideoStream:setSync	TODO	Added since 0.10.0
VideoStream:tell	Gets the current playback position of video stream.	Added since 0.10.0

Supertypes

- Object

See Also

- love.video
- Video

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:getFilename

VideoStream:getFilename · revision 25373 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets filename of video stream.

Function

Synopsis

```
filename = VideoStream:getFilename( )
```

Arguments

None.

Returns

`string filename`

The filename of video stream

See Also

- VideoStream

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:isPlaying

VideoStream:isPlaying · revision 25376 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whatever the video stream is playing.

Function

Synopsis

```
playing = VideoStream:isPlaying( )
```

Arguments

None.

Returns

`boolean playing`

Whatever video stream is playing.

See Also

- VideoStream
- VideoStream:play
- VideoStream:pause

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:pause

VideoStream:pause · revision 28632 · last edited 2026-04-06

On this page
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0
This function is not supported in earlier versions.

Pauses video stream.

Function

Synopsis

```
VideoStream:pause( )
```

Arguments

None.

Returns

Nothing.

See Also

- VideoStream
- VideoStream:play
- VideoStream:isPlaying

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:play

VideoStream:play · revision 28636 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Plays video stream.

Function

Synopsis

```
VideoStream:play( )
```

Arguments

None.

Returns

Nothing.

See Also

- VideoStream
- VideoStream:pause
- VideoStream:isPlaying

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:rewind

VideoStream:rewind · revision 28634 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Rewinds video stream. Synonym to `VideoStream:seek(0)`

Function

Synopsis

```
VideoStream:rewind( )
```

Arguments

None.

Returns

Nothing.

See Also

- VideoStream
- VideoStream:seek
- VideoStream:tell

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:seek

VideoStream:seek · revision 28783 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets the current playback position of the Video.

Function

Synopsis

```
VideoStream:seek( offset )
```

Arguments

`number offset`
: The time in seconds since the beginning of the Video.

Returns

Nothing.

See Also

- VideoStream
- VideoStream:rewind
- VideoStream:tell

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:setSync

VideoStream:setSync · revision 28776 · last edited 2023-07-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.



This page is still incomplete.

TODO

Function

TODO

Synopsis

```
VideoStream:setSync( source )
```

Arguments

`Source source`

TODO

Returns

Nothing.

Function

TODO

Synopsis

```
VideoStream:setSync( source )
```

Arguments

`VideoStream source`

TODO

Returns

Nothing.

Function

TODO

Synopsis

```
VideoStream:setSync(  )
```

Arguments

None.

Returns

Nothing.

See Also

- VideoStream

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

VideoStream:tell

VideoStream:tell · revision 25379 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the current playback position of video stream.

Function

Synopsis

```
seconds = VideoStream:tell( )
```

Arguments

None.

Returns

`number seconds`

The seconds since the beginning of the video.

See Also

- VideoStream
- VideoStream:seek
- VideoStream:rewind

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Wasx

Wasx · revision 27965 · last edited 2023-07-05

Wasx is a very versatile input manager for LÖVE

There is a lot of "quality of life" features in this library that make your code less redundant and a lot cleaner. Find the repository on GitHub (https://github.com/PhytoEpidemic/Wasx)

WeldJoint

WeldJoint · revision 10253 · last edited 2023-07-05

Available since LÖVE 0.8.0

This type is not supported in earlier versions.

A WeldJoint essentially glues two bodies together.

Constructors

love.physics.newWeldJoint	A WeldJoint essentially glues two bodies together.	Added since 0.8.0

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
WeldJoint:getDampingRatio	Returns the damping ratio of the joint.	Added since 0.8.0	
WeldJoint:getFrequency	Returns the frequency.	Added since 0.8.0	
WeldJoint:getReferenceAngle	Gets the reference angle.	Added since 0.10.2	
WeldJoint:setDampingRatio	Sets a new damping ratio.	Added since 0.8.0	
WeldJoint:setFrequency	Sets a new frequency.	Added since 0.8.0	

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WeldJoint:getDampingRatio

WeldJoint:getDampingRatio · revision 8676 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the damping ratio of the joint.

Function

Synopsis

```
ratio = WeldJoint:getDampingRatio( )
```

Arguments

None.

Returns

`number ratio`

The damping ratio.

See Also

- WeldJoint
- WeldJoint:setDampingRatio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WeldJoint:getFrequency

WeldJoint:getFrequency · revision 8678 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the frequency.

Function

Synopsis

```
freq = WeldJoint:getFrequency( )
```

Arguments

None.

Returns

`number freq`

The frequency in hertz.

See Also

- WeldJoint
- WeldJoint:setFrequency

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

WeldJoint:getReferenceAngle

WeldJoint:getReferenceAngle · revision 26825 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the reference angle.

Function

Synopsis

```
angle = WeldJoint:getReferenceAngle( )
```

Arguments

None.

Returns

`number angle`

The reference angle in radians.

See Also

- WeldJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WeldJoint:setDampingRatio

WeldJoint:setDampingRatio · revision 18793 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new damping ratio.

Function

Synopsis

```
WeldJoint:setDampingRatio( ratio )
```

Arguments

`number ratio`
: The new damping ratio.

Returns

Nothing.

See Also

- WeldJoint
- WeldJoint:getDampingRatio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WeldJoint:setFrequency

WeldJoint:setFrequency · revision 8679 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new frequency.

Function

Synopsis

```
WeldJoint:setFrequency( freq )
```

Arguments

`number freq`
: The new frequency in hertz.

Returns

Nothing.

See Also

- WeldJoint
- WeldJoint:getFrequency

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint

WheelJoint · revision 16463 · last edited 2023-07-05

Restricts a point on the second body to a line on the first body.



Constructors

love.physics.newWheelJoint	Creates a wheel joint.	Added since 0.8.0

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
WheelJoint:enableMotor	Starts and stops the joint motor.	Added since 0.8.0	Removed in 0.9.0
WheelJoint:getAxis	Gets the world-space axis vector of the Prismatic Joint.	Added since 0.10.2	
WheelJoint:getJointSpeed	Returns the current joint translation speed.	Added since 0.8.0	
WheelJoint:getJointTranslation	Returns the current joint translation.	Added since 0.8.0	
WheelJoint:getMaxMotorTorque	Returns the maximum motor torque.	Added since 0.8.0	
WheelJoint:getMotorSpeed	Returns the speed of the motor.	Added since 0.8.0	
WheelJoint:getMotorTorque	Returns the current torque on the motor.	Added since 0.8.0	
WheelJoint:getSpringDampingRatio	Returns the damping ratio.	Added since 0.8.0	
WheelJoint:getSpringFrequency	Returns the spring frequency.	Added since 0.8.0	
WheelJoint:isMotorEnabled	Checks if the joint motor is running.	Added since 0.8.0	
WheelJoint:setMaxMotorTorque	Sets a new maximum motor torque.	Added since 0.8.0	
WheelJoint:setMotorEnabled	Starts and stops the joint motor.	Added since 0.9.0	
WheelJoint:setMotorSpeed	Sets a new speed for the motor.	Added since 0.8.0	
WheelJoint:setSpringDampingRatio	Sets a new damping ratio.	Added since 0.8.0	
WheelJoint:setSpringFrequency	Sets a new spring frequency.	Added since 0.8.0	

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:enableMotor

WheelJoint:enableMotor · revision 13731 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to WheelJoint:setMotorEnabled.

Starts and stops the joint motor.

Function

Synopsis

```
WheelJoint:enableMotor( on )
```

Arguments

`boolean on`

True turns the motor on and false turns it off.

Returns

Nothing.

See Also

- WheelJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getAxis

WheelJoint:getAxis · revision 18118 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the world-space axis vector of the Wheel Joint.

Function

Synopsis

```
x, y = WheelJoint:getAxis( )
```

Arguments

None.

Returns

number **x**
: The x-axis coordinate of the world-space axis vector.

number **y**
: The y-axis coordinate of the world-space axis vector.

See Also

- WheelJoint
- love.physics.newWheelJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getJointSpeed

WheelJoint:getJointSpeed · revision 8664 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the current joint translation speed.

Function

Synopsis

```
speed = WheelJoint:getJointSpeed( )
```

Arguments

None.

Returns

`number speed`

The translation speed of the joint in meters per second.

See Also

- WheelJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getJointTranslation

WheelJoint:getJointTranslation · revision 8665 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the current joint translation.

Function

Synopsis

```
position = WheelJoint:getJointTranslation( )
```

Arguments

None.

Returns

`number position`

The translation of the joint in meters.

See Also

- WheelJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getMaxMotorTorque

WheelJoint:getMaxMotorTorque · revision 8666 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the maximum motor torque.

Function

Synopsis

```
maxTorque = WheelJoint:getMaxMotorTorque( )
```

Arguments

None.

Returns

`number maxTorque`

The maximum torque of the joint motor in newton meters.

See Also

- WheelJoint
- WheelJoint:setMaxMotorTorque

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getMotorSpeed

WheelJoint:getMotorSpeed · revision 8667 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the speed of the motor.

Function

Synopsis

```
speed = WheelJoint:getMotorSpeed( )
```

Arguments

None.

Returns

`number speed`

The speed of the joint motor in radians per second.

See Also

- WheelJoint
- WheelJoint:setMotorSpeed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getMotorTorque

WheelJoint:getMotorTorque · revision 8668 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the current torque on the motor.

Function

Synopsis

```
torque = WheelJoint:getMotorTorque( invdt )
```

Arguments

`number invdt`
: How long the force applies. Usually the inverse time step or 1/dt.

Returns

`number torque`
: The torque on the motor in newton meters.

See Also

- WheelJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getSpringDampingRatio

WheelJoint:getSpringDampingRatio · revision 8669 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the damping ratio.

Function

Synopsis

```
ratio = WheelJoint:getSpringDampingRatio( )
```

Arguments

None.

Returns

`number ratio`
The damping ratio.

See Also

- WheelJoint
- WheelJoint:setSpringDampingRatio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:getSpringFrequency

WheelJoint:getSpringFrequency · revision 8670 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the spring frequency.

Function

Synopsis

```
freq = WheelJoint:getSpringFrequency( )
```

Arguments

None.

Returns

`number freq`

The frequency in hertz.

See Also

- WheelJoint
- WheelJoint:setSpringFrequency

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:isMotorEnabled

WheelJoint:isMotorEnabled · revision 8671 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Checks if the joint motor is running.

Function

Synopsis

```
on = WheelJoint:isMotorEnabled( )
```

Arguments

None.

Returns

`boolean on`

The status of the joint motor.

See Also

- WheelJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:setMaxMotorTorque

WheelJoint:setMaxMotorTorque · revision 8672 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new maximum motor torque.

Function

Synopsis

```
WheelJoint:setMaxMotorTorque( maxTorque )
```

Arguments

`number maxTorque`
: The new maximum torque for the joint motor in newton meters.

Returns

Nothing

See Also

- WheelJoint
- WheelJoint:getMaxMotorTorque

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:setMotorEnabled

WheelJoint:setMotorEnabled · revision 18821 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This method is not supported in earlier versions.

Starts and stops the joint motor.

Function

Synopsis

```
WheelJoint:setMotorEnabled( enable )
```

Arguments

`boolean enable`

True turns the motor on and false turns it off.

Returns

Nothing.

See Also

- WheelJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:setMotorSpeed

WheelJoint:setMotorSpeed · revision 8673 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new speed for the motor.

Function

Synopsis

```
WheelJoint:setMotorSpeed( speed )
```

Arguments

`number speed`
: The new speed for the joint motor in radians per second.

Returns

Nothing.

See Also

- WheelJoint
- WheelJoint:getMotorSpeed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:setSpringDampingRatio

WheelJoint:setSpringDampingRatio · revision 8674 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new damping ratio.

Function

Synopsis

```
WheelJoint:setSpringDampingRatio( ratio )
```

Arguments

`number ratio`
: The new damping ratio.

Returns

Nothing.

See Also

- WheelJoint
- WheelJoint:getSpringDampingRatio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WheelJoint:setSpringFrequency

WheelJoint:setSpringFrequency · revision 8675 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new spring frequency.

Function

Synopsis

```
WheelJoint:setSpringFrequency( freq )
```

Arguments

`number freq`
: The new frequency in hertz.

Returns

Nothing.

See Also

- WheelJoint
- WheelJoint:getSpringFrequency

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World

World · revision 22731 · last edited 2023-07-05

A world is an object that contains all bodies and joints.

Constructors

love.physics.newWorld	Creates a new World.

Functions

Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
World:destroy	Destroys the world.	Added since 0.8.0	
World:getAllowSleeping	Returns the sleep behaviour of the world.	Added since 0.8.0	Removed in 0.9.0
World:getBodies	Returns a table with all bodies.	Added since 11.0	
World:getBodyCount	Returns the number of bodies in the world.		
World:getBodyList	Returns a table with all bodies.	Added since 0.8.0	Deprecated in 11.0
World:getCallbacks	Returns functions for the callbacks during the world update.		
World:getContactCount	Returns the number of contacts in the world.	Added since 0.8.0	
World:getContactFilter	Returns the function for collision filtering.	Added since 0.8.0	
World:getContactList	Returns a table with all contacts.	Added since 0.8.0	Deprecated in 11.0
World:getContacts	Returns a table with all Contacts.	Added since 11.0	
World:getGravity	Get the gravity of the world.		
World:getJointCount	Returns the number of joints in the world.		
World:getJointList	Returns a table with all joints.	Added since 0.8.0	Deprecated in 11.0
World:getJoints	Returns a table with all joints.	Added since 11.0	
World:isAllowSleep	Get the sleep behaviour of the world.		Removed in 0.8.0
World:isDestroyed	Gets whether the World is destroyed.	Added since 0.9.2	
World:isLocked	Returns if the world is updating its state.	Added since 0.8.0	
World:isSleepingAllowed	Gets the sleep behaviour of the world.	Added since 0.9.0	
World:queryBoundingBox	Calls a function for each fixture inside the specified area.	Added since 0.8.0	
World:rayCast	Casts a ray and calls a function with the fixtures that intersect it.	Added since 0.8.0	
World:setAllowSleep	Set the sleep behaviour of the world.		Removed in 0.8.0
World:setAllowSleeping	Sets the sleep behaviour of the world.	Added since 0.8.0	Removed in 0.9.0
World:setCallbacks	Sets functions to be called when shapes collide.	Added since 0.8.0	
World:setContactFilter	Sets a function for collision filtering.	Added since 0.8.0	
World:setGravity	Set the gravity of the world.		
World:setMeter	Set the scale of the world.		Removed in 0.8.0
World:setSleepingAllowed	Sets the sleep behaviour of the world.	Added since 0.9.0	
World:translateOrigin	Translates the World's origin.	Added since 0.9.0	
World:update	Update the state of the world.		

Supertypes

- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

World:destroy

World:destroy · revision 25123 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Destroys the world, taking all bodies, joints, fixtures and their shapes with it.

An error will occur if you attempt to use any of the destroyed objects after calling this function.

Function

Synopsis

```
World:destroy( )
```

Arguments

None.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getAllowSleeping

World:getAllowSleeping · revision 11716 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to World:isSleepingAllowed.

Returns the sleep behaviour of the world.

Function

Synopsis

```
allowSleep = World:getAllowSleeping( )
```

Arguments

None.

Returns

`boolean allowSleep`

True if the bodies are allowed to sleep or false if not.

See Also

- World
- World:setAllowSleeping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

World:getBodies

World:getBodies · revision 24180 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has renamed from World:getBodyList.

Returns a table with all bodies.

Function

Synopsis

```
bodies = World:getBodies( )
```

Arguments

None.

Returns

`table bodies`

A sequence with all bodies.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getBodyCount

World:getBodyCount · revision 8435 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the number of bodies in the world.

Function

Synopsis

```
n = World:getBodyCount( )
```

Arguments

None.

Returns

number n
: The number of bodies in the world.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getBodyList

World:getBodyList · revision 22733 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

It has been renamed to World:getBodies.

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns a table with all bodies.

Function

Synopsis

```
bodies = World:getBodyList( )
```

Arguments

None.

Returns

`table bodies`

A sequence with all bodies.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getCallbacks

World:getCallbacks · revision 8565 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Returns functions for the callbacks during the world update.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
beginContact, endContact, preSolve, postSolve = World:getCallbacks( )
```

Arguments

None.

Returns

`function beginContact`
: Gets called when two fixtures begin to overlap.

`function endContact`
: Gets called when two fixtures cease to overlap.

`function preSolve`
: Gets called before a collision gets resolved.

`function postSolve`
: Gets called after the collision has been resolved.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
add, persist, remove, result = World:getCallbacks( )
```

Arguments

None.

Returns

`function add`
: Called when two shapes first collide.

`function persist`
: Called each frame, if collision lasts more than 1 frame.

`function remove`
: Called when two shapes finish colliding.

`function result`

No idea. Never seems to be called...

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getContactCount

World:getContactCount · revision 8659 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the number of contacts in the world.

Function

Synopsis

```
n = World:getContactCount( )
```

Arguments

None.

Returns

`number n`

The number of contacts in the world.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getContactFilter

World:getContactFilter · revision 8658 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the function for collision filtering.

Function

Synopsis

```
contactFilter = World:getContactFilter( )
```

Arguments

None.

Returns

`function contactFilter`

The function that handles the contact filtering.

See Also

- World
- World:setContactFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getContactList

World:getContactList · revision 22734 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages


Not stable in 0.8.0. You may experience crashing.

Deprecated in LÖVE 11.0

This function has been renamed to World:getContacts.

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns a table with all contacts.

Function

Synopsis

```
contacts = World:getContactList( )
```

Arguments

None.

Returns

`table contacts`

A sequence with all contacts.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getContacts

World:getContacts · revision 22736 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from World:getContactList.

Returns a table with all Contacts.

Function

Synopsis

```
contacts = World:getContacts( )
```

Arguments

None.

Returns

`table contacts`

A sequence with all Contacts.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

World:getGravity

World:getGravity · revision 6169 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the gravity of the world.

Function

Synopsis

```
x, y = World:getGravity( )
```

Arguments

None.

Returns

number x
: The x component of gravity.

number y
: The y component of gravity.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getJointCount

World:getJointCount · revision 8425 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Returns the number of joints in the world.

Function

Synopsis

```
n = World:getJointCount( )
```

Arguments

None.

Returns

`number n`
: The number of joints in the world.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getJointList

World:getJointList · revision 22728 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function has been renamed to World:getJoints.

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns a table with all joints.

Function

Synopsis

```
joints = World:getJointList( )
```

Arguments

None.

Returns

`table joints`

A sequence with all joints.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getJoints

World:getJoints · revision 22729 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from World:getJointList.

Returns a table with all joints.

Function

Synopsis

```
joints = World:getJoints( )
```

Arguments

None.

Returns

table joints

A sequence with all joints.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

World:getMeter

World:getMeter · revision 29205 · last edited 2024-05-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

It has been replaced by love.physics.getMeter.

Get the scale of the world.

The world scale is the number of pixels per meter. Try to keep your shape sizes less than 10 times this scale.

This is important because the physics in Box2D is tuned to work well for objects of size 0.1m up to 10m. All physics coordinates are divided by this number for the physics calculations.

Function

Synopsis

```
scale = World:getMeter( )
```

Arguments

None.

Returns

`number scale`

The size of 1 meter in pixels.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:getShapesInArea

World:getShapesInArea · revision 29139 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Gets a list of Shapes inside the specified area by searching for any overlapping bounding box (Shape:getBoundingBox).

Function

Synopsis

```
shapes = World:getShapesInArea( topLeftX, topLeftY, bottomRightX, bottomRightY )
```

Arguments

number topLeftX
: The x position of the top-left point.

number topLeftY
: The y position of the top-left point.

number bottomRightX
: The x position of the bottom-right point.

number bottomRightY
: The y position of the bottom-right point.

Returns

table shapes
: A sequence with all shapes in the area.

See Also

- World
- World:queryShapesInArea

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:isAllowSleep

World:isAllowSleep · revision 9932 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the sleep behaviour of the world.

A sleeping body is much more efficient to simulate than when awake.

If sleeping is allowed, any body that has come to rest will sleep.

Function

Synopsis

```
permission = World:isAllowSleep( )
```

Arguments

None.

Returns

`boolean permission`

Permission for any body to sleep.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:isDestroyed

World:isDestroyed · revision 14644 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets whether the World is destroyed. Destroyed worlds cannot be used.

Function

Synopsis

```
destroyed = World:isDestroyed( )
```

Arguments

None.

Returns

`boolean destroyed`

Whether the World is destroyed.

See Also

- World
- World:destroy

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:isLocked

World:isLocked · revision 8655 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns if the world is updating its state.

This will return true inside the callbacks from World:setCallbacks.

Function

Synopsis

```
locked = World:isLocked( )
```

Arguments

None.

Returns

`boolean locked`

Will be true if the world is in the process of updating its state.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:isSleepingAllowed

World:isSleepingAllowed · revision 11718 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from World:getAllowSleeping.

Gets the sleep behaviour of the world.

Function

Synopsis

```
allow = World:isSleepingAllowed( )
```

Arguments

None.

Returns

`boolean allow`

True if bodies in the world are allowed to sleep, or false if not.

See Also

- World
- World:setSleepingAllowed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:queryBoundingBox

World:queryBoundingBox · revision 15126 · last edited 2023-07-05

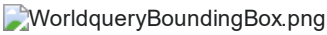
On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Calls a function for each fixture inside the specified area by searching for any overlapping bounding box (Fixture:getBoundingBox).



Function

Synopsis

```
World:queryBoundingBox( topLeftX, topLeftY, bottomRightX, bottomRightY, callback )
```

Arguments

`number topLeftX`
: The x position of the top-left point.

`number topLeftY`
: The y position of the top-left point.

`number bottomRightX`
: The x position of the bottom-right point.

`number bottomRightY`
: The y position of the bottom-right point.

`function callback`
: This function gets passed one argument, the fixture, and should return a boolean. The search will continue if it is true or stop if it is false.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:queryShapesInArea

World:queryShapesInArea · revision 29136 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has replaced World:queryBoundingBox.

Calls a function for each Shape inside the specified area by searching for any overlapping bounding box (Shape:getBoundingBox).

Function

Synopsis

```
World:queryShapesInArea( topLeftX, topLeftY, bottomRightX, bottomRightY, callback )
```

Arguments

`number topLeftX`
: The x position of the top-left point.

`number topLeftY`
: The y position of the top-left point.

`number bottomRightX`
: The x position of the bottom-right point.

`number bottomRightY`
: The y position of the bottom-right point.

`function callback`
: This function gets passed one argument, the Shape, and should return a boolean. The search will continue if it is true or stop if it is false.

Returns

Nothing.

See Also

- World
- World:getShapesInArea

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:rayCast

World:rayCast · revision 31366 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Callback
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Casting a ray over some random shapes.
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Casts a ray and calls a function for each fixture it intersects.

Function

Synopsis

```
World:rayCast( x1, y1, x2, y2, callback )
```

Arguments

number x1
: The x position of the starting point of the ray.

number y1
: The y position of the starting point of the ray.

number x2
: The x position of the end point of the ray.

number y2
: The y position of the end point of the ray.

function callback
: A function called for each fixture intersected by the ray. The function gets six arguments and should return a number as a control value. The intersection points fed into the function will be in an arbitrary order. If you wish to find the closest point of intersection, you'll need to do that yourself within the function. The easiest way to do that is by using the fraction value.

Returns

Nothing.

Callback

Synopsis

```
control = callback( fixture, x, y, xn, yn, fraction )
```

Arguments

Fixture fixture
: The fixture intersecting the ray.

number x
: The x position of the intersection point.

number y
: The y position of the intersection point.

number xn
: The x value of the surface normal vector of the shape edge.

number yn
: The y value of the surface normal vector of the shape edge.

number fraction

The position of the intersection on the ray as a number from 0 to 1 (or even higher if the ray length was changed with the return value).

Returns

number control

The ray can be controlled with the return value. A positive value sets a new ray length where 1 is the default value. A value of 0 terminates the ray. If the callback function returns -1, the intersection gets ignored as if it didn't happen.

Notes

There is a bug in LÖVE 0.8.0 where the normal vector passed to the callback function gets scaled by love.physics.getMeter.

Examples

Casting a ray over some random shapes.

```
function worldRayCastCallback(fixture, x, y, xn, yn, fraction)
    local hit = {}
    hit.fixture = fixture
    hit.x, hit.y = x, y
    hit.xn, hit.yn = xn, yn
    hit.fraction = fraction

    table.insert(Ray.hitList, hit)

    return 1 -- Continues with ray cast through all shapes.
end

function createStuff()
    -- Cleaning up the previous stuff.
    for i = #Terrain.Stuff, 1, -1 do
        Terrain.Stuff[i].Fixture:destroy()
        Terrain.Stuff[i] = nil
    end

    -- Generates some random shapes.
    for i = 1, 30 do
        local p = {}

        p.x, p.y = math.random(100, 700), math.random(100, 500)
        local shapetype = math.random(3)
        if shapetype == 1 then
            local w, h, r = math.random() * 10 + 40, math.random() * 10 + 40, math.random() * math.pi * 2
            p.Shape = love.physics.newRectangleShape(p.x, p.y, w, h, r)
        elseif shapetype == 2 then
            local a = math.random() * math.pi * 2
            local x2, y2 = p.x + math.cos(a) * (math.random() * 30 + 20), p.y + math.sin(a) * (math.random() * 30 + 20)
            p.Shape = love.physics.newEdgeShape(p.x, p.y, x2, y2)
        else
            local r = math.random() * 40 + 10
            p.Shape = love.physics.newCircleShape(p.x, p.y, r)
        end

        p.Fixture = love.physics.newFixture(Terrain.Body, p.Shape)

        Terrain.Stuff[i] = p
    end
end

function love.keypressed()
    createStuff()
end

function love.load()
    -- Setting this to 1 to avoid all current scaling bugs.
    love.physics.setMeter(1)
```

```
    -- Start out with the same random stuff each start.
    math.randomseed(0xfacef00d)

    World = love.physics.newWorld()

    Terrain = {}
    Terrain.Body = love.physics.newBody(World, 0, 0, "static")
    Terrain.Stuff = {}
    createStuff()

    Ray = {
        x1 = 0,
        y1 = 0,
        x2 = 0,
        y2 = 0,
        hitList = {}
    }
end

function love.update(dt)
    local now = love.timer.getTime()

    World:update(dt)

    -- Clear fixture hit list.
    Ray.hitList = {}

    -- Calculate ray position.
    local pos = (math.sin(now/4) + 1.2) * 0.4
    Ray.x2, Ray.y2 = math.cos(pos * (math.pi/2)) * 1000, math.sin(pos * (math.pi/2)) * 1000

    -- Cast the ray and populate the hitList table.
    World:rayCast(Ray.x1, Ray.y1, Ray.x2, Ray.y2, worldRayCastCallback)
end

function love.draw()
    -- Drawing the terrain.
    love.graphics.setColor(1, 1, 1)
    for i, v in ipairs(Terrain.Stuff) do
        if v.Shape:getType() == "polygon" then
            love.graphics.polygon("line", Terrain.Body:getWorldPoints( v.Shape:getPoints() ))
        elseif v.Shape:getType() == "edge" then
            love.graphics.line(Terrain.Body:getWorldPoints( v.Shape:getPoints() ))
        else
            local x, y = Terrain.Body:getWorldPoints(v.x, v.y)
            love.graphics.circle("line", x, y, v.Shape:getRadius())
        end
    end

    -- Drawing the ray.
    love.graphics.setLineWidth(3)
    love.graphics.setColor(1, 1, 1, .4)
    love.graphics.line(Ray.x1, Ray.y1, Ray.x2, Ray.y2)
    love.graphics.setLineWidth(1)

    -- Drawing the intersection points and normal vectors if there were any.
    for i, hit in ipairs(Ray.hitList) do
        love.graphics.setColor(1, 0, 0)
        love.graphics.print(i, hit.x, hit.y) -- Prints the hit order besides the point.
        love.graphics.circle("line", hit.x, hit.y, 3)
        love.graphics.setColor(0, 1, 0)
        love.graphics.line(hit.x, hit.y, hit.x + hit.xn * 25, hit.y + hit.yn * 25)
    end
end
```



Screenshot of the example.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:rayCastAny

World:rayCastAny · revision 29243 · last edited 2024-06-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Casts a ray and returns an intersection point if the ray touched any Shape in the world.

This will not always return the closest intersection point, use World:rayCastClosest or World:rayCast if that's desired. World:rayCastAny is more efficient than World:rayCastClosest.

Function

Synopsis

```
shape, x, y, xn, yn, fraction = World:rayCastAny( x1, y1, x2, y2, categoryMaskBits )
```

Arguments

number x1
: The x position of the starting point of the ray.

number y1
: The y position of the starting point of the ray.

number x2
: The x position of the end point of the ray.

number y2
: The y position of the end point of the ray.

number categoryMaskBits (0xFFFF)
: A bit-mask of categories the raycast can intersect with. Shapes with no categories that match any of this mask's bits will be ignored. This uses the same bit mask format as Shape:setFilterData. The default value intersects with all Shapes.

Returns

Shape shape
: The Shape intersecting the ray, or nil if there was no intersection.

number x
: The x position of the intersection point, or nil if there was no intersection.

number y
: The y position of the intersection point, or nil if there was no intersection.

number xn
: The x value of the surface normal vector of the shape edge, or nil if there was no intersection.

number yn
: The y value of the surface normal vector of the shape edge, or nil if there was no intersection.

number fraction
: The position of the intersection on the ray as a number from 0 to 1, or nil if there was no intersection.

See Also

- World
- World:rayCastClosest

- World:rayCast

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:rayCastClosest

World:rayCastClosest · revision 29244 · last edited 2024-06-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Casts a ray and returns the intersection point of the closest Shape touched by the ray, if any.

World:rayCastAny is more efficient than World:rayCastClosest but will not always return the closest Shape.

Function

Synopsis

```
shape, x, y, xn, yn, fraction = World:rayCastClosest( x1, y1, x2, y2, categoryMaskBits )
```

Arguments

number x1
: The x position of the starting point of the ray.

number y1
: The y position of the starting point of the ray.

number x2
: The x position of the end point of the ray.

number y2
: The y position of the end point of the ray.

number categoryMaskBits (0xFFFF)
: A bit-mask of categories the raycast can intersect with. Shapes with no categories that match any of this mask's bits will be ignored. This uses the same bit mask format as Shape:setFilterData. The default value intersects with all Shapes.

Returns

Shape shape
: The Shape intersecting the ray, or nil if there was no intersection.

number x
: The x position of the intersection point, or nil if there was no intersection.

number y
: The y position of the intersection point, or nil if there was no intersection.

number xn
: The x value of the surface normal vector of the shape edge, or nil if there was no intersection.

number yn
: The y value of the surface normal vector of the shape edge, or nil if there was no intersection.

number fraction
: The position of the intersection on the ray as a number from 0 to 1, or nil if there was no intersection.

See Also

- World
- World:rayCastAny

- World:rayCast

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:setAllowSleep

World:setAllowSleep · revision 8687 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Set the sleep behaviour of the world.

A sleeping body is much more efficient to simulate than when awake.

If sleeping is allowed, any body that has come to rest will sleep.

Function

Synopsis

```
World:setAllowSleep( permission )
```

Arguments

`boolean permission`

Permission for any body to sleep.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:setAllowSleeping

World:setAllowSleeping · revision 11715 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to World:setSleepingAllowed.

Sets the sleep behaviour of the world.

Function

Synopsis

```
World:setAllowSleeping( allowSleep )
```

Arguments

`boolean allowSleep`

True if the bodies are allowed to sleep or false if not.

Returns

Nothing.

See Also

- World
- World:getAllowSleeping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

World:setCallbacks

World:setCallbacks · revision 28080 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Sets functions for the collision callbacks during the world update.

Four Lua functions can be given as arguments. The value nil removes a function.

When called, each function will be passed three arguments. The first two arguments are the colliding fixtures and the third argument is the Contact between them. The postSolve callback additionally gets the normal and tangent impulse for each contact point. See notes.

The Contacts may be destroyed immediately after the callback is called, so if you want to store their data for use later in love.update, you must cache it.

If you are interested to know when exactly each callback is called, consult a Box2d manual (http://www.iforce2d.net/b2dtut/collision-anatomy)


Making changes to a World is not allowed inside of the beginContact, endContact, preSolve, and postSolve callback functions, as Box2D locks the world during these callbacks.

Function

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Synopsis

```
World:setCallbacks( beginContact, endContact, preSolve, postSolve )
```

Arguments

`function beginContact`
: Gets called when two fixtures begin to overlap.

`function endContact`
: Gets called when two fixtures cease to overlap. This will also be called outside of a world update, when colliding objects are destroyed.

`function preSolve`
: Gets called before a collision gets resolved.

`function postSolve`
: Gets called after the collision has been resolved.

Returns

Nothing.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
World:setCallbacks( add, persist, remove, result )
```

Arguments

function add
: Called when two shapes first collide.

function persist
: Called each frame, if collision lasts more than 1 frame.

function remove
: Called when two shapes finish colliding.

function result
: Called after a collision has been calculated. Note: This callback is not properly bound in LOVE at the time of writing, as a result, this callback does not get called, nor do proper arguments get passed for it.

Returns

Nothing.

Notes

Below are the parameters for the postSolve callback. Note that the numbers of normal and tangent impulse correspond with the numbers of contact points.

```
function postSolve(fixture1, fixture2, contact, normal_impulse1, tangent_impulse1, normal_impulse2, tangent_impulse2)
-- do stuff
end
```

See Also

- love.physics
- World
- Contact
- Tutorial:PhysicsCollisionCallbacks

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:setContactFilter

World:setContactFilter · revision 8781 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a function for collision filtering.

If the group and category filtering doesn't generate a collision decision, this function gets called with the two fixtures as arguments. The function should return a boolean value where true means the fixtures will collide and false means they will pass through each other.

Function

Synopsis

```
World:setContactFilter( filter )
```

Arguments

`function filter`
: The function handling the contact filtering.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:setGravity

World:setGravity · revision 6175 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Set the gravity of the world.

Function

Synopsis

```
World:setGravity( x, y )
```

Arguments

number x
: The x component of gravity.

number y
: The y component of gravity.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:setMeter

World:setMeter · revision 29204 · last edited 2024-05-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

It has been replaced by love.physics.setMeter.

Set the scale of the world.

The world scale is the number of pixels per meter. Try to keep your shape sizes less than 10 times this scale.

The default scale for new worlds is 30 pixels per meter.

This is important because the physics in Box2D is tuned to work well for objects of size 0.1m up to 10m. All physics coordinates are divided by this number for the physics calculations.

Function

Synopsis

```
World:setMeter( scale )
```

Arguments

`number scale`
: The size of 1 meter in pixels.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:setSleepingAllowed

World:setSleepingAllowed · revision 11717 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from World:setAllowSleeping.

Sets the sleep behaviour of the world.

Function

Synopsis

```
World:setSleepingAllowed( allow )
```

Arguments

`boolean allow`

True if bodies in the world are allowed to sleep, or false if not.

Returns

Nothing.

See Also

- World
- World:isSleepingAllowed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:translateOrigin

World:translateOrigin · revision 13358 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Translates the World's origin. Useful in large worlds where floating point precision issues become noticeable at far distances from the origin.

Function

Synopsis

```
World:translateOrigin( x, y )
```

Arguments

number x
: The x component of the new origin with respect to the old origin.

number y
: The y component of the new origin with respect to the old origin.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

World:update

World:update · revision 22107 · last edited 2025-09-30

On this page
- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Update the state of the world.

Function

Synopsis

```
World:update( dt, velocityiterations, positioniterations )
```

Arguments

number dt
: The time (in seconds) to advance the physics simulation.

Available since LÖVE 11.0

number velocityiterations (8)
: The maximum number of steps used to determine the new velocities when resolving a collision.

number positioniterations (3)
: The maximum number of steps used to determine the new positions when resolving a collision.

Returns

Nothing.

See Also

- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

WrapMode

WrapMode · revision 28283 · last edited 2026-04-19

How the image wraps inside a Quad with a larger quad size than image size. This also affects how Meshes with texture coordinates which are outside the range of [0, 1] are drawn, and the color returned by the `Texel` Shader function when using it to sample from texture coordinates outside of the range of [0, 1].



Constants

clamp
: Clamp the texture. Appears only once. The area outside the texture's normal range is colored based on the edge pixels of the texture.

repeat
: Repeat the texture. Fills the whole available extent.

Available since LÖVE 0.9.2

mirroredrepeat
: Repeat the texture, flipping it each time it repeats. May produce better visual results than the `repeat` mode when the texture doesn't seamlessly tile.

Available since LÖVE 0.10.0

clampzero
: Clamp the texture. Fills the area outside the texture's normal range with transparent black (or opaque black for textures with no alpha channel.)

Available since LÖVE 12.0

clampone
: Clamp the texture. Fills the area outside the texture's normal range with opaque white.

Notes

The `clampzero` and `clampone` modes are not available on most mobile devices, and will fall back to the `clamp` mode when unsupported. Check the `clampzero` and `clampone` GraphicsFeature constants by calling love.graphics.getSupported.

See Also

- love.graphics
- Texture:setWrap
- Texture:getWrap

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Yonder

Yonder · revision 26725 · last edited 2023-07-05

YONDER

A very simple game state manager.

https://github.com/thenerdie/Yonder ↗

TUTORIAL

First, clone the repo and open it up in your favorite code editor. This library is designed so that you can build straight off a fresh clone of the repo, because it has a fully functional main.lua file that can be extended off of very easily.

Next, go into the ScreenManager.lua file and look for the array titled `gameStates`. This is where you will add your screens.

To add a screen, simply add a new item in the array with the index pointing to what you want the screen to be called (this screenkey is used when you try to switch states) and the value being the required screen module.

For example:

```
local gameStates = { -- this is where you set the directories of your screens
--  SCREENKEY      REQUIRED SCREEN
    ["MainMenu"] = require("Screens/mainmenu")
}
```

Now for the actual contents of the screen, I highly recommend using the template module found here:
https://github.com/thenerdie/Yonder/blob/master/exampleScreen.lua ↗

This template likely all the functionality you would need in any given screen. If you need more, all you have to do is add a new method in ScreenManager.lua that calls the method you want in your screen(s). Then call whatever method you set up in **ScreenManager.lua** in main.lua. Pretty simple, huh?

This should be all the info you need to get started on some serious gamedev awesomeness! If I missed anything, just let me know and I'll add it.

Zoetrope

Zoetrope · revision 28936 · last edited 2023-09-22

Zoetrope is designed to let you hit the ground running with LÖVE. It's similar in design to Flixel (http://flixel.org/), so if you have any experience with that, you should be able to able to figure out Zoetrope quickly. It provides basic functionality for:

- creating classes, subclassing existing ones, reusing objects
- animated sprites, including alpha fading, color tinting, and rotation
- user input via keyboard, mouse, and gamepads
- framerate-independent physics (velocity, acceleration, and drag)
- collision checking
- static and animated tilemaps, loaded from either CSV or Tiled (http://mapeditor.org/) files
- tweening object properties
- recording user input and debugging

Project site (https://github.com/klembot/zoetrope)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Zuli

Zuli · revision 14792 · last edited 2021-03-14

"Zuli is a fun, but challenging arcade game where you take the role of a little albino bat who has to escape from the cave he lives in.

Avoid the deadly foes that fly around the cave and eat all the bugs to release Zuli's power (press Space).

Make sure that any of the enemies you encounter doesn't touch you or you'll have to start the level all over again. Have fun!"

Forum Thread: http://love2d.org/forums/viewtopic.php?f=5&t=77467&p=164653&hilit=Zuli#p164585

Download: https://bitbucket.org/MinusOneGames/zuli-project/downloads ↗

ZURUECKgefilmt

ZURUECKgefilmt · revision 7994 · last edited 2021-03-14

ZURUECKgefilmt

ZURÜCKgefilmt was developed in only 9 hours during the Devmania 2009 Overnight Contest by 3 people.

We also won the Contest with our game :-).

Um was gehts: Ihr kennt sicher alle das Prügelvideo von der "Freiheit statt Angst" Demo in Berlin vom 12.Sep.2009 und genau um dieses Thema ging es bei der diesjährigen Devmania. In diesem Spiel, müsst Ihr als Reporter die prügelnden Polizisten filmen. Für jedes Foto erhaltet Ihr einen Punkt. Aber seid vorsichtig, Rache könnte sich gegen euch richten ;-). Falls euch die Kamera abgenommen wird, könnt Ihr eine neue am Pressestand abholen. Hier könnt Ihr auch zwischenzeitlich eure bereits geschossenen Fotos abgeben, damit Sie euch nicht von der Polizei abgenommen werden. Gewonnen hat, wer zum Schluss am meisten Punkte hat (Punktestand zusammenrechnen).

Achja, in den letzten 10 Sekunden läuft die Polizei Amok.

Also aufpassen :-)!



Downloads for Windows/Linux/MacOSX

http://gamejams.schattenkind.net/2011/10/zurueckgefilmt.html (http://gamejams.schattenkind.net/2011/10/zurueckgefilmt.html)

ZURUECKgefilmt